

目录

前言.....	8
1. 第一章 Java 开发环境搭建.....	9
1.1 章节目标与知识框架.....	9
1.1.1 章节目标.....	9
1.1.2 知识框架.....	9
1.2 Java 语言概述（了解）.....	9
1.3 Java 语言发展史（了解）.....	10
1.4 Java 语言特性（了解）.....	13
1.5 JDK、JRE、JVM 三者关系（理解）.....	14
1.6 初步了解 Java 的加载与执行（了解）.....	15
1.7 开发前的准备工作（掌握）.....	17
1.7.1 windows 显示文件扩展名.....	17
1.7.2 windows 系统的常用 DOS 命令.....	18
1.7.3 文本编辑器的安装与配置.....	22
1.7.4 常用文本编辑快捷键.....	26
1.8 开发第一个 Java 程序（掌握）.....	27
1.8.1 JDK 的下载与安装.....	27
1.8.2 JDK 的 bin 目录.....	32
1.8.3 编写 HelloWorld.....	33
1.8.4 编译 HelloWorld 程序.....	33
1.8.5 运行 HelloWorld 程序.....	41
1.9 对 HelloWorld 程序的解释（理解）.....	47
1.10 Java 中的注释（掌握）.....	48
1.10.1 注释的作用.....	48
1.10.2 注释的三种方式.....	48
1.10.3 注释应该怎么写.....	49
1.10.4 为 HelloWorld 提供注释.....	49
1.11 public class 和 class 的区别（掌握）.....	50

1.12 章节小结.....	53
1.13 难点解惑.....	53
1.13.1 JDK 常用基本组件.....	53
1.13.2 运行时出现“无法加载主类”.....	53
1.14 章节习题.....	54
1.15 习题答案.....	54
2. 第二章 标识符与关键字.....	55
2.1 章节目标与知识框架.....	55
2.1.1 章节目标.....	55
2.1.2 知识框架.....	56
2.2 标识符概述（了解）.....	56
2.3 标识符详解.....	57
2.3.1 标识符都可以标识什么（理解）.....	57
2.3.2 标识符命名规则（掌握）.....	57
2.3.3 标识符命名规范（掌握）.....	57
2.4 关键字（理解）.....	59
2.5 章节小结.....	62
2.6 难点解惑.....	62
2.7 章节习题.....	65
2.8 习题答案.....	65
3. 第三章 变量.....	65
3.1 章节目标与知识框架.....	65
3.1.1 章节目标.....	65
3.1.2 知识框架.....	66
3.2 字面量（理解）.....	66
3.3 变量.....	70
3.3.1 变量概述（理解）.....	70
3.3.2 使用变量（掌握）.....	72
3.3.3 变量分类（了解）.....	81
3.3.4 变量作用域（理解）.....	81

3.4 章节小结.....	85
3.5 难点解惑.....	85
3.6 章节习题.....	87
3.7 习题答案.....	88
4. 第四章 数据类型	89
4.1 章节目标与知识框架.....	89
4.1.1 章节目标.....	89
4.1.2 知识框架.....	90
4.2 数据类型概述（理解）	90
4.3 字符编码（理解）	93
4.4 数据类型详解	94
4.4.1 字符型详解（理解）	95
4.4.2 整数型详解（理解）	100
4.4.3 布尔型详解（理解）	114
4.4.4 浮点型详解（理解）	116
4.5 基本数据类型转换（理解）	118
4.6 章节小结.....	122
4.7 难点解惑.....	123
4.8 章节习题.....	123
4.9 习题答案.....	123
5. 第五章 运算符.....	124
5.1 章节目标与知识框架.....	124
5.1.1 章节目标.....	124
5.1.2 知识框架.....	124
5.2 运算符概述（了解）	125
5.3 运算符详解.....	126
5.3.1 算术运算符（掌握）	126
5.3.2 关系运算符（掌握）	129
5.3.3 逻辑运算符（掌握）	130
5.3.4 赋值运算符（掌握）	133

5.3.5 条件运算符（掌握）	135
5.3.6 字符串连接运算符（掌握）	136
5.4 章节小结.....	139
5.5 难点解惑.....	139
5.6 章节习题.....	140
5.7 习题答案.....	143
6. 第六章 控制语句	145
6.1 章节目标与知识框架.....	145
6.1.1 章节目标.....	145
6.1.2 知识框架.....	145
6.2 控制语句概述（理解）	145
6.3 选择语句.....	146
6.3.1 if（掌握）	146
6.3.2 switch（掌握）	155
6.4 循环语句.....	160
6.4.1 for（掌握）	161
6.4.2 while（掌握）	173
6.4.3 do...while（掌握）	177
6.5 转向语句.....	179
6.5.1 break（掌握）	179
6.5.2 continue（掌握）	183
6.6 章节小结.....	185
6.7 难点解惑.....	185
6.8 章节习题.....	186
6.9 习题答案.....	187
7. 第七章 方法.....	191
7.1 章节目标与知识框架.....	191
7.1.1 章节目标.....	191
7.1.2 知识框架.....	192
7.2 方法（掌握）	192

7.2.1 方法的本质以及作用（理解）	192
7.2.2 方法的定义以及调用（掌握）	194
7.2.3 方法返回值详解（掌握）	198
7.2.4 栈数据结构（理解）	204
7.2.5 方法执行过程中内存的变化（理解）	207
7.3 方法重载/overload（掌握）	210
7.4 方法递归（理解）	216
7.5 章节小结.....	224
7.6 难点解惑.....	224
7.7 章节习题.....	225
7.8 习题答案.....	225
8. 第八章 认识面向对象.....	227
8.1 章节目标与知识框架.....	227
8.1.1 章节目标.....	227
8.1.2 知识框架.....	228
8.2 面向过程和面向对象的区别（了解）	228
8.3 面向对象三大特征（了解）	230
8.4 类.....	230
8.4.1 类和对象的概念（理解）	230
8.4.2 类的发现和设计（了解）	231
8.4.3 类的定义（掌握）	232
8.5 章节小结.....	233
8.6 难点解惑.....	234
8.7 章节习题.....	234
8.8 习题答案.....	234
9. 第九章 对象的创建和使用.....	236
9.1 章节目标与知识框架.....	236
9.1.1 章节目标.....	236
9.1.2 知识框架.....	236
9.2 对象的创建和使用（掌握）	236

9.2.1 对象的创建.....	236
9.2.2 对象的使用.....	238
9.3 对象创建和使用的深层次解密	240
9.3.1 java 虚拟机内存管理（理解）	240
9.3.2 构造方法 Constructor（掌握）	248
9.3.3 空指针异常（掌握）	259
9.3.4 当实例变量是一个引用（理解）	262
9.3.5 方法调用时参数的传递问题（理解）	268
9.4 章节小结.....	273
9.5 难点解惑.....	274
9.6 章节习题.....	274
9.7 习题答案.....	274
10. 第十章 封装.....	276
10.1 章节目标与知识框架.....	276
10.1.1 章节目标.....	276
10.1.2 知识框架.....	277
10.2 封装（掌握）	277
10.2.1 封装的理解.....	277
10.2.2 不封装存在的问题.....	278
10.2.3 怎么封装.....	279
10.3 章节小结.....	284
10.4 难点解惑.....	284
10.5 章节习题.....	284
10.6 习题答案.....	285
11. 第十一章 this 和 static.....	287
11.1 章节目标与知识框架.....	287
11.1.1 章节目标.....	287
11.1.2 知识框架.....	287
11.2 this（掌握）	287
11.2.1 this 是什么	287

11.2.2 this 使用在实例方法中	289
11.2.3 this 使用在构造方法中	302
11.3 static（掌握）	308
11.3.1 static 概述.....	308
11.3.2 静态变量.....	308
11.3.3 静态代码块.....	315
11.3.4 静态方法.....	318
11.4 章节小结.....	321
11.5 难点解惑.....	322
11.6 章节习题.....	322
11.7 习题答案.....	324
12. 第十二章 继承（Inheritance）	325
12.1 章节目标与知识框架.....	325
12.1.1 章节目标.....	325
12.1.2 知识框架.....	325
12.2 继承.....	325
12.2.1 继承概述（理解）	325
12.2.2 如何继承（掌握）	326
12.2.3 继承的相关特性（掌握）	330
12.2.4 对继承自 Object 类的方法的测试（理解）	331
12.3 章节小结.....	332
12.4 难点解惑.....	332
12.5 章节习题.....	333
12.6 习题答案.....	334
13. 第十三章 方法覆盖和多态（Polymorphism）	336
13.1 章节目标与知识框架.....	336
13.1.1 章节目标.....	336
13.1.2 知识框架.....	337
13.2 方法覆盖 Override.....	337
13.2.1 什么时候需要方法覆盖（理解）	337

13.2.2 怎么进行方法覆盖（掌握）	339
13.2.3 方法覆盖的条件及注意事项（掌握）	341
13.3 多态	343
13.3.1 多态基础语法（掌握）	343
13.3.2 多态在开发中的作用（理解）	352
13.4 章节小结	361
13.5 难点解惑	362
13.6 章节习题	362
13.7 习题答案	363
14. 第十四章 super	372
14.1 章节目标与知识框架	372
14.1.1 章节目标	372
14.1.2 知识框架	373
14.2 super	373
14.2.1 super 概述（理解）	373
14.2.2 super 使用在构造方法中（掌握）	376
14.2.3 super 使用在实例方法中（掌握）	384
14.3 章节小结	390
14.4 难点解惑	390
14.5 章节习题	391
14.6 习题答案	392

前言

本书籍为实战型教材，由动力节点-Java 研究院出品，适合 Java 编程零基础的学员阅读；书中理论与实践相结合，采用一个知识点对应一个经典小案例进行讲解，深入浅出；

本书对知识点进行了三个级别的描述：了解、理解、掌握。了解的内容知道即可，掌握的内容不但要求理解，而且能够将代码实现。

在此强调一下，本书对知识点不会进行学术性的研究，比较偏向实战；通过本书的学习，可以让您快速入门 Java 编程。

【注：动力节点成立于 2009 年，多年来针对 Java 教育产品深耕细作，对 Java 教育行业进行垂直生态布局，覆盖所有 Java 人群；目前有两大大事业部（线下事业部和线上事业部），旗下产品包括：Java 实验班、Java 就业班、Java 夜校班、Java 架构师班、Java 同步直播班、专注 Java 在线教育平台：蛙课网等。】

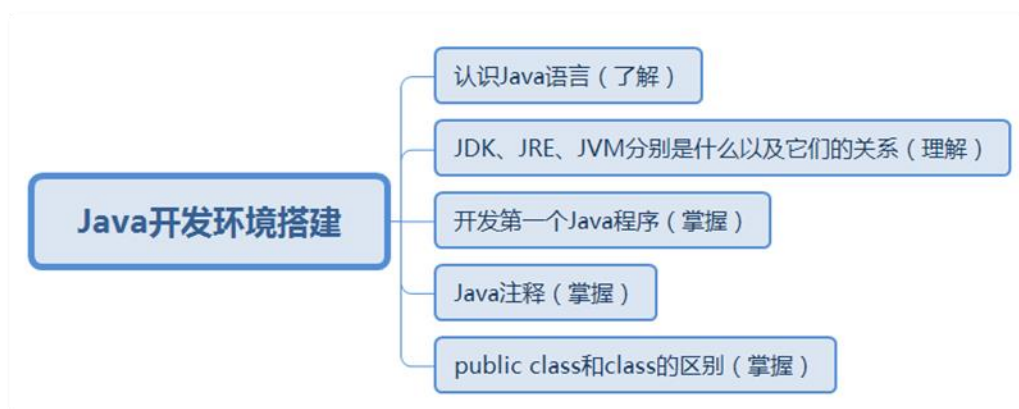
1. 第一章 Java 开发环境搭建

1.1 章节目标与知识框架

1.1.1 章节目标

掌握 Java 的开发环境搭建，会编写 HelloWorld 程序，并能够准确的进行编译和运行；理解 path 和 classpath 环境变量并可以自行配置。

1.1.2 知识框架



1.2 Java 语言概述（了解）

Java 编程语言是 Sun Microsystems 公司的 James Gosling 在 1990 年创建的，于 1995 年公布于世（一般说 Java 诞生于 1995 年）。Java 是一门面向对象的编程语言，不仅吸收了 C++ 语言的各种优点，还摒弃了 C++ 里难以理解的多继承、指针等概念，因此 Java 语言具有功能强大

和简单易用两个特征。Java 语言作为面向对象编程语言的代表，极好地实现了面向对象理论，允许程序员以优雅的思维方式进行复杂的编程。

目前在世界各地都有 Java 程序员的存在，你走到哪里也不再孤单，因为你的 Java 编程小伙伴在陪伴着你。我们一起来看看世界编程语言排行榜 TIOBE (<https://www.tiobe.com/tiobe-index/>) 给出的统计数据。

May 2018	May 2017	Change	Programming Language	Ratings	Change
1	1		Java	16.380%	+1.74%
2	2		C	14.000%	+7.00%
3	3		C++	7.668%	+2.92%
4	4		Python	5.192%	+1.64%
5	5		C#	4.402%	+0.95%
6	6		Visual Basic .NET	4.124%	+0.73%
7	9	▲	PHP	3.321%	+0.63%
8	7	▼	JavaScript	2.923%	-0.15%
9	-	▲▲	SQL	1.987%	+1.99%
10	11	▲	Ruby	1.182%	-1.25%

图 1-1: 2018 年 5 月编程语言排行榜

上图是 TIOBE 排行榜给出的 2018 年 5 月份的编程语言排行榜的统计数据，如果你一直在关注编程语言排行榜，那么你应该很容易发现 Java 的排名多年以来一直在前两名徘徊，并且多数情况下都是以排名第一的形式存在。

1.3 Java 语言发展史（了解）

1990 年末，Sun 公司准备为下一代智能家电（电视机，微波炉，电话）编写一个通用的控制系统。该团队最初考虑使用 C++ 语言，很多成员包括 Sun 公司的首席科学家 Bill Joy，发现 C++ 语言在某些方面复杂，系统资源极其有限，缺少垃圾回收系统等，于是 Bill Joy 决定开发一种新的语言：Oak。

1992 年夏天，Green 计划已经完成新平台的部分功能，包括 Green 操作系统，Oak 的程序

设计语言、类库等。同年 11 月，Green 计划被转成“FirstPerson 有限公司”，一个 Sun 公司的全资子公司。该团队致力于创建一种高度互动的设备。

1994 年夏天，互联网和浏览器的出现不仅给广大互联网的用户带来了福音，也给 Oak 语言带来了新的生机。James Gosling（Java 之父）立即意识到，这是一个机会，于是对 Oak 进行了小规模改造。

1994 年秋，小组中的 Naughton 和 Jonathan payne 完成了第一个 Java 语言的网页浏览器：WebRunner。Sun 公司实验室主任 Bert Sutherland 和技术总监 Eric Schmidt 观看了该网页的演示并给予了高度的评价。当时 Oak 这个商标已经被注册了，于是将 Oak 改名为 Java。

1995 年初，Sun 公司发布 Java 语言，Sun 公司直接把 Java 放到互联网上，免费给大家使用，甚至连源代码也不保密，也放在互联网公开。几个月后，Java 成了互联网上最热门的宝贝。各种各样的小程序层出不穷，Java 终于扬眉吐气，成为了一种广为人知的编程语言。

1996 年底，Flash 问世了，这是一种更加简单的动画设计软件：使用 Flash 几乎无须任何编程语言知识，就可以做出丰富多彩的动画。Flash 逐渐蚕食了 Java 在网页上的应用。

1997 年 2 月 18 日，Sun 公司发布了 JDK1.1，增加了即时编译器 JIT。

1995 年 Java 诞生到 1998 年底，Java 语言虽然成为了互联网上广泛使用的编程语言，但它没有找到准确的定位。

1998 年 12 月，Sun 发布了 Java 历史上最重要的 JDK 版本：JDK1.2。并将 Java 分成了 J2EE（提供了企业应用开发相关的完整解决方案）、J2SE（整个 Java 技术的核心和基础）、J2ME（主要用于控制移动设备和信息家电等有限存储的设备）三个版本。

2002 年 2 月，Sun 发布了 JDK 历史上最为成熟的版本，JDK1.4。

2004 年 10 月，Sun 发布了万众期待的 JDK1.5。JDK1.5 增加了诸如泛型、增强的 for 语句、可变数量的形参、注释、自动拆箱和装箱等。

2005 年，Java 诞生十周年，J2SE/J2EE/J2ME 分别改名为：JavaSE/JavaEE/JavaME。

2006 年 12 月，Sun 发布了 JDK1.6。

2009 年 4 月 20 日，Oracle 甲骨文公司宣布将以每股 9.5 美元的价格收购 Sun。Oracle 通过收购 Sun 获得了两项资产：Java 和 Solaris。

2007 年 11 月，Google 宣布推出一款基于 Linux 平台的开源手机操作系统：Android。Android 使用 Java 语言来开发应用程序。Android 平台的流行，让 Java 语言获得了在客户端程序上大展拳脚的机会。

2011 年 7 月 28 日，Oracle 发布了 Java SE7，这次版本升级耗时将近 5 年时间。引入二进制整数、支持字符串的 switch 语句等。

2014 年 3 月 18 日，Oracle 发布了 Java SE 8。

2017 年 7 月，Oracle 发布了 JavaSE 9。

2018 年 3 月 20 日，Oracle 发布了正式版 JavaSE 10。同一年 9 月 25 日发布了 Java11。

2019 年 3 月 19 日，Oracle 发布了 Java12。

以上的描述就是 Java 一路走来发展历程，我们只能说：Java，你好坚挺啊！在这个发展的过程中一代语言的兴起又衰败，又兴起又衰败，但 Java 这 24 年来一直立于不败之地。并且渗透到每个行业，已然根深蒂固。

在以上的描述中，我们提到了 Java 包括三大块，分别是 JavaSE、JavaEE、JavaME，这三者之间存在什么样的关系呢？请看下图你就明白了：

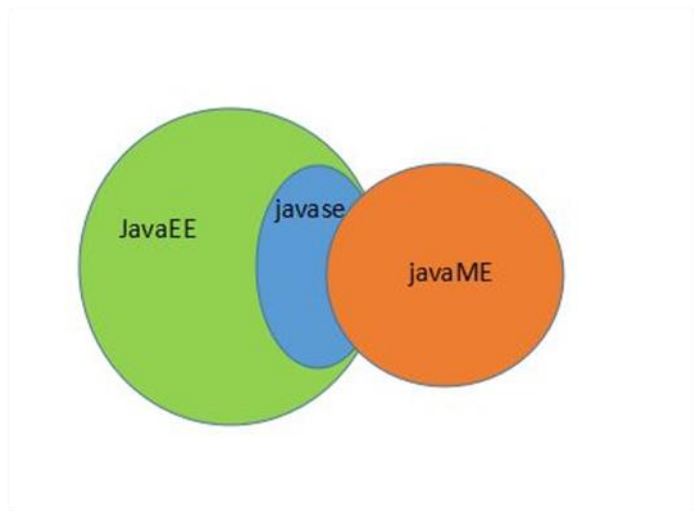


图 1-2：JavaSE、JavaEE、JavaME 三者关系

如上图所示，JavaEE 和 JavaME 都包含 JavaSE。实际上，这三大块就是 Java 的三大版本，JavaSE 是 Java 的标准版，是学习 JavaEE 和 JavaME 的基础，JavaEE 是企业版，JavaME 是微型版。

Java SE (Java Platform, Standard Edition)。Java SE 以前称为 J2SE。它允许开发和部署在桌面、服务器、嵌入式环境和实时环境中使用的 Java 应用程序。Java SE 包含了支持 Java Web 服务开发的类，并为 Java Platform, Enterprise Edition (Java EE) 提供基础。

Java EE (Java Platform, Enterprise Edition)。这个版本以前称为 J2EE。企业版本帮助开发和部署可移植、健壮、可伸缩且安全的服务器端 Java 应用程序。Java EE 是在 Java SE 的基础上构建的，它提供 Web 服务、组件模型、管理和通信 API，可以用来实现企业级的面向服务体系结构 (service-oriented architecture, SOA) 和 Web 2.0 应用程序。

Java ME (Java Platform, Micro Edition)。这个版本以前称为 J2ME。Java ME 为在移动设备和嵌入式设备 (比如手机、PDA、电视机顶盒和打印机) 上运行的应用程序提供一个健壮且灵活的环境。Java ME 包括灵活的用户界面、健壮的安全模型、许多内置的网络协议以及对可以动态下载的连网和离线应用程序的丰富支持。基于 Java ME 规范的应用程序只需编写一次，就可以用于许多设备，而且可以利用每个设备的本机功能。

综上所述，我们用几句话简单概括一下它们之间的区别。Java 标准版，主要做一般的 Java 应用，比如应用软件/ QQ 之类的通信软件等等。Java 企业版，主要做企业应用，比如公司网站，企业解决方案等。Java 微型版，主要面向嵌入式等设备应用的开发，比如手机游戏等。

1.4 Java 语言特性（了解）

sun 公司对 Java 的描述: "Java is a simple, object-oriented, distributed, interpreted, robust, secure, architecture neutral, portable, high-performance, multithreaded, and dynamic language". 翻译起来就是: “Java 是一门简单的, 面向对象, 分布式, 解释性, 健壮的, 安全的, 结构中立的, 便捷的, 高性能的, 多线程的, 动态的语言”。那么, 在学习 Java 编程语言之前, 让我们一起来看看它有哪些特性吧?

① 简单性: Java 语言底层采用 C++语言实现, 相对于 C++来说, Java 是简单的, 在 Java 语言中程序员不需要再操作复杂的指针(指针的操作是很复杂的), 继承方面也是只支持单继承(C++语言是一种半面向对象的编程语言, 支持多继承, 多继承会导致关系很复杂), 在很多方面进行了简化。

② 面向对象: Java 中提供了封装、继承、多态等面向对象的机制。

③ **健壮性**: 在 C++程序当中的无用数据/垃圾数据需要编程人员手动释放, 当忘记释放内存的时候, 会导致内存使用率降低, 影响程序的执行; 在 Java 语言当中这种问题得到了解决, 因为 Java 语言引入了自动垃圾回收机制(GC 机制), Java 程序启动了一个单独的垃圾回收线程, 时刻监测内存使用情况, 在特定时机会回收/释放垃圾数据, 这样会让内存时刻处于最好的状态。

④ 多线程: Java 语言支持多个线程同时并发执行, 同时也提供了多线程环境下的安全机制。

⑤ **可移植性/跨平台**: 可移植性/跨平台表示 Java 语言只需要编写/编译一次, 即可处处运行。Java 代码既可以运行在 windows 的环境下, 又可以运行在 Linux 的环境下, 而不需要修改 Java 源程序, 那么它是怎么做到的呢? 功劳全在于“Java 虚拟机 (Java Virtual Machine, 简称 JVM)”这种机制, 实际上 Java 程序运行的时候并不是直接运行在操作系统上面的, 而是在操作系统上先安装了一个 JVM, 把 Java 程序放到了 JVM 当中运行, JVM 屏蔽了各操作系统之间的差异, 这样就完成了跨平台。但是, JVM 的出现虽然搞定了跨平台, 同时也带来了一些问题, 比如要想运行 Java 程序就必须先安装 JVM, 没有 JVM, Java 程序是运行不了的, 就像你要在网页上看视频, 结果浏览器却提示你需要安装 Flash 插件, 这一点你是不是感觉非常不爽呀! 那么 Java 程序、Java 虚拟机、操作系统之间是怎样的关系呢? 请看下图:

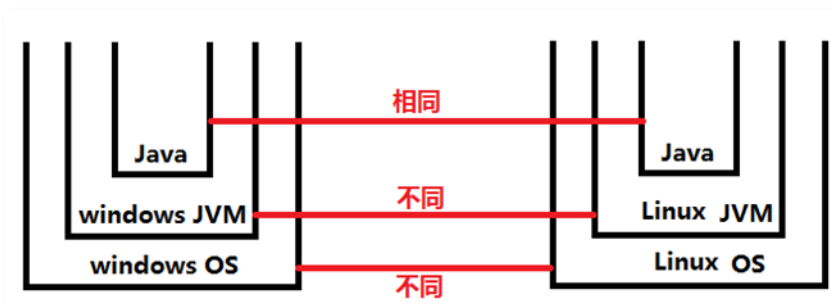


图 1-3: Java 程序、JVM、OS 之间的关系

通过上图我们可以看到不同的操作系统中安装的 JVM 肯定也是不同的，windows 操作系统则必须安装 windows 版本的 JVM，Linux 操作系统则必须安装 Linux 版本的 JVM。这是因为 JVM 是和操作系统直接打交道的，windows 和 Linux 操作系统本身的执行原理不同，所以 JVM 肯定也必须是定制的，不能通用。但是 Java 程序放到 windows 的 JVM 上和放到 Linux 的 JVM 上最终执行效果是完全相同的。这是因为：虽然 JVM 版本不同，但是所有版本的 JVM 的实现都是遵守 sun 制定的 JVM 规范的，这样就可以达到编写一次到处运行的效果，有没有感觉很神奇呀！

当然，Java 语言除了以上的特性之外还有很多其它的特性，我在这里就不再一一赘述了，大家对以上的特性来说重点知道 Java 的跨平台性以及垃圾回收机制即可，其它的作为了解。

1.5 JDK、JRE、JVM 三者关系（理解）

在学习 Java 之前，我们需要对一些专业术语有一定的了解，在 Java 中常见的专业术语包括：JDK、JRE、JVM 等，它们分别是什么，它们之间的关系又是怎样的呢，请看下图：



图 1-4: JDK、JRE、JVM 三者关系

从上图中我们可以看到，JDK、JRE、JVM 之间存在这样的包含关系：JDK 包含 JRE，JRE 又包含 JVM。换句话说，只要安装了 JDK，JRE 和 JVM 则自动就安装了。那么它们分别代表什么呢：

① **JDK**：JDK（Java Development Kit）是 Java 语言的软件开发工具包（SDK）。它是每一个 Java 软件开发人员必须安装的。JDK 安装之后，它会自带一个 JRE，因为软件开发人员编写完代码之后总是要运行的。注意：如果只是在这台机器上运行 Java 程序，则不需要安装 JDK，只需要安装 JRE 即可（JRE 是有独立安装包的，这个大家可以从 Oracle 官网上找一

下)。

② JRE: JRE (Java Runtime Environment, Java 运行环境), 运行 JAVA 程序所必须的环境的集合, 包含 JVM 标准实现及 Java 核心类库。

③ JVM: JVM 是 Java Virtual Machine (Java 虚拟机) 的缩写, JVM 是一种用于计算设备的规范, 它是一个虚构出来的计算机, 是通过在实际的计算机上仿真模拟各种计算机功能来实现的。JVM 是实现 Java 语言跨平台的法宝。

在这里我需要重点强调一下, 不同的操作系统需要安装不同版本的 JDK, 有专门的 windows 版 JDK, Linux 版 JDK, Mac 版 JDK 等, 当然不同版本的 JDK 安装之后会对应不同版本的 JRE 和 JVM。

1.6 初步了解 Java 的加载与执行 (了解)

在编写第一个 Java 程序之前, 我们应当对 Java 的加载与执行提前有一个简单的了解, 请看下图:

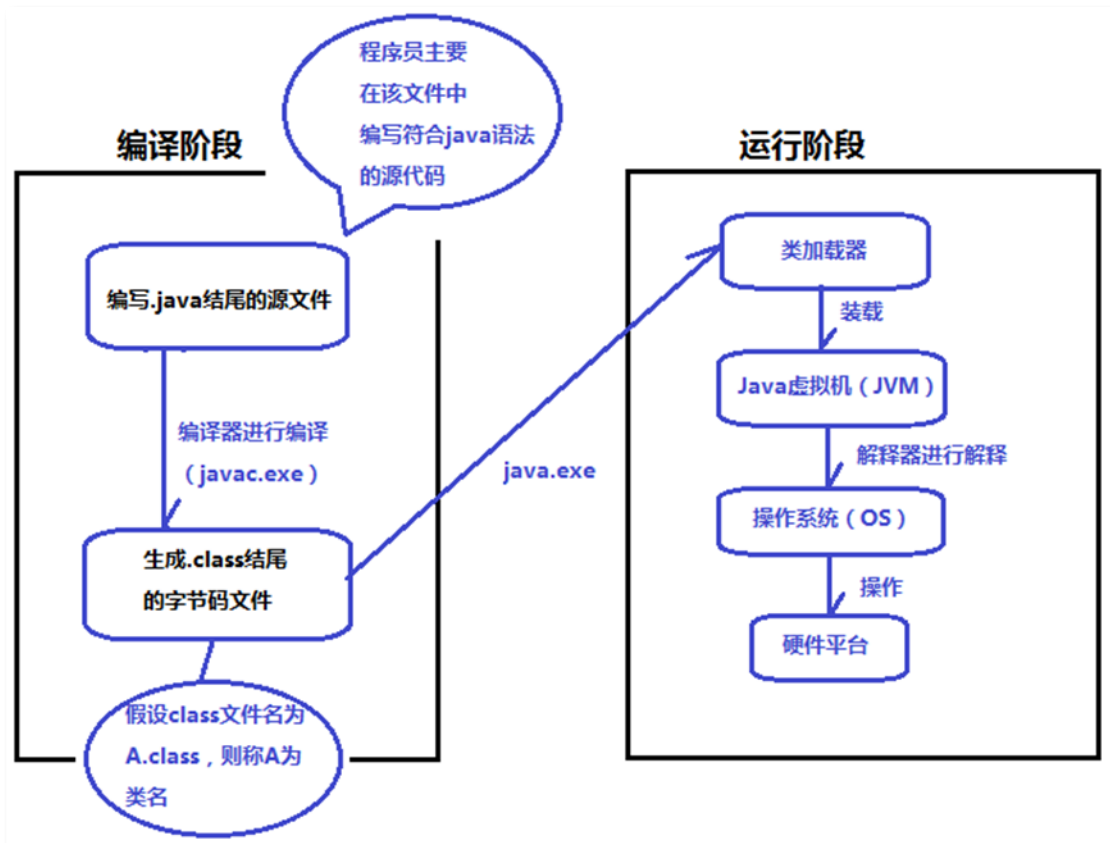


图 1-5: Java 的加载与执行

通过上图，我们可以看到 Java 程序从开发到最终运行是这样进行的：

第一步（写代码）：在任意位置创建一个.java 结尾的文件，程序员在该文件中编写符合 Java 语法的源代码，这个文件被称为 Java 源文件。

第二步（编译）：使用“javac”命令对 java 源文件进行编译，如果 Java 源文件符合 Java 语法规则，则编译生成 1 个或者多个以“.class”结尾的文件。“.class”结尾的文件我们称之为字节码文件，注意字节码文件不是普通文本文件，使用记事本等文本编辑器是无法打开的，但该文件内容也不是二进制，如果是二进制形式的话，操作系统是可以直接执行的，这个文件的内容我们称之为字节码。注意：这里有一个“类名”的概念，假设生成的文件是 A.class，则表示类名为 A，B.class 则表示类名为 B。

第三步（运行）：使用“java”命令运行已编译的 Java 程序，假设编译之后的字节码文件是 A.class，则在 dos 命令窗口中执行 java A 命令，这个时候 Java 虚拟机（JVM）就启动了，Java 虚拟机通过“类装载机 ClassLoader”从硬盘中找到 A.class 文件并装载，字节码文件被装载到 Java 虚拟机之后，Java 虚拟机解释器对字节码进行解释，最终解释为二进制码，然后操作系统通过执行二进制指令来和硬件平台完成交互。

以上则是 Java 程序加载与执行的过程，接下来我给大家总结 7 条规则，大家可以理解并记忆一下：

① Java 程序从开发到运行包括编译和运行两个阶段，这两个阶段可以在不同的操作系统中完成，例如在 windows 环境下进行编译，在 Linux 环境下运行，这是因为有 JVM 机制的存在，做到了一次编译到处运行（跨平台/可移植）。

② 编译阶段需要使用 javac.exe（安装 JDK 之后该命令就存在了）命令，运行阶段需要使用 java.exe（安装 JRE 之后该命令就存在了）命令。

③ 一个 Java 源文件可能会编译生成多个 class 文件。

④ Java 源文件中的源代码如果不符合 Java 的语法机制则编译时编译器会提示错误信息，并且无法生成 class 文件。反之则生成 class 文件，而 class 文件才是最终要执行的程序，此时将 Java 源文件删除是不会影响 Java 程序运行的（当然，我们也不必删除 java 源文件，因为在运行 class 文件之后，如果没有达到预期的运行效果，这个时候还需要将 Java 源代码修改，重新编译，以达到最终的运行效果）。

⑤ 若生成的字节码文件名为 A.class，那么我们称 A 为一个类的名字（这个先记住就行，后面的内容会讲）。

⑥ 当编译阶段完成之后，我们可以使用 JRE 中的 java.exe 命令运行程序，例如执行命令“java A”，该命令执行后会启动类加载器，类加载器去硬盘上搜索 A.class 文件，找到该字节码文件之后，将其加载到 JVM 当中，JVM 中的解释器会将 A.class 字节码文件解释为操作系统可以执行的二进制码，然后操作系统通过执行二进制码和硬件平台交互。

⑦ 运行 Java 程序的前提是当前操作系统上已经安装了对应版本的 JVM（JVM 不是单独安装的，安装 JRE 即可，不同的操作系统需要安装不同版本的 JRE，不同版本的 JRE 对应不同版本的 JVM）。

在了解了 Java 程序的加载与执行之后，我们开始动手实现第一个 Java 程序吧！

1.7 开发前的准备工作（掌握）

1.7.1 windows 显示文件扩展名

Java 源文件要求文件扩展名必须为.java，不能使用其他扩展名，有些操作系统默认不显示文件扩展名，大家可按照以下步骤，将文件的扩展名显示出来（以 win7 系统为例），请看以下步骤：

第一步：打开电脑桌面上的“计算机”，在菜单栏上找“工具”。

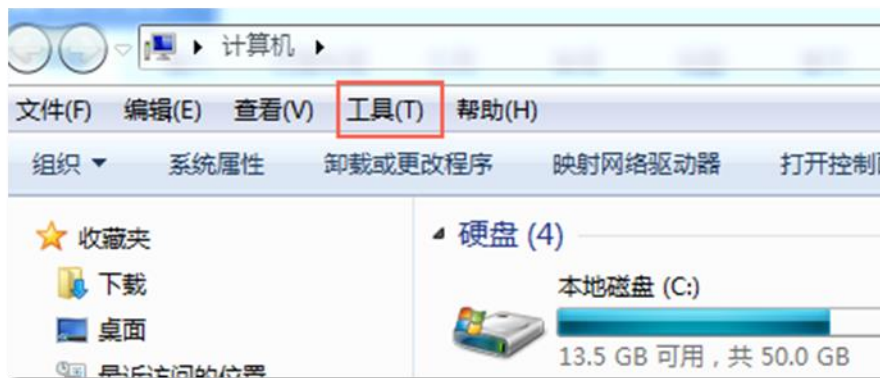


图 1-6：打开桌面上的计算机，菜单栏上的“工具”

第二步：点击“工具”，下拉列表中点击“文件夹选项(O)...”。



图 1-7：文件夹选项

第三步：在弹出的“文件夹选项”对话框中点击“查看”选项卡，在列表中找到“隐藏已知文件类型的扩展名”项，将该项前的“对勾”去掉，最后点击“应用”，点击“确定”即可。

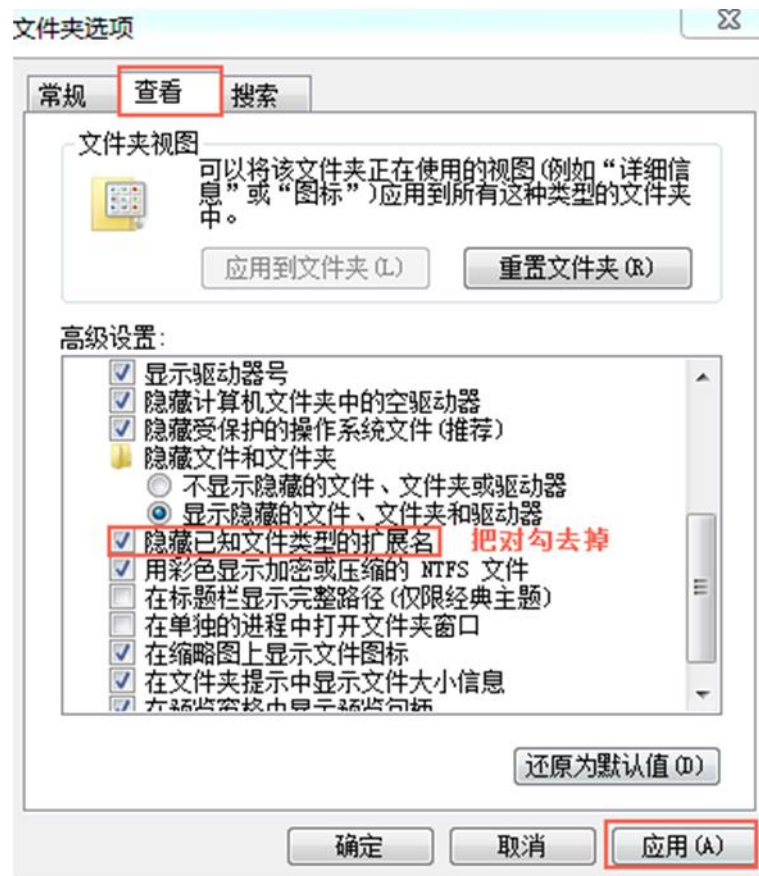


图 1-8：点击“查看”，把上图红框的对勾去掉

以上是 win7 操作系统显示文件扩展名的步骤，其它 windows 系列版本的系统和以上操作类似，大家可以自己找一下，并设置好。

1.7.2 windows 系统的常用 DOS 命令

初次学习 Java 编程最好不要一上来就使用集成开发环境 IDE，不利于代码的掌握，建议使用文本编辑器，甚至可以使用记事本编写。在这种情况下，我们就需要熟悉一些 dos 命令，接下来我们来看几个常见的 dos 命令吧：

打开 DOS 命令窗口，使用快捷键：win + r，输入 cmd，回车。

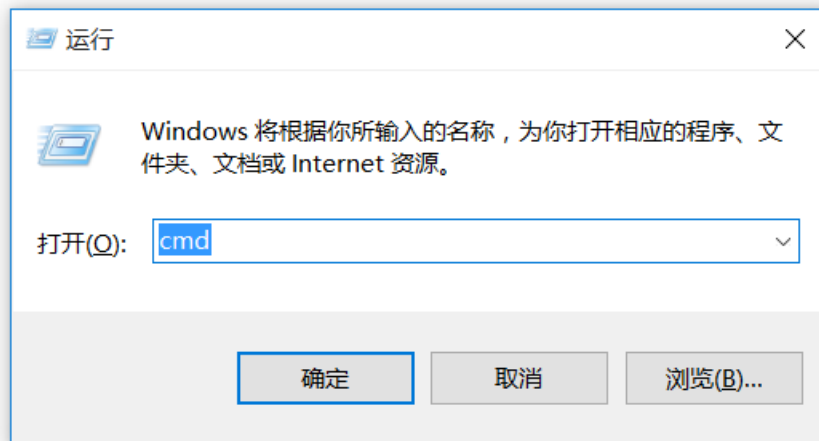


图 1-9: win+r 打开运行窗口, 输入 cmd, 点击确定

dir 命令: 查看当前目录下所有的子文件或子目录。



图 1-10: dir 命令查看当前目录下有哪些子目录或子文件

cd 命令: 切换路径, 使用方法是: cd+目录路径, 需要注意的是路径包括相对路径和绝对路径, 对于 windows 来说从硬盘的根路径下开始的路径都是绝对路径, 例如: C:\Program Files、C:\Program Files\Java 等, 所有的相对路径都是从当前所在目录作为起点开始查找的路径。另外 cd .. 切换到上级目录, cd \ 切换到根目录。

```
C:\Users\Administrator>cd C:\Program Files\Java 绝对路径
C:\Program Files\Java>cd .. 回到上级
C:\Program Files>cd \ 回到根
C:\>dir 查看当前都有啥
驱动器 C 中的卷没有标签。
卷的序列号是 84B7-F81F

C:\ 的目录

2017/09/29 15:26 <DIR>      apache-tomcat-7.0.82
2017/12/28 10:48 <DIR>      eclipse
2017/03/08 17:56 <DIR>      Intel
2018/01/15 19:51 <DIR>      Program Files
2018/02/08 17:31 <DIR>      Program Files (x86)
2017/12/21 13:33 <DIR>      temp
2017/04/13 20:04 <DIR>      Tencent
2017/03/08 22:11 <DIR>      tomcat9
2017/03/08 14:53 <DIR>      Users
2018/02/10 08:57 <DIR>      Windows
                0 个文件          0 字节
                10 个目录 14,530,560,000 可用字节

C:\>cd Windows\Boot 相对路径
C:\Windows\Boot>
```

图 1-10: cd 命令使用办法

切换盘符：直接输入 c:，或者 d:，然后回车即可。切换盘符不需要 cd 命令。

```
C:\windows\system32\cmd.exe

C:\Users\Administrator>d:

D:\>
```

图 1-11: 切换盘符

del 命令：删除文件或目录，例如：del *.class，删除当前目录下所有的.class 文件。


```
C:\Windows\System32\cmd.exe

E:\test>dir
驱动器 E 中的卷是 软件
卷的序列号是 54BE-3C19

E:\test 的目录

2019/05/16  12:02    <DIR>        .
2019/05/16  12:02    <DIR>        ..
2019/05/16  12:02                0 A. java
2019/05/16  12:02                0 B. java
2019/05/16  12:02                0 C. java
                3 个文件                0 字节
                2 个目录 310,967,435,264 可用字节

E:\test>del A. java

E:\test>dir
驱动器 E 中的卷是 软件
卷的序列号是 54BE-3C19

E:\test 的目录

2019/05/16  12:03    <DIR>        .
2019/05/16  12:03    <DIR>        ..
2019/05/16  12:02                0 B. java
2019/05/16  12:02                0 C. java
                2 个文件                0 字节
                2 个目录 310,967,435,264 可用字节

E:\test>del *. java

E:\test>dir
驱动器 E 中的卷是 软件
卷的序列号是 54BE-3C19

E:\test 的目录

2019/05/16  12:03    <DIR>        .
2019/05/16  12:03    <DIR>        ..
                0 个文件                0 字节
```

图 1-12: del 命令删除文件

ipconfig 命令：查看 IP 地址等信息，查看更详细的信息使用 ipconfig /all。

ping 命令：查看两台计算机是否可以正常通信，例如：ping 192.168.1.100，正常情况下发送数据包和接收数据包数量相同并且没有延迟的时候表示通信正常，ping 192.168.1.100 -t 表示一直发送数据包和接收数据包，ping www.baidu.com 可以查看电脑是否可以正常上网。

mkdir 命令：创建目录，例如：mkdir abc 表示在“当前目录”下新建 abc 目录。

```
C:\Windows\System32\cmd.exe

E:\test>dir
驱动器 E 中的卷是 软件
卷的序列号是 54BE-3C19

E:\test 的目录
2019/05/16  12:03    <DIR>          .
2019/05/16  12:03    <DIR>          ..
               0 个文件                0 字节
               2 个目录 310,967,435,264 可用字节

E:\test>mkdir abc

E:\test>dir
驱动器 E 中的卷是 软件
卷的序列号是 54BE-3C19

E:\test 的目录
2019/05/16  16:11    <DIR>          .
2019/05/16  16:11    <DIR>          ..
2019/05/16  16:11    <DIR>          abc
               0 个文件                0 字节
               3 个目录 310,967,435,264 可用字节
```

图 1-13: mkdir 命令创建目录

cls 命令：清屏。

exit 命令：退出 DOS 命令窗口

以上的命令需要大家多敲多练才可以记住，那就辛苦大家敲一下吧！

1.7.3 文本编辑器的安装与配置

任何文本编辑器都可以进行 Java 代码的编写，例如：记事本、editplus、notepad++等，记

事本的文本编辑能力稍弱，没有高亮显示，这里选择 EditPlus。这里安装的 EditPlus 版本为：

 EditPlus_3.12.602_sc.exe ，双击之后，一直点击下一步就可以了。

在使用该工具之前，我们对该工具进行一个简单的配置，例如：取消自动备份、设置字体大小、设置缩进长度等。首先打开“首选项(P)...”，如下图所示：

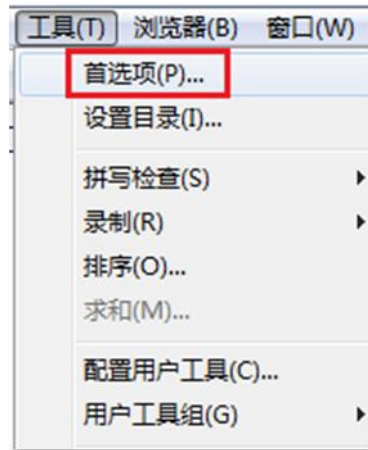


图 1-14：菜单栏“工具”，然后“首选项”

打开首选项之后，从左边“类别”中可以找到“常规”，在常规下有字体的设置，这里就不再赘述了，另外除了常规之外，还有“文件”，选中“文件”之后，可以看到右边有“保存时创建备份文件”，将前面的对勾去掉，然后点击应用，这样就可以取消自动备份功能了。如下图所示：

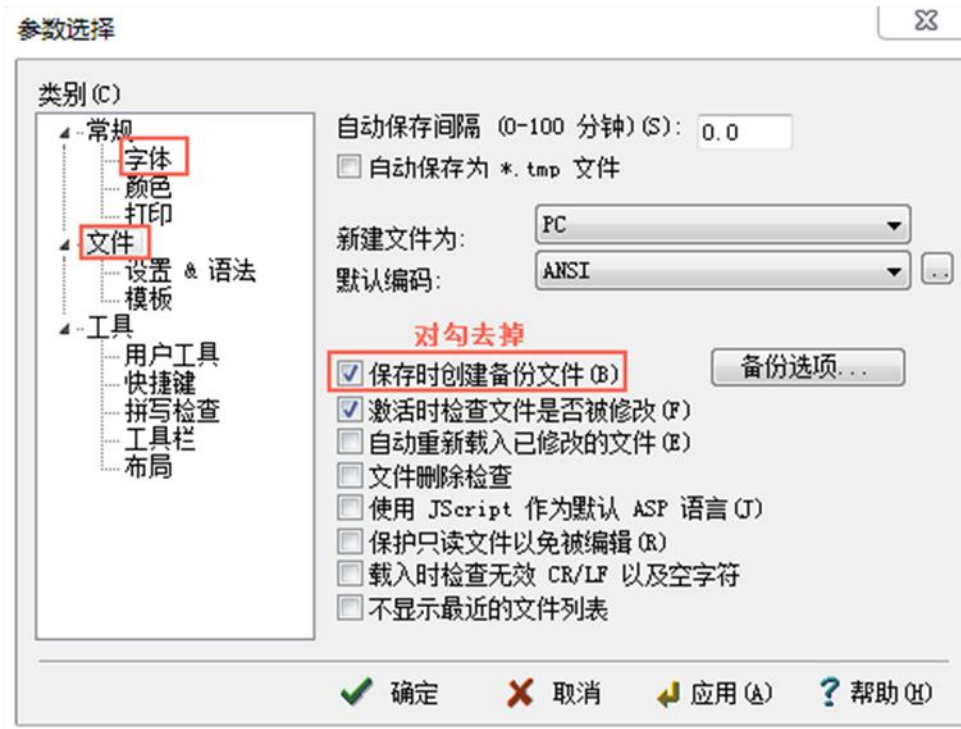


图 1-15：设置字体和取消自动备份

接下来，我们一起来看一下怎么设置制表符的长度，默认是 8 个长度，太长了，代码看起来不是很协调，可以修改一下，点击类别“文件”下的“设置&语法”选项，然后可以看到右侧有“制表符/缩进”，如下图所示：

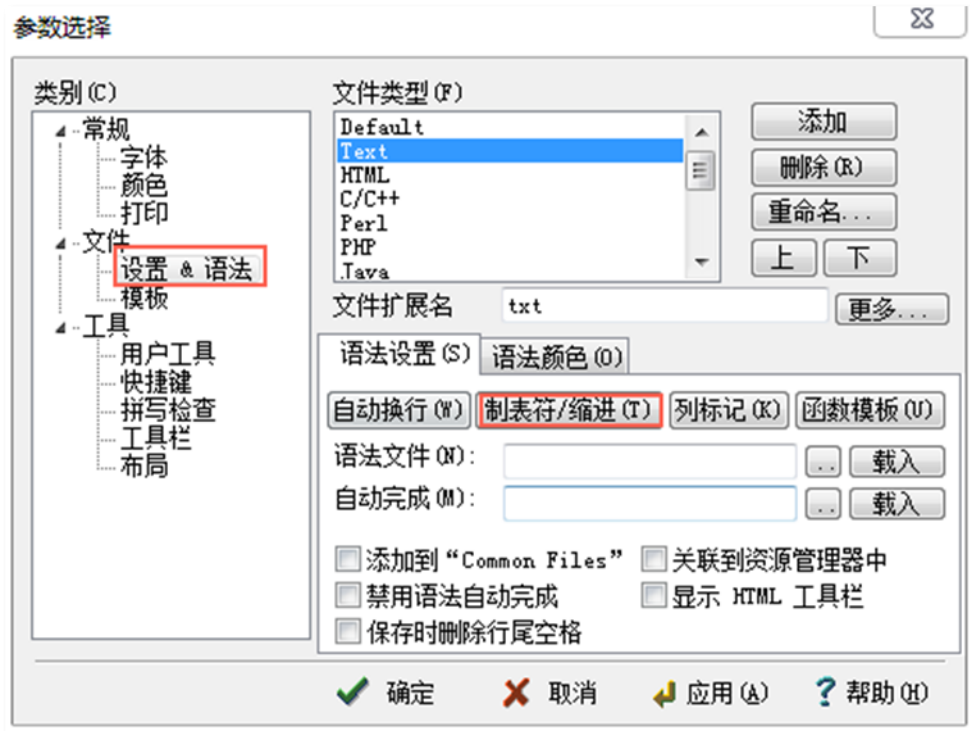


图 1-16：设置缩进长度

然后再点击“制表符/缩进”，弹出以下对话框，这时将长度修改为 3，如下图所示：

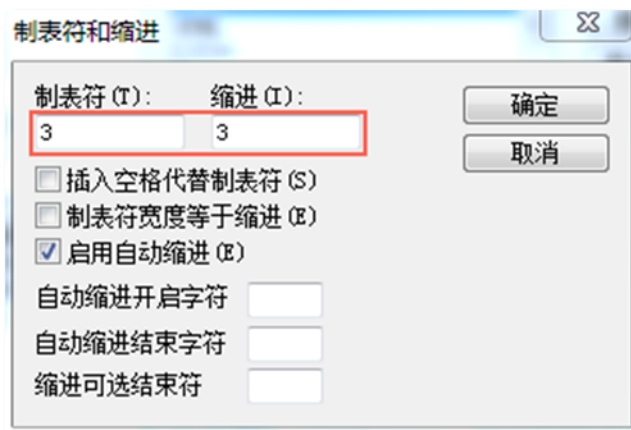


图 1-17：这里设置缩进长度为 3

到此为止，文本编辑器 EditPlus 安装并且配置完成了。当然，EditPlus 还有更强悍的配置，比如 EditPlus 可以配置属于自己的 Java 环境，在这里我们就不再配置了，接下来的学习，Java

程序的编译和运行，我们统一采用手工的方式进行，这样有利于大家熟练掌握开发步骤。

1.7.4 常用文本编辑快捷键

使用快捷键可以大大提高开发效率，从现在起我们应该尽可能使用快捷键来代替鼠标，接下来，我给大家列举一下通用的文本编辑快捷键。当然，如果大家想熟练掌握，还是需要后期不断的使用，请看下面列表：

快捷键	作用
ctrl+c	复制
ctrl+v	粘贴
ctrl+x	剪切
ctrl+s	保存
ctrl+z	撤销
ctrl+y	重做
ctrl+a	全选
键盘上的 home 键	光标回到行首
键盘上的 end 键	光标回到行尾
ctrl + home 键	光标回到文章开始
ctrl + end 键	光标回到文章末尾
shift + home 或 end 键	选中一行
鼠标双击	选中一个单词
ctrl + shift + 右箭头或左箭头	选中一个单词
ctrl + f	查找

以上是一些比较常见的快捷键，只要大家能够勤加练习，就能够提高操作速度，进而提高编程速度，最开始的时候可能会不太适应，只要坚持就对了。

1.8 开发第一个 Java 程序（掌握）

1.8.1 JDK 的下载与安装

Java 语言由 Sun（Sun Microsystems）公司开发，2009 年被 Oracle（甲骨文）公司收购，所以目前 JDK 需要从 Oracle 下载，Oracle 官网地址是 <https://www.oracle.com>，这里我们学习 Java10 版本，所以下载 JDK10，具体下载过程如下图所示：

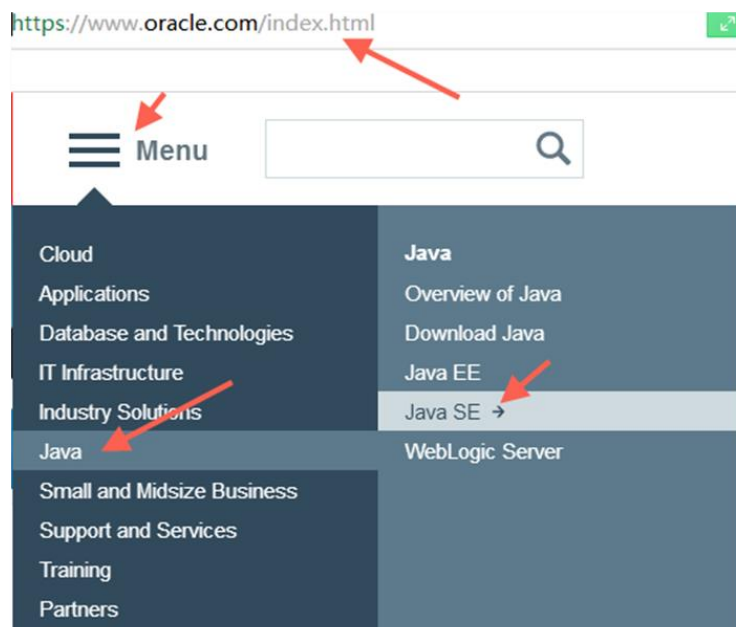


图 1-18：选择 JavaSE

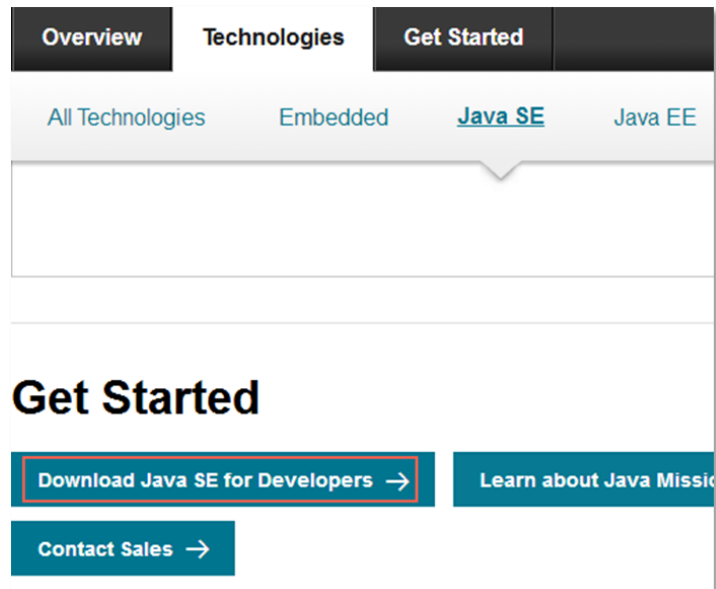


图 1-19: 点击 “Download Java SE...”



图 1-20: 点击 “DOWNLOAD”



图 1-21：选择“Accept...”，然后选择“windows”版

注意：JDK10 要求 windows 操作系统必须是 64bit 的。下载完成之后，直接双击 `jdk-10_windows-x64_bin.exe` 进行安装，具体步骤如下所示：



图 1-22：点击“下一步”



图 1-23：选择 JDK10 的安装目录，点击下一步

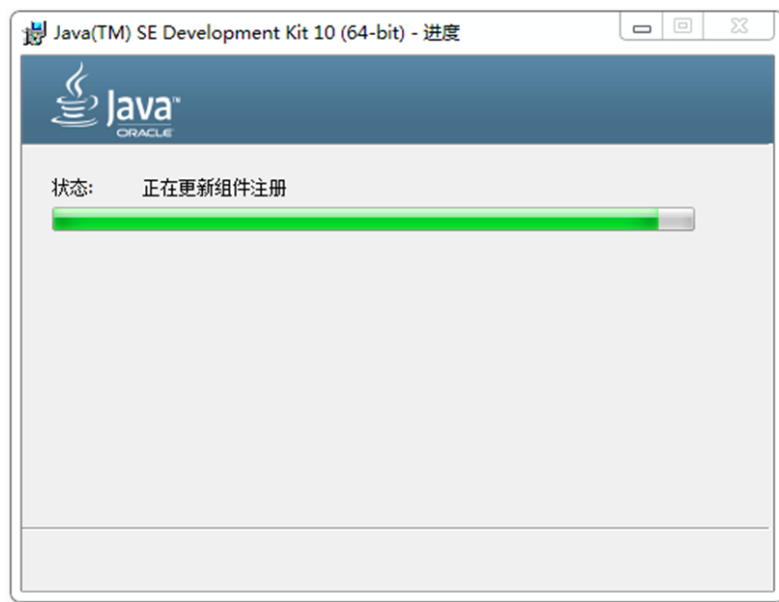


图 1-24：正在安装 JDK10



图 1-25：选择 JRE10 的安装目录，点击“下一步”



图 1-26：正在安装 JRE10



图 1-27: 点击关闭, 安装结束

到此, JDK 的下载与安装就完成了。

1.8.2 JDK 的 bin 目录

JDK 安装结束之后, 安装目录下有很多子目录, 这里就不再一一赘述, 后面用到的时候我们再看, 这里重点看一下 JDK 的 bin 目录, bin 目录下存放的都是运行 Java 程序相关的命令, 在 windows 操作系统中, 以 exe 后缀的文件都是可执行文件, 或者叫做命令文件, 请看下图:



图 1-28: java 和 javac 命令的存放位置

通过上图我们可以看到，在 bin 目录下有 javac 和 java 两个命令，其中 javac 命令是用来编译 Java 源程序的，而 java 命令是用来运行 Java 程序的。其它命令目前我们还用不上，后期用上的话我们再看。

1.8.3 编写 HelloWorld

以上所说的环境都准备好之后，大家就可以开始编写你的第一个 Java 程序了，由于 Java 程序的基础语法还没有讲解，第一个程序大家只能严格按照以下代码照抄了，以下程序为什么要这么写，大家可以先不需要去了解，因为这第一个程序的最主要目的是为了测试你机器上的 Java 环境是否搭建好了。我们首先新建 `HelloWorld.java` 文件，具体代码如下所示：

```
1 public class HelloWorld {  
2     public static void main(String[] args) {  
3         System.out.println("动力节点-口口相传的 Java 黄埔军校！");  
4     }  
5 }
```

在这里大家需要注意的是，在照抄以上程序的时候，一定要注意标点符号都是半角形式，还需要注意字母的大小写。大家编写好以上程序之后，使用快捷键 `ctrl + s` 完成保存。

1.8.4 编译 HelloWorld 程序

编写完以上的 Java 程序并保存之后，接下来我们对以上的程序进行编译，编译 java 程序需要使用 JDK bin 目录下的 javac.exe 命令，我们先来看看这个命令是否可以在 DOS 命令窗口中使用。打开 DOS 命令窗口，输入 javac，然后回车执行，如下图所示：

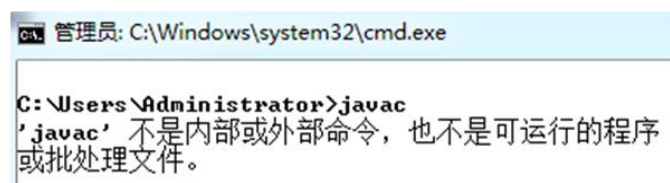


图 1-29：测试 javac 命令

根据以上测试结果，可以看到 javac 命令是无法执行的，出现的错误提示信息是“javac’不是内部或外部命令，也不是可运行的程序或批处理文件”。这说明 windows 操作系统没有找到 javac.exe 文件。接下来我们将所在的 “C:\Users\Administrator” 目录切换到 “C:\Program Files\Java\jdk-10\bin” 目录下再来测试，如下图所示：

```

管理员: C:\Windows\system32\cmd.exe

C:\Users\Administrator>cd C:\Program Files\Java\jdk-10\bin

C:\Program Files\Java\jdk-10\bin>dir javac.exe
驱动器 C 中的卷没有标签。
卷的序列号是 84B7-F81F

C:\Program Files\Java\jdk-10\bin  的目录
2018/02/21  16:54                16,936 javac.exe
               1 个文件                16,936 字节
               0 个目录 13,105,631,232 可用字节

C:\Program Files\Java\jdk-10\bin>javac
用法: javac <options> <source files>
其中, 可能的选项包括:
  -c <filename>           从文件读取选项和文件名
  -Akey[=value]           传递给注释处理程序的选项
  --add-modules <模块>[.<模块>]*
                          除了初始模块之外要解析的根模块; 如果 <module>
                          为 ALL-MODULE-PATH, 则为模块路径中的所有模块。
  --boot-class-path <path>, -bootclasspath <path>
                          覆盖引导类文件的位置
  --class-path <path>, -classpath <path>, -cp <path>
                          指定查找用户类文件和注释处理程序的位置
  -d <directory>          指定放置生成的类文件的位置
  -deprecation             输出使用已过时的 API 的源位置
  -encoding <encoding>    指定源文件使用的字符编码
  -endorseddirs <dirs>    覆盖签名的标准路径的位置
  -extdirs <dirs>         覆盖所安装扩展的位置
  -g                      生成所有调试信息
  -g:{lines,vars,source}  只生成某些调试信息
  
```

图 1-30: 切换到 bin 目录下再次测试 javac 命令

通过上图我们可以看到，当我们将所在的当前目录切换到 bin 目录之后，查看了 bin 目录下确实存在 javac.exe 文件，然后执行 javac 命令，结果 javac 命令找到了并且也运行了，根据测试结果可以得出这样的结论：windows 操作系统默认是从当前所在的路径下查找可执行命令文件的。换句话说，以后我们每一次使用 javac 这个命令的时候都需要先切换到 bin 目录下。这岂不是很麻烦，有什么解决办法吗？我们先来研究一下 ipconfig 命令的执行原理，请看下图：

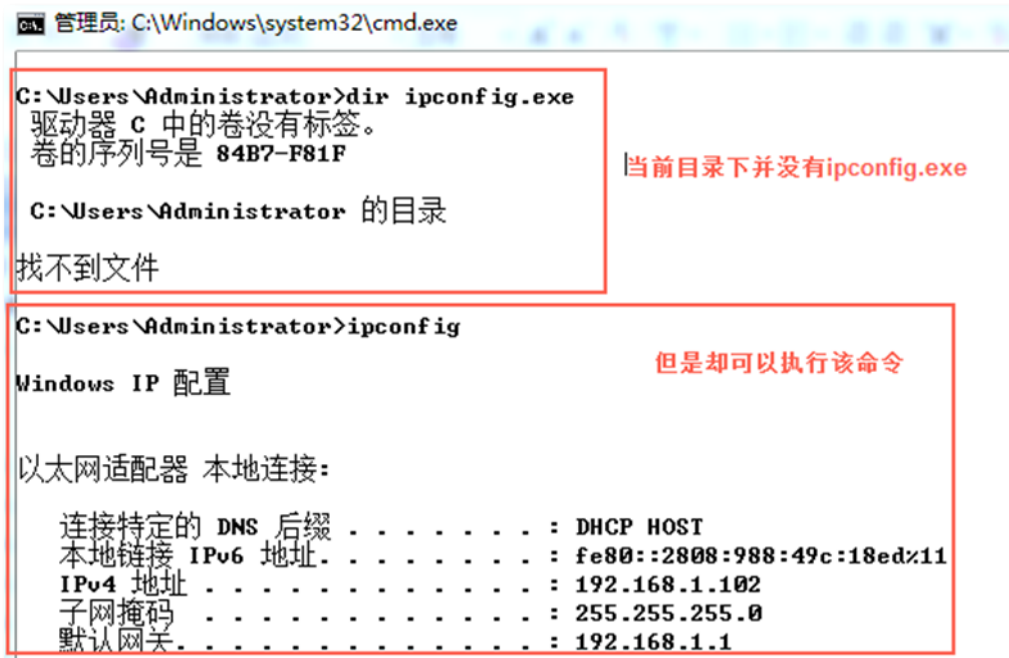


图 1-31: 测试 ipconfig 命令

通过上图我们可以看到，先将目录切换到“C:\Users\Administrator”下，并且使用 dir 命令查看 ipconfig.exe 文件，发现当前目录下 ipconfig.exe 文件并不存在，然后输入 ipconfig 命令并执行，发现该命令是可以执行的，换句话说 windows 操作系统可以找到 ipconfig.exe 文件，那么 ipconfig.exe 文件在哪个目录下呢，请看下图：

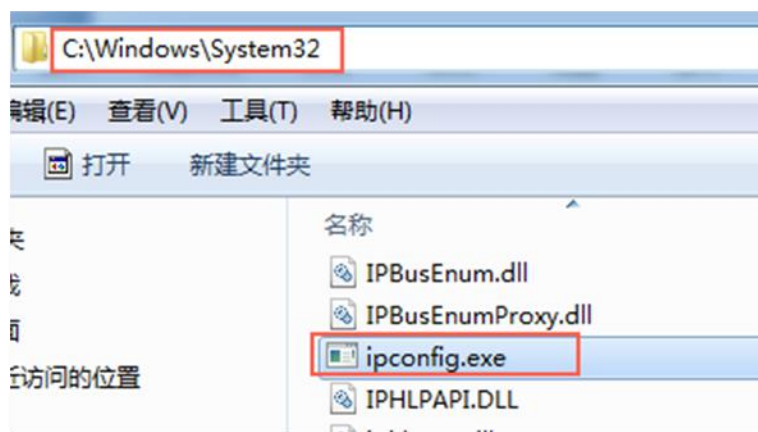


图 1-32: ipconfig.exe 在这里

我们可以看到 ipconfig.exe 文件实际上是在“C:\Windows\System32”目录下。那么 ipconfig 命令为什么在命令窗口可以直接执行而不需要切换到命令所在目录呢？

实际上这是因为 windows 系统中的环境变量 path 在起作用。我们打开环境变量（在桌面上“计算机”图标上点击右键->属性->高级系统设置->环境变量），可以看到 windows 系统有以下默认配置，请看下图：



图 1-33: windows 自带的 path 环境变量配置

我们可以看到 windows 系统有一个默认的系统变量 path，path 变量中有“C:\Windows\System32;”路径。

我们来做一个实验，把上图 path 环境变量当中的“C:\Windows\System32;”删掉（鼠标双击 Path，将“C:\Windows\System32;”删除，注意：System32 路径后面的半角分号也需要删除），然后点击确定。将所有的 DOS 命令窗口全部关闭（注意：修改了任何环境变量，DOS 命令窗口都需要关闭之后重新打开，这样新的环境变量才会生效），打开一个新的 DOS 命令窗口，测试 ipconfig，如下图所示：

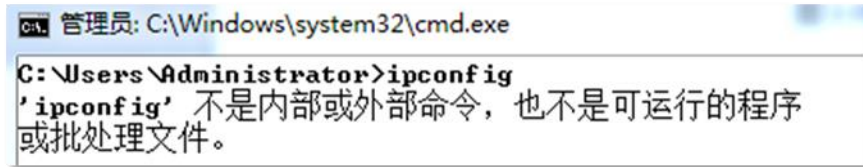


图 1-34: 测试 ipconfig 命令

根据以上测试，ipconfig 命令无法执行了，windows 操作系统已经找不到 ipconfig.exe 文件了，也就是说 windows 是从环境变量 path 中指定的路径下查找命令的。

接下来我们再重新恢复 path 环境变量的配置，配置环境变量 path 的步骤是这样的：在桌面  图标上点击鼠标右键，继续点击 **属性(R)**，在弹出的窗口上选择  **高级系统设置**，然后在弹出的 **系统属性** 窗口上点击 **环境变量(N)...**，此时会弹出以下窗口：

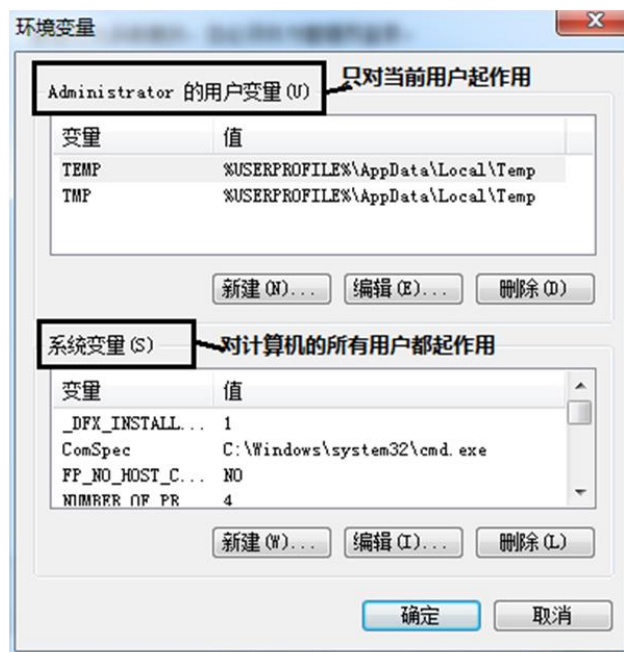


图 1-35: 配置环境变量的窗口

通过上图我们可以看到环境变量包括用户变量和系统变量两种，在这里简单说明一下，配置用户变量表示只对当前用户有效，例如在“Administrator 的用户变量”中配置环境变量的话，这个环境变量只对“Administrator”用户有效。配置系统变量则表示对使用该计算机的所有用

查看编译器的版本，如下图所示：



图 1-38：查看编译器版本

到这里，大家应该掌握环境变量 `path` 的作用以及配置了吧！

通过以上内容的学习，大家应该思考这样一个问题：`path` 环境变量是隶属于 `java` 的吗，它和 `java` 有关系吗？答案是：`path` 环境变量隶属于 `windows` 操作系统，不属于 `java` 范畴，是 `windows` 操作系统搜索某个命令文件的路径依据。

`windows` 操作系统到底是如何搜索命令文件的呢？实际上它会先在当前路径下找，找不到的时候会自动去环境变量 `path` 的路径中查找，找到则执行该命令，找不到则在 `DOS` 窗口中提示错误信息。希望大家以后遇到类似的“在 `DOS` 命令窗口中输入某个命令时出现找不到命令”这样的问题都能够独立的解决。

通过以上的配置，`javac` 命令已经可以使用了，那么它具体怎么使用才能编译 `java` 程序呢，接下来我们详细的讲解一下 `javac` 命令的具体使用办法。

`javac` 命令的语法格式是：“`javac 源文件路径`”，非常简单，就是 `javac` 命令后面加上 `java` 源文件的路径，之前我们讲解 `cd` 命令的时候说过路径包括绝对路径和相对路径。也就是说 `javac` 后面的源文件路径可以是绝对的也可以是相对的，如下图所示：

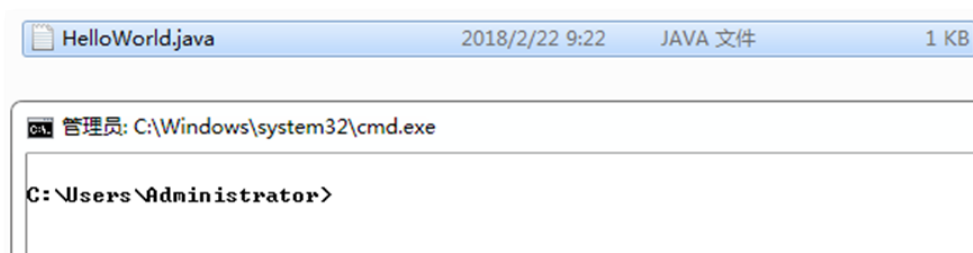


图 1-39：编译之前

通过上图，我们可以看到，在 `java` 源程序编译之前，只有一个文件“`HelloWorld.java`”。继续看下图：



图 1-40：编译之后（绝对路径方式）

我们可以看到，javac 命令后面使用了 java 源文件的绝对路径，编译之后，生成了一个新的文件“HelloWorld.class”，我们称为字节码文件。另外，这也说明 java 源文件没有语法错误，通过了编译。那么，除了这种绝对路径的方式，相对路径的方式应该怎么做呢？请看下图：

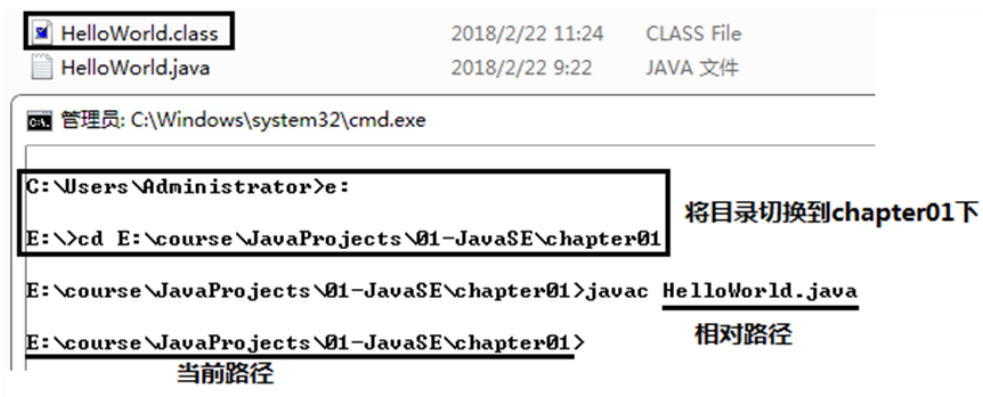


图 1-41：编译之后（相对路径方式）

我们可以看到先使用 cd 命令将目录切换到 chapter01 目录下，为什么要切换到该目录下呢，这是因为 HelloWorld.java 文件就在 chapter01 目录下。这样一来，当前目录下存在 HelloWorld.java 文件，则直接使用“javac HelloWorld.java”进行编译。这种方式就表示使用了相对路径，记住：相对路径表示从当前所在的路径下作为起点开始找。细心的同学应该可以看到新生成的 HelloWorld.class 文件的最后修改时间变成了 11:24，这说明编译通过了。

我们把 HelloWorld.java 文件中的程序故意修改让其出现错误，请看以下代码：

```
1 public class HelloWorld {
```



```

2    public static void main(Strin[] args) {
3        System.out.println("动力节点-口口相传的 Java 黄埔军校");
4    }
5 }

```

从以上的代码中可以看到 main 方法的参数应该是(String[] args)，结果写成了(Strin[] args)，这种情况下语法是错误的，那么此时编译会发生什么呢，请看下图：

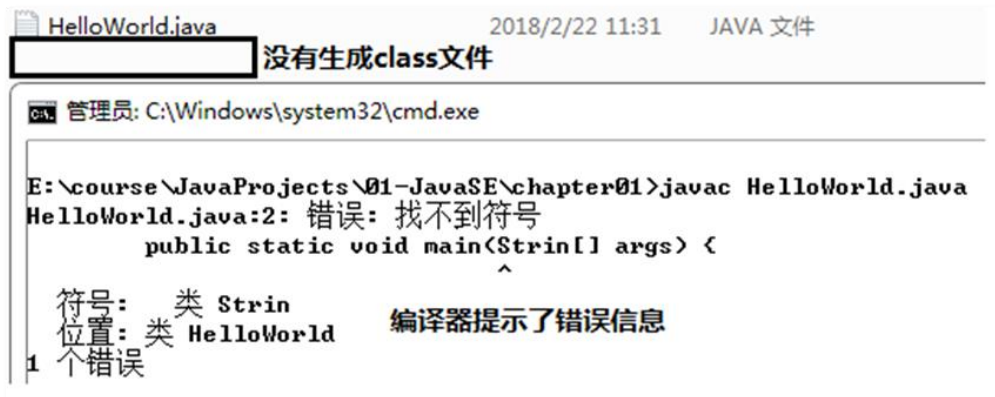


图 1-42：编译器报错测试

我们可以看到，编译器报错了，字节码文件并没有生成，并且提示了很详细的错误信息，“错误：找不到符号”就表示某种类型未定义。以后开发中大家可能还会经常遇到这样的错误。

通过以上的测试，可以得出这样的结论：java 源代码中存在语法错误，在编译的时候编译器会提示错误消息，并且不会生成 class 字节码文件。在以后的学习过程当中，编译错误的这些提示信息可以积累一下，有利于大家以后的开发。

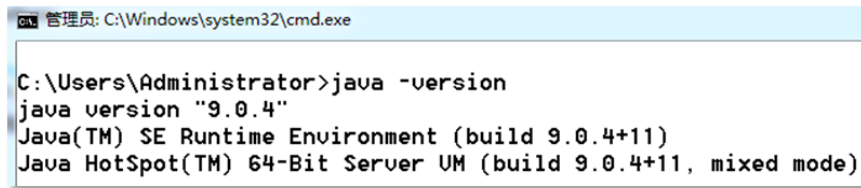
小结：通过本小节的学习，每位同学必须掌握 path 环境变量的作用，以及它是怎么配置的，还有怎么使用 javac 编译 java 源程序。

1.8.5 运行 HelloWorld 程序

程序通过以上的编译之后，接下来我们就可以运行程序了。在这里先给大家普及一下，在 Java 中有一个“类名”的概念，什么是类名呢，假设字节码是 A.class，则类名为 A，字节码是 B.class，则类名为 B。之前生成的字节码文件是 `HelloWorld.class`，那么类名则为 HelloWorld。

程序怎么运行呢，这个时候就需要借助 JDK bin 目录下的 java.exe 命令了，我们先来测试

这个命令是否可以在 DOS 窗口中使用，请看下图：



```
管理员: C:\Windows\system32\cmd.exe
C:\Users\Administrator>java -version
java version "9.0.4"
Java(TM) SE Runtime Environment (build 9.0.4+11)
Java HotSpot(TM) 64-Bit Server VM (build 9.0.4+11, mixed mode)
```

图 1-43：测试 java.exe 是否可用

经过测试，我们看到 java.exe 命令是可以在 DOS 窗口中直接使用的，其实只要 javac.exe 可以使用，那么 java.exe 就一定可以使用，因为 javac.exe 和 java.exe 都在 JDK 的 bin 目录下，之前的课程中这个目录已经配置到环境变量 path 当中，所以这里 java.exe 肯定也是可以使用的。

那么 java.exe 具体怎么使用呢，这里需要大家记忆语法，语法格式为：“java 类名”，需要注意的是 java 命令后面是类名，而不是 class 文件的名称，也不是 class 文件的路径，不能这样写 java HelloWorld.class，也不能这样写 java E:\course\JavaProjects\01-JavaSE\chapter01\HelloWorld.class，只能这样写：java HelloWorld。

那么它的运行原理是什么呢？实际上是这样的：在命令窗口中输入“java HelloWorld”回车之后，先启动的是类加载器（类加载器 ClassLoader 主要的作用是将类名所对应的 class 文件装载到 JVM 当中，这里不再赘述，以后再详细学习类加载器），类加载器从硬盘上查找 HelloWorld.class 字节码文件（为什么会查找 HelloWorld.class 呢？为什么不是查找 A.class 或者 B.class 文件呢？这是因为运行的命令是 java HelloWorld，命令中指定的是 HelloWorld 类，则类加载器查找的就是 HelloWorld.class 文件），默认情况下类加载器只从当前路径下查找，查找到之后则将 HelloWorld.class 文件加载到 JVM 并执行，如果没有查找到则会出现错误信息，请看下图：

```

管理员: C:\Windows\system32\cmd.exe

C:\Users\Administrator>e:
E:\>cd E:\course\JavaProjects\01-JavaSE\chapter01
E:\course\JavaProjects\01-JavaSE\chapter01>dir HelloWorld.class
驱动器 E 中的卷是 文档
卷的序列号是 6EAB-ABA8

E:\course\JavaProjects\01-JavaSE\chapter01 的目录
2018/02/22  15:35                458 HelloWorld.class
               1 个文件                458 字节
               0 个目录 97,974,464,512 可用字节

E:\course\JavaProjects\01-JavaSE\chapter01>java HelloWorld
动力节点-口口相传的Java黄埔军校
  
```

切换到HelloWorld.class
文件所在目录

当前目录下确实有
HelloWorld.class
文件

运行

图 1-44: 运行 HelloWorld 类（正常）

我们可以看到，先使用 `cd` 命令将路径切换到 `HelloWorld.class` 字节码文件所在的目录，并使用 `dir` 命令查看一下，确认当前路径下存在 `HelloWorld.class` 文件，然后执行 `java HelloWorld` 命令，执行结果是向控制台输出了“动力节点-口口相传的 Java 黄埔军校”。

由于默认情况下类加载器只从当前所在的路径下加载字节码文件，如果该字节码文件不存在会提示什么错误信息呢？请看下图：

```

管理员: C:\Windows\system32\cmd.exe

E:\course\JavaProjects\01-JavaSE\chapter01>dir A.class
驱动器 E 中的卷是 文档
卷的序列号是 6EAB-ABA8

E:\course\JavaProjects\01-JavaSE\chapter01 的目录
找不到文件

E:\course\JavaProjects\01-JavaSE\chapter01>java A
错误: 找不到或无法加载主类 A
原因: java.lang.ClassNotFoundException: A
  
```

当前目录下
A.class文件
不存在

运行报错了

图 1-45: 运行 A 类（错误）

我们可以看到上图中是在 `chapter01` 目录下使用 `dir` 命令查看 `A.class` 文件是否存在，结果是 `A.class` 文件在当前目录下是不存在的，我们使用“`java A`”来执行，发现出错了，错误信息是：找不到或无法加载主类 `A`。换句话说也就是类加载器在硬盘上找不到 `A.class` 文件导致的

错误。

在上面我们已经说过了：类加载器默认从当前路径下加载字节码，那么可以让类加载器从指定的目录下加载 class 文件吗？答案是可以的，这个时候就需要借助 classpath 这个环境变量了（classpath 环境变量隶属于 java 语言，专门给类加载器指路的），接下来我们来设置一下 classpath 环境变量，此处把 classpath 设置为 E:\，如下图所示：



图 1-46：设置环境变量 classpath

配置环境变量 classpath 的时候，这个变量不像 path 是已经存在的，我们只需要修改就行了，而 classpath 是不存在的，这里我们需要新建。在系统变量栏下点击新建即可，变量名填写 classpath，变量值填写 E:\，然后点击确定即可。设置完成后关闭所有 DOS 窗口，重新开启新窗口，接下来按照下图操作：

```

管理员: C:\Windows\system32\cmd.exe

C:\Users\Administrator>e:
E:\>cd E:\course\JavaProjects\01-JavaSE\chapter01
E:\course\JavaProjects\01-JavaSE\chapter01>dir HelloWorld.class
驱动器 E 中的卷是 文档
卷的序列号是 6EAB-ABA8

E:\course\JavaProjects\01-JavaSE\chapter01 的目录
2018/02/22  15:35                458 HelloWorld.class
               1 个文件                458 字节
               0 个目录 97,974,464,512 可用字节

E:\course\JavaProjects\01-JavaSE\chapter01>java HelloWorld
错误: 找不到或无法加载主类 HelloWorld
原因: java.lang.ClassNotFoundException: HelloWorld
  
```

图 1-47: 测试环境变量 classpath

我们可以看到先使用 `cd` 命令将目录切换到 `chapter01` 下，并使用 `dir` 命令查看，确认 `HelloWorld.class` 文件在当前目录下是存在的，然后使用 `java HelloWorld` 运行程序，结果出错了。错误信息之前我们已经见过了，表达的意思就是 `HelloWorld.class` 文件没找到。这是为什么呢？这是因为当环境变量 `classpath` 设置为固定的 `E:\` 路径之后，类加载器只会从 `E:\` 路径下查找字节码文件，不再从当前路径下加载。我们来试试把 `HelloWorld.class` 文件放到 `E:\` 目录下呢？请看下图：

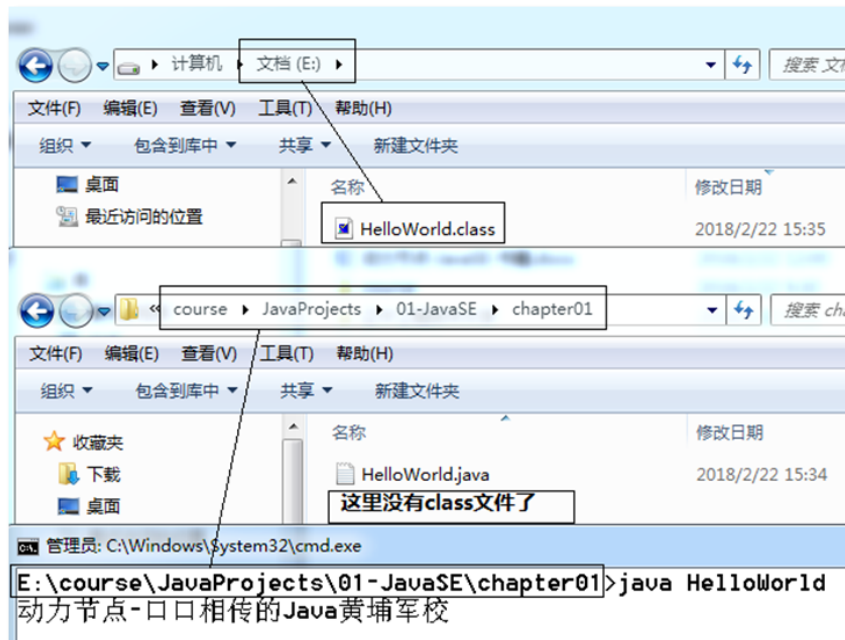


图 1-48: 测试环境变量 classpath

通过上图我们可以看到，将 HelloWorld.class 文件移动到了 E:\ 目录下，然后再执行 java HelloWorld，我们可以看到程序正常执行了。

通过以上的测试可以得出，当设置环境变量 classpath=E:\ 的时候，类加载器只去 E:\ 目录下加载 class 文件了，不再从当前目录下加载，也不会去其它目录下加载。大家在这里思考一下，如果将 classpath 配置成一个指定的路径，例如 classpath=E:\，这样我们以后的开发会不会很麻烦呢？

答案是非常麻烦，这是因为每一次编译生成的 class 文件都要放到 E:\ 目录下类加载器才能找到，所以目前来说 classpath 环境变量是不需要配置的，因为 classpath 在没有配置的情况下，类加载器默认会从当前所在的目录下加载 class，也就是说以后要想运行 class 首先要将 DOS 窗口的目录切换（cd 命令）到 class 文件所在的位置，然后再运行。当然我们也可以把环境变量 classpath 配置为：classpath=., 因为.代表当前路径。

在本课中我们就不再设置环境变量 classpath 了，大家可以将 classpath 环境变量删除。或者您要是想配置 classpath 的话，就把 classpath 配置为.就行了。有同学认为，既然是这样我们为什么还要学习 classpath 环境变量呢，这是因为在 java 开发中有很多第三方的类库需要在我们的项目中引入，等需要引入其它类库的时候，我们就需要将这些类库的路径配置到 classpath 当中了。并且 classpath 可以配置多个路径，注意路径和路径之间采用半角分号分隔。例如以后学到 java 连接数据库 JDBC 的时候，环境变量就需要配置为 classpath=.;xxxx.jar，它表示的含义是类加载器可以从当前路径下加载，也可以去指定的 jar 包中加载字节码文件了。这里不再赘述，学到 JDBC 的时候再说。

小结: 通过以上内容的学习, 我们知道 classpath 环境变量不属于 windows 操作系统, 是 java 编程语言当中的一种机制, 这种机制是专门为类加载器加载 class 文件时提供路径依据的。最终的结论是 classpath 环境变量目前是不需要配置! 当然, 随着后面内容的学习, 大家会知道 classpath 环境变量总有一天是需要配置的, 到那个时候大家可别忘了将“当前路径.”配置到 classpath 当中, 例如: classpath=.;path1;path2;; 如果没有把.配置到 classpath 当中, 那么类加载器就不再从当前路径下加载 class 了。

到此为止大家必须掌握两个重要环境变量, 一个是 windows 操作系统的 path 环境变量, 另一个是 java 语言的 classpath 环境变量。至于有些参考资料上还提到了 JAVA_HOME 等环境变量, 其实这些环境变量对于我们目前来说是不需要配置的, 以后用到的时候再说吧。

1.9 对 HelloWorld 程序的解释（理解）

HelloWorld 程序的代码如下所示:

1	public class HelloWorld {
2	public static void main(String[] args) {
3	System.out.println("动力节点-口口相传的 Java 黄埔军校");
4	}
5	}

接下来, 我们对这个代码进行简单的解释, 这里只是一个简单的说明, 要彻底弄明白还需要后面课程的铺垫, 大家耐心等待。对于以上的程序我要说这么几点:

第一: public 表示公开的 (关键字, 固定写法)

第二: class 用来声明一个类 (关键字, 固定写法)

第三: HelloWorld 是一个类名 (既然是一个名字, 就可以改成其它的名字)

第四: public class HelloWorld 表示声明一个公共的类 HelloWorld

第五: 在 java 编程中, 一定要注意成对儿的符号要成对儿写, 以上 HelloWorld 当中成对儿的符号包括: 小括号(), 中括号[], 大括号{}, 双引号"。这些符号在编写的时候建议成对儿编写。

第六: 最初学习 java 编程的时候一定要注意代码的格式, 要有合理的缩进, 什么时候缩进呢? 大家需要记住: 只要“我”这个大括号{}包含着“你”, 那么“你”就应该比“我”低一级, 此时“你”应缩进。

第七：类体的概念，在以上程序中 HelloWorld 后面的大括号{}，这个大括号{}里被称为类体。如下所示：

```
public class HelloWorld {  
    // 类体  
}
```

第八：程序入口，java 中规定程序的入口是一个固定的写法，必须像以下代码一样，不这样写，程序无法执行：

```
public static void main(String[] args) {  
    // 方法体  
}
```

第九：以上程序的入口又叫做 main 方法，或者叫做主方法。大家记住固定写法即可。另外在 main 方法后面的大括号{}我们称之为方法体，方法体也是由大括号括起来的。

第十：方法体由一条一条 java 语句构成，每一条 java 语句必须以“;”结束。方法体当中的代码遵循自上而下的顺序依次逐行执行。

以上对 java 的入门程序 HelloWorld 进行了简单说明，有一些内容现在无法彻底搞明白，学习后面内容之后大家就理解了。

1.10 Java 中的注释（掌握）

1.10.1 注释的作用

注释是对代码的解释和说明，其目的是让程序员能够更加快速的理解代码。它是编写程序时，写程序的人给一个语句、程序段等的解释或提示，能提高程序代码的可读性。我认为添加注释，是为了程序更容易理解与维护，特别是维护，更是对自己代码负责的一种体现。

注释在编译的时候不会生成到 class 字节码文件当中，它只在 java 源文件中保留。

1.10.2 注释的三种方式

Java 语言的注释包括三种方式，它们分别是：

第一种：单行注释，语法格式如下：

```
// 单行注释，两个正斜杠后面的内容被注释
```

第二种：多行注释，语法格式如下：

```
/*
```


* 这里的注释信息为多行注释:

* 第 1 行注释信息

* 第 2 行注释信息

*/

第三种: javadoc 注释。

/**

* 这里的信息是 javadoc 注释

* @author 作者名字

* @version 版本号

* @since 自从哪个版本号开始就存在了

*/

注意: 对于 javadoc 注释来说, 这里的注释会被 JDK bin 目录下的 javadoc.exe 命令解析并生成帮助文档 (生成帮助文档后期做项目的时候大家会接触到的)。

1.10.3 注释应该怎么写

编写注释是程序员最基本的素质, 养成编写注释的好习惯, 要有编写注释的意识。当然, 写注释也是有技巧的, 不是所有位置都写, 不是把写的代码原版翻译过来, 老程序员往往在写注释的时候, 不多不少, 能够做到恰到好处, 几句话就可以描述清楚程序的核心功能。

通常要在类和接口上写注释, 这一部分注释是必须的。在这里, 我们需要使用 javadoc 注释, 需要标明: 创建者, 创建时间, 版本, 以及该类的作用。在方法中, 我们需要对入参, 出参, 以及返回值, 均要标明。对常量, 我们需要使用多行注释, 进行标明该常量的用途。在关键算法上, 添加注释并且按照顺序依次标明, 写明白该方法为什么这么做。

记住: 注释的作用不在于表示代码的含义, 而在于表示代码的功能。希望在以后的课程当中通过慢慢的培养, 能够写一手漂亮的注释, 当然, 目前大家只需要掌握注释有哪几种, 分别写到什么符号里就行了。

1.10.4 为 HelloWorld 提供注释

接下来我们为 HelloWorld 程序提供简单的注释信息, 来练习一下注释的编写:

1	/**
---	-----

```
2  * 我的第一个 Java 程序问世了
3  * @author bjpowernode
4  * @version 1.0
5  * @since 1.0
6  */
7  public class HelloWorld {
8      /**
9       * 这是程序的入口
10      * @param args 参数
11      */
12      public static void main(String[] args) {
13          // 向控制台输出一段话
14          System.out.println("动力节点-口口相传的 Java 黄埔军校!");
15      }
16  }
```

通过以上代码我们可以看到，HelloWorld 类上写了一个 javadoc 注释，在 javadoc 注释中提供了对这个类的整体描述信息、作者信息、版本号信息等。在 main 方法上提供了 javadoc 注释，对这个方法进行了说明，对参数进行了说明等。在输出信息的那行代码上提供了单行注释，说明了这行代码的作用。

1.11 public class 和 class 的区别（掌握）

在以上的程序中，我们看到 HelloWorld 类在定义的时候使用了关键字 public，那么一个类声明的时候可以不使用 public 吗？我们一起来看看它们有什么区别？

我们先来进行一个实验，看看一个 java 源文件中是否可以定义多个 class，请看下图：

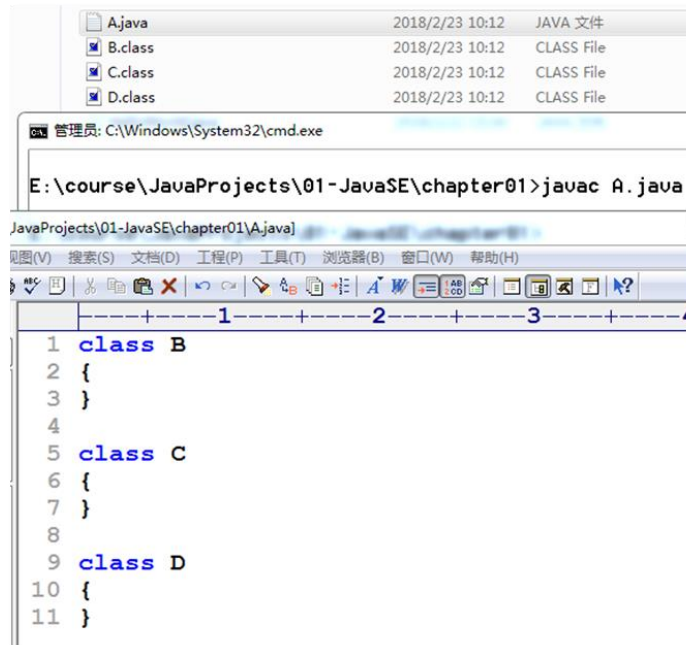


图 1-49：一个 java 源文件中可定义多个 class

我们可以看到创建了一个 A.java 源文件，在该文件中定义了三个类，分别是 B 类、C 类和 D 类，使用 javac 命令编译之后生成了三个字节码，分别是 B.class、C.class、D.class。

通过以上的测试可以得出：一个 java 源文件中可以定义多个 class，并且在编译的时候一个 class 会对应编译生成一个 class 字节码文件。还有，public 的 class 可以没有。

接下来，我们在 A.java 源代码中继续定义一个“公开的类 E”，请看下图：

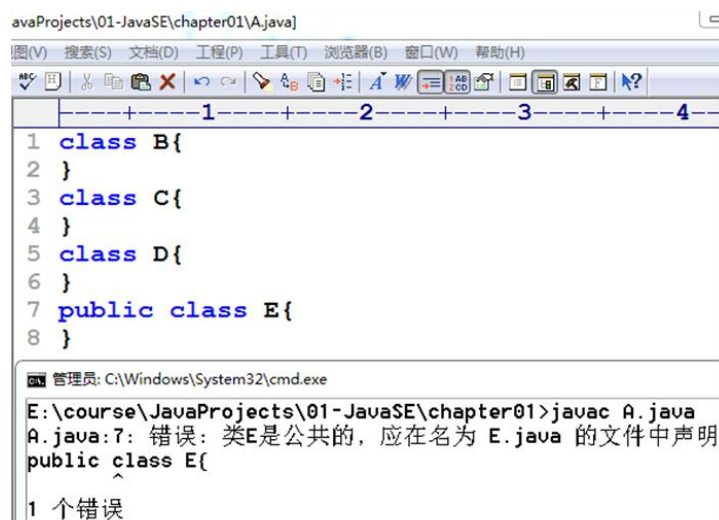


图 1-50: public class 的类名要求和 java 源文件名一致

我们可以看到，定义公开的类 E 之后，再次编译，编译器报错了。并且提示的错误信息是：类 E 是公共的，应在名为 E.java 的文件中声明。换句话说在 A.java 文件中定义的公共的类的名字必须是 A，不能是其它名称。也间接说明在同一个 java 文件中公共的类只能有一个（注意：在同一个 java 文件中类名不能重名）。

通过以上的测试可以得出：如果定义 public class 的类，只能定义一个，并且要求此类名必须和 java 源文件名保持一致。（这是规则记住就行，学计算机编程语言有很多知识点在学习的时候很难理解，只能靠记忆，随着后面内容的学习，大家会对以前困惑的知识点有所理解）。

接下来，我们在每一个类的类体当中都定义 main 方法，都写上程序的入口，看看是否可以编译和运行：

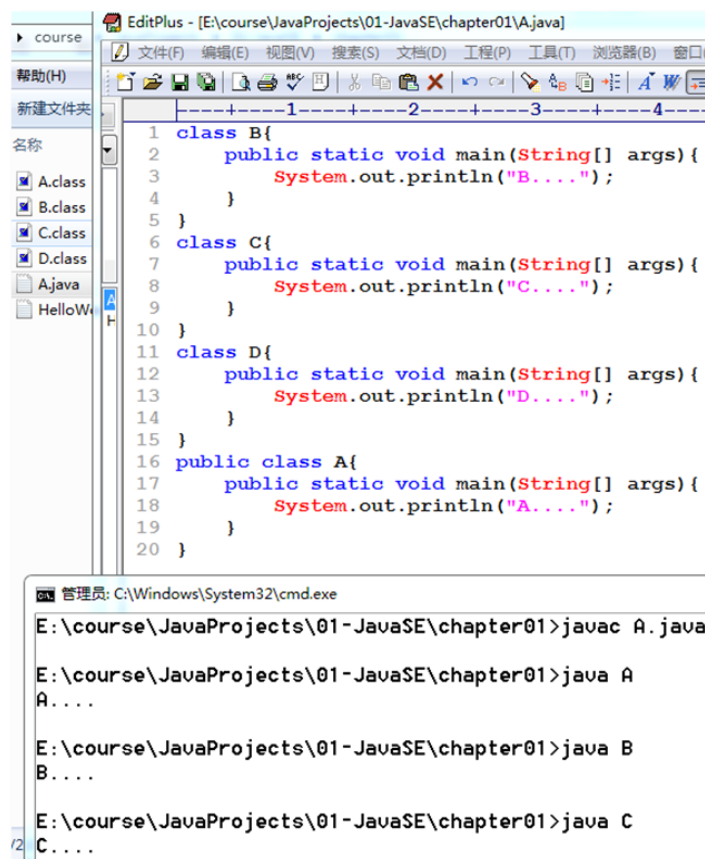


图 1-51: 每个类中都可以定义 main 方法

我们可以看到，在每一个 class 中都可以编写 main 方法，想让程序从哪个入口进去执行则加载该类即可。

通过以上的测试可以得出：任何一个 class 中都可以设定程序入口，也就是说任何一个 class 中都可以写 main 方法（主方法），想从哪个入口进去执行，则让类加载器先加载对应的类即可，例如：想让 A 类中的 main 方法执行，则执行：java A，想让 B 类中的 main 方法执行，则执行：java B。但实际上，对于一个完整的独立的应用来说，只需要提供一个入口，也就是说只需要定义一个 main 方法即可。

还有，在实际的开发中，虽然一个 java 源文件可以定义多个 class，实际上这是不规范的，比较规范的写法是一个 java 源文件中只定义一个 class。

1.12 章节小结

本章节的主要内容是带领大家搭建 Java 的开发环境，编写第一个 Java 程序。在这个过程中经历了 JDK 的安装，环境变量 path 和 classpath 的配置，Java 程序的编写、编译和运行。其中重点是需要大家理解 path 和 classpath 环境变量的作用以及如何配置。另外能够顺利的默写 HelloWorld 程序（不参考任何代码）。能够给 HelloWorld 程序提供简单的注释信息。

1.13 难点解惑

1.13.1 JDK 常用基本组件

JDK 常用的基本组件包括：javac（编译器）、java（运行 java 程序）、javadoc（提取 java 程序的注释信息并生成帮助文档）、jar（打 jar 包）、jdb（查错工具）、javap（反编译器）、jconsole（系统调试和内存监控工具）等。

以上有一些组件目前还没有接触到，随着后面内容的学习，大家会接触到的。

1.13.2 运行时出现“无法加载主类”

遇到这种情况，如下图所示：



图 1-52：无法加载主类

可能有以下几方面原因：

第一：在运行 java 程序时，目录没有切换到 class 文件所在的路径下。运行时，先使用 dir 命令查看当前路径下是否存在 xxx.class 文件。

第二：如果切换到 class 文件所在的路径下，还是出现以上问题，说明手动配置了环境变量 classpath，并且所配置的环境变量 classpath 当中没有当前路径“.”。要么将 classpath 删除，要么在 classpath 环境变量中添加当前路径“.”。

1.14 章节习题

第一题：编写 Java 程序，输出学生的基本信息，输出结果如下图所示：

```
学号：11111111  
姓名：动力节点  
生日：2009年6月1日  
家庭住址：北京市大兴区亦庄大族企业湾10号楼A座三层
```

图 1-53：输出学生信息

第二题：编写 Java 程序，输出京东商城商品列表信息，输出结果如下图所示：

```
-----  
|手机商品列表|  
-----  
|商品编号|商品名称|单价(元)|  
-----  
|1309203929|荣耀10青春版|1299.00|  
-----  
|1309203920|红米Redmi Note7|1199.00|  
-----  
|1309203921|HUAWEI P30 Pro|5988.00|  
-----  
|1309203922|荣耀10 GT游戏加速|1899.00|  
-----
```

图 1-54：输出商品列表信息

1.15 习题答案

第一题答案：

```
public class StudentInfo {
    public static void main(String[] args) {
        System.out.println("学号: 11111111");
        System.out.println("姓名: 动力节点");
        System.out.println("生日: 2009年6月1日");
        System.out.println("家庭住址: 北京市大兴区亦庄大族企业湾10号楼A座三层");
    }
}
```

第二题答案:

```
public class ProductListInfo {
    public static void main(String[] args) {
        System.out.println("-----");
        System.out.println("| 手机商品列表 |");
        System.out.println("-----");
        System.out.println("| 商品编号 | 商品名称 | 单价 (元) |");
        System.out.println("-----");
        System.out.println("| 1309203929 | 荣耀10青春版 | 1299.00 |");
        System.out.println("-----");
        System.out.println("| 1309203920 | 红米Redmi Note7 | 1199.00 |");
        System.out.println("-----");
        System.out.println("| 1309203921 | HUAWEI P30 Pro | 5988.00 |");
        System.out.println("-----");
        System.out.println("| 1309203922 | 荣耀10 GT游戏加速 | 1899.00 |");
        System.out.println("-----");
    }
}
```

2. 第二章 标识符与关键字

2.1 章节目标与知识框架

2.1.1 章节目标

了解构成 java 源程序的标识符和关键字都是什么, 掌握标识符的命名规则以及规范。能够识别标识符是否合法。

2.1.2 知识框架



2.2 标识符概述（了解）

标识符（identifier）是指用来标识某个实体的一个符号，在不同的应用环境下有不同的含义。在计算机编程语言中，标识符是用户编程时使用的名字，用于给变量、常量、函数、语句块等命名，以建立起名称与使用之间的关系。标识符通常由字母和数字以及其它字符构成。

在编程语言中，标识符就是程序员自己规定的代表一定含义的单词（java 源程序当中凡是程序员自己有权利修改的名字），比如类名，属性名，变量名等。如以下代码所示：

1	public class Student {
2	int age;
3	public void setAge(int a){
4	age = a;
5	}
6	}

其中，Student 是一个类名，表示学生类；age 是一个属性名表示学生的年龄属性，setAge 是一个方法名，a 表示一个变量名。这些都是标识符。

2.3 标识符详解

2.3.1 标识符都可以标识什么（理解）

在 java 源程序当中，标识符可以用来标识：

- 类名，例如：Student 学生类、User 用户类、Product 商品类、Order 订单类等。
- 接口名，例如：Runnable 可运行的、Comparable 可比较的等。
- 变量名，例如：name 名字、age 年龄、birth 生日、length 长度等。
- 方法名，例如：login 登录、logout 登出、eat 吃、drink 喝等。
- 常量名，例如：LOGIN_SUCCESS、ACCESS_TOKEN 等。

除了标识以上之外，还可以标识其他的，这里就不再一一列举，大家主要先把以上的了解一下。总之标识符就是起名字。

2.3.2 标识符命名规则（掌握）

标识符主要用来起名字，那么可以随便起名吗，有没有什么命名规则呢，答案是：有的，而且还得必须遵守，当编写源程序的时候如果标识符违背命名规则，编译时会报错。那么 java 中的标识符命名规则有哪些呢？请看以下规则：

- ① 标识符只能由数字、字母、下划线“_”、美元符号“\$”组成，不能含有其它符号。
- ② 标识符不能以数字开始。
- ③ java 关键字和保留字不能作为标识符。
- ④ 标识符严格区分大小写。
- ⑤ 标识符理论上没有长度限制。

以上几点需要大家在以后不断的练习中进行掌握，不需要死记硬背。

2.3.3 标识符命名规范（掌握）

遵守了标识符的命名规则之后，一起来看一看标识符有没有相关的命名规范呢，有同学可能问了：命名规则和命名规范有何不同呢？我在这里给大家解释一下，命名规则是一种语法上的要求，如果违背了，则表示语法错误，程序是无法正常编译的。而命名规范在一个团队中进行协同开发时尤为重要，如果大家都按照统一的命名规范书写代码，那么代码看起来就会像是同一个人编写的一样，能够很大程度上提高代码的可读性。换句话说，命名规范就是一个团队的编码约定。不过，当程序没能遵守命名规范的话，是不会影响程序的正常编译的。那么 java 中标识符的命名规范有哪些呢？请看以下通用的规范：

① 见名知意：看到这个单词就知道它表示什么，增强程序的可读性，例如：Student 则表示学生类型，User 则表示用户类型；

② 遵循驼峰命名方式：可以很好的分隔单词，每个单词之间会划清界限，同样也是增强程序的可读性，例如：getName 则表示获取名字，UserService 则表示用户业务类；

③ 类名、接口名首字母大写，后面每个单词首字母大写，这是遵守驼峰命名方式的；

④ 变量名、方法名首字母小写，后面每个单词首字母大写，这也是遵守驼峰命名方式的；

⑤ 常量名全部大写，单词和单词之间使用“_”衔接，为了表示清楚含义，不要怕单词长，例如：INT_MAX_VALUE 则表示 int 类型最大值。

以上的命名规范是大部分 java 开发团队通用的，但有一些团队可能要求更严格，和大家分享一段阿里巴巴的开发规约：



1. 代码中的命名均不能以下划线或美元符号开始，也不能以下划线或美元符号结束。

- 类名使用 UpperCamelCase 风格，必须遵从驼峰形式，但以下情形例外：（领域模型的相关命名）DO / BO / DTO / VO 等。
- 方法名、参数名、成员变量、局部变量都统一使用 lowerCamelCase 风格，必须遵从驼峰形式。
- 常量命名全部大写，单词间用下划线隔开，力求语义表达完整清楚，不要嫌名字长。
- 抽象类命名使用 Abstract 或 Base 开头；异常类命名使用 Exception 结尾；测试类命名以它要测试的类的名称开始，以 Test 结尾。
- POJO 类中布尔类型的变量，都不要加 is，否则部分框架解析会引起序列化错误。
反例：定义为基本数据类型 `boolean isSuccess;` 的属性，它的方法也是 `isSuccess()`，RPC 框架在反向解析的时候，“以为”对应的属性名称是 `success`，导致属性获取不到，进而抛出异常。

图 2-1：阿里巴巴 java 开发规约（部分截图）

通过上图，我们可以看到阿里巴巴的开发规约更严格一些，比如第一条：代码中的命名均不能以下划线或美元符号开始，也不能以下划线或美元符号结束。另外还有“POJO 类中布尔类型的变量，都不要加 is，否则部分框架解析会引起序列化错误……”。

在实际的开发中，每个团队都有自己的开发规约，大家写代码的时候一定要遵守自己团队的开发规约。这样才能增强程序的可读性。

2.4 关键字（理解）

Java 关键字是编程语言里事先定义的，有特殊意义的单词，Java 中所有的关键字都是小写的英语单词。

Java 的关键字对 Java 的编译器有特殊的意义，它们用来表示一种数据类型，或者表示程序的结构等，关键字不能用作标识符。常见的关键字有哪些呢？请看以下表格：

byte	short	int	long	float
double	boolean	char	if	for
else	while	do	continue	break
public	default	protected	private	throw
throws	try	catch	final	finally
case	switch	transient	package	new
static	this	abstract	strictfp	native
goto	super	extends	implements	import
instanceof	return	synchroniz ed	void	const
class	enum	assert	interface	volatile

接下来，我们一起看一下每个关键字代表的大致含义，请看以下表格：

关键字	大致含义
abstract	表明类或者成员方法具有抽象属性
assert	断言，用来进行程序调试
boolean	基本数据类型之一，声明布尔类型的关键字
break	提前跳出一个块
byte	基本数据类型之一，字节类型

case	用在 switch 语句之中，表示其中的一个分支
catch	用在异常处理中，用来捕捉异常
char	基本数据类型之一，字符类型
class	声明一个类
const	保留关键字，没有具体含义
continue	回到一个块的开始处
default	默认，例如，用在 switch 语句中，表明一个默认的分支
do	用在 do-while 循环结构中
double	基本数据类型之一，双精度浮点数类型
else	用在条件语句中，表明当条件不成立时的分支
enum	枚举
extends	表明一个类型是另一个类型的子类型，这里常见的类型有类和接口
final	表示不可变，最终的
finally	用于处理异常情况，用来声明一个基本肯定会被执行到的语句块
float	基本数据类型之一，单精度浮点数类型
for	一种循环结构的引导词
goto	保留关键字，没有具体含义
if	条件语句的引导词
implements	表明一个类实现了给定的接口
import	表明要访问指定的类或包
instanceof	用来测试一个对象是否是指定类型的实例对象

int	基本数据类型之一，整数类型
interface	接口
long	基本数据类型之一，长整数类型
native	用来声明一个方法是由与计算机相关的语言（如 C/C++ 语言）实现的
new	用来创建新实例对象
package	包
private	一种访问控制方式：私用模式
protected	一种访问控制方式：保护模式
public	一种访问控制方式：共用模式
return	从成员方法中返回数据
short	基本数据类型之一，短整数类型
static	表明具有静态属性
strictfp	用来声明 <code>FP_strict</code> （单精度或双精度浮点数）表达式遵循 IEEE 754 算术规范
super	表明当前对象的父类型的引用或者父类型的构造方法
switch	分支语句结构的引导词
synchronized	表明一段代码需要同步执行
this	指向当前实例对象的引用
throw	抛出一个异常
throws	声明在当前定义的成员方法中所有需要抛出的异常
transient	声明不用序列化的成员域
try	尝试一个可能抛出异常的程序块

void	声明当前成员方法没有返回值
volatile	表明两个或者多个变量必须同步地发生变化
while	用在循环结构中

以上关键字以及关键字的大致含义，大家目前先对其进行一个简单的了解，对于关键字不需要去死记硬背，随着后面内容的学习，每一天的积累，不断的敲代码，慢慢的就掌握了。接下来我们一起来看一下以下程序中有哪些单词是关键字，请看以下代码：

1	public class HelloWorld {
2	public static void main(String[] args){
3	System.out.println("Hello World!");
4	}
5	}

通过以上代码，我们可以看到其中 `public`、`class`、`static`、`void` 为关键字。有的同学说：我为什么没看出来哪些是关键字呢？我想说的是：别着急同学，后面慢慢的你就会找关键字了，也能很快的区分出哪些是标识符了，这需要一个过程，拭目以待吧。

2.5 章节小结

本章节主要是带领大家一起来看看 Java 程序的重要组成成分：标识符和关键字。其中标识符中重点掌握标识符的命名规则，以及命名规范，养成一个良好的编写规范，能够大大提高程序的可读性。另外，还有当给出相关的标识符，能够很快的判断其合法性。关键字这块重点要知道 Java 中的每一个关键字全部都是小写的英文单词，每个关键字都有特殊意义，并且不能拿关键字作为标识符。至于每个关键字所代表的含义，以及这个关键字如何编写，建议随着课程一边学习一边练习，然后一边再记忆。

2.6 难点解惑

之前我们在学习标识符的时候，其中有这样一条规则，那就是：标识符不能以数字开头。大家一起来思考一个这样的问题：新建一个 java 文件，起名“123Test.java”这样可以吗？接下来我们进行一个简单的测试，新建一个 123Test.java，打开文件，编写代码，定义类 T，如

下图所示：

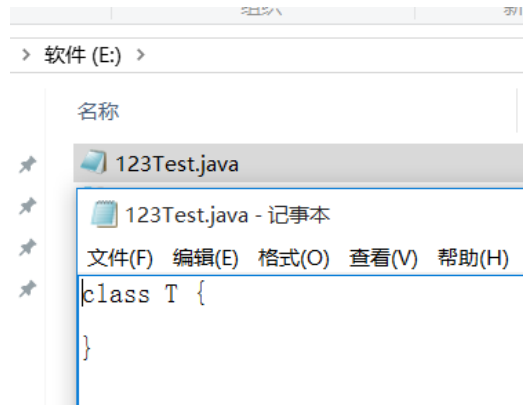


图 2-2：新建文件 123Test.java，定义 T 类

保存以上程序之后，进行编译，结果如下图所示：

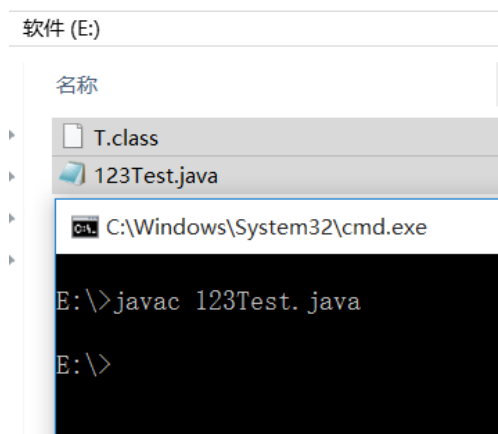


图 2-3：编译结果

我们可以看到 123Test.java 文件编译通过了，并且生成了 T.class 字节码文件。我相信有些同学看到这里的时候会恍然大悟。这是为什么呢？这是因为 123Test.java 中的“123Test”不是一个标识符，“123Test.java”只是一个普通的文件名而已。也就是说这里的“123Test”并不是作为一个类名的形式出现的，它还不是一个类的名字。

那么接下来大家再继续思考另一个问题，以上的“123Test.java”文件中能够定义一个公开的类吗？答案是：不能。这是为什么呢？因为我们之前学习过这样一条规则：公开的类的类名

必须和 Java 源文件名保持一致，换句话说，如果我们这里定义公开的类的话，类名必须是 123Test，这就尴尬了，123Test 作为类名出现时，则是一个标识符，而标识符命名规则中规定不能以数字开始，所以是不能的。我们进行一个简单的测试，将以上 123Test.java 文件中的代码全部删除，然后定义一个公开的类，起名 123Test，如下图所示：

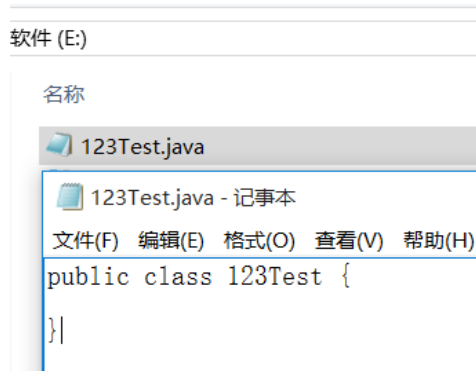


图 2-4：定义公开的类 123Test

接下来我们对以上程序进行编译，来看看会出现什么问题，请看下图：

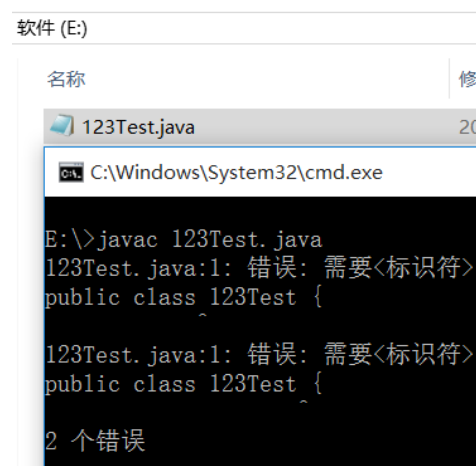


图 2-5：编译结果

通过以上的编译结果可以清楚的看到，“需要<标识符>”错误的出现，也就是说此时的 123Test 不是一个合法的标识符。

通过本难点的学习，你是否能够掌握到 java 源文件的名字并不是类名，并且也不需要符合标识符的命名规则呢。

2.7 章节习题

第一题：分析以下单词哪些是合法的标识符，哪些不合法，并说明原因：

myName, 字, My_name, Points, \$points, _sys_ta, OK, _23b, _3_, #name, 25name, class, &time, if, Hello World

2.8 习题答案

第一题答案：

myName, 字, My_name, Points, \$points, _sys_ta, OK, _23b, _3_ 都是合法的标识符。

#name: 不合法，标识符不能包含#

25name: 不合法，标识符不能以数字开始

class: 不合法，class 是关键字，不能做标识符

&time: 不合法，标识符不能包含&

if: 不合法，if 是关键字，不能做标识符

Hello World: 不合法，标识符不能包含空格

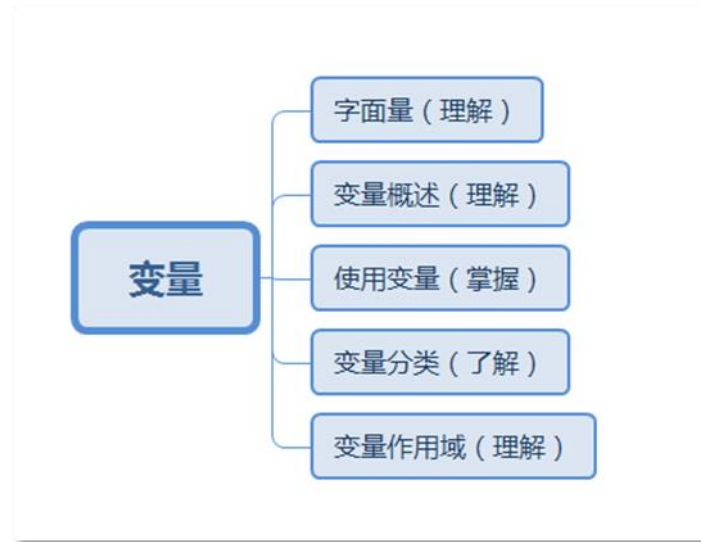
3. 第三章 变量

3.1 章节目标与知识框架

3.1.1 章节目标

理解变量本质是什么，在开发中有什么用？变量三要素是什么？怎么声明变量？怎么给变量赋值？变量是如何分类的？变量的作用域？

3.1.2 知识框架



3.2 字面量 (理解)

字面量就是数据/数值，例如：1234，true，"abc"，'中'，3.14。在现实生活中每天都会接触到数据，例如：你今天的体重是 86Kg，你今天花了 500 元，买了个西瓜重量是 8.6Kg，外面明明是晴天，你说说狂风暴雨，你说的是假话（false），你明明喜欢她，却嘴上说不喜欢，撒谎（false）。

软件其实是为了解决现实生活当中的问题，解决生活当中的问题其实就是处理生活当中的数据，一门编程语言首先要能够表示数据才可以处理数据，所以 Java 程序通过字面量来表示数据。

在编程语言中数据一般会被分门别类，所以每个数据都是有数据类型的，不同的数据类型会分配不同大小的内存空间去存储它。

数据被分为：整数型、浮点型、字符型、布尔型、字符串型等。

- 整数型（数字）：1、2、100、-2
- 浮点型（数字，带小数）：1.0、2.0、3.14
- 字符型（文字，单个字符）：'a'、'中'
- 布尔型（真假）：true、false
- 字符串型（文字，多个字符）："你好呀童鞋，欢迎来到动力节点！"

需要注意的是，java 中规定字符型字面量必须采用半角的单引号括起来，而字符串型字面量必须使用半角双引号括起来。这是一个语法规则，不然编译器就报错了。

接下来，我们一起来找出以下代码中哪些是字面量吧？

```
1 public class ConstTest1 {
2     public static void main(String[] args) {
3         System.out.println("小明的体重="+ 86 + "kg");
4         System.out.println("圆周率="+ 3.1415926);
5         System.out.println(true);
6         System.out.println(false);
7         System.out.println('男');
8         System.out.println('a');
9         System.out.println("你的对手在看书！");
10        System.out.println("你的闺蜜在减肥！");
11        System.out.println("你的仇人在磨刀！");
12        System.out.println("隔壁老王在练腰！");
13        System.out.println("你还不赶紧抓紧时间，还在这愣着干啥呀！！！");
14    }
15 }
```

通过以上的代码我们可以看到这些数据，或者说可以看到这些字面量：

- "小明的体重="：字符串型字面量
- 86：整数型字面量
- "kg"：字符串型字面量
- "圆周率="：字符串型字面量
- 3.1415926：浮点型字面量
- true、false：都是布尔型字面量，表示真和假
- '男'、'a'：都是字符型字面量
- "你的对手在看书！"、"你的闺蜜在减肥！"、"你的仇人在磨刀！"、"隔壁老王在练腰！"、"你还不赶紧抓紧时间，还在这愣着干啥呀！！！"：这些都是字符串型字面量

以上程序的运行结果如下图所示：

```
小明的体重=86kg  
圆周率=3.1415926  
true  
false  
男  
a  
你的对手在看书!  
你的闺蜜在减肥!  
你的仇人在磨刀!  
隔壁老王在练腰!  
你还不赶紧抓紧时间,还在这愣着干啥呀!!!
```

图 3-1：字面量

接下来，我们再看一段代码，找出以下程序中字面量属于哪种类型，请看代码：

```
1 public class ConstTest2 {  
2     public static void main(String[] args){  
3         System.out.println('9');  
4         System.out.println(9);  
5         System.out.println("100");  
6         System.out.println(100);  
7         System.out.println("3.1415926");  
8         System.out.println(3.1415926);  
9         System.out.println("true");  
10        System.out.println(true);  
11    }  
12 }
```

通过代码我们可以看到这些字面量：'9'、9、"100"、100、"3.1415926"、3.1415926、"true"、true，其中'9'由于带有单引号则属于字符型字面量，9 属于整数型字面量；"100"由于带有双引号则属于字符串型字面量，100 属于整数型字面量；"3.1415926"由于带有双引号则属于字符串型字面量，3.1415926 属于浮点型字面量；"true"带有双引号则属于字符串型字面量，true 属于

布尔型字面量。

接下来，再看一段代码，分析以下程序存在什么问题？

1	public class ConstTest3 {
2	public static void main(String[] args){
3	System.out.println(abc);
4	}
5	}

以上程序的编译结果如下图所示：

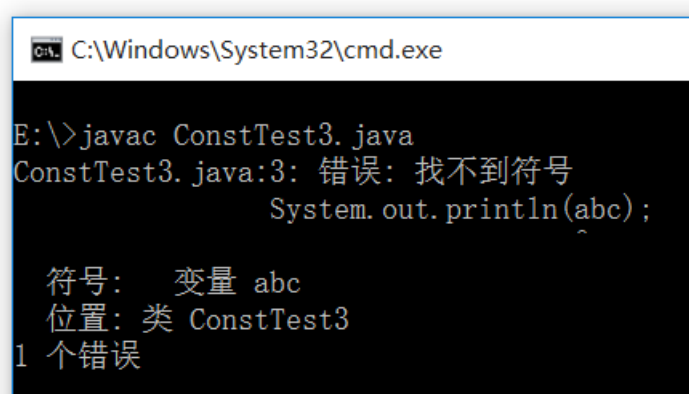


图 3-2：编译报错，找不到符号

以上程序为什么会编译报错呢，因为在程序中 `abc` 不是一个合法的字面量，`abc` 不是数字，也不是布尔型，那我们只能将其看做字符串型，但 Java 中规定字符串型字面量必须使用双引号括起来，这里没有，所以程序是不符合语法规则的。故编译报错。

接下来，我们再一起来看一段程序，请看以下代码：

1	public class ConstTest4 {
2	public static void main(String[] args){
3	System.out.println('ab');
4	}

5	}
---	---

我们对以上的程序进行编译，编译结果请看下图：



```
C:\Windows\System32\cmd.exe

E:\>javac ConstTest4.java
ConstTest4.java:3: 错误: 未结束的字符文字
    System.out.println(' ab');
                        ^
ConstTest4.java:3: 错误: 需要';'
    System.out.println(' ab');
                        ^
ConstTest4.java:3: 错误: 未结束的字符文字
    System.out.println(' ab');
                        ^
3 个错误
```

图 3-3：编译 ConstTest4.java 程序的结果

通过上图我们可以看到编译报错了，错误信息的第一条是：未结束的字符文字，这是为什么呢，因为在 java 语言中字符型只能是单个字符，多个字符则是字符串，应该使用双引号括起来。以上程序中编译器检测到'ab'之后，发现以单引号开始，会认为后面是一个字符，于是去 a 后面找另一半单引号，结果未找到结束的单引号（因为结束的单引号在 b 后面），所以编译器报错了，并且错误信息是“未结束的字符文字”。

通过本小节的学习，大家需要理解的是：什么是字面量。能够知道字面量就是数据，我们软件处理的就是数据，不同类型的数据在程序中有不同的编写方式，例如：字符型字面量必须是单个字符，并且使用半角的单引号括起来。字符串型字面量必须是使用半角的双引号括起来。布尔类型字面量只有两个值，写法是 true 和 false，true 表示真，false 表示假。而浮点型字面量则带有小数点。

3.3 变量

3.3.1 变量概述（理解）

变量是内存当中存储数据最基本的单元，将数据（字面量）放到内存当中，给这块内存空

间起一个名字，这就是变量。所以变量就是内存当中的一块空间，这块空间有名字、有类型、有值，这也是变量必须具备的三要素。变量在内存中的抽象图形可以参考下图：

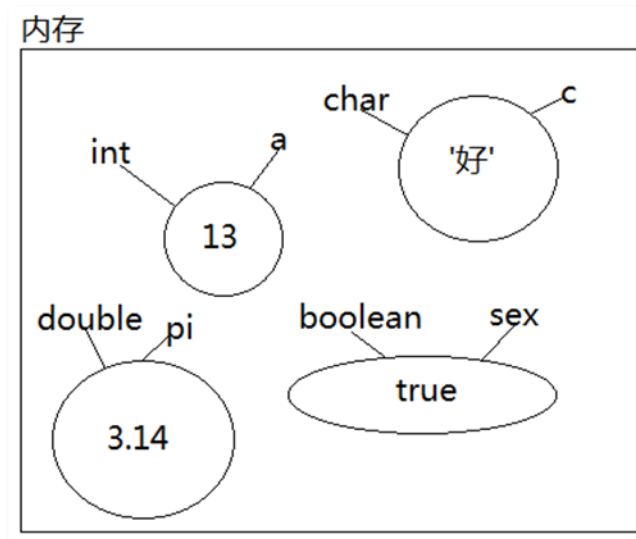


图 3-4: 变量内存抽象图

在上图当中每一个抽象的椭圆就代表一个变量，其中 a、c、pi、sex 是 4 个变量的名字（变量名只要是合法的标识符即可），13、'好'、3.14、true 是 4 个变量中分别存储的数据（字面量），int、char、double、boolean 是 4 个变量分别对应的数据类型（int、char、double、boolean 等都是 java 的关键字，声明变量时用来指定变量的数据类型）。

数据类型在任何一门编程语言当中都很重要，因为程序在运行的过程中会通过不同的数据类型给数据分配不同大小的空间。有的数据类型占用的空间较小，但有的数据类型占用的空间就会很大。这也是符合现实的，在现实生活中有些数据较大，有些数据则较小。

变量要求“变量的数据类型”和变量中存储的“数据（字面量）”必须类型是一致的，换句话说，冰箱是用来存放小食品的，也就是说冰箱只能存放小食品，大象不能往冰箱里放，原因是放不下，空间不合适。例如：int 类型就只能存下 4 个字节大小的整数，再长一点儿放不下，比如 long 类型的整数占有 8 个字节，这样的数据肯定是无法放到 int 类型的变量当中的。

所谓变量：可变化的量。它的意思是变量中存储的数据不是一成不变的，是可以被改变的，假设变量 i 中之前存储的数据是 10，我们可以将 10 换成 100，变量就是这个意思。

通过以上内容的学习，大家需要掌握一个变量是有三要素组成的，分别是：数据类型、变量名、存储的值。其中存储的值就是上一节讲到的字面量。

3.3.2 使用变量（掌握）

我们在使用变量之前需要先进行变量的声明，那么声明变量的语法格式是什么呢？请看：

数据类型 变量名；

以上则是声明变量的语法格式，其中数据类型我们在下一章节会详细讲解，目前我们以“int”这种数据类型为例进行学习，int 表示整数类型（注意：int 是关键字，不能随意写，必须全部小写）。变量名只要符合标识符命名规则即可，当然也要见名知意，命名规范中还要求变量名首字母小写，后面每个单词首字母大写。请看以下代码则表示声明一个 int 类型的变量 age 用来存储年龄数据：

1	public class VarTest01 {
2	public static void main(String[] args){
3	int age; // 年龄
4	}
5	}

大家可以看到上面的代码中这个 age 变量的三要素当中只具备了两个要素：数据类型和变量名，此时的 age 变量并没有存储数据（或者说没有赋值），那么这个 age 变量可以访问吗，我们来试一下，请看代码：

1	public class VarTest01 {
2	public static void main(String[] args){
3	int age; // 年龄
4	System.out.println(age);
5	}
6	}

我们对上面的程序进行编译，请看下图：

```
C:\>javac VarTest01.java
VarTest01.java:4: 错误: 可能尚未初始化变量age
    System.out.println(age);
                        ^
1 个错误
```

图 3-5: 编译 VarTest01.java

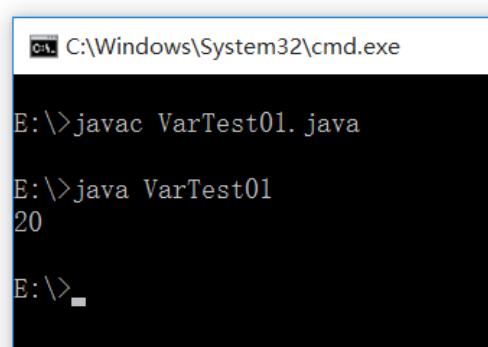
以上编译错误信息为：可能尚未初始化变量 `age`，这句话的意思是 `age` 变量还没有初始化（没有赋值），也就是说变量 `age` 中还没有数据，空间还没有开辟出来，可见，java 语言中的变量要求必须先声明，再赋值才能访问（这个规则大家一定要记住）。那么 java 语言中怎么给变量赋值呢？在 Java 语言中给变量赋值需要采用赋值运算符“=”，请看赋值的语法格式：

变量名 = 值;

在以上的语法当中，等号右边的值其实就是数据，我们之前所学的字面量就可以当做“值”。其中的等号“=”是一种运算符，它被称为赋值运算符，赋值运算符右边的表达式优先级较高，所以等号右边先执行，将执行结果赋给左边的变量。（注意：java 语言中的单等号不是用来判断是否相等的，是为了完成赋值运算的。）接下来我们给 `age` 变量赋值，请看以下代码：

1	public class VarTest01 {
2	public static void main(String[] args){
3	int age; // 年龄
4	age = 20;
5	System.out.println(age);
6	}
7	}

我们对以上的代码进行编译并运行，请看下图结果：



```
C:\Windows\System32\cmd.exe

E:\>javac VarTest01.java

E:\>java VarTest01
20

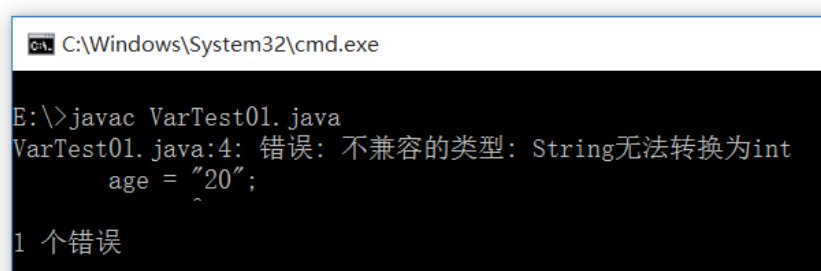
E:\>
```

图 3-6: VarTest01 程序的运行结果

可以看到，以上程序的运行结果是在控制台输出了：20。这里需要注意的是，当在 Java 程序中输出某个变量的时候，会自动输出变量中所保存的值。以上的测试说明了我们的赋值是没有问题的。那么赋值运算还有其他注意事项吗？有的。在进行赋值运算的时候，Java 中规定“值”的数据类型必须和“变量”的数据类型保持一致，也就是说 int 类型的变量只能存储 int 类型的数据，不能存储其他类型的数据，我们来进行一个简单的测试，请看以下代码：

1	public class VarTest01 {
2	public static void main(String[] args){
3	int age;
4	age = "20";
5	}
6	}

我们对以上的程序进行编译，请看下图的编译结果：



```
C:\Windows\System32\cmd.exe

E:\>javac VarTest01.java
VarTest01.java:4: 错误: 不兼容的类型: String无法转换为int
    age = "20";
           ^
1 个错误
```

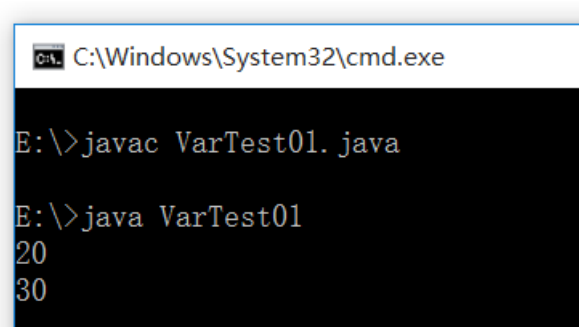
图 3-7: 编译 VarTest01.java 程序的结果

通过以上的编译我们可以看出，程序的第四行出错了，错误信息是：类型不兼容，这是因为编译器检测到 `age` 变量是 `int` 类型，只能存储 `int` 类型的数据，结果赋给 `age` 变量的数据不是一个 `int` 类型的数据，而是一个带有双引号的字符串。通过以上的测试证实了在进行赋值运算的时候，“值”的数据类型必须和“变量”的数据类型一致才可以。

变量赋值之后可以再次重新赋值吗？当然可以，要不然怎么能叫变量呢。请看以下代码：

```
1 public class VarTest01 {  
2     public static void main(String[] args){  
3         int age;  
4         age = 20;  
5         System.out.println(age);  
6         age = 30;  
7         System.out.println(age);  
8     }  
9 }
```

我们对以上的代码进行编译和运行，结果如下图所示：



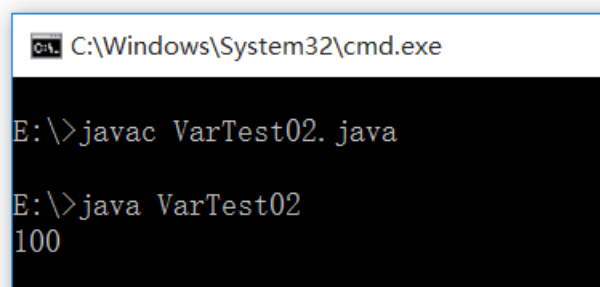
```
C:\Windows\System32\cmd.exe  
E:\>javac VarTest01.java  
E:\>java VarTest01  
20  
30
```

图 3-8: VarTest01 的运行结果

通过测试可以看到变量是可以重新赋值的。在以上的程序当中，我们看到变量的声明和赋值是分两行代码完成的，那么变量的声明和赋值可以在一行上完成吗，我们再进行一个简单的测试，请看以下代码：

```
1 public class VarTest02 {  
2     public static void main(String[] args){  
3         int count = 100;  
4         System.out.println(count);  
5     }  
6 }
```

我们对以上的程序进行编译和运行，请看运行结果：



```
C:\Windows\System32\cmd.exe  
E:\>javac VarTest02.java  
E:\>java VarTest02  
100
```

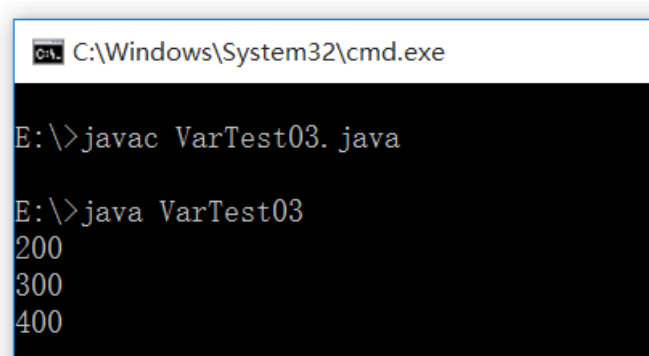
图 3-9: VarTest02 的运行结果

通过以上的测试，可以看出，变量的声明和赋值是可以在一行上完成的，声明变量的同时可以完成赋值运算。那么 Java 允许一次声明多个同类型的变量吗（C++是允许的，Java 可以吗）？请看以下代码：

```
1 public class VarTest03 {  
2     public static void main(String[] args) {  
3         int a = 200, b = 300, c = 400;  
4         System.out.println(a);  
5         System.out.println(b);  
6         System.out.println(c);  
7     }
```


8	}
---	---

以上程序当中，我们看到第三行代码，一次声明了 3 个 int 类型的变量，并且分别进行了赋值。我们对以上的程序进行编译和运行，结果如下图所示：



```
C:\Windows\System32\cmd.exe

E:\>javac VarTest03.java

E:\>java VarTest03
200
300
400
```

图 3-10: VarTest03 的运行结果

通过运行结果可以看出，Java 是允许一次声明多个同类型的变量的。我们将以上的代码进行一个简单的修改，请看修改之后的代码：

1	public class VarTest03 {
2	public static void main(String[] args) {
3	int a, b, c = 400;
4	System.out.println(a);
5	System.out.println(b);
6	System.out.println(c);
7	}
8	}

对以上的程序进行编译，请看以下编译结果：

```
C:\Windows\System32\cmd.exe

E:\>javac VarTest03.java
VarTest03.java:4: 错误: 可能尚未初始化变量a
    System.out.println(a);
                       ^
VarTest03.java:5: 错误: 可能尚未初始化变量b
    System.out.println(b);
                       ^
2 个错误
```

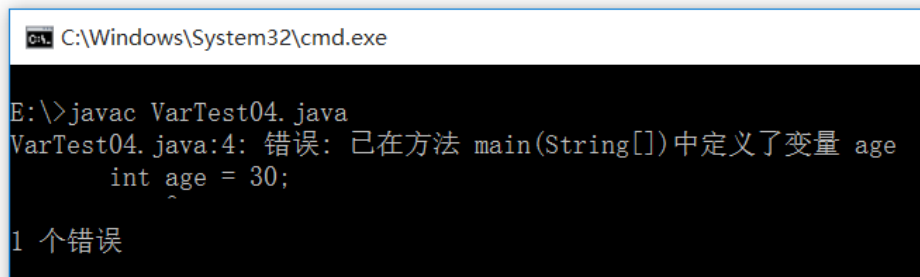
图 3-11: VarTest03.java 的编译结果

我们可以看到程序编译报错了，而错误信息中显示 a 变量和 b 变量并没有赋值，这说明了代码 “int a, b, c = 400;” 表示声明了三个 int 类型的变量，分别起名 a、b、c，其中 c 变量赋值 400，而 a 和 b 是没有赋值的，这里一定要注意，以上代码并不表示给 a、b、c 三个变量同时赋值 400。

接下来我们再来研究一下，在同一个大括号当中，可以声明两个同名的变量吗？请看以下代码：

1	public class VarTest04 {
2	public static void main(String[] args) {
3	int age = 20;
4	int age = 30;
5	System.out.println(age);
6	}
7	}

我们对以上的代码进行编译，请看下图结果：



```
C:\Windows\System32\cmd.exe

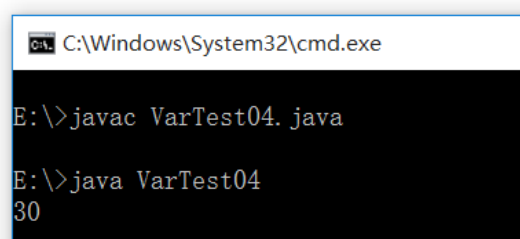
E:\>javac VarTest04.java
VarTest04.java:4: 错误: 已在方法 main(String[])中定义了变量 age
    int age = 30;
    ^
1 个错误
```

图 3-12: 编译 VarTest04.java

通过以上的编译结果，可以看到第 4 行出错了，结果也证实了在同一个大括号当中不能同时声明多个同名的变量。这是因为在同一个大括号当中表示在内存的同一个域当中，在同一块区域上面有两个名字都叫做 age 的变量，这是不允许的，因为这样程序在运行的时候，java 虚拟机也不知道该访问哪个变量了（好比说，你有两个朋友，他们的俩的名字都叫张三，当你们三个人同时在场的时候，你呼张三，其实他俩也不知道你在叫谁呢！）。所以，同一个域中变量名不能重名，但是记住：变量是可以重新赋值的。比如以下代码：

1	public class VarTest04 {
2	public static void main(String[] args) {
3	int age = 20;
4	age = 30;
5	System.out.println(age);
6	}
7	}

我们将以上代码编译并运行，请看下图结果：



```
C:\Windows\System32\cmd.exe

E:\>javac VarTest04.java

E:\>java VarTest04
30
```

图 3-13: 运行 VarTest04

最后我们再来看一下，在方法体当中的代码是否存在执行顺序，变量可以先访问，后声明吗？请看以下代码：

1	public class VarTest05 {
2	public static void main(String[] args) {
3	System.out.println(age);
4	int age = 20;
5	}
6	}

对以上程序进行编译，请看下图编译结果：



```
C:\Windows\System32\cmd.exe
E:\>javac VarTest05.java
VarTest05.java:3: 错误: 找不到符号
        System.out.println(age);
                        ~
符号:   变量 age
位置:  类 VarTest05
1 个错误
```

图 3-14: VarTest05.java 的编译结果

以上测试结果中显示第 3 行报错了，错误信息是找不到符号，换句话说 **age** 变量是不存在的，通过这个测试得知，方法中的代码是有执行顺序的，遵循自上而下的顺序依次逐行执行，也说明了变量必须先声明，然后才能使用。

通过本小节的学习，大家需要掌握的是在 Java 语言中变量如何声明，怎么赋值。另外也要知道变量是可以重新赋值的，还有声明和赋值可以分开完成，也可以一起完成，并且也可以一次声明多个变量。还有就是在同一个域当中变量名是不能重名的。方法当中的代码是有执行顺序的，遵循自上而下的顺序依次逐行执行。

3.3.3 变量分类（了解）

变量根据声明的位置不同可以分为：局部变量和成员变量。在方法体当中声明的变量以及方法的每一个参数都是局部变量。在方法体外，类体内声明的变量称为成员变量，成员变量声明时如果使用 `static` 关键字修饰的为静态成员变量（简称静态变量），如果没有 `static` 关键字修饰则称为实例成员变量（简称实例变量），请看以下代码：

1	<code>public class VarTest06 {</code>
2	<code> int x = 20; // 实例变量</code>
3	<code> static int y = 200; // 静态变量</code>
4	<code> public static void sum(int a, int b){ // 局部变量 a 和 b</code>
5	<code> int firstNum = 100; // 局部变量</code>
6	<code> }</code>
7	<code>}</code>

在以上代码当中，`sum` 是和 `main` 相似的方法，在 `sum` 方法当中 `a`、`b` 都是方法上的参数属于局部变量，在 `sum` 的方法体当中声明的 `firstNum` 属于局部变量。大家也可以看到在 `sum` 方法体外面声明了两个变量，一个是 `x`，一个是 `y`，这两个变量都属于成员变量，`x` 变量没有使用 `static` 修饰属于实例变量，而 `y` 属于静态变量。

局部变量只在方法体当中有效，方法开始执行的时候局部变量的内存才会被分配，当方法执行结束之后，局部变量的内存就释放了。所以局部变量的生命周期非常短暂。

在本小节当中，需要大家掌握的是，看到程序能够找出哪些是局部变量，哪些是实例变量，哪些是静态变量即可，至于这个变量什么时候声明为局部的，什么时候声明为成员的，后面的课程当中会详细介绍。

3.3.4 变量作用域（理解）

所谓变量的作用域就是变量的有效范围。通过后面内容的学习大家会更加明白，实际上局部变量、实例变量、静态变量它们存储在 Java 虚拟机的不同内存区域上，所以变量是有作用域的。关于变量的有效范围，在这里我给大家总结了一个口诀：出了大括号就不认识了。虽然这句话属于大白话，但很实用。我们来测试一下，请看以下代码：

1	public class VarTest07 {
2	public static void main(String[] args){
3	int k = 100;
4	}
5	public static void m(){
6	System.out.println(k);
7	}
8	}

在以上代码中我们可以看到在 main 方法中声明了一个变量 k，根据之前所学得知 k 变量是局部变量，只在本方法中有效（main 方法），那么在 m 方法中是否可以访问局部变量 k 呢，我们对以上程序进行编译，请看下图结果：

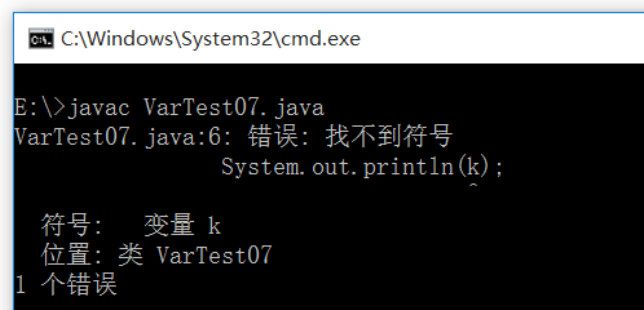


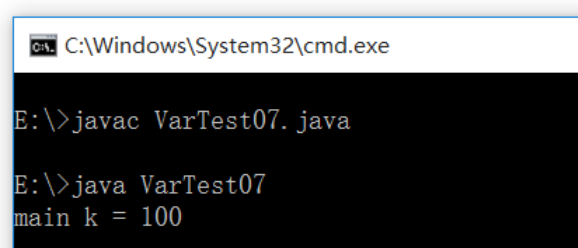
图 3-15: VarTest07.java 的编译结果

通过以上的编译结果可以看到第 6 行出错了，错误信息找不到符号，也就是说 k 变量是不存在的无法访问的。这也印证了之前所说的那句话：出了大括号就不认识了。接下来我们对以上程序进行修改，如果将变量声明为静态变量呢（这里不要纠结为什么声明成静态变量，后面会讲），变量的作用域会发生改变吗，请看以下代码：

1	public class VarTest07 {
2	static int k = 100;
3	public static void main(String[] args){
4	System.out.println("main k = " + k);

5	}
6	public static void m(){
7	System.out.println(k);
8	}
9	}

我们对以上代码进行编译，并运行，请看下图结果：



```
C:\Windows\System32\cmd.exe

E:\>javac VarTest07.java

E:\>java VarTest07
main k = 100
```

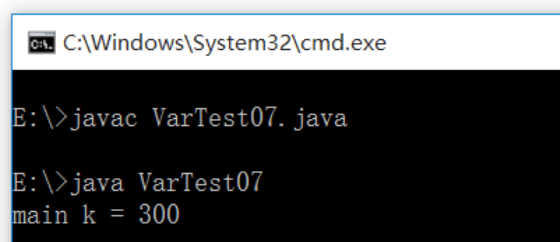
图 3-16：VarTest07 的运行结果

我们可以看到编译通过了，并且程序也正常运行了。有同学说，为什么只输出了 `main` 方法中的 `k`，而 `m` 方法中的 `k` 没有输出呢？这是因为 `m` 方法并没有执行，程序的执行是从 `main` 方法作为入口进来执行的，在 `main` 方法中并没有手动调用 `m` 方法，所以 `m` 方法是不会执行的。不过这个测试也印证了那句话：出了大括号就不认识了，只要没有出大括号，变量就是有效的可访问的，以上程序声明 `k` 变量的位置在类体当中，方法体在类体当中包含，所以在以上的 `main` 方法和 `m` 方法中都是可以访问 `k` 变量的。接下来我们对以上的程序进一步修改，请看以下代码：

1	public class VarTest07 {
2	static int k = 100;
3	public static void main(String[] args){
4	int k = 300;
5	System.out.println("main k = " + k);
6	}
7	public static void m(){
8	System.out.println(k);

9	}
10	}

看到以上代码，我们发现在 `main` 方法当中声明了一个局部变量 `k`，有同学可能会说，有两个同名的变量 `k`，这个程序编译会报错的，因为变量名不能重名，如果你要是这么想的就错了，我们之前确实讲过变量名不能重名，但是我们指的是“在同一个域”当中变量名不能重名，以上程序声明的两个变量 `k` 处在不同的域当中，一个是类体域，一个是方法体域。那么我们对以上程序进行编译并运行，结果会是怎样呢，请看下图结果：



```
C:\Windows\System32\cmd.exe

E:\>javac VarTest07.java

E:\>java VarTest07
main k = 300
```

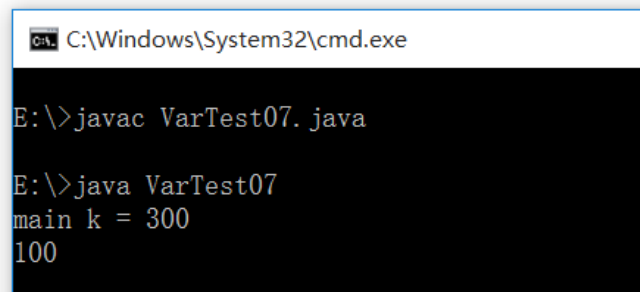
图 3-17: VarTest07 运行结果

通过上图我们看到程序不但编译通过了，而且还运行了，并且运行的结果不是 100，而是 300，这说明了 Java 程序遵守“就近原则”，其实这个就近原则不止是 Java 语言中存在，大部分的编程语言都符合这个原则。也就是说以上程序在 `main` 方法中访问变量 `k` 的话，访问的是局部变量 `k`，而不是静态变量 `k`。那么有同学就要问了，`m` 方法中的 `k` 访问的是哪个呢？我们尝试在 `main` 方法中调用 `m` 方法，让 `m` 方法执行，来测试一下输出结果，请看以下代码：

1	public class VarTest07 {
2	static int k = 100;
3	public static void main(String[] args){
4	int k = 300;
5	System.out.println("main k = " + k);
6	m();
7	}
8	public static void m(){
9	System.out.println(k);

10	}
11	}

以上 main 方法当中调用 m 方法这段代码大家可以先大概理解一下，方法调用这块的知识点会在后续的内容当中讲解。我们对以上程序进行编译并运行，请看下图结果：



```
C:\Windows\System32\cmd.exe

E:\>javac VarTest07.java

E:\>java VarTest07
main k = 300
100
```

图 3-18: VarTest07 运行结果

通过以上的测试我们可以看到在 m 方法中访问的是静态变量 k，因为输出结果是 100，而不是 300。

在本小节当中需要大家掌握的是变量的有效范围，还是需要大家记住那句话：出了大括号就不认识了。另外 Java 遵循就近原则。

3.4 章节小结

通过本章内容的学习，大家需要掌握变量应该如何声明，如何赋值，如何使用。需要理解变量就是内存当中存储数据的一块空间，它包括三要素：数据类型、变量名、值。另外要知道变量根据声明的位置可以分为成员变量和局部变量，并且声明位置不同作用域也是不同的。还需要注意在同一个域当中变量名不能重名，不同的域，变量名可以相同，只不过 Java 遵循就近原则，会自动访问离它最近的数据。

3.5 难点解惑

本章节内容整体比较简单，没有太多的难点，不过对于初学者来说，还是有一定难度的，尤其是变量的作用域这一小节，关键是对变量作用域的理解。请看以下代码：

1	public class VarTest08 {
2	public static void main(String[] args){
3	int num = 100;
4	if(num > 99){
5	int i = 10;
6	}
7	System.out.println(i);
8	}
9	}

我们对以上程序进行编译，请看下图编译结果：



```
C:\Windows\System32\cmd.exe
E:\>javac VarTest08.java
VarTest08.java:7: 错误: 找不到符号
        System.out.println(i);
                        ^
符号:   变量 i
位置: 类 VarTest08
1 个错误
```

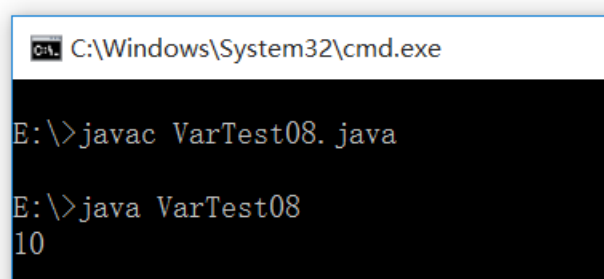
图 3-19：编译 VarTest08.java

通过以上的编译结果可以看出，程序的第 7 行出现错误，错误信息表示变量 `i` 不存在，这是因为变量 `i` 的声明位置是 `if` 语句的大括号当中，还是我们之前所说的那句话：出了大括号就不认识了。这里的 `i` 变量在 `if` 语句的大括号执行结束之后，内存就会自动释放，它的作用域是语句块级别的，生命周期更短。也就是说 `if` 语句的大括号之外都不能访问到变量 `i`。那么，如果想在后续的程序中继续使用变量 `i` 应该怎么办呢？我们可以将 `i` 变量的声明位置修改一下，请看代码：

1	public class VarTest08 {
2	public static void main(String[] args){
3	int num = 100;

```
4      int i = 0;
5      if(num > 99){
6          i = 10;
7      }
8      System.out.println(i);
9  }
10 }
```

我们对以上的代码进行编译并运行，请看下图结果：



```
C:\Windows\System32\cmd.exe

E:\>javac VarTest08.java

E:\>java VarTest08
10
```

图 3-20：运行 VarTest08

以上程序之所以可以访问 `i` 变量，是因为 `i` 变量的作用域修改成了方法体域的级别。扩大了它的作用范围。

3.6 章节习题

第一题：通过变量来描述学生的信息，学生信息包括：学号、姓名、性别、身高。其中学号采用整数，姓名采用字符串，性别采用字符型，身高采用浮点型（提示：查阅相关资料，在 `java` 中如何定义字符串类型的变量，如何定义字符型的变量，还有如何定义浮点型的变量）。具体的学生数据有两份，第一个学生信息是：学号 110，姓名张三，性别男，身高 1.85 米。第二个学生信息是：学号 120，姓名李四，性别女，身高 1.65 米。要求最终将学生的信息输出到控制台。输出结果如下图所示：

```
学号: 110  
姓名: 张三  
性别: 男  
身高: 1.85  
-----  
学号: 120  
姓名: 李四  
性别: 女  
身高: 1.65
```

图 3-21: 练习题要求的输出结果

3.7 习题答案

第一题答案: 代码如下所示:

```
1 public class VarPractice{  
2     public static void main(String[] args){  
3         int stuNo1 = 110;  
4         String stuName1 = "张三";  
5         char stuSex1 = '男';  
6         double stuHeight1 = 1.85;  
7         System.out.println("学号: " + stuNo1);  
8         System.out.println("姓名: " + stuName1);  
9         System.out.println("性别: " + stuSex1);  
10        System.out.println("身高: " + stuHeight1);  
11        System.out.println("-----");  
12        int stuNo2 = 120;  
13        String stuName2 = "李四";  
14        char stuSex2 = '女';  
15        double stuHeight2 = 1.65;  
16        System.out.println("学号: " + stuNo2);  
17        System.out.println("姓名: " + stuName2);  
18        System.out.println("性别: " + stuSex2);
```

19	System.out.println("身高: " + stuHeight2);
20	}
21	}

运行结果如下所示:

```
C:\Windows\System32\cmd.exe

E:\>javac VarPractice.java

E:\>java VarPractice
学号: 110
姓名: 张三
性别: 男
身高: 1.85
-----
学号: 120
姓名: 李四
性别: 女
身高: 1.65
```

图 3-22: VarPractice 程序的运行结果

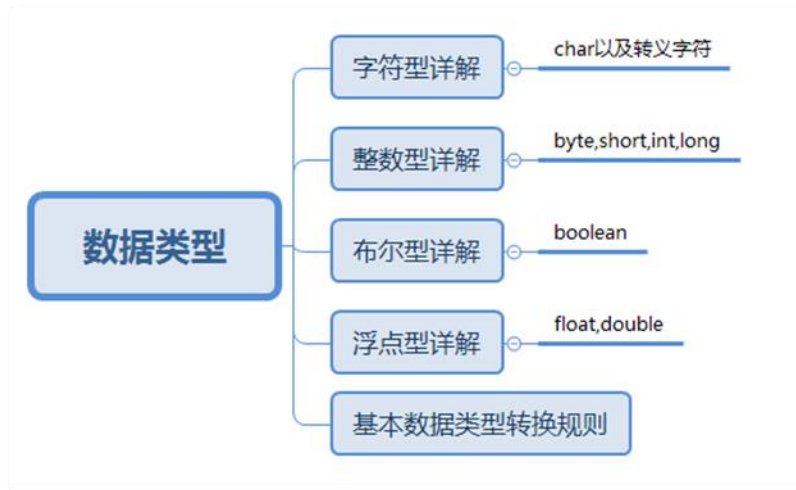
4. 第四章 数据类型

4.1 章节目标与知识框架

4.1.1 章节目标

本章节的目标是要求大家理解数据类型的作用，八种基本数据类型各是什么，常见数据类型的取值范围，怎么使用它们声明变量，各数据类型使用时的注意事项，另外要知道在实际开发中怎么选择合适的数据类型，还有这八种基本数据类型之间的相互转换。

4.1.2 知识框架



4.2 数据类型概述（理解）

在上一章节中我们学习了变量，我们知道任何一个变量都包括三要素，分别是：数据类型、变量名、值。其中数据类型尤为重要，目前我们已经接触过一种数据类型，那就是表示整数型的 `int`。接下来我们一起来学习一下其他的数据类型。

在学习其他数据类型之前我们先来思考一个问题，数据类型在程序中起到什么作用呢？实际上是这样的，软件的存在主要是进行数据的处理，现实生活中的数据有很多，所以编程语言对其进行了分门别类，然后就产生了数据类型，不同数据类型的数据会为其分配不同大小的空间进行存储。也就是说，数据类型作用就是决定程序运行阶段给该变量分配多大的内存空间。这就是数据类型的主要作用。那么 `java` 中的数据类型都包括哪些呢？实际上 `Java` 中的数据类型就包括两大类，一类是基本数据类型，另一类是引用数据类型（引用数据类型后面学习），其中，基本数据类型又包括 4 类 8 种：

- 第 1 类：整数型（不带小数的数字）：`byte, short, int, long`
- 第 2 类：浮点型（带小数的数字）：`float, double`
- 第 3 类：字符型（文字，单个字符）：`char`
- 第 4 类：布尔型（真和假）：`boolean`

大家可以看到，在以上的基本数据类型范畴中未发现字符串类型（带双引号的是字符串），所以，在这里我要告诉大家，`Java` 中的字符串属于引用数据类型，不属于基本数据类型的范畴。通过以上的学习，我们知道八种基本数据类型指的是：`byte`、`short`、`int`、`long`、`float`、`double`、`boolean`、`char`。接下来我们来看一下八种基本数据类型的详细信息，请看下表：

数据类型	占字节数	取值范围	缺省默认值
byte（字节型）	1	$[-2^7 \sim 2^7 - 1]$ 、 $[-128 \sim 127]$	0
short（短整型）	2	$[-2^{15} \sim 2^{15} - 1]$ 、 $[-32768 \sim 32767]$	0
int（整型）	4	$[-2^{31} \sim 2^{31} - 1]$ 、 $[-2147483648 \sim 2147483647]$	0
long（长整型）	8	$[-2^{63} \sim 2^{63} - 1]$	0L
float（单精度）	4	$[-2^{31} \sim 2^{31} - 1]$	0.0f
double（双精度）	8	$[-2^{63} \sim 2^{63} - 1]$	0.0
boolean（布尔型）	1	true、false	false
char（字符型）	2	$[0 \sim 2^{16} - 1]$ 、 $[0 \sim 65535]$	'\u0000'

通过上表我们可以看出八种基本数据类型中 byte 占用 1 个字节，short 占用 2 个字节，int 占用 4 个字节，long 占用 8 个字节，float 占用 4 个字节，double 占用 8 个字节，boolean 占用 1 个字节，char 占用 2 个字节。那么字节是什么呢，这个大家要知道 1 个字节是 8 个比特位，那么又有同学问了，1 个比特位是什么，1 个比特位就是一个 1 或 0，或者说 1 个比特位就是一个二进制位。也就是说 1 个字节是由 8 个 1 和 0 组成的二进制数字串。

接下来我们先普及一下计算机基础知识吧，实际上计算机在任何情况下都只能识别二进制，什么是二进制呢？计算机毕竟是一台通电的机器，电流只有正极、负极，所以只能表示两种情况，也就是 1 和 0。对于一串由 1 和 0 组成的数字来说就是二进制，所谓的二进制就是满 2 进 1，请看以下十进制和二进制的对照表：

十进制	0	1	2	3	4	5	6	7	8	9	10	11
二进制	0	1	10	11	100	101	110	111	1000	1001	1010	1011

其实十进制和二进制之间是存在转换规则的，如下所示：

- 十进制转换成二进制：比方说十进制数 65 转换成二进制，我们可以使用短除法，65 对 2 整除商 32 余数为 1，把 1 写在旁边，接着 32 对 2 整除商 16 余数为 0，把 0 写在旁边，用 16 整除 2 商 8 余数为 0，把 0 写在旁边，这样进行下去直至商为 0 时为止。然后把余数逆序排列就得到了 65 的二进制。如下图所示：

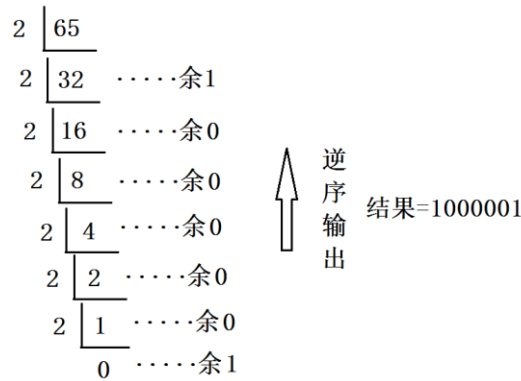


图 4-1：十进制转换成二进制

- 二进制转换成十进制：比方说二进制代码为 1000001 的十进制数是多少呢？可以采用按权相加的方法，对于二进制代码 1000001 首先从右边第一位起对应 2 的零次方，第二位对应 2 的 1 次方，以此类推，把相应的数位与权值相乘得到的积相加即可，即 $1 \times 2^0 + 0 \times 2^1 + 0 \times 2^2 + 0 \times 2^3 + 0 \times 2^4 + 0 \times 2^5 + 1 \times 2^6 = 65$

好了，计算机二进制的小插曲我们就先说到这里，言归正传，接下来我们继续学习八种基本数据类型。

当我们对二进制有一定的了解之后，来看一下 byte 类型的取值范围，为什么最大值只能取到 127 呢：首先数字是有正负之分，在二进制位当中最左边的二进制位是符号位，0 表示正数，1 表示负数，byte 属于字节型，占用空间大小是 1 个字节，1 个字节是 8 个 bit 位，所以 byte 类型最大值是左边一个 0 右边七个 1：01111111，这个二进制位实际上是 $2^7 - 1$ ，也就是 127。byte 类型最小值是 -128，那么，这也说明 1 个字节最多可以表示 256 种不同的情况（-128 到 127，中间有一个 0，共 256 个不同的数字）。

接下来我再给大家普及一下计算机的容量单位换算：

1byte = 8bit（1 个字节是 8 个比特位）

1KB = 1024byte

1MB = 1024KB

1GB = 1024MB

1TB = 1024GB

到这里，我相信各位会去想，我自己的硬盘是多大空间，可以存储多少个二进制位，你可以算算哦！

对于以上的八种基本数据类型，其中 byte、short、int、long 属于整数型，代表现实世界中的整数，只不过容量大小不同，细分的话，byte 叫做字节型，short 叫做短整型，int 叫做整型，long 叫做长整型，在实际的开发中，为了兼顾到开发效率，选择数据类型的时候也不会太斤斤

计较，这四种类型中 `int` 最为常用。

八种基本数据类型中，除了整数型可表示数字之外，浮点型也可以表示数字，并且浮点型表示的数字是带有小数的，其中包括 `float`、`double`，`float` 叫做单精度浮点数，`double` 叫做双精度浮点数，根据容量大小我们要知道 `double` 可以表示更精确的数字。在实际的开发中，`double` 更常用一些。不过对于财务系统来说，涉及到钱的问题，`double` 的精度也是远远不够的，后期我们会学到 Java 基础库中的 `BigDecimal` 类，该类可表示超大精度的数据。比较适合财务系统的开发。不过，`BigDecimal` 则不属于基本数据类型的范畴了。

八种基本数据类型中以上六种所描述的都是数字，除了数字之外，还可以表示文字，那就是基本数据类型 `char`，`char` 在 Java 中占用两个字节，一个汉字正好是两个字节，所以 `char` 类型完全可以存储一个汉字，`char` 类型的字面量要求使用半角的单引号括起来，例如：'`a`'、'`A`'、'`国`'等。`char` 类型和 `short` 类型都是占用 2 个字节，所以它们可表示的种类数量是相同的，不过由于 `char` 类型表示文字，没有负数这一说，所以相对来说 `char` 类型可以取到更大的正整数。通过上表我们也可以看出，`short` 类型取值范围是`[-32768~32767]`，而 `char` 类型取值范围是`[0~65535]`，它们可表示的种类数量都是 65536 种，只不过 `char` 可以取到最大值为 65535。

八种基本数据类型中有数字，还有文字，数字和文字占了七种，还有一种类型叫做布尔型，关键字是 `boolean`，这种类型在 Java 语言中只有两个值：`true` 和 `false`，没有其他值，用来表示现实世界中的真和假。布尔类型的数据在实际的开发中，尤其在业务逻辑判断方面起到非常重要的作用。并且使用较为频繁。布尔类型占用 1 个字节，`true` 和 `false` 其实在计算机底层是使用 1 和 0 来表示的。

通过本小节的学习，大家需要知道八种基本数据类型分别包括哪些，每种类型占用几个字节，取值范围是怎样的。对于二进制和十进制的转换，大家作为一个了解即可。

4.3 字符编码（理解）

对于以上的八种基本数据类型来说，其中七种类型 `byte`、`short`、`int`、`long`、`float`、`double`、`boolean` 计算机表示起来是很容易的，因为这七种类型底层直接就是数字，十进制的数字和二进制之间有固定的转换规则，所以计算机可直接表示和处理。但是大家别忘了，除了以上的七种数据类型之外，还有一种类型叫做字符型 `char`，这个对于计算机来说表示起来就不是那么容易了，因为字符毕竟是现实世界当中的文字，而文字每个国家又是不同的，计算机是如何表示文字的呢？

实际上，起初的时候计算机只支持数字，因为计算机最初就是为了科学计算，随着计算机的发展，为了让计算机起到更大的作用，因此我们需要让计算机支持现实世界当中的文字，一些标准制定的协会就制定了字符编码（字符集），字符编码其实就是一张对照表，在这个对照表上描述了某个文字与二进制之间的对应关系。

最初的时候美国标准协会制定了 ASCII 码，ASCII（American Standard Code for Information Interchange：美国信息交换标准代码）是基于拉丁字母的一套电脑编码系统，主要用于显示现代英语和其他西欧语言。它是现今最通用的信息交换标准，并等同于国际标准 ISO/IEC 646。ASCII 码采用 1 个字节编码，1 个字节可以表示 256 种不同的形式（前面说过了），对于英文来说这个足够了，因为英文单词就是由 26 个英文字母拼凑而成，大小写全部才 52 个，再加上数字和标点符号也不会超过 256 个。但 ASCII 码对于中文来说那就不够了，因为汉字不止 256 个。

常见的 ASCII 码需要大家能够记住几个，在 ASCII 码中规定'a'对应 97，'b'对应 98，以此类推，'A'对应 65，'B'对应 66，以此类推，'0'字符对应 48，'1'字符对应 49，以此类推，这些常见的编码还是需要大家记住的。

在字符编码当中，有这样两个常见的术语需要大家了解一下：编码和解码，它们都是什么，我们拿字符'a'来解释一下：'a'是 97，97 对应的二进制是 01100001，那么从'a'到二进制 01100001 的转换过程称为编码，从二进制 01100001 到'a'的转换过程称为解码。大家一定要注意：编码和解码要采用同一种字符编码方式（要采用同一个对照表），不然会出现乱码。这也是乱码出现的本质原因。

随着计算机的不断发展，为了让计算机支持更多国家的语言，国际标准组织又制定了 ISO-8859-1 字符集，又被称为 latin-1，向上兼容 ASCII 码，仍不支持中文，主要支持西欧语言。再后来，计算机慢慢的开始支持简体中文、繁体中文、日语、朝鲜语等，其中支持简体中文的字符集包括：GB2312、GBK、GB18030，它们的容量大小不同，其中 GB2312 < GBK < GB18030。支持繁体中文的是大五码 Big5 等。后来，在上世纪 90 年代初，国际组织制定了一种字符编码方式，叫做 Unicode 编码，这种编码方式统一了全球所有国家的文字，具体的实现包括：UTF-8，UTF-16，UTF-32 等。

Java 为了国际化，为了支持所有国家的语言，所以 Java 采用的编码方式为 Unicode 编码。例如字符'中'对应的 Unicode 码是'\u4e2d'。在实际开发中几乎所有的团队都会使用 Unicode 编码方式，因为这种方式更通用，兼容性更好。

通过本小节的学习，大家需要理解字符编码是什么，有什么作用，常见的 ASCII 码要知道一些，另外要理解什么是编码，什么是解码，要知道编码和解码采用的字符编码方式不同时会出现乱码，还要知道国际通用的编码方式为 ISO-8859-1，支持简体中文的编码方式包括 GB2312、GBK、GB18030，而 Java 采用 unicode 编码，目前在实际的开发中大部分团队都会选择 UTF-8 的编码方式。

4.4 数据类型详解

以上我们对数据类型有初步的了解之后，我们来详细的学习一下每一种数据类型。

4.4.1 字符型详解（理解）

字符型 char 在 Java 语言中占用 2 个字节，char 类型的字面量必须使用半角的单引号括起来，取值范围为[0-65535]，char 和 short 都占用 2 个字节，但是 char 可以取到更大的正整数，因为 char 类型没有负数。

Java 语言中的 char 类型变量可以容纳一个汉字。请看以下程序：

1	public class CharTest01{
2	public static void main(String[] args){
3	char c = '中';
4	System.out.println(c);
5	}
6	}

我们对以上的程序编译并运行，请看下图结果：

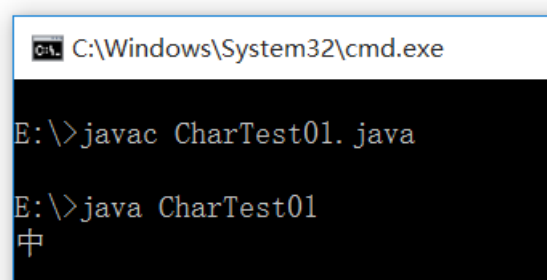


图 4-2：CharTest01 运行结果

我们可以看到 Java 中的 char 类型确实可以存储一个汉字。我们再来看以下程序，假设字符我们采用双引号括起来会怎样：

1	public class CharTest01{
2	public static void main(String[] args){
3	char c = "中";
4	System.out.println(c);

5	}
6	}

我们对以上的程序进行编译，请看下图编译结果：

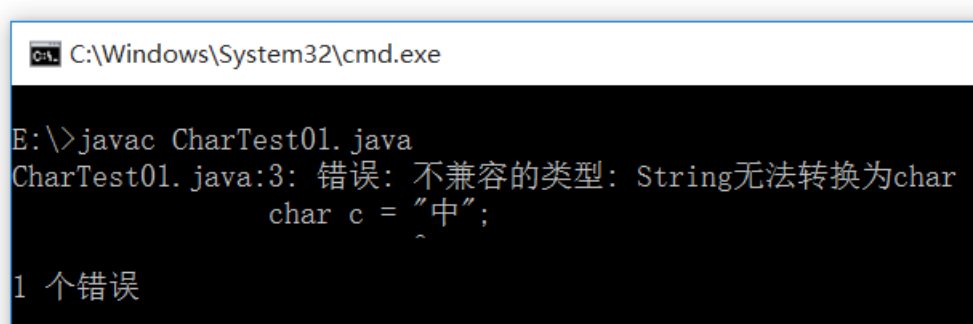


图 4-3: CharTest01.java 程序的编译

我们看到编译器报错了，并且提示的错误信息是“不兼容的类型”，这是因为双引号括起来不是 char 类型，而是 String 类型，其实 String 类型就是 Java 中的字符串类型，但大家要知道字符串不属于基本数据类型，而是引用数据类型。所以类型不兼容。接下来我们来测试一下两个或多个字符是否可以使用单引号括起来，请看以下代码：

1	public class CharTest01{
2	public static void main(String[] args){
3	char c = 'ab';
4	System.out.println(c);
5	}
6	}

我们对以上的程序进行编译，请看下图编译结果：


```
C:\Windows\System32\cmd.exe

E:\>javac CharTest01.java
CharTest01.java:3: 错误: 未结束的字符文字
    char c = 'ab';
              ^
CharTest01.java:3: 错误: 未结束的字符文字
    char c = 'ab';
              ^
CharTest01.java:3: 错误: 不是语句
    char c = 'ab';
              ^
3 个错误
```

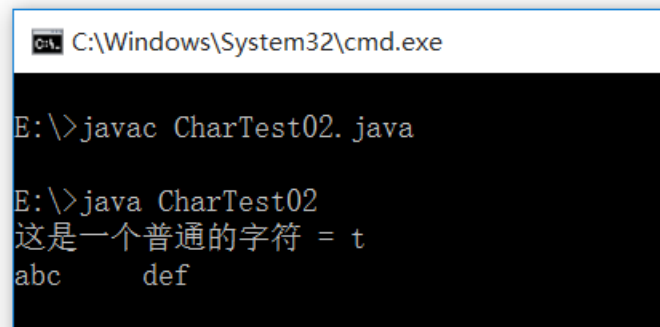
图 4-4: CharTest01.java 程序编译结果

我们可以看出，编译器报错了，错误信息是“未结束的字符文字”，这是因为Java中有规定，字符型只能是单个字符，当编译器检测到'ab'的时候，左边以单引号开始，继续检测到a字符，然后编译器会继续检查下一个字符是否为另一半单引号，结果不是，而是b，所以编译器报错了。这也说明了Java中的字符只能是单个字符，不能是多个字符。

接下来，我们再来看一下关于转义字符，转义字符指用一些普通的字符组合代表一些特殊的字符，由于组合用的字符改变了原意,称为转义字符。Java中的转义字符以\开始，常见的转义字符有：\t、\n、\u、\\、\'、\"，其中\t代表制表符，\n是换行符，\\表示一个普通的\字符，\'表示一个普通的'，\"表示一个普通的"。请看以下代码：

1	public class CharTest02 {
2	public static void main(String[] args) {
3	char c1 = 't';
4	System.out.println("这是一个普通的字符 = "+ c1);
5	char c2 = '\t';
6	System.out.println("abc" + c2 + "def");
7	}
8	}

我们对以上的程序进行编译并运行，请看下图结果：



```
C:\Windows\System32\cmd.exe

E:\>javac CharTest02.java

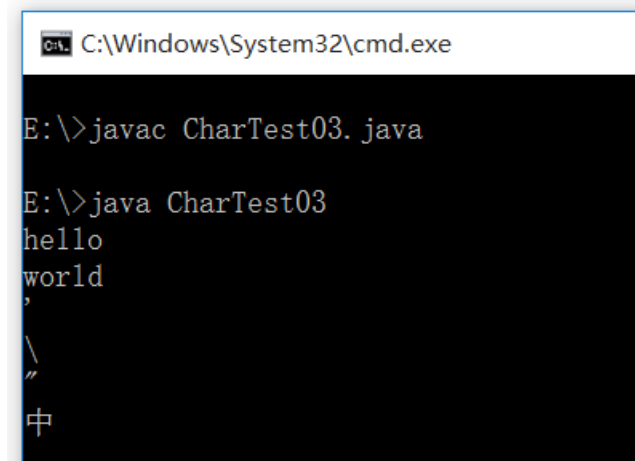
E:\>java CharTest02
这是一个普通的字符 = t
abc      def
```

图 4-5: CharTest02 运行结果

对于以上程序，表面看起来\t是由两个字符构成，按说应该编译报错，因为它毕竟是两个字符组成的字符串，可是最终的结果编译通过了，并且正常运行了，这说明了\t表示1个字符，所以\具有转义功能，根据以上输出结果可以看出\t是制表符（abc和def之间的空白就是制表符）。接下来我们来看一看其它的转义字符。请看以下程序：

1	public class CharTest03 {
2	public static void main(String[] args) {
3	System.out.println("hello\nworld");
4	System.out.println("\");
5	System.out.println("\");
6	System.out.println("\");
7	System.out.println("\u4e2d");
8	}
9	}

我们对以上的程序进行编译并运行，请看下图运行结果：



```
C:\Windows\System32\cmd.exe

E:\>javac CharTest03.java

E:\>java CharTest03
hello
world
'
\
"
中
```

图 4-6: CharTest03 程序运行结果

通过以上代码的测试，hello 和 world 之间换行了，所以 `\n` 代表一个换行符。

对于以上代码的第 4 行 `"\'` 来说，这里的 `\` 是不能去掉的，如果去掉之后代码就变成了 `"'`，那么此时编译器会报错，因为单引号在 Java 中有特殊含义，在这个代码当中第一个单引号会主动和第二个单引号配对，此时最后的单引号就是多余的了，不符合 Java 的语法，所以会导致编译错误，如果这个时候在第二个单引号前面添加 `\` 进行转义，代码是 `"\'`，那么此时第二个单引号 `'` 就是一个不具备特殊含义的普通单引号字符，这个时候第一个单引号会和最后一个单引号配对。编译和运行就正常了。

对于以上代码第 6 行来说和第 4 行的原理是相同的，代码 `\"` 表示普通的双引号字符。

对于第 5 行代码来说，代码 `\\` 联合起来表示一个普通的 `\` 字符，在 Java 中 1 个 `\` 字符不是普通的 `\` 字符，具有特殊的作用就是转义，我们想让其变成一个普通的 `\` 字符需要使用两个 `\` 来表示，代码 `\\` 中，第一个 `\` 具有转义功能，第一个 `\` 将第二个 `\` 转换成普通的 `\` 字符，以此类推，如果代码是 `\\\\` 这样，则表示结果是 `\\`。

对于第 7 行代码来说，如果代码修改为 `'u4e2d'` 必然报错，因为 `u4e2d` 在这个时候是一个普通的字符串，字符串是不能使用单引号括起来的，如果在 `u` 字符前添加转义字符 `\` 则表示的含义就不同了，`\u` 转义之后表示后面的 `4e2d` 是一个 unicode 码，根据 unicode 字符集可以查询到汉字'中'的 unicode 码就是 `4e2d`，所以最终输出结果是汉字'中'。

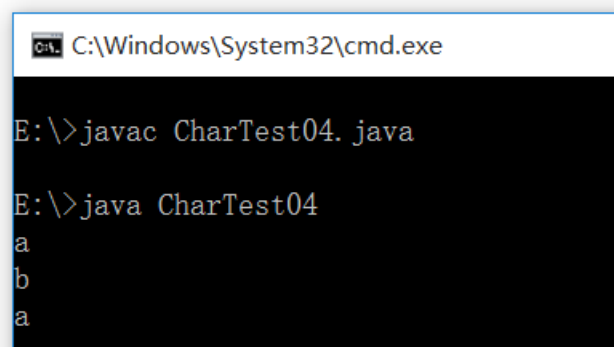
综上所述，`\n` 表示换行符，`\'` 表示普通的单引号字符，`\\` 表示一个普通的 `\` 字符，`\"` 表示一个普通的双引号字符，`\u` 后面的十六进制是文字的 unicode 编码。

通过本小节的学习，我们需要掌握的是，在 Java 语言中，字符 `char` 类型的数据只能使用单引号括起来，并且在 Java 语言中 `char` 类型完全可以存储一个汉字，另外还需要知道在 Java 语言中 `\` 具有转义功能，常见的转义字符要知道代表什么含义。

最后再给大家留两行代码，思考一下它的运行结果，在后面我们再详细讲解该内容，请看以下代码：

```
1 public class CharTest04 {  
2     public static void main(String[] args) {  
3         char c1 = 97;  
4         System.out.println(c1);  
5         char c2 = 'a' + 1;  
6         System.out.println(c2);  
7         char c3 = '0' + '1';  
8         System.out.println(c3);  
9     }  
10 }
```

我们对以上的程序进行编译并运行，请看下图运行结果：



```
C:\Windows\System32\cmd.exe  
E:\>javac CharTest04.java  
E:\>java CharTest04  
a  
b  
a
```

图 4-7：CharTest04 运行结果

在这里我简单提示一下，以上程序中的“+”运算符起到的作用是求和并不是字符串连接运算，剩下的大家思考一下吧，后面我们再看。

4.4.2 整数型详解（理解）

整数型数据在 java 中有 4 种表示方式，分别是十进制、八进制、十六进制、二进制。不过

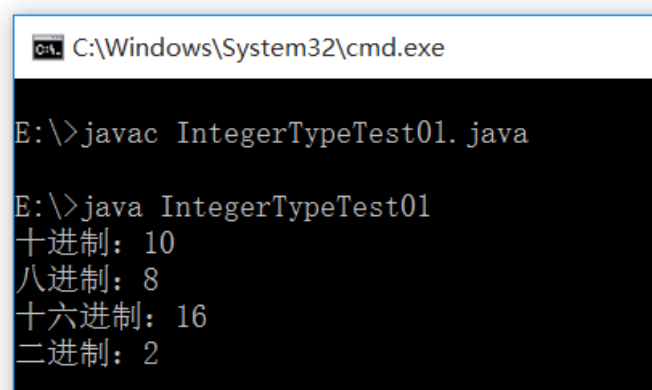
要注意的是二进制写法是在 Java7 中引入的，对于 Java7 之前的版本不支持该语法。默认为十进制，以 0 开始表示八进制，以 0x 开始表示十六进制，以 0b 开始表示二进制。十进制、八进制、十六进制有什么区别，请看：

- 十进制：0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17...
- 八进制：0,1,2,3,4,5,6,7,10,11,12,13,14,15,16,17,20,21...
- 十六进制：0,1,2,3,4,5,6,7,8,9,a,b,c,d,e,f,10,11...

接下来我们在代码中试验一下以上的几种写法，请看以下代码：

```
1 public class IntegerTypeTest01{
2     public static void main(String[] args){
3         System.out.println("十进制: " + 10);
4         System.out.println("八进制: " + 010);
5         System.out.println("十六进制: " + 0x10);
6         System.out.println("二进制: " + 0b10);
7     }
8 }
```

我们对以上的程序进行编译并运行，请看下图运行结果：



```
C:\Windows\System32\cmd.exe

E:\>javac IntegerTypeTest01.java

E:\>java IntegerTypeTest01
十进制: 10
八进制: 8
十六进制: 16
二进制: 2
```

图 4-8: IntegerTypeTest01 的运行结果

通过测试确实可以看到八进制 10 是 8，十六进制的 10 是 16，二进制的 10 是 2。在实际的开发中大部分还是直接使用十进制的方式，因为十进制对于我们来说更直观，程序的可读性会更好一些。

在 java 语言当中，整数型字面量被当做 `int` 类型处理，也就是说在程序中只要遇到整数型的数字，该数字会默认被当做 `int` 类型来处理，如果想表示 `long` 类型则需要在字面量后面添加 `L/l`，建议大写 `L`，因为小写 `l` 和 `1` 不好区分。请看以下程序：

1	<code>public class IntegerTypeTest02{</code>
2	<code> public static void main(String[] args){</code>
3	<code> int a = 10;</code>
4	<code> long b = 10L;</code>
5	<code> }</code>
6	<code>}</code>

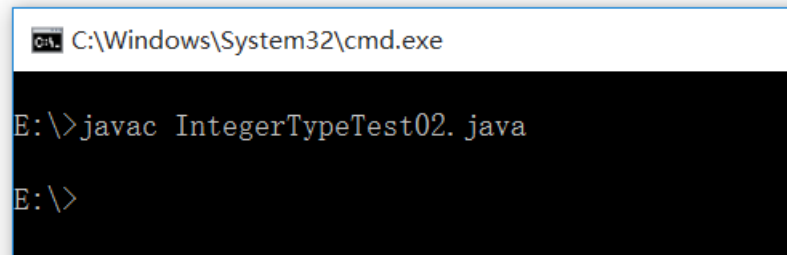
在以上的代码中 `int a = 10;` 表示声明一个 `int` 类型的变量 `a`，然后给 `a` 变量赋值 `10`，其中 `10` 是一个整数型字面值，根据以上规则，`10` 默认被当做 `int` 类型来处理，那么 `int a = 10;` 就表示 `int` 类型的字面量 `10` 赋值给 `int` 类型的变量 `a`，这个过程是不存在类型转换的。

另外在以上代码中 `long b = 10L;` 表示声明一个 `long` 类型的变量 `b`，然后给 `b` 变量赋值 `10L`，由于 `10` 后面添加有 `L`，则编译器会将 `10L` 当做 `long` 类型来处理，`long` 类型的字面量 `10L` 赋值给 `long` 类型的变量 `b`，这个过程也是不存在类型转换的。

接下来我们在以上代码的基础之上继续编写，请看以下代码：

1	<code>public class IntegerTypeTest02{</code>
2	<code> public static void main(String[] args){</code>
3	<code> int a = 10;</code>
4	<code> long b = 10L;</code>
5	<code> long c = 10;</code>
6	<code> System.out.println(c);</code>
7	<code> }</code>
8	<code>}</code>

我们可以看到在第 5 行新增了一行代码：`long c = 10;`，这行代码是什么原理呢？我们先来编译，看看是否符合 Java 的语法规则，请看下图编译结果：



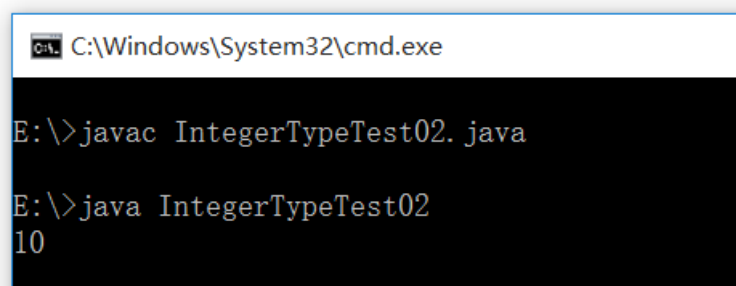
```
C:\Windows\System32\cmd.exe

E:\>javac IntegerTypeTest02. java

E:\>
```

图 4-9: IntegerTypeTest02.java 的编译结果

接下来我们再运行一下，请看下图运行结果：



```
C:\Windows\System32\cmd.exe

E:\>javac IntegerTypeTest02. java

E:\>java IntegerTypeTest02
10
```

图 4-10: IntegerTypeTest02 运行结果

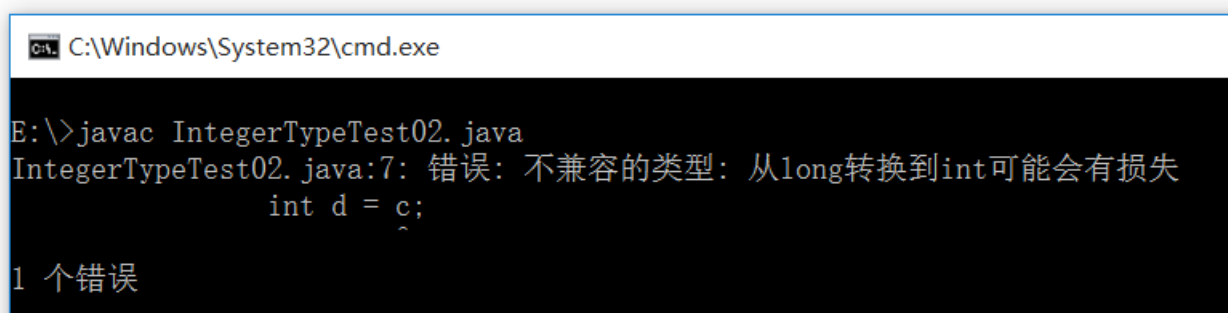
通过以上的测试我们可以看到 `long c = 10;` 这种编写方式符合语法规则，并且也可以正常运行，我们来分析一下，`10` 是整数型字面量，按照以上规则，Java 中会将 `10` 默认当做 `int` 类型处理，而 `c` 变量是 `long` 类型，`int` 类型可以赋值给 `long` 类型的变量吗？答案是可以的，因为我们已经测试过了。这是因为 `int` 占用 4 个字节，而 `long` 占用 8 个字节，在 Java 中小容量可以直接赋值给大容量，这个过程被称为自动类型转换。

接下来我们对以上代码继续编写，请看以下代码：

1	<code>public class IntegerTypeTest02{</code>
2	<code> public static void main(String[] args){</code>
3	<code> int a = 10;</code>
4	<code> long b = 10L;</code>
5	<code> long c = 10;</code>

6	System.out.println(c);
7	int d = c;
8	}
9	}

我们可以看到在第 7 行新增了一行代码：int d = c;，我们先对以上代码进行编译，请看以下编译结果：



```
C:\Windows\System32\cmd.exe

E:\>javac IntegerTypeTest02.java
IntegerTypeTest02.java:7: 错误: 不兼容的类型: 从long转换到int可能会有损失
    int d = c;
           ^
1 个错误
```

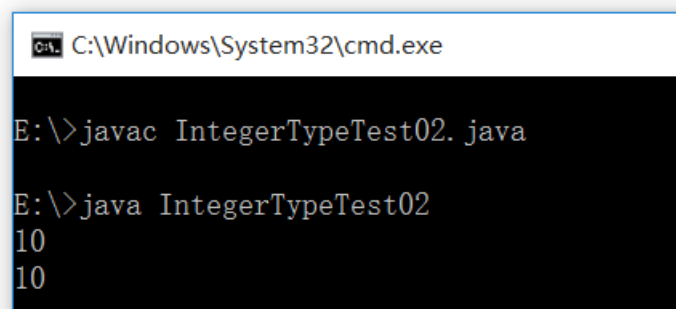
图 4-11: IntegerTypeTest02.java 的编译结果

我们可以看到编译器报错了，也就是以上编写方式不符合 Java 语法规则，我们来看一看为什么，首先我们看到错误提示信息是：不兼容的类型，从 long 转换到 int 可能会有损失。这是因为 c 变量是 long 类型占用 8 个字节，而负责接收的 d 变量是 int 类型占用 4 个字节，很明显是大容量转换成小容量，好比一大玻璃杯中的水倒向小玻璃杯，最终的结果可能会使水溢出，因为小玻璃杯可能放不下。编译器检测到这种情况的时候就不会自作主张了，需要程序员来指定，因为数据的损失需要程序员知晓，毕竟数据损失是一件很严重的事情，而编译器自身是不会负责的，于是 Java 中规定大容量如果需要转换成小容量，则程序员必须手动添加强制类型转换符才能编译通过，这个过程我们称为强制类型转换。我们对以上代码进行修改，请看以下修改之后的代码：

1	public class IntegerTypeTest02{
2	public static void main(String[] args){
3	int a = 10;
4	long b = 10L;
5	long c = 10;

6	System.out.println(c);
7	int d = (int)c;
8	System.out.println(d);
9	}
10	}

我们可以看到将第 7 行的代码修改为: `int d = (int)c;`, 这就是强制类型转换, 语法格式是在需要强转的数据前添加小括号, 小括号中写上要转换的类型, 我们对以上的程序编译并运行, 请看下图结果:



```
C:\Windows\System32\cmd.exe

E:\>javac IntegerTypeTest02.java

E:\>java IntegerTypeTest02
10
10
```

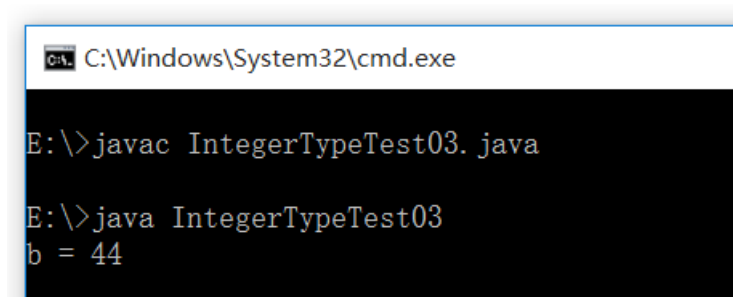
图 4-12: IntegerTypeTest02 运行结果

通过以上的测试我们得出这样一条结论: 一个数据在赋值给一个变量的时候存在三种不同的情况, 第一种情况是类型一致, 不存在类型转换; 第二种情况是小容量可以自动赋值给大容量, 称为自动类型转换; 第三种情况是大容量不能直接赋值给小容量, 大容量如果一定要赋值给小容量的话, 必须添加强制类型转换符进行强制类型转换操作。不过需要注意的是, 强制类型转换在使用的时候一定要谨慎, 因为可能会导致精度损失, 因为大杯水倒入小杯中, 可能会导致水的溢出, 不过这也不全都是, 也可能精度不会损失, 如果大杯中的水很少, 这个时候倒入小杯中也可能是不溢出的。就像以上的运行结果, 虽然进行了强制类型转换, 但并没有损失精度。接下来我们一起来看看精度损失是什么情况, 请看以下代码:

1	public class IntegerTypeTest03 {
2	public static void main(String[] args) {
3	int a = 300;
4	byte b = (byte)a;

5	System.out.println("b = " + b);
6	}
7	}

我们可以看到在以上代码中，int 类型的变量 a 强转为 byte 类型，我们对以上代码进行编译并运行，请看下图运行结果：



```
C:\Windows\System32\cmd.exe

E:\>javac IntegerTypeTest03.java

E:\>java IntegerTypeTest03
b = 44
```

图 4-13: IntegerTypeTest03 运行结果

4 个字节的 int 类型 300 强转为 1 个字节的 byte 类型，最终的结果是 44，为什么呢？这是因为首先 int 类型的 300 对应的二进制码是：00000000 00000000 00000001 00101100，强制类型转换的时候会变成 1 个字节，这个时候底层是将前 3 个字节砍掉了，也就是最后的二进制码是：00101100，这个二进制码对应的是 44。所以精度损失之后的结果就是 44 了。接下来我们再来看一下精度损失之后成为负数的情况，请看以下代码：

1	public class IntegerTypeTest04 {
2	public static void main(String[] args) {
3	int a = 150;
4	byte b = (byte)a;
5	System.out.println("b = " + b);
6	}
7	}

我们对以上的代码进行编译和运行，请看下图结果：

```
C:\Windows\System32\cmd.exe

E:\>javac IntegerTypeTest04.java

E:\>java IntegerTypeTest04
b = -106
```

图 4-14: IntegerTypeTest04 运行结果

为什么以上的运行结果是-106 呢？这是因为计算机在任何情况下都是采用二进制补码的形式存储数据的（为什么采用二进制补码形式存储数据，这里就不再赘述了，不做学术研究）。计算机二进制编码方式包括原码、反码、补码。对于正数来说原码、反码、补码是同一个。对于负数来说呢？负数的反码是在其原码的基础上，符号位不变，其余各个位取反，例如：-15 的原码是：10001111，-15 反码是：11110000。负数的补码是其反码再加 1。例如：-15 的补码是 11110000 加 1：11110001。换句话说-15 最终在计算机上会采用 11110001 二进制来表示。

我们再来看看以上的程序：`int a = 150`。4 个字节的 150 对应的二进制是：00000000 00000000 00000000 10010110，强转时前 3 个字节砍掉，最终计算机存储的二进制为：10010110，我们之前说过最终存储在计算机中的是二进制补码形式，也就是说 10010110 现在是二进制补码形式，我们通过补码推出原码，负数的补码是反码+1，所以 10010110 减 1 就是反码 10010101，反码的符号位不变，其余位取反就能得出原码：11101010，而这个值就是-106。对于以上原理大家了解即可，在实际的开发中很少使用。

接下来我们再来看一段程序，分析以下程序错在哪里，为什么以及怎么解决？

1	public class IntegerTypeTest05 {
2	public static void main(String[] args) {
3	long num = 2147483648;
4	}
5	}

我们对以上的程序进行编译，请看下图结果：

```
C:\Windows\System32\cmd.exe

E:\>javac IntegerTypeTest05.java
IntegerTypeTest05.java:3: 错误: 过大的整数: 2147483648
    long num = 2147483648;
                  ^
1 个错误
```

图 4-15: IntegerTypeTest05.java 的编译结果

我们可以看到，编译报错了，为什么呢？原因是：java 程序见到 2147483648 这个整数的时候，默认将其当做 int 类型来处理，但这个数字本身已经超出了 int 类型的取值范围（int 类型最大值是 2147483647），所以编译报错了，注意：这里编译报错的原因并不是说 long 类型存不下，long 类型的变量完全可以存储这个数字，以上程序出现的错误是在赋值之前，还没有进行到赋值运算，数字本身已经超出 int 类型范围，自己崩掉了。怎么解决以上的问题呢？其实很简单，我们只要让 java 程序认为 2147483648 是一个 long 类型的数据就行了，也就是说在该数字后面添加 L 问题就解决了（long num = 2147483648L;）。

接下来，一起来看一下以下程序是否可以编译通过，请看代码：

1	public class IntegerTypeTest06 {
2	public static void main(String[] args) {
3	byte b = 1;
4	System.out.println(b);
5	}
6	}

我们来分析一下以上的代码：byte b = 1;，1 是整数型字面量，在 java 中默认被当做 int 类型来处理，int 类型占用 4 个字节，b 变量是 byte 类型占用 1 个字节，根据上面所学，大容量无法直接赋值给小容量，要想赋值需要进行强制类型转换，这里没有强转，所以按理说编译是报错的，接下来我们来看一下编译结果，请看下图：

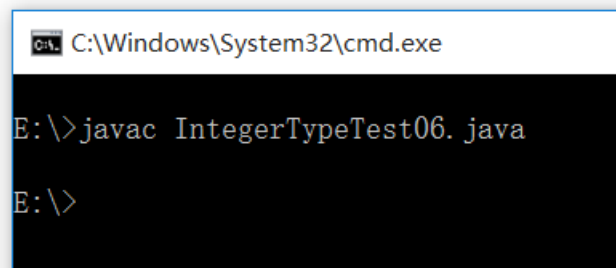
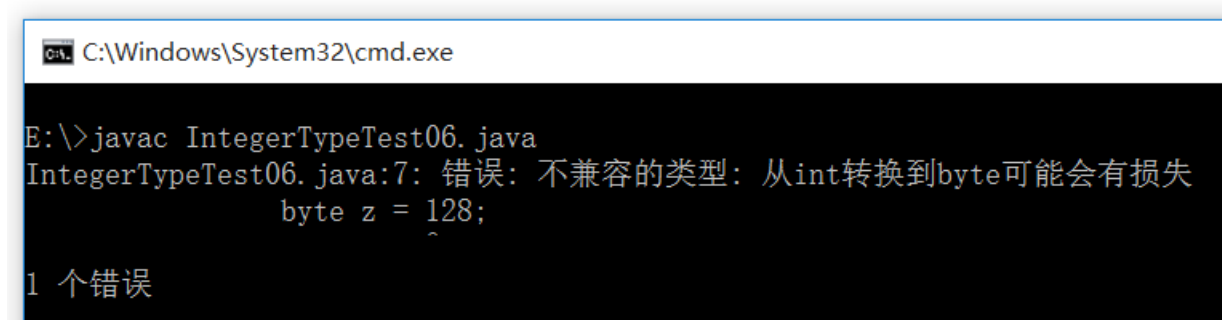


图 4-16: IntegerTypeTest06.java 编译结果

编译结果让我们意外了，编译通过了，这是为什么呢？我来给大家解释一下，这是因为在 java 语言有这样一条规定，大家记住就行了，如果当一个整数型字面量没有超出 byte 类型取值范围时，可以直接赋值给 byte 类型变量。那么如果整数型字面量超出 byte 类型取值范围会怎样呢？我们来测试一下，请看以下代码：

1	public class IntegerTypeTest06 {
2	public static void main(String[] args) {
3	byte b = 1;
4	System.out.println(b);
5	byte x = 127;
6	System.out.println(x);
7	byte z = 128;
8	System.out.println(z);
9	}
10	}

我们对以上的代码进行编译，请看下图编译结果：



```
C:\Windows\System32\cmd.exe

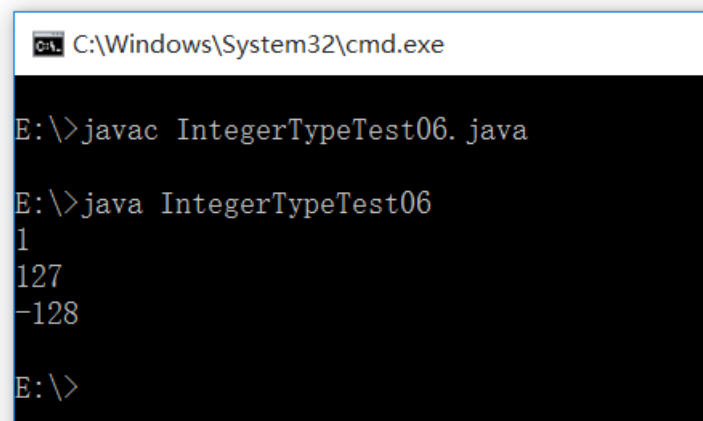
E:\>javac IntegerTypeTest06.java
IntegerTypeTest06.java:7: 错误: 不兼容的类型: 从int转换到byte可能会有损失
    byte z = 128;
              ~
1 个错误
```

图 4-17: IntegerTypeTest06.java 的编译结果

我们可以看到编译报错了，错误信息是第 7 行：不兼容的类型，从 `int` 转换到 `byte` 可能会有损失。对于以上程序的第 5 行并没有报错。针对这个错误信息我们之前在学习强制类型转换的时候接触过，也就是说以上程序要想编译通过必须进行强制类型转换，请看以下代码：

1	public class IntegerTypeTest06 {
2	public static void main(String[] args) {
3	byte b = 1;
4	System.out.println(b);
5	byte x = 127;
6	System.out.println(x);
7	byte z = (byte)128;
8	System.out.println(z);
9	}
10	}

我们对以上的程序进行编译并运行，请看下图结果：



```
C:\Windows\System32\cmd.exe

E:\>javac IntegerTypeTest06.java

E:\>java IntegerTypeTest06
1
127
-128

E:\>
```

图 4-18: IntegerTypeTest06 的运行结果

我们通过测试结果可以看出程序正常编译并运行了，这也印证了我们上面所说：当整数型字面量没有超出 `byte` 类型取值范围时，可以直接赋值。不过，如果超出了 `byte` 类型的取值范围，在使用时必须进行强制类型转换。但需要注意的是强制类型转换会导致精度的损失，例如以上代码中 `int` 类型的 128 强转为 `byte` 之后结果是 -128（这是因为计算机以二进制补码形式存储数字），还是要谨慎使用。

其实除了 `byte` 类型有这样的规则之外，`short` 和 `char` 也具有同样的规则，接下来我们先对 `short` 进行测试，代码如下所示：

1	<code>public class IntegerTypeTest07 {</code>
2	<code> public static void main(String[] args) {</code>
3	<code> short s1 = 32767;</code>
4	<code> System.out.println(s1);</code>
5	<code> short s2 = 32768;</code>
6	<code> System.out.println(s2);</code>
7	<code> }</code>
8	<code>}</code>

我们对以上的代码进行编译，请看下图编译结果：


```
C:\Windows\System32\cmd.exe

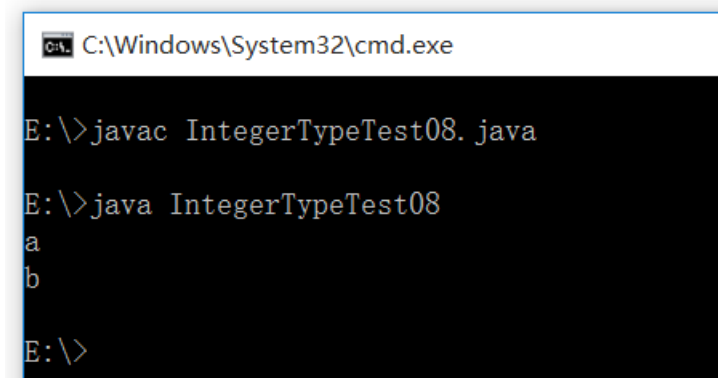
E:\>javac IntegerTypeTest07.java
IntegerTypeTest07.java:5: 错误: 不兼容的类型: 从int转换到short可能会有损失
    short s2 = 32768;
                  ^
1 个错误
```

图 4-19: IntegerTypeTest07.java 的编译结果

通过以上的结果可以看出第 3 行代码编译通过了，但是第 5 行编译报错了，这是因为 short 类型最大值是 32767。对于第 5 行的 32768 已经超出了 short 类型取值范围，同样如果要使用的话需要进行强制类型转换，这里就不再演示了。接下来我们再来看一看 char 类型，char 同样满足以上规则，当没有超出 char 类型取值范围时，可以直接赋值，请看以下代码：

1	public class IntegerTypeTest08 {
2	public static void main(String[] args) {
3	char c1 = 97;
4	char c2 = 98;
5	System.out.println(c1);
6	System.out.println(c2);
7	}
8	}

我们对以上的程序进行编译并运行，请看下图结果：



```
C:\Windows\System32\cmd.exe

E:\>javac IntegerTypeTest08.java

E:\>java IntegerTypeTest08
a
b

E:\>
```

图 4-20: IntegerTypeTest08 的运行结果

通过以上的测试可以看出当没有超出 char 类型取值范围的时候, 整数型字面量是可以直接赋值给 char 类型变量的, 但结果为什么会是字符 a 和 b 呢? 这是因为程序 char c1 = 97; 在实际执行的时候存在隐式的类型转换, 会自动将 int 转换成 char, 由于 char 最终是一个字符, 而 97 正好是字符 a 的 ASCII 码, 所以最终结果是字符 a 和 b。那么如果超出 char 类型取值范围会怎样呢 (char 最大值是 65535)? 请看以下代码:

1	public class IntegerTypeTest08 {
2	public static void main(String[] args) {
3	char c1 = 97;
4	char c2 = 98;
5	System.out.println(c1);
6	System.out.println(c2);
7	char c3 = 65536;
8	}
9	}

我们对以上代码进行编译, 请看下图编译结果:

```
C:\Windows\System32\cmd.exe

E:\>javac IntegerTypeTest08.java
IntegerTypeTest08.java:7: 错误: 不兼容的类型: 从int转换到char可能会有损失
    char c3 = 65536;
                ^
1 个错误

E:\>
```

图 4-21: IntegerTypeTest08.java 的编译结果

通过以上测试我们同样看到一旦超出 char 类型取值范围时就不能直接赋值了,要修改以上的错误也是需要进行强制类型转换操作,这里就不再演示了。

综上所述,大家记住一个结论: 当一个整数型的字面量没有超出 byte,short,char 的取值范围,可以将该字面量直接赋值给 byte,short,char 类型的变量,如果超出范围则需要添加强制类型转换符。

通过本小节的学习,我们需要掌握以下几个内容: 第一, Java 中的整数型字面量有四种表示方式,但最常用的还是十进制; 第二, 整数型字面量被当做 int 处理,如果想当做 long 处理,需要在后面添加 L 或 l; 第三, 小容量转换为大容量被称为自动类型转换; 第四, 大容量转换成小容量称为强制类型转换,强转时需要添加强制类型转换符,但要注意强转可能损失精度; 第五, 当整数型字面量没有超出 byte、short、char 的取值范围,可直接赋值。

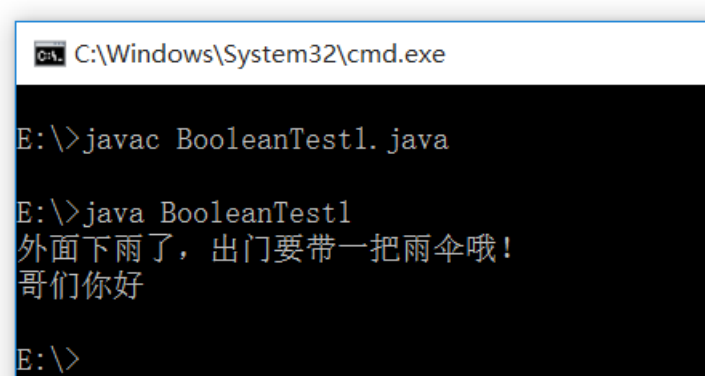
4.4.3 布尔型详解（理解）

在 Java 语言中布尔类型的值只包括 true 和 false,没有其他值,不包括 1 和 0,布尔类型的数据在开发中主要使用在逻辑判断方面,例如: 如果外面在下雨,我出门带一把雨伞。如果明天休息,咱们就一起出去玩耍吧。请看一段程序(以下程序中使用了控制语句,后面会详细讲,先大概了解一下):

1	public class BooleanTest1 {
2	public static void main(String[] args) {
3	boolean isRain = true;
4	if(isRain){

```
5      System.out.println("外面下雨了，出门要带一把雨伞哦！");
6      }else{
7          System.out.println("外面天气晴朗，走起吧！");
8      }
9      boolean sex = true;
10     if(sex){
11         System.out.println("哥们你好");
12     }else{
13         System.out.println("姐们你好");
14     }
15 }
16 }
```

我们对以上程序进行编译并运行，请看下图运行结果：



```
C:\Windows\System32\cmd.exe
E:\>javac BooleanTest1.java
E:\>java BooleanTest1
外面下雨了，出门要带一把雨伞哦！
哥们你好
E:\>
```

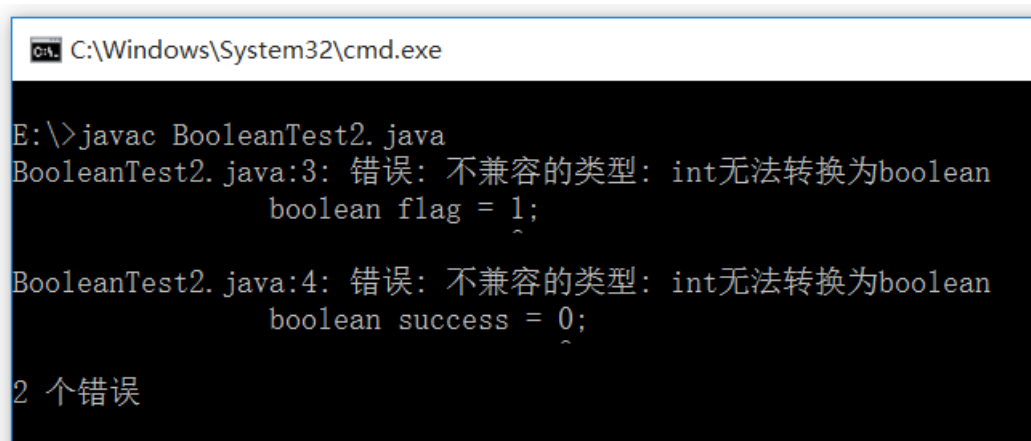
图 4-22: BooleanTest1 的运行结果

接下来对以上程序进行一个简单的解释：其中第 3 行代码表示定义一个布尔类型的变量 `isRain` 来表示是否下雨了，给其赋值 `true`，以下的判断逻辑是如果 `isRain` 为 `true` 则输出"外面下雨了，出门要带一把雨伞哦！"，反之则输出"外面天气晴朗，走起吧！"。第 9 行代码表示定义一个布尔类型的变量 `sex` 来表示性别，判断逻辑是如果 `sex` 为 `true` 则输出"哥们你好"，反之则输出"姐们你好"。

接下来，我们再来看一段代码，布尔类型变量的值是否可以使用 1 和 0：

```
1 public class BooleanTest2 {  
2     public static void main(String[] args) {  
3         boolean flag = 1;  
4         boolean success = 0;  
5     }  
6 }
```

我们对以上的程序进行编译，请看下图结果：



```
C:\Windows\System32\cmd.exe  
  
E:\>javac BooleanTest2.java  
BooleanTest2.java:3: 错误: 不兼容的类型: int无法转换为boolean  
        boolean flag = 1;  
                        ^  
BooleanTest2.java:4: 错误: 不兼容的类型: int无法转换为boolean  
        boolean success = 0;  
                        ^  
  
2 个错误
```

图 4-23: BooleanTest2.java 的编译结果

通过以上的测试结果可以看出，在 Java 中布尔类型的变量值不能使用 1 和 0，只能使用 true 和 false。

通过本小节的学习，大家需要掌握的是在 Java 语言中 boolean 类型的数据只有两个值，分别是 true 和 false，没有其他值，并且 boolean 类型在开发中主要使用在逻辑判断方面。

4.4.4 浮点型详解（理解）

浮点型数据实际上在内存中存储的时候大部分情况下都是存储了数据的近似值，为什么呢？这是因为在现实世界中存在无穷的数据，例如：3.3333333333333333..，数据是无穷的，但是内存是有限的，所以只能存储近似值，float 单精度占 4 个字节，double 双精度占 8 个字节，相对来说 double 精度要高一些。由于浮点型数据存储的是近似值，所以一般判断两个浮点型

数据是否相等的操作很少。

在 java 语言中有这样的一条规定：只要是浮点型的字面量，例如 1.0、3.14 等默认会被当做 double 类型来处理，如果想让程序将其当做 float 类型来处理，需要在字面量后面添加 f/F。请看以下代码：

```
1 public class DoubleTest {  
2     public static void main(String[] args) {  
3         float f = 3.0;  
4     }  
5 }
```

编译报错了：

```
C:\>javac DoubleTest.java  
DoubleTest.java:3: 错误: 不兼容的类型: 从double转换到float可能会有损失  
    float f = 3.0;  
                ^  
1 个错误
```

图 4-24：编译错误提示信息

为什么会编译报错呢？那是因为 3.0 默认被当做 double 类型来处理，占用 8 个字节，前面的 f 变量是 float 类型占用 4 个字节，大容量无法直接赋值给小容量。怎么修改呢？请看代码：

```
1 public class DoubleTest {  
2     public static void main(String[] args) {  
3         //float f = 3.0;  
4         //第一种方案  
5         float f = 3.0F;  
6         System.out.println("f = " + f);  
7         //第二种方案  
8         float f1 = (float)3.0;  
9         System.out.println("f1 = " + f1);  
}
```

10	}
11	}

运行结果如下图所示：

```
f = 3.0  
f1 = 3.0
```

图 4-25：运行结果

以上程序的第一种方案在 3.0 后面添加了 F，3.0F 被当做 float 类型来处理。第二种方案是进行了强制类型转换，第二种方案可能会存在精度损失。

4.5 基本数据类型转换（理解）

基本数据类型之间是存在固定的转换规则的，现总结出以下 6 条规则，无论是哪个程序，将这 6 个规则套用进去，问题迎刃而解：

- 八种基本数据类型中，除 boolean 类型不能转换，剩下七种类型之间都可以进行转换；
- 如果整数型字面量没有超出 byte,short,char 的取值范围，可以直接将其赋值给 byte,short,char 类型的变量；
- 小容量向大容量转换称为自动类型转换，容量从小到大的排序为：byte < short(char) < int < long < float < double，其中 short 和 char 都占用两个字节，但是 char 可以表示更大的正整数；
- 大容量转换成小容量，称为强制类型转换，编写时必须添加“强制类型转换符”，但运行时可能出现精度损失，谨慎使用；
- byte,short,char 类型混合运算时，先各自转换成 int 类型再做运算；
- 多种数据类型混合运算，各自先转换成容量最大的那一种再做运算；

接下来，根据以上的 6 条规则，我们来看一下以下代码，指出哪些代码编译报错，以及怎么解决（大家注意看代码的注释信息）：

1	public class TypeConversionTest {
2	public static void main(String[] args) {
3	byte b1 = 1000;


```
4      byte b2 = 20;
5      short s = 1000;
6      int c = 1000;
7      long d = c;
8      int e = d;
9      int f = 10 / 3;
10     long g = 10;
11     int h = g / 3;
12     long m = g / 3;
13     byte x = (byte)g / 3;
14     short y = (short)(g / 3);
15     short i = 10;
16     byte j = 5;
17     short k = i + j;
18     int n = i + j;
19     char cc = 'a';
20     System.out.println("cc = " + cc);
21     System.out.println((byte)cc);
22     int o = cc + 100;
23     System.out.println(o);
24 }
25 }
```

编译报错，错误信息如下所示：

```
C:\>javac TypeConversionTest.java
TypeConversionTest.java:3: 错误: 不兼容的类型: 从int转换到byte可能会有损失
    byte b1 = 1000;
                ^
TypeConversionTest.java:8: 错误: 不兼容的类型: 从long转换到int可能会有损失
    int e = d;
           ^
TypeConversionTest.java:11: 错误: 不兼容的类型: 从long转换到int可能会有损失
    int h = g / 3;
           ^
TypeConversionTest.java:13: 错误: 不兼容的类型: 从int转换到byte可能会有损失
    byte x = (byte)g / 3;
                ^
TypeConversionTest.java:17: 错误: 不兼容的类型: 从int转换到short可能会有损失
    short k = i + j;
              ^
5 个错误
```

图 4-26: 类型转换编译错误提示信息

如何修改，请看以下代码：

```
1 public class TypeConversionTest {
2     public static void main(String[] args) {
3         //1000 超出 byte 取值范围，不能直接赋值
4         //byte b1 = 1000;
5         /*
6          * 如果想让上面程序编译通过，可以手动强制
7          * 类型转换，但程序运行时会损失精度
8          */
9         byte b1 = (byte)1000;
10        //20 没有超出 byte 取值范围，可以直接赋值
11        byte b2 = 20;
12        //1000 没有超出 short 取值范围，可以直接赋值
13        short s = 1000;
14        //1000 本身就是 int 类型，以下程序不存在类型转换
15        int c = 1000;
16        //小容量赋值给大容量属于自动类型转换
```

```
17      long d = c;
18      //大容量无法直接赋值给小容量
19      //int e = d;
20      //加强制类型转换符
21      int e = (int)d;
22      //int 类型和 int 类型相除最后还是 int 类型，所以结果是 3
23      int f = 10 / 3;
24      long g = 10;
25      /*
26       * g 是 long 类型，long 类型和 int 类型最终结果是 long 类
27       * 型，无法赋值给 int 类型
28       */
29      //int h = g / 3;
30      //添加强制类型转换符
31      int h = (int)(g / 3);
32      //long 类型赋值给 long 类型
33      long m = g / 3;
34      /*
35       * g 先转换成 byte，byte 和 int 运算，最后是 int 类型，
36       * 无法直接赋值给 byte
37       */
38      //byte x = (byte)g / 3;
39      //将以上程序的优先级修改一下
40      byte x = (byte)(g / 3);
41      short y = (short)(g / 3);
42      short i = 10;
43      byte j = 5;
44      /*
45       * short 和 byte 运算时先各自转换成 int 再做运算，结
46       * 果是 int 类型，无法赋值给 short
```

```
47      */
48      //short k = i + j;
49      int n = i + j;
50      char cc = 'a';
51      System.out.println("cc = " + cc);
52      //将字符型 char 转换成数字, 'a'对应的 ASCII 是 97
53      System.out.println((byte)cc);
54      /*
55       * char 类型和 int 类型混合运算, char 类型先转换成
56       * int 再做运算, 最终 197
57       */
58      int o = cc + 100;
59      System.out.println(o);
60  }
61 }
```

运行结果如下图所示:

图 4-27: 类型转换测试

通过本小节的学习,大家主要掌握基本数据类型转换的6条规则,以后遇到类似的问题直接套用规则即可。

4.6 章节小结

通过本章节内容的学习,需要理解数据类型在程序中的作用,需要掌握在Java语言中数据类型包括哪些,其中基本数据类型又包括哪些,每个基本数据类型所占用的字节数量,byte类型取值范围等。另外需要理解字符编码在程序中的作用。还有每一种数据类型特有的语法机制,包括整数型字面值默认当做int类型处理,如果以long形式表示,需要在字面值后添加L

或1；浮点型字面量默认被当做 double 处理，后面添加 F/f 才可以被当做 float 类型；而布尔型在 Java 中只有 true 和 false 没有其他值；字符型变量完全可以存储1个汉字等。除了以上描述之外，还有相对来说难度较大的类型之间的相互转换，这个就需要大家记住相应的转换规则了。

4.7 难点解惑

本章节的难点主要还是数据类型相互转换这块，例如：`byte a = 3;`这行代码为什么可以编译通过呢？按说3是 int 类型，而 a 变量是 byte 类型，大容量转换成小容量不是应该使用强制类型转换符吗，这里没有使用，也可以编译通过。这是因为 Java 中固定的语法规则规定的，当一个整数没有超出 byte 类型取值范围时，可以直接赋值给 byte 类型的变量，其实这一设计也是为了方便程序员写代码。

另外，我们如果基于以上代码再添加这行代码：`byte b = a + 4;`这行代码为什么又编译报错了呢？按说 a 是 3，3+4 是 7，这个 7 并没有超出 byte 取值范围，为什么编译报错呢？这是因为 byte 类型的 a 和 int 类型的 4 求和，结果为 int 类型，并且对于以上代码来说 a 是一个变量，变量就是一个不确定的值，所以编译器会认为 a + 4 可能会超出 byte 取值范围，所以编译报错了。

如果我们程序是这样写的：`byte b = 3 + 4;`对于这行代码来说，编译又通过了，这是因为 3 和 4 不是变量，都是确定的值，编译器会直接检测出 3+4 等于 7，这个 7 并没有超出范围，所以编译又通过了。

4.8 章节习题

- `short s1 = 1; s1 = s1 + 1;`有什么错？
- char 类型变量能不能储存一个中文的汉字，为什么？
- `float f = 1.0` 有什么错？
- `long a = 2147483648` 有什么错？
- `int i = 0xffff` 有问题吗？
- `char c = 65536` 有问题吗，为什么？

4.9 习题答案

- `short s1 = 1; s1 = s1 + 1;`有什么错？
s1 是 short 类型，1 是 int 类型，short 和 int 混合运算的时候 short 会自动转换为 int 类型，所以 s1 + 1 编译器检测出是 int 类型，int 类型无法赋值给 short 类型的变量 s1。这样修改：`s1 = (short)(s1 + 1);`
- char 类型变量能不能储存一个中文的汉字，为什么？
java 中的文字采用 unicode 编码，一个中文占用 2 个字节，char 类型在 java 中就是占用两

个字节，所以 java 中的 char 类型完全可以容纳一个汉字。

➤ float f = 1.0 有什么错？

1.0 字面量被当做 double 类型处理，大容量 double 无法赋值给小容量 float。这样修改，两种方案：第一种方案是 1.0 后面添加 f/F。第二种方案是强制类型转换：float f = (float)1.0;

➤ long a = 2147483648 有什么错？

不是 long 类型存不下 2147483648，而是 java 把 2147483648 当做 int 类型来处理，但本身已经超出 int 类型范围。这样修改：long a = 2147483648L;

➤ int i = 0xffff 有问题吗？

没有问题：0xffff 以 0x 开始表示十六进制表示方式，ffff 转换成十进制是：65535

➤ char c = 65536 有问题吗，为什么？

65536 已经超出 char 类型取值范围，不能直接赋值，这样修改：char c = (char)65536;

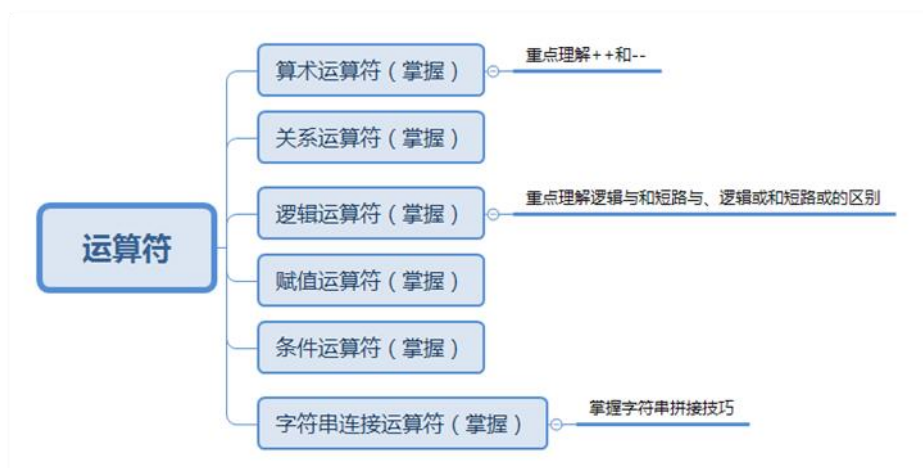
5. 第五章 运算符

5.1 章节目标与知识框架

5.1.1 章节目标

掌握常见的 Java 运算符的使用，包括算术运算符、关系运算符、逻辑运算符、赋值运算符、条件运算符、字符串连接运算符。

5.1.2 知识框架



5.2 运算符概述（了解）

运算符是指对操作数的运算方式。组成表达式的 Java 操作符有很多种（什么是操作数和操作符，例如 $1+2$ ，其中 1 和 2 都是操作数，+ 是操作符，操作符和操作数联合起来构成表达式）。运算符按照其要求的操作数数目来分，可以有单目运算符（1 个操作数）、双目运算符（2 个操作数）和三目运算符（3 个操作数）。运算符按其功能来分，有算术运算符、赋值运算符、关系运算符、逻辑运算符、位运算符、条件运算符、字符串连接运算符和其他运算符。常见的运算符如下所示：

算术运算符	$+$ 、 $-$ 、 $*$ 、 $/$ 、 $\%$ （取模）、 $++$ （自加 1【单目】）、 $--$ （自减 1【单目】）
关系运算符	$>$ 、 $>=$ 、 $<$ 、 $<=$ 、 $==$ 、 $!=$
逻辑运算符	$\&$ （逻辑与）、 $ $ （逻辑或）、 $!$ （逻辑非）、 $\&\&$ （短路与）、 $\ \ $ （短路或）
赋值运算符	$=$ 、 $+=$ 、 $-=$ 、 $*=$ 、 $/=$ 、 $\%=$ 、 $\wedge=$ 、 $\&=$ 、 $ =$ 、 $<<=$ 、 $>>=$
位运算符	$\&$ （按位与）、 $ $ （按位或）、 $\wedge$ （按位异或）、 $\sim$ （按位取反【单目】）、 $<<$ （左移）、 $>>$ （带符号右移）、 $>>>$ （无符号右移）
条件运算符	布尔表达式?表达式 1:表达式 2 （三目）
字符串连接运算符	$+$
其它运算符	<code>instanceof</code> 、 <code>new</code>

每个编程语言当中都有运算符，基本上都是通用的，这么多的运算符，它们同时出现的时候有优先级吗？答案是有的。那么如果不确定它们的优先级怎么办，其实很简单，直接加小括号就可以了，添加有小括号优先级一定是高的，所以优先级不需要死记硬背，不确定就加小括号，例如： $1 + 2 * 3$ ，想确保先求和，你就需要这样写： $(1 + 2) * 3$ 。

5.3 运算符详解

5.3.1 算术运算符（掌握）

算术运算符包括：+（两个数字求和）、-（两个数字相减）、*（两个数字乘积）、/（两个数字相除）、%（两个数字取模或者求余）、++（单目运算符，对数字自加 1）、--（单目运算符，对数字自减 1）。对于初学者来说，可能%、++、--比较生疏一些，我们重点来学习一下，请看以下代码（注意代码中的注释信息）：

```
1 public class ArithmeticOperatorTest01 {
2     public static void main(String[] args) {
3         int a = 10;
4         int b = 3;
5         System.out.println(a + b); //13
6         System.out.println(a - b); //7
7         System.out.println(a * b); //30
8         /*
9          * 在 java 语言中 10 除以 3 结果不是 3.3333..
10        * 因为 java 中有一个语法规则：int 类型和 int 类型
11        * 做运算，最终的结果还是 int 类型，所以 10 除以 3
12        * 的结果就是 3，这个过程会取整数位。
13        */
14        System.out.println(a / b); //3
15        /*
16        * 10 对 3 求余数，3 乘 3 得 9，余 1。
17        */
```

```
18      System.out.println(a % b); //1
19      /*
20          * ++和--都是单目运算符，++负责自加 1，--负责自减 1
21          * 以下程序重点研究++运算符，--运算符可以依照++运算
22          * 符进行类推。
23          */
24      int i = 10;
25      i++;
26      System.out.println("i = " + i); //11
27      int j = 10;
28      ++j;
29      System.out.println("j = " + j); //11
30      /*
31          * 根据以上的程序可以得出结论：++可以出现在变量前，也
32          * 可以出现在变量后，无论是变量前还是变量后，只要执行完
33          * ++，最终的结果都会让变量自加 1。那么++出现在变量前
34          * 和变量后到底有什么区别呢？请看以下程序
35          */
36      int k = 10;
37      int m = k++;
38      /*
39          * 根据以上结论，++无论是出现在变量前还是变量后，只要++执行
40          * 变量一定会自加 1，那么 k 变量一定会自加 1，所以下输出 k=11
41          */
42      System.out.println("k = " + k); //11
43      /*
44          * 以上 int m = k++;代码中++和=两个运算符同时存在，
45          * 那么此时这两个运算符哪个优先级高呢？java 语法中是这样
46          * 规定的，当++运算符出现在变量后，会先做赋值运算，再自
47          * 加 1,所以最后 m = 10
```

```
48      */
49      System.out.println("m = " + m); //10
50      int x = 10;
51      int y = ++x;
52      //上行代码执行结束后,我们可以确定 x 变量一定自加 1 了,所以 x=11
53      System.out.println("x = " + x); //11
54      /*
55       * java 语法中又有这样的规定, ++出现在变量前, 会先进行自加 1
56       * 的操作, 然后再做赋值运算, 所以 y = 11
57       */
58      System.out.println("y = " + y); //11
59  }
60 }
```

运行结果如下图所示:

```
13
7
30
3
1
i = 11
j = 11
k = 11
m = 10
x = 11
y = 11
```

图 5-1: 运行结果

通过以上代码的学习,我们得知,第一:两个 `int` 类型数据进行数学运算之后的结果一定是 `int` 类型;第二: `++`或`--`可以出现在变量前也可以是变量后;第三: `++`无论出现在变量前还是变量后,只要执行了`++`,那么变量必然会自加 1;第四: `++`出现在变量后会先进行赋值运算,再自加 1;第五: `++`出现在变量前会先自加 1,再进行赋值运算。那么,大家思考一下以下代

码的执行结果是什么？

1	int c = 100;
2	System.out.println(c++);
3	int d = 100;
4	System.out.println(++d);

5.3.2 关系运算符（掌握）

关系运算符主要是完成数据和数据之间的比较，比如：5>3，结果是 true（真），5>10，结果是 false（假），那么关系运算符都有哪些呢？>、>=、<、<=、==、!=。关系运算符是比较简单容易理解的，我们来看一段代码（注意代码中的注释）：

1	class RelationOperatorTest01 {
2	public static void main(String[] args) {
3	int a = 10;
4	int b = 10;
5	System.out.println(a > b);//false
6	System.out.println(a >= b);//true
7	System.out.println(a < b);//false
8	System.out.println(a <= b);//true
9	System.out.println(a == b);//true
10	System.out.println(a != b);//false
11	/*
12	* 比较两个字符串是否相等，不能使用“==”，
13	* 必须调用 equals 方法，equals 方法以后会
14	* 讲解的，现在先记住吧
15	*/
16	System.out.println("abc".equals("abc"));//true
17	}

18 }

通过以上代码可以看出，任何一个关系运算符的运算结果都是布尔类型，最后的结果不是 true 就是 false，没有其他值，并且我们也看到 a 变量和 b 变量在比较的时候是拿着变量当中保存的值进行比较。也就是说 a == b 实际上是拿着 a 变量中保存的 10 和 b 变量中保存的 10 进行比较，看它们是否相等。

5.3.3 逻辑运算符（掌握）

逻辑运算符主要包括逻辑与（&），逻辑或（|），逻辑异或（^），短路与（&&），短路或（||）。所有逻辑运算符的特点是操作数都是布尔类型，并且最终的运算结果也是布尔类型。逻辑运算符的基本运算规则如下表所示：

操作符	例子	结果	描述
&	true&true	TRUE	(5>3) & (5>4): 5 大于 3 并且 5 大于 4，有道理
	true false	TRUE	(5>3) (5>10): 5 大于 3 或者 5 大于 10，没毛病
!	!false	TRUE	false 取反是 true，true 取反就是 false
^	true^false	TRUE	异或运算符，只要两边的操作数不同，结果就是 true
&&	true&&true	TRUE	短路与（&&）和逻辑与（&）实际上最终的运行结果是完全相同的，只不过短路与（&&）会存在短路现象。
	true false	TRUE	短路或（ ）和逻辑或（ ）实际上最终的运行结果是完全相同的，只不过短路或（ ）会存在短路现象。

接下来我们来研究一段代码，重点看看短路是如何发生的（注意看注释信息）：

```
1 public class LogicalOperatorTest01 {
2     public static void main(String[] args) {
3         System.out.println(5 > 3 & 5 > 4); //true
4         System.out.println(5 > 100 | 5 > 4); //true
5         System.out.println(!(5 > 3)); //false
6         System.out.println(!(5 < 3)); //true
7         System.out.println(true & true); //true
8         System.out.println(true & false); //false
9         System.out.println(true | false); //true
10        System.out.println(!false); //true
11        System.out.println(!true); //false
12        System.out.println(true ^ false); //true
13        System.out.println(true ^ true); //false
14        System.out.println(false ^ false); //false
15        //重点研究逻辑与（&）和短路与（&&）的区别
16        int x = 100;
17        int y = 98;
18        int z = 99;
19        System.out.println(x > y & x > z); //true
20        System.out.println(x > y && x > z); //true
21        /*
22         * 通过测试得出，逻辑与（&）和短路与（&&）最终运行
23         * 结果相同那么它们有什么区别呢？请看以下代码。
24         */
25        int m = 99;
26        int n = 100;
27        System.out.println(m > n & m > n++); //false
28        /*
29         * 逻辑与（&）运算符只有当两边的操作数都是 true 的时候，
```

```
30      * 结果才是 true，只要有一个是 false，结果必然是 false。
31      * 以上程序逻辑左边的表达式(m > n)显然结果是 false，
32      * 通过左边的表达式已经可以得出结果是 false 了，右边的
33      * 表达式没有执行的必要，那我们来看一下右边的表达式
34      * 有没有执行，只要输出 n 就可以了断定。
35      */
36      System.out.println("n = " + n); //101
37      /*
38      * 以上程序的运行结果是 101，足以说明 n++已经执行了。
39      * 也说明了逻辑与运算符无论左边表达式结果是 true 还
40      * 是 false，右边表达式一定会执行。
41      */
42      //那我们来看一下短路与（&&）是怎样的？
43      int k = 99;
44      int f = 100;
45      System.out.println(k > f && k > f++); //false
46      System.out.println("f = " + f); //100
47      /*
48      * 通过以上测试得出 f++并没有执行，因为左边表达式(k > f)
49      * 为 false，右边的表达式就不再执行了，这种现象被称为短路
50      * 现象。也就是说对于短路与来说，左边的表达式只要为 false，
51      * 就会发生短路，右边表达式不再执行了。
52      */
53      }
54 }
```

通过以上的测试，可以得出短路与（&&）在左边的表达式结果为 false 的时候，右边的表达式则不再执行，这种现象被称为短路现象，这种机制也显得短路与比较智能一些，效率更高一些，所以在实际开发中短路与（&&）的使用率要比逻辑与高一些。但这并不是绝对的，有的时候也可能会选择使用逻辑与（&），这取决于你是否期望右边的表达式一定执行。

大家思考一个问题，短路或（||）在什么情况下会发生短路现象呢？

5.3.4 赋值运算符（掌握）

赋值运算符目前也是只需要掌握=、+=、-=、*=、/=、%=，其它和二进制相关的内容也是到后面遇到的时候再详细学习。赋值类的运算符包括基本赋值运算符（=）和扩展的赋值运算符（+=、-=、*=、/=、%=）。我们来看一段代码：

```
1 public class AssignmentOperatorTest01 {
2     public static void main(String[] args) {
3         //基本的赋值运算符
4         int i;
5         i = 10;
6         System.out.println("i = " + i);
7         i = 100;
8         System.out.println("i = " + i);
9         //扩展的赋值运算符
10        int x = 10;
11        x += 1; //等同于 x = x + 1
12        System.out.println("x = " + x); //11
13        int y = 10;
14        y -= 1; //等同于 y = y - 1
15        System.out.println("y = " + y); //9
16        int z = 10;
17        z *= 2; //等同于 z = z * 2
18        System.out.println("z = " + z); //20
19        int m = 10;
20        m /= 3; //等同于 m = m / 3
21        System.out.println("m = " + m); //3
22        int n = 10;
23        n %= 3; //等同于 n = n % 3
```

24	System.out.println("n = " + n); //1
25	}
26	}

$x += 1$ 和 $x = x + 1$ 真的是完全相同吗？我们来看下面的代码：

1	class AssignmentOperatorTest02 {
2	public static void main(String[] args) {
3	byte b = 10;
4	//以下程序编译报错，编译器提示错误信息为：
5	//Type mismatch: cannot convert from int to byte
6	/*
7	* 编译没有通过的原因：b 是 byte 类型，1 是 int 类型，根据之前
8	* 讲解的类型转换规则得知，byte 和 int 混合运算最后结果是
9	* int 类型，int 类型的值无法直接赋值给 byte 类型的变量 b，
10	* 所以编译报错。
11	*/
12	//b = b + 1;
13	b += 1; //编译通过并可以正常运行
14	System.out.println("b = " + b); //11
15	//通过以上的测试得出：b = b + 1 和 b += 1 是不一样的
16	//那么 b += 1 等同于什么呢？
17	/*
18	* 实际上 java 对于扩展类的赋值运算符进行了特殊的处理，
19	* 所有的扩展赋值运算符，最终都不会改变运算的结果类
20	* 型，假设前面的变量是 byte 类型，那么后面的表达式运
21	* 算之后的结果还是 byte 类型。所以实际上 b += 1 等同于：
22	*/
23	b = (byte)(b + 1);
24	System.out.println("b = " + b); //12

```
25      b += 1000; //编译通过，并且可以正常运行
26      /*
27      * 以上代码实际上等同于：b = (byte)(b + 1000);
28      * 分析得出，显然结果已经超出了 byte 类型取值范围，
29      * 所以精度一定会损失，最终的结果需要对计算机
30      * 二进制的原码反码补码进行研究。
31      */
32      System.out.println("b = " + b); // -12
33  }
34 }
```

根据以上代码测试得出，对于扩展类的赋值运算符在运算的过程中不会改变运算的结果类型，也就是说 `byte b = 100; b += 1000;` `b` 变量最初是 `byte` 类型，最后的运算结果还是一个 `byte` 类型。这是一条固定的语法规则，大家记住就行了，以后在使用扩展类赋值运算符的时候要谨慎，不小心就会精度损失的。

5.3.5 条件运算符（掌握）

条件运算符属于三目运算符，它的语法结构是：布尔表达式?表达式1:表达式2。它的运行原理是这样的，先判断布尔表达式的结果是 `true` 还是 `false`，如果是 `true`，则选择表达式1的结果作为整个表达式的结果，反之则选择表达式2的结果作为整个表达式的结果。来看一段代码：

```
1  public class ConditionalOperatorTest01 {
2      public static void main(String[] args) {
3          //编译报错：这不是一个语句
4          //10;
5          boolean flag = true;
6          //编译报错：这不是一个语句
7          //flag ? 1 : 0;
8          //以上如果是一条完整的语句应该这样写
9          int k = flag ? 1 : 0;
```

```
10      System.out.println("k = " + k); //1
11      //三目运算符最经典的用法
12      boolean sex = true;
13      //当布尔变量 sex 为 true 则结果是'男'，反之'女'
14      char gender = sex ? '男' : '女';
15      System.out.println("性别: " + gender); //男
16      sex = false;
17      gender = sex ? '男' : '女';
18      System.out.println("性别: " + gender); //女
19      //又如
20      int x = 100;
21      int y = 100;
22      System.out.println(x==y?"x 和 y 相等":"x 和 y 不相等");
23  }
24 }
```

实际上条件运算符和后期要学习的控制语句 if 可以达到同等效果。在实际开发中灵活运用条件运算符会让你的代码看起来更加简洁清凉，达到意想不到的效果。

5.3.6 字符串连接运算符（掌握）

在 java 语言中所有的字符串都使用半角双引号括起来的，字符串属于引用数据类型，不属于基本数据类型的范畴，怎么定义一个字符串的变量呢？例如：`String name = "jack"`；这就类似于 `int i = 10`；是一样的，`int` 是一种整数类型，`i` 是变量，`10` 是整数型字面量。那么 `String` 则是一种字符串类型，`name` 是变量，`"jack"` 是字符串型字面量。在 java 编程中对字符串的操作是非常频繁的，例如字符串的连接操作，此时就需要使用 “+” 字符串连接运算符了。

实际上 “+” 运算符在 java 语言中有两个作用，作用一是对数字进行求和运算，作用二就是字符串连接运算，那么它在什么时候是进行求和，什么时候又进行字符串连接呢？大家可以这样进行区分，当 “+” 运算的时候，两边的操作数都是数字的话，一定会进行求和运算，只要其中有一个操作数是字符串类型，那么一定会进行字符串拼接运算，字符串拼接之后的结果还是字符串类型。需要注意的是，当一个表达式当中有多个 “+”，并且在没有小括号的前提下，遵循自左向右的顺序依次执行。我们来看一段程序：

```
1 class PlusTest01 {
2     public static void main(String[] args) {
3         int a = 10;
4         int b = 20;
5         //加号两边都是 int 类型，执行求和运算
6         int c = a + b;
7         /*
8          * 以下代码虽然有两个加号，第一个加号在双引号当中，
9          * 不起任何作用，只是一个普通字符串，第二个加号由于
10         * 左边操作数是一个字符串，所以这个加号一定会进行
11         * 字符串连接，连接之后的结果还是一个字符串。
12         */
13         System.out.println("a + b = " + c); //a + b = 30
14         /*
15          * 分析以下程序的输出结果？哪些加号进行求和，
16          * 哪些加号进行字符串连接
17          */
18         System.out.println(a + " + " + b + " = " + a + b);
19         /*
20          * 分析以下程序的输出结果？哪些加号进行求和，
21          * 哪些加号进行字符串连接
22          */
23         System.out.println(a + " + " + b + " = " + (a + b));
24         //定义一个字符串类型变量
25         String name = "jack";
26         /*
27          * 字符串拼接在实际开发中很常用，拼接的时候是有口诀，
28          * 大家记住，将变量放到字符串当中进行拼接，可以这样
29          * 做：1、在拼接的位置添加一个英文双引号；2、在双引
```

```

30      * 号中间添加两个加号；3、把字符串变量放到两个加号
31      * 中间。
32      */
33      System.out.println("登录成功，欢迎"+ name + "回来！");
34  }
35 }

```

对于程序 `System.out.println(a + " + " + b + " = " + a + b);` 的分析见下图：

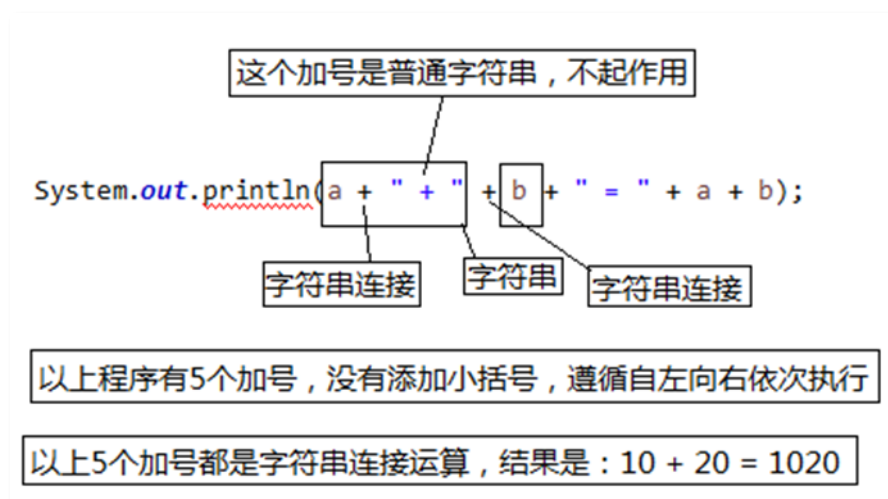


图 5-2：字符串连接运算

对于程序 `System.out.println(a + " + " + b + " = " + (a + b));` 的分析见下图：

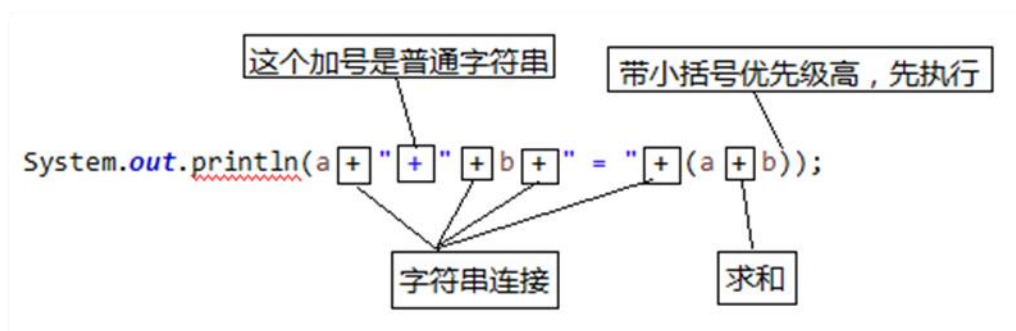


图 5-3：字符串连接运算

总之，使用“+”进行字符串拼接在开发中使用太频繁了，大家一定要将其掌握，尤其是如何将一个变量放到字符串当中，你还记得以上程序的口诀吗？

除了以上所讲的运算符之外，实际上还有其它运算符，例如位运算符主要操作二进制位的，还有运算符 `instanceof`、`new` 等。这里就不再赘述了，当前使用较少，后期使用到的时候我们再看。

5.4 章节小结

本章节内容需要大家掌握开发中常用运算符的使用，包括算术运算符、关系运算符、逻辑运算符、赋值运算符、条件运算符、字符串连接运算符。实际上除了以上所讲运算符之外，还有其它运算符，比如位运算符、`instanceof`、`new` 等运算符，在当下使用较少，后期用到的时候再进行学习。

本章节内容具有通用性，不仅仅是在 Java 中可以使用，在其他的编程语言中也可以使用。运算符在实际的开发中使用频率很高，因为软件存在的目的就是为了解决数据的处理。在这里想和大家说的是，学好每一个运算符的用法，为后面的学习打下扎实的基础。

5.5 难点解惑

本章节的难点主要还是++运算符的使用，例如以下代码：

1	<code>public class Test{</code>
2	<code> public static void main(String[] args){</code>
3	<code> int k = 10;</code>
4	<code> System.out.println(k++);</code>
5	<code> System.out.println(++k);</code>
6	<code> }</code>
7	<code>}</code>

我们对以上程序进行分析，`k` 变量等于 10，输出 `k++`，我们可以将 `System.out.println(k++)` 这行代码进行拆解，可以拆解为两行代码：`int temp = k++; System.out.println(temp);`，也就是说实际上输出 `k++` 就是输出 `temp`，根据之前所学规则，`temp` 等于 10，所以 `System.out.println(k++)` 会输出 10。不过当这行代码执行结束之后 `k` 就不再是 10 了，变成了 11。

我们继续再向下分析：System.out.println(++k)。这行代码同样也可以拆分为两行代码：int temp2 = ++k; System.out.println(temp2);。其中 k 已经等于 11 了，根据以上所学规则 temp2 的值是 12，所以最终输出 12。

我们对以上的程序进行编译并运行，来看一下我们分析是否正确，请看下图：

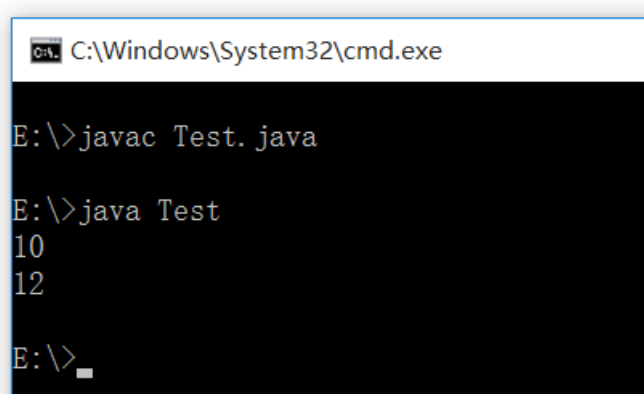


图 5-4：Test 程序的编译和运行结果

通过以上程序的运行我们可以看出以上的分析是没有问题的，所以对于初学者来说，这块内容确实是一个不太好理解的地方。

5.6 章节习题

第一题：判断以下程序的输出结果。

1	int x = 10;
2	int a = x+ x++;
3	System.out.println("a =" + a);
4	System.out.println("x =" + x);
5	int b = x + ++x;
6	System.out.println("b =" + b);
7	System.out.println("x =" + x);
8	int c = x + x--;
9	System.out.println("c =" + c);
10	System.out.println("x =" + x);
11	int d = x + --x;

12	System.out.println("d =" + d);
13	System.out.println("x =" + x);

第二题：判断以下程序的输出结果。

1	int a = 15;
2	int b = 2;
3	double c = 2;
4	System.out.println(a + "/" + b + "=" + (a / b));
5	System.out.println(a + "%" + b + "=" + (a % b));
6	System.out.println(a + "/" + c + "=" + (a / c));
7	System.out.println(a + "%" + c + "=" + (a % c));

第三题：判断以下程序的输出结果。

1	boolean x, y, z;
2	int a = 15;
3	int b = 2;
4	x = a > b; // true;
5	y = a < b; // false;
6	z = a != b; // true;
7	System.out.println("x =" + x);
8	System.out.println("y =" + y);
9	System.out.println("z =" + z);

第四题：判断以下程序的输出结果。

1	int x;
2	double y;

3	x = (int) 22.5 + (int) 34.7;
4	y = (double) x;
5	System.out.println("x = " + x);
6	System.out.println("y = " + y);

第五题：判断以下程序的输出结果。

1	int i = 5;
2	int j = 5;
3	int m = 5;
4	int n = 5;
5	i++;
6	j = j + 1;
7	m--;
8	n = n - 1;
9	System.out.println(i);
10	System.out.println(i++);
11	System.out.println(++i);
12	System.out.println(i--);
13	System.out.println();
14	System.out.println(j);
15	System.out.println(j++);
16	System.out.println(j--);
17	System.out.println(--j);
18	System.out.println();
19	System.out.println(m);
20	System.out.println(n);

第六题：判断以下程序的输出结果。

1	int i = 0;
2	int j = 0;
3	System.out.println(i);
4	System.out.println(j);
5	i++;
6	++j;
7	System.out.println(i);
8	System.out.println(j);
9	System.out.println("-----");
10	System.out.println(i++);
11	System.out.println(++j);
12	System.out.println("-----");
13	System.out.println(i);
14	System.out.println(j);

5.7 习题答案

第一题答案：

```
a =20
x =11
b =23
x =12
c =24
x =11
d =21
x =10
```

第二题答案：

```
15/2=7
15%2=1
15/2.0=7.5
15%2.0=1.0
```

第三题答案：

```
x =true  
y =false  
z =true
```

第四题答案:

```
x = 56  
y = 56.0
```

第五题答案:

```
6  
6  
8  
8  
  
6  
6  
7  
5  
  
4  
4
```

第六题答案:

```
0  
0  
1  
1  
-----  
1  
2  
-----  
2  
2
```

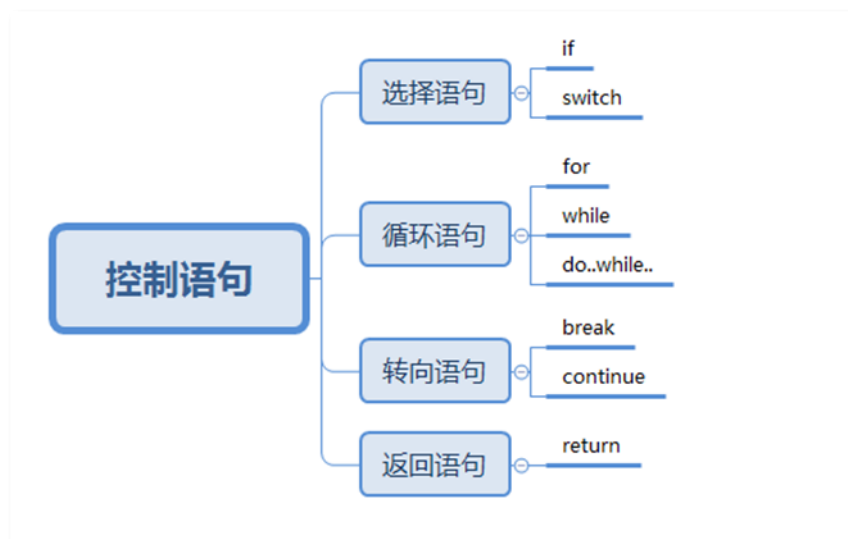
6. 第六章 控制语句

6.1 章节目标与知识框架

6.1.1 章节目标

记住所有控制语句的语法格式，理解不同的控制语句分别在哪些不同的情况下使用，并且能够熟练编写这些控制语句。

6.1.2 知识框架



6.2 控制语句概述（理解）

在大部分编程语言当中都会存在控制语句，控制语句是一个程序的灵魂，我们只依靠“标识符、关键字、变量、运算符”等零散的知识点是无法进行流程控制的，无法实现一个具体的功能或业务，所以控制语句这一章节非常重要。

现实生活中哪些业务需要使用控制语句呢？比如说“小孩身高如果高于 1.2 米，则乘坐交通工具就需要收费了，反之则免费”，再比如“A 账户向 B 账户转账 10000 元，首先需要判断 A 账户的余额是否大于等于 10000 元，如果余额充足则可以转账，不足则无法转账”等等，这些业务当中都需要使用控制语句进行控制才能完成。

什么是控制语句，官方的解释是这样的：控制语句即用来实现对程序流程的选择、循环、转向和返回等进行控制。Java 语言中共有 8 种控制语句，可以分为“选择语句”，“循环语句”，“转向语句”，“返回语句”四类。那么，这 8 种控制语句分别是什么呢？if 语句、switch 语句、for 循环、while 循环、do..while 循环、break 语句、continue 语句、return 语句。其中 if 和 switch 语句属于选择语句，for、while、do..while 语句属于循环语句，break 和 continue 语句属于转向语句，return 属于返回语句。接下来让我们认真的学习每一个语句。

6.3 选择语句

选择语句又称为分支语句，它通过对给定的条件进行判断，从而决定执行两个或多个分支中的哪一支。因此，在编写选择语句之前，应该首先明确判断条件是什么，并确定当判断结果为“真”或“假”时应分别执行什么样的操作/算法。在 Java 语言中选择语句主要提供了两个，一个是 if 语句，另一个则是 switch 语句。

6.3.1 if（掌握）

我们先来看 if 语句，if 语句的编写方式归纳为以下四种，请看下图：

<pre>if(布尔表达式){ java语句; }</pre>	第一种
<pre>if(布尔表达式){ java语句; }else{ java语句; }</pre>	第二种
<pre>if(布尔表达式){ java语句; }else if(布尔表达式){ java语句; }else if(布尔表达式){ java语句; }else if(布尔表达式){ java语句; }...</pre>	第三种
<pre>if(布尔表达式){ java语句; }else if(布尔表达式){ java语句; }else if(布尔表达式){ java语句; }else if(布尔表达式){ java语句; }else{ java语句; }</pre>	第四种

图 6-1: if 语句语法格式

if 语句是非常简单容易理解的，if 被翻译为如果，else 被翻译为其它，我们针对以上第二种写法进行说明：如果布尔表达式的结果是“真”，则执行它后面的分支，反之则执行 else 对应的分支。好比说“如果外面下雨了（真），出门则拿一把雨伞，反之则不带雨伞”。再比如“如果你吃饱了（真）就不需要再吃了，反之则需要继续吃东西”。那么以上两个业务用代码应该如何实现呢？请看下面程序：

1	boolean raining = false;
2	if(raining){
3	System.out.println("外面下雨了，出门记得带一把伞哦！");
4	}else{
5	System.out.println("天气不错哦！");
6	}

```
7 boolean eatFull = true;
8 if(eatFull){
9     System.out.println("吃的好饱啊！");
10 }else{
11     System.out.println("还没有吃饱，再来份米饭！");
12 }
```

上图中第一种写法对应的原理图：

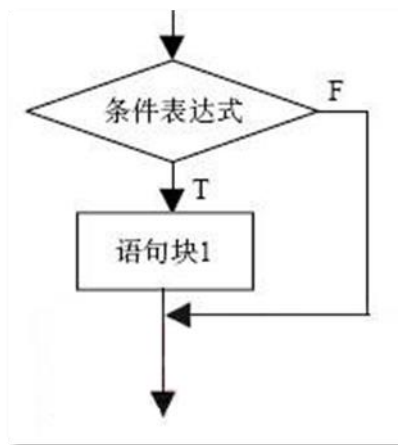


图 6-2： 6-1 图中第一种写法对应的原理图

上图中第二种写法对应的原理图：

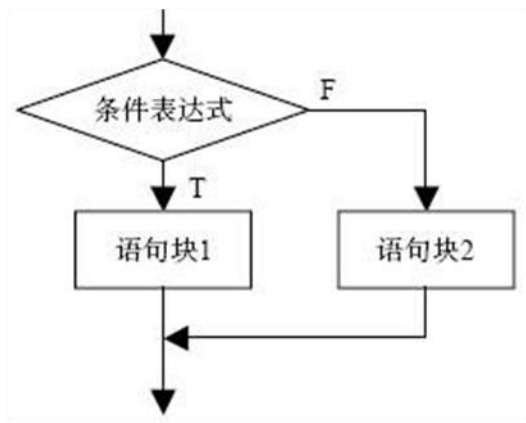


图 6-3: 6-1 图中第二种写法对应的原理图

上图中第三种写法对应的原理图:

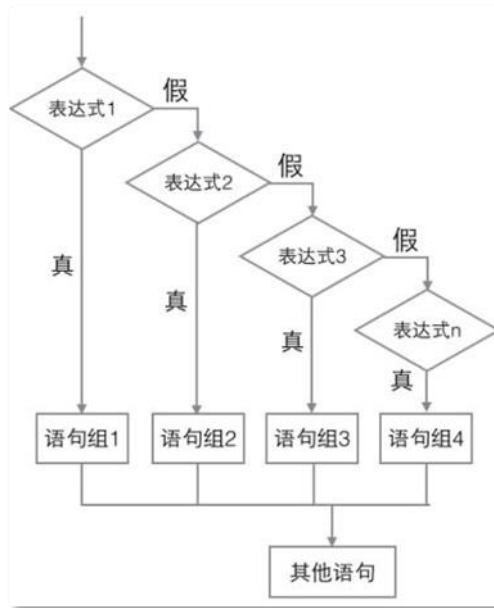


图 6-4: 6-1 图中第三种写法对应的原理图

上图中第四种写法对应的原理图:

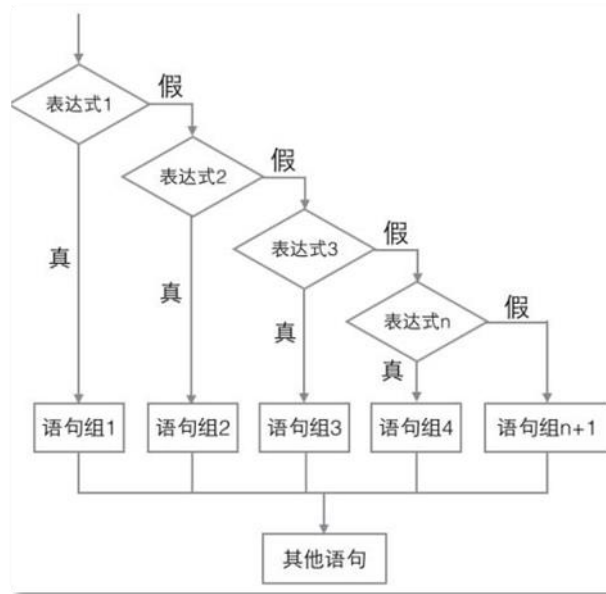


图 6-5: 6-1 图中第四种写法对应的原理图

通过以上原理的分析，我们可以得出 if 语句的任何一种形式在执行的时候只要有一个分支执行，则所对应的整个 if 语句就全部结束了，换句话说，对于一个独立的完整的 if 语句来说，最多只能有一个分支执行。

另外，我们可以得出“图 6-1”中的四种写法，其中第二种和第四种都带有 else 分支，所以这两种写法我们可以确保肯定会有一个分支执行，另外两种写法则无法保证，也就是说第一种和第三种这两种写法可能一个分支都不会执行，这两种写法的分支是否执行取决于条件是否为 true。

还有，Java 语言中有这样一条规定：当分支中只有一条 java 语句话，大括号可以省略不写，如以下程序：

1	boolean sex = true;
2	if(sex){
3	System.out.println("男");
4	}else{
5	System.out.println("女");
6	}
7	//也可以这样写
8	if(sex)

9	System.out.println("男");
10	else
11	System.out.println("女");

虽然大括号可以省略，但是大家编写程序的时候一定要小心，分析以下程序错在哪里？

1	if(sex)
2	System.out.println("男");
3	System.out.println("hello world!");
4	else
5	System.out.println("女");

显然 if 是没有任何问题的，输出“男”也是没问题的，输出“hello world!”也是正常的，只不过负责输出“hello world!”的那一行代码不在 if 语句的分支当中，那么下面的 else 则缺少了 if 语句而编译报错。换句话说，else 以上的代码都没有问题，错误出现在 else 缺少了 if。

虽然 java 规定当分支中只有一条 java 语句的话大括号可以省略，但是为了程序具有很强的可读性，所以建议在实际开发中还是不要省略大括号会比较好。

那么，接下来我们给大家列举几个 if 语句的案例，帮助大家更好的理解和运用 if 语句。

示例代码 1：业务背景：根据用户输入不同的数字来判断用户不同性别，当输入 1 时性别为男，当输入 0 时性别为女，输入其它则不合法。

1	java.util.Scanner scan = new java.util.Scanner(System.in);
2	System.out.print("请输入数字：");
3	int i = scan.nextInt();
4	if(i == 1){
5	System.out.println("性别：男");
6	}else if(i == 0){
7	System.out.println("性别：女");
8	}else{
9	System.out.println("对不起，您输入不合法！");

10	}
----	---

示例代码 2：业务背景：键盘接收一个学生的成绩，范围在[0-100]，根据成绩判断等级，[90-100]等级为“优秀”，[80-90)等级为“良好”，[70-80)等级为“中等”，[60-70)等级为“及格”，[0-60)等级为“大笨蛋”，输入其它数字表示成绩不合法。

1	java.util.Scanner scan = new java.util.Scanner(System.in);
2	System.out.print("输入考试成绩: ");
3	int score = scan.nextInt();
4	if(score < 0 score > 100){
5	System.out.println("对不起，您输入的成绩不合法！");
6	}else if(score >= 90 && score <= 100){
7	System.out.println("成绩等级: 优秀");
8	}else if(score < 90 && score >= 80){
9	System.out.println("成绩等级: 良好");
10	}else if(score < 80 && score >= 70){
11	System.out.println("成绩等级: 中等");
12	}else if(score < 70 && score >= 60){
13	System.out.println("成绩等级: 及格");
14	}else if(score < 60 && score >= 0){
15	System.out.println("成绩等级: 大笨蛋");
16	}

对于初学者来说以上程序是可以的，但是对于老程序员来说，以上的代码就太冗余了，我们将其进行优化，看以下代码：

1	java.util.Scanner scan = new java.util.Scanner(System.in);
2	System.out.print("输入考试成绩: ");
3	int score = scan.nextInt();
4	String grade = "大笨蛋";

```
5  if(score < 0 || score > 100){
6      grade = "对不起，您输入的成绩不合法！";
7  }else if(score >= 90){
8      grade = "优秀";
9  }else if(score >= 80){
10     grade = "良好";
11 }else if(score >= 70){
12     grade = "中等";
13 }else if(score >= 60){
14     grade = "及格";
15 }
16 System.out.println("成绩等级： "+ grade);
```

通过以上程序我们可以了解到一个功能的实现代码有多种编写形式，大家在学习的时候千万别机械性的一行一行对抄代码，实际上这种学习方法是非常痛苦，并且低效的，软件开发主要培养的是编程思想/编程思路，使用编程的思路解决现实生活中的问题。当然，对于初学者来说我们不可能刚开始就写出简单而高效的程序，这需要不断的模仿，不断的积累，尤其我们可以多参考高手写的程序（例如：JDK 源代码），当你的代码达到一定量的时候，你就可以出山了。

示例代码3：业务背景：系统首先接收天气参数，1 表示下雨天，0 表示晴天，如果天气是晴天，输出“走起，一起出去玩耍喽！”，当天气是下雨天，则继续接收性别参数，1 表示男，0 表示女，当性别为女，则拿一把小花伞出去玩，当性别为男，则拿一把大黑伞出去玩。

```
1  java.util.Scanner scann = new java.util.Scanner(System.in);
2  System.out.print("请输入天气指数，1 表示下雨天，0 表示晴天：");
3  int weather = scann.nextInt();
4  if(weather == 1){
5      System.out.print("请再输入您的性别，1 表示男，0 表示女：");
6      int sex = scann.nextInt();
7      if(sex == 0){
8          System.out.println("女士，外面下雨了，您需要带一把小花伞");
```

```
9      }else if(sex == 1){
10          System.out.println("哥们，外面下雨了，您需要带一把大黑伞");
11      }else{
12          System.out.println("对不起，您输入性别参数出错了！");
13      }
14  }else if(weather == 0){
15      System.out.println("走起，一起出去玩耍喽！");
16  }else{
17      System.out.println("对不起，您输入天气参数出错了！");
18  }
```

运行结果如下图所示：

```
请输入天气指数，1表示下雨天，0表示晴天：1
请再输入您的性别，1表示男，0表示女：0
女士，外面下雨了，您需要带一把小花伞
```

```
请输入天气指数，1表示下雨天，0表示晴天：1
请再输入您的性别，1表示男，0表示女：1
哥们，外面下雨了，您需要带一把大黑伞
```

图 6-6: if 语句嵌套演示

通过以上的程序我们得知，if 语句是可以进行嵌套使用的，也就是说 if 语句的某个分支当中还可以继续编写其它 if 语句。实际上所有的控制语句都是可以进行嵌套使用的。语句嵌套这个知识点本来是不需要拿来进行特殊强调的，专门强调之后反而会让初学者特殊对待，我希望大家不要特殊对待嵌套。虽然被嵌套的是一个控制语句，但这个控制语句也是一段普通的 java 程序，你说呢！如下图：

```

if(weather == 1){
    System.out.print("请再输入您的性别, 1表示男, 0表示女: ");
    int sex = scann.nextInt();
    if(sex == 0){
        System.out.println("女士, 外面下雨了, 您需要带一把小花伞");
    }else if(sex == 1){
        System.out.println("哥们, 外面下雨了, 您需要带一把大黑伞");
    }else{
        System.out.println("对不起, 您输入性别参数出错了! ");
    }
}
}else if(weather == 0){
    System.out.println("走起, 一起出去玩耍喽! ");
}
}else{
    System.out.println("对不起, 您输入天气参数出错了! ");
}
}

```

把这部分代码看做普通的java语句, 不要把它当做控制语句, 不要把它特殊化。

图 6-7: 控制语句嵌套

总之, 选择语句之 if 语句, 在实际开发中使用频率非常高, 大家一定要掌握 if 语句的编写方式以及运行原理, 也可以自己想一些现实生活中的小例子加以练习。

6.3.2 switch（掌握）

switch 语句和 if 语句一样, 都属于选择语句（分支语句）, 不再赘述, 我们直接来看一下一个比较完整的 switch 语句结构是怎样的, 请看下图:

```

switch(int/String类型数据){
case int/String类型数据:
    java语句;...
    break;
case int/String类型数据:
    java语句;...
    break;
case int/String类型数据:
    java语句;...
    break;
default:
    java语句;...
}

```

图 6-8: switch 语句完整结构

switch 运行原理是怎样的呢，请看下图：

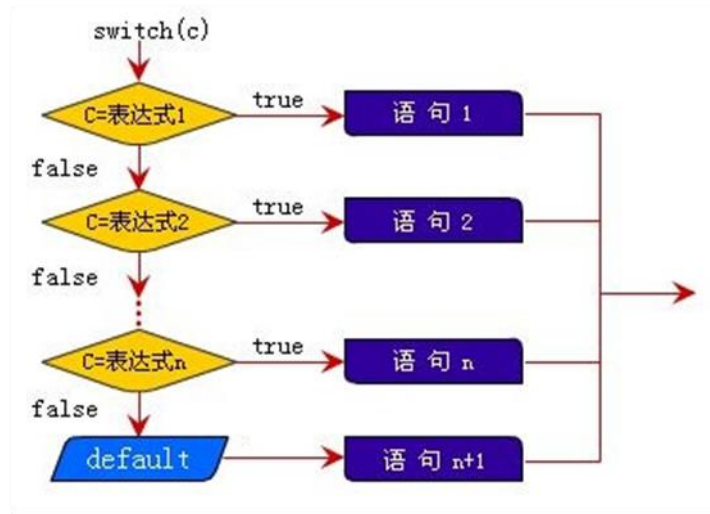


图 6-9: switch 语句运行原理图

switch 运行原理文字性描述是这样的（以上图为例）：拿着 c 和表达式 1 进行比较，如果相等，则执行语句 1，如果不相等，继续拿着 c 和表达式 2 进行比较，如果相等，则执行语句 2，如果不相等，继续...，如果 c 和所有的分支表达式都不相等，此时 default 分支“语句 n+1”则执行。

以上描述是 switch 语句的一般性执行流程，实际上在执行过程中还有很多因素的影响，接下来我将列出使用 switch 语句的一些注意事项：

① switch 语句后面的小括号“()”当中都可以出现什么，在 JDK1.6 之前（包括 1.6 在内）switch 只能探测 int 类型的数据，但是在 JDK1.7 之后（包括 1.7 在内），switch 语句除了支持 int 类型之外，还支持 String 类型。也就是说从 Java7 开始 switch 后面小括号中可以出现字符串。

② switch 虽然只能探测 int 类型，但是也可以将 byte,short,char 类型放到小括号当中，因为这些类型会自动转换成 int 类型（小容量向大容量转换称为自动类型转换）。

③ switch 语句当中 case 是可以进行合并的，例如以下代码：

1	switch(90){
2	case 100 : case 90 : case 80 : case 70 :


```
3      System.out.println("过关了");
4      break;
5  case 60:
6      System.out.println("失败了");
7      break;
8  default :
9      System.out.println("出错了");
10 }
```

④ switch 语句当中当某个分支匹配成功，则开始执行此分支当中的 java 语句，当遇到当前分支中的“break;”语句，switch 语句就结束了，但是如果当前分支中没有“break;”语句，则会发生 case 穿透现象，也就是说下一个分支也不再进行匹配，直接进入下一个分支执行，直到遇到“break;”为止。例如以下代码：

```
1  char c = 'A';
2  switch(c){
3  case 'A' :
4      System.out.println(1);
5  case 'B' :
6      System.out.println(2);
7      break;
8  default :
9      System.out.println(3);
10 }
```

执行结果如下所示：



```
1
2
```

图 6-10: switch 的 case 穿透测试

接下来我们来看几个 switch 的经典案例，请看代码：

示例代码 1：从键盘接收一个数字[1-7]，当数字是 1~5 的时候，控制台输出工作日，当数字是 6~7 的时候，控制台输出休息日，其他数字提示错误信息。

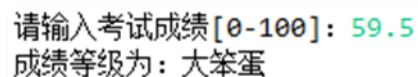
```
1 java.util.Scanner scan = new java.util.Scanner(System.in);
2 System.out.print("请输入[1-7]的整数数字: ");
3 int dayOfWeek = scan.nextInt();
4 switch(dayOfWeek){
5     case 1: case 2: case 3: case 4: case 5:
6         System.out.println("今天是工作日哦! ");
7         break;
8     case 6: case 7:
9         System.out.println("今天是休息日哦! ");
10        break;
11    default :
12        System.out.println("对不起，您的输入有误");
13 }
```

示例代码 2：业务背景：键盘接收一个学生的成绩（假设成绩是合法的），范围在[0-100]，根据成绩判断等级，[90-100]等级为“优秀”，[80-90)等级为“良好”，[70-80)等级为“中等”，[60-70)等级为“及格”，[0-60)等级为“大笨蛋”。只允许使用 switch，不允许使用 if 语句。

```
1 java.util.Scanner scan = new java.util.Scanner(System.in);
2 System.out.print("请输入考试成绩[0-100]: ");
3 double score = scan.nextDouble();
4 String grade = "大笨蛋";
5 switch((int)(score / 10)){
6     case 10: case 9:
7         grade = "优秀";
```

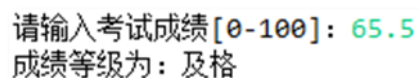
8	break;
9	case 8:
10	grade = "良好";
11	break;
12	case 7:
13	grade = "中等";
14	break;
15	case 6:
16	grade = "及格";
17	break;
18	}
19	System.out.println("成绩等级为: "+ grade);

运行结果如下图所示:



```
请输入考试成绩[0-100]: 59.5
成绩等级为: 大笨蛋
```

图 6-11: switch 判断成绩等级



```
请输入考试成绩[0-100]: 65.5
成绩等级为: 及格
```

图 6-12: switch 判断成绩等级

以上程序在编写时候的核心技巧是: [90-100]分的所有数字, 包括浮点型数字, 让其除以 10 之后强转为 int 类型, 结果一定是 9 或者 10。[80-90)分的所有数字, 除以 10 之后取整, 结果一定是 8, 以此类推。没关系同学们, 大家在起初学习的时候不一定会马上写出来以上程序, 这个过程是需要积累的, 下去之后先自己模仿我的程序, 慢慢的你就会写了。

if 和 switch 都是条件判断语句(分支语句/选择语句), 在效果上是没有什么差别的。而不同, 我们可以从效率上来看一看, 举一个很简单的例子: if(a > b) { a = b; } else { b = a; } 如果将这个例子放在 switch 语句中, 也并不是不能实现, 但是实现起来可能会比较麻烦一些, 因为在

switch 语句中 case 要求的是常量，一般是不能进行逻辑判断的，所以这也是 if 语句优于 switch 语句的地方。但是如果您判断的都是几个常量的数据，我建议您最好采用 switch 语句，虽然 if 语句也是能实现的，但是性能就比 switch 差的有点远了。

6.4 循环语句

在不少实际问题中有许多具有规律性的重复操作，因此在程序中就需要重复执行某些语句。一组被重复执行的语句称之为循环体，能否继续重复，取决于循环的终止条件。循环结构是在一定条件下反复执行某段程序的流程结构，被反复执行的程序被称为循环体。循环语句是由循环体及循环的终止条件两部分组成的。

为了帮助大家理解循环语句存在的意义，我们来看一段代码：

1	System.out.println("hello world!");
2	System.out.println("hello world!");
3	System.out.println("hello world!");
4	System.out.println("hello world!");
5	System.out.println("hello world!");
6	System.out.println("hello world!");
7	System.out.println("hello world!");
8	System.out.println("hello world!");
9	System.out.println("hello world!");
10	System.out.println("hello world!");
11	//...
12	//...

以上程序的业务背景是：输出 100 行 “hello world!”，如果我们像以上方式进行代码编写的话，代码将无法得到重复使用，大家也可以看到 “System.out.println(“hello world!”)” 这行代码重复出现，直到输出 100 个为止。显然程序应该找一种更好的实现方式，这个时候我们就可以借助 java 语言中的循环语句来实现了。java 中的循环语句共包括三个，分别是：for 循环、while 循环、do..while 循环。

6.4.1 for（掌握）

接下来我们先来学习 for 循环。for 循环的语法格式如下图所示：

```
for(初始化表达式; 布尔表达式; 更新表达式){  
    循环体;  
}
```

图 6-13: for 循环语法结构

对以上 for 循环语法结构进行说明：初始化表达式最先执行，并且在整个循环当中初始化表达式只执行一次，布尔表达式结果是 true 或者 false，true 表示循环可以继续执行，false 表示循环结束，循环体当中的代码就是需要反复执行的代码，更新表达式一般是用来更新某个变量的值，更新操作之后可能布尔表达式的结果就不再是 true 了。那么 for 循环的执行顺序以及原理如下图所示：

```
1      2      4  
for(初始化表达式; 布尔表达式; 更新表达式){  
    循环体; 3  
}
```

执行顺序：

1 - 2(true) - 3 - 4 ,
2(true) - 3 - 4 ,
2(true) - 3 - 4 ,
2(true) - 3 - 4 ,
2(true) - 3 - 4 ,
... ..
2(true) - 3 - 4 ,
2(false) 【结束】

图 6-14: for 循环语句执行顺序及原理

对于 for 循环来说，初始化表达式、布尔表达式、更新表达式都不是必须的，当布尔表达式缺失的时候，没有条件控制的前提下，会形成死循环，我们来看一下最简单的 for 循环形式：

1	for(;;){
2	System.out.print("死循环、");
3	}

执行结果如下图所示：

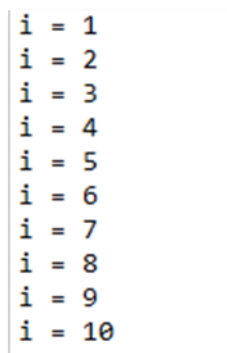
死循环、死循环、死循环、死循环、死循环、死循环、死循环、死循环、死循

图 6-15：死循环输出结果

最常见的 for 循环是怎样的呢？看以下代码：

1	for(int i = 1; i <= 10; i++){
2	System.out.println("i = " + i);
3	}

运行结果如下图所示：



```
i = 1
i = 2
i = 3
i = 4
i = 5
i = 6
i = 7
i = 8
i = 9
i = 10
```

图 6-16：for 循环输出 1~10

对以上代码进行分析：首先执行 `int i = 1`，并且只执行一次，定义初始化变量 `i`，赋值 `1`，然后判断 `i <= 10` 结果为 `true`，则执行循环体打印 `i = 1`，循环体结束之后，执行 `i++`，然后 `i` 变成了 `2`，继续判断 `i <= 10` 结果为 `true`，则执行循环体打印 `i = 2`，如此循环执行，直到打印 `i = 10` 之后，再执行 `i++` 让 `i` 变成了 `11`，然后判断 `i <= 10` 结果为 `false`，循环结束，这样就完成了 `1~10` 的输出，当然程序不是固定的，大家也可以把条件 `i <= 10` 修改为 `i < 11`，这样也是可以的。

关于 `for` 循环的使用我们还需要注意初始化变量的作用域，在 `for` 循环当中声明的变量只在 `for` 循环中有效，当 `for` 循环结束之后，初始化变量的内存就释放了/消失了，所以在 `for` 循环之外是无法访问该变量的，例如以下代码在编译的时候就会报错了：

1	//在 for 循环内部声明的变量 i 只在 for 循环中可见/有效
2	for(int i = 1; i <= 10; i++){
3	System.out.println("i = " + i);
4	}
5	//这行编译报错，i 变量不存在
6	System.out.println("i = " + i);

当然，这样写就不会报错了：

1	public static void test(){
2	//变量的声明位置放到了 for 循环之外
3	int i = 1;
4	for(; i <= 10; i++){
5	System.out.println("i = " + i);
6	}
7	//这里就可以访问变量 i 了
8	System.out.println("i = " + i);
9	}

为什么这样写就可以了呢？那是因为 `i` 变量的作用域已经被扩大了，不再只是 `for` 循环中可以使用，`for` 循环之外也可以使用，换句话说，以上的 `for` 循环结束之后 `i` 变量的内存并不会被释放。后续的程序可以继续使用。`i` 变量的作用域是在整个 `test()` 方法中都有效，直到 `test()`

方法执行结束的时候才会释放 i 变量的内存空间。

接下来我们再来看看 for 循环还有哪些其它的写法：

1	for(int i = 10; i > 0; i--){
2	System.out.println("i = " + i);
3	}
4	System.out.println("-----分割线-----");
5	for(int i = 10; i > 0; i -= 2){
6	System.out.println("i = " + i);
7	}
8	System.out.println("-----分割线-----");
9	for(int i = 100; i >= 10; i /= 3){
10	System.out.println("i = " + i);
11	}

以上程序运行结果，请看下图：


```
i = 10
i = 9
i = 8
i = 7
i = 6
i = 5
i = 4
i = 3
i = 2
i = 1
-----分割线-----
i = 10
i = 8
i = 6
i = 4
i = 2
-----分割线-----
i = 100
i = 33
i = 11
```

图 6-17: for 循环的其它编写方式执行结果

接下来，我们再使用 for 循环实现 1~100 所有数字的求和，实现思路是：定义变量 i，初始值从 1 开始，每循环一次加 1，这样就可以取到 1~100 的每一个整数了，那么求和怎么做呢？求和就是将 1~100 的每一个整数累加，这必然需要提前定义一个变量，使用变量实现累加，例如：a += 1, a += 2, a += 3...，代码如下所示：

1	int sum = 0;
2	for(int i = 1; i <= 100; i++){
3	sum += i;
4	}
5	System.out.println("sum = " + sum);

运行结果如下所示：

```
sum = 5050
```

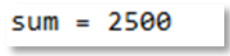
图 6-18: 1~100 所有整数求和

通过以上程序我们可以学到什么？编程语言当中的累加可以使用扩展类赋值运算符+=来实现，另外 sum 变量为什么定义到 for 循环外面，而不是定义到循环体当中呢？那是因为当定义到循环体内之后，每一次循环体在执行的时候，都会重新定义 sum 变量，这样会让 sum 变量归 0，无法达到累加的效果。

接下来，我们在以上程序的基础之上实现 1~100 所有奇数的和，编程思路是：在累加之前先判断变量 i 是否为奇数，如果是奇数则累加，这就需要在 sum += i 外面套一个 if 语句进行控制，代码如下所示：

1	int sum = 0;
2	for(int i = 1; i <= 100; i++){
3	if(i % 2 != 0){
4	sum += i;
5	}
6	}
7	System.out.println("sum = " + sum);

运行结果如下所示：



sum = 2500

图 6-19: 1~100 所有奇数和的执行结果

其实以上的方式是将每一个数字取出来，然后再判断是否为奇数，这种方式会导致循环次数达到 100 次，实际上可以换成另外一种解决方案，假设从 1 开始，每次累加 2，那么每次取出的数字为 1,3,5...，这样正好每次取出的数字都是奇数，可以直接累加了，这样循环的次数基本上会减半，效率则提高了，这种代码既优雅，又高效。请看以下代码：

1	int sum = 0;
2	for(int i = 1; i < 100; i += 2){

3	sum += i;
4	}
5	System.out.println("sum = " + sum);

运行结果如下所示：



图 6-20：1~100 所有奇数和的执行结果

以上演示的所有循环都是单层的，循环当中可以嵌套循环吗？答案是：当然可以，之前我们就说过所有控制语句都是可以嵌套使用的，当循环 A 中嵌套循环 B 的时候就等于在 A 循环体中的代码是 B 循环。其实大家在学习循环嵌套的时候完全没必要特殊化对待，完全可以把 A 循环体当中的 B 循环看做是一段普通的 java 代码。接下来我们看一段代码：

1	//循环 5 次（B 循环）
2	for(int i = 1;i <= 5; i++){
3	System.out.print("i = " + i + ",");
4	}
5	
6	System.out.println();
7	
8	//循环 2 次（A 循环）
9	for(int j = 1;j <= 2; j++){
10	System.out.print("j = " + j + ",");
11	}
12	
13	System.out.println();
14	
15	//将 B 循环放到 A 循环体当中

```
16 for(int j = 1;j <= 2;j++){ //A 循环(2 次)
17
18     for(int i = 1;i <= 5;i++){ //B 循环(5 次)
19
20         System.out.print("i = " + i + ",");
21
22     }
23 }
```

运行结果如下图所示：

```
i = 1,i = 2,i = 3,i = 4,i = 5,
j = 1,j = 2,
i = 1,i = 2,i = 3,i = 4,i = 5,i = 1,i = 2,i = 3,i = 4,i = 5,
```

图 6-21: for 循环嵌套

分析以上 for 循环嵌套，请看下图：

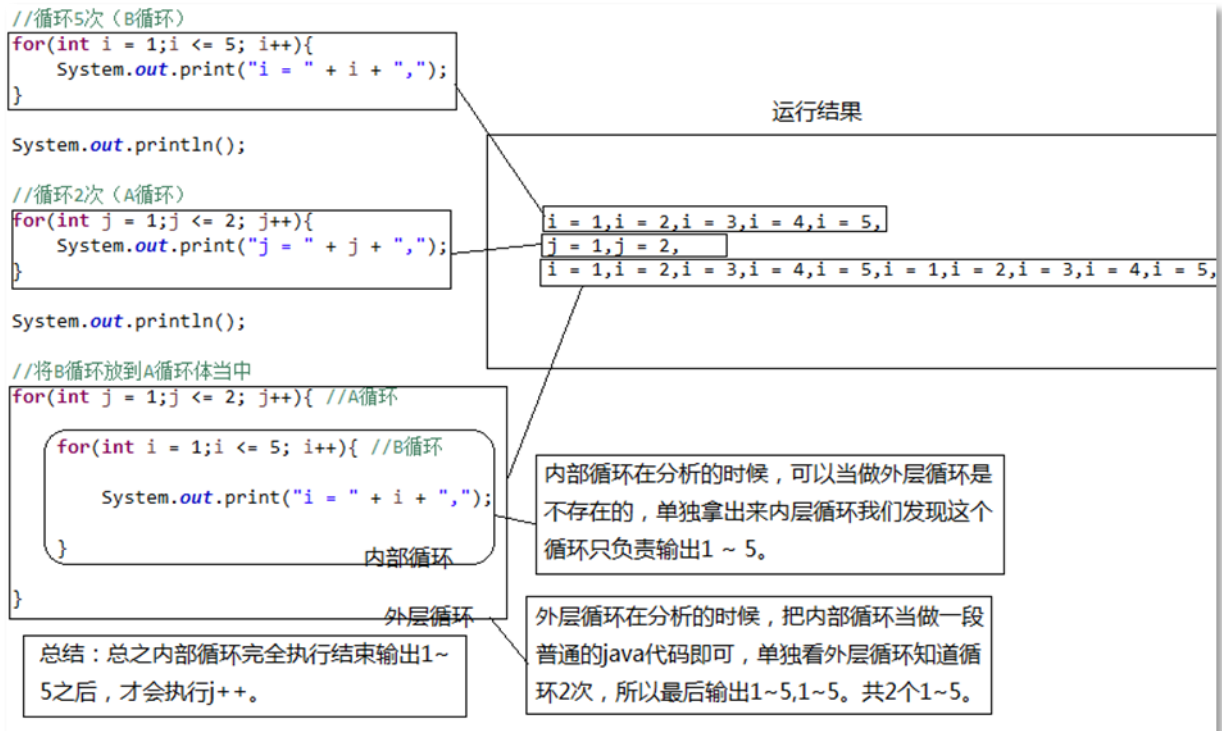


图 6-22: for 循环嵌套分析

分析循环嵌套的小窍门, 如下图所示:

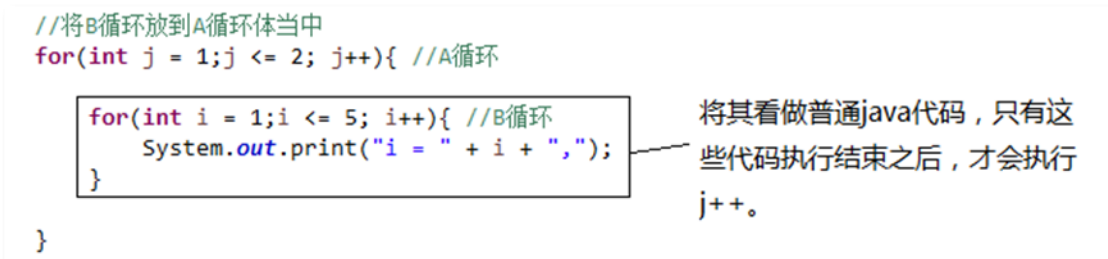


图 6-23: for 循环嵌套小窍门

学习了 for 循环嵌套的使用, 我们一起来写一下经典的九九乘法表, 九九乘法表的结构大家还记得吗, 我们一起来回顾一下小学的知识 (嘿嘿):

1*1=1									
2*1=2	2*2=4								
3*1=3	3*2=6	3*3=9							
4*1=4	4*2=8	4*3=12	4*4=16						
5*1=5	5*2=10	5*3=15	5*4=20	5*5=25					
6*1=6	6*2=12	6*3=18	6*4=24	6*5=30	6*6=36				
7*1=7	7*2=14	7*3=21	7*4=28	7*5=35	7*6=42	7*7=49			
8*1=8	8*2=16	8*3=24	8*4=32	8*5=40	8*6=48	8*7=56	8*8=64		
9*1=9	9*2=18	9*3=27	9*4=36	9*5=45	9*6=54	9*7=63	9*8=72	9*9=81	

图 6-24：九九乘法表结构

通过观察九九乘法表结构我们可以看出来它有 9 行，所以可以得出一定需要以下代码：

```

1 public static void main(String[] args) {
2     for(int i = 1; i <= 9; i++){
3         System.out.println("i = " + i);
4     }
5 }
```

运行结果如下：

```

i = 1
i = 2
i = 3
i = 4
i = 5
i = 6
i = 7
i = 8
i = 9
```

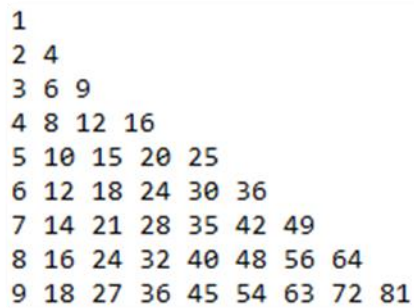
图 6-25：九九乘法表 9 行

观察上图我们可以得出 i 是行号，那么再次观察九九乘法表的规律得知，第 1 行 1 个式子，

第2行2个式子，第3行3个式子...，第9行9个式子，那么结论是第*i*行有*i*个式子，以上循环确定为外层循环，共循环9次，外层循环每循环一次要保证能够输出九九乘法表的1整行。那么输出九九乘法表1整行的时候又需要一个循环进行控制，而且这个循环被放到内部，循环的次数随着行号的变化而变化。代码如下所示：

1	for (int i = 1; i <= 9; i++) {
2	for (int j = 1; j <= i; j++) {
3	System.out.print(i * j + " ");
4	}
5	System.out.println();
6	}

运行结果如下所示：



```
1
2 4
3 6 9
4 8 12 16
5 10 15 20 25
6 12 18 24 30 36
7 14 21 28 35 42 49
8 16 24 32 40 48 56 64
9 18 27 36 45 54 63 72 81
```

图 6-26：九九乘法表每个表达式的执行结果

分析以上代码，请看下图：

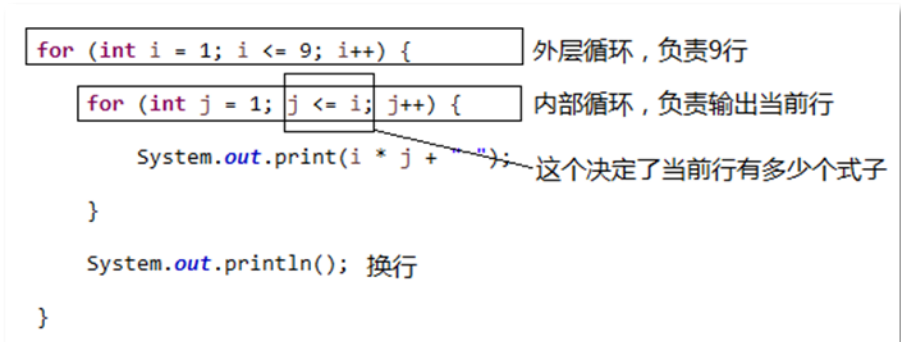


图 6-27：分析九九乘法表程序

接下来我们在每一个“结果”前添加“i*j=”，代码如下所示：

1	for (int i = 1; i <= 9; i++) {
2	for (int j = 1; j <= i; j++) {
3	System.out.print(i + "*" + j + "=" + i * j + " ");
4	}
5	System.out.println();
6	}

运行结果如下所示：

```

1*1=1
2*1=2 2*2=4
3*1=3 3*2=6 3*3=9
4*1=4 4*2=8 4*3=12 4*4=16
5*1=5 5*2=10 5*3=15 5*4=20 5*5=25
6*1=6 6*2=12 6*3=18 6*4=24 6*5=30 6*6=36
7*1=7 7*2=14 7*3=21 7*4=28 7*5=35 7*6=42 7*7=49
8*1=8 8*2=16 8*3=24 8*4=32 8*5=40 8*6=48 8*7=56 8*8=64
9*1=9 9*2=18 9*3=27 9*4=36 9*5=45 9*6=54 9*7=63 9*8=72 9*9=81

```

图 6-28：九九乘法表运行结果

通过以上代码的学习，需要每位读者能够掌握什么是循环，为什么要使用循环，for 循环的语法结构是什么，for 循环的执行顺序以及原理是什么，嵌套的 for 循环又应该怎么理解，大

家也可以挑战一下三层 for 循环嵌套，或者更多。总之 for 循环在实际开发中使用非常频繁，大家务必掌握。

6.4.2 while（掌握）

循环语句除了 for 循环之外还有 while 和 do..while，接下来我们先来看一下 while 循环，首先学习 while 循环的语法结构，如下图所示：

```
while(布尔表达式){  
    循环体;  
}
```

图 6-29: while 循环语法结构

通过上图可以看出 while 循环的语法结构非常简单，它的执行顺序以及原理是这样的，先判断布尔表达式的结果，如果是 true，则执行循环体，循环体结束之后，再次判断布尔表达式的结果，如果是 true，再执行循环体，循环体结束之后，再判断布尔表达式的结果，直到结果为 false 的时候，while 循环结束。如果起初第一次判断布尔表达式结果的时候就是 false，那么 while 循环体执行次数就为 0 了。它的执行顺序以及原理也可以参见下图：

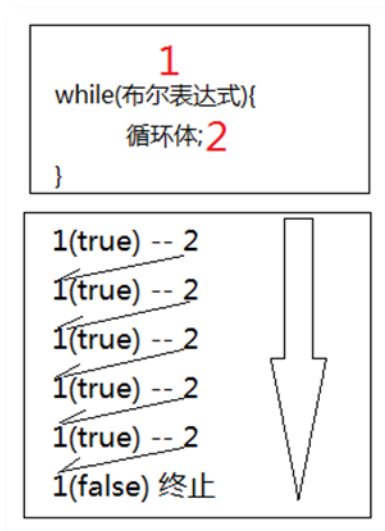


图 6-30: while 循环执行顺序以及原理

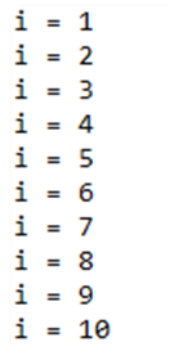
通过 while 循环的执行原理可以得出 while 循环的循环体执行次数可能是 0 次,也可能是 N 次。那么要想使用 while 循环实现一个死循环,代码应该怎么写呢?

```
1 while(true){  
2     System.out.println("死循环");  
3 }
```

运行结果就不再给大家展示了,要使用 while 实现一个死循环是非常简单的,让判断条件一直为 true 就可以了。那么使用 while 循环实现输出 1~10 应该怎么做呢?

```
1 int i = 0;  
2 while(i < 10){  
3     i++;  
4     System.out.println("i = " + i);  
5 }
```

运行结果如下图所示:



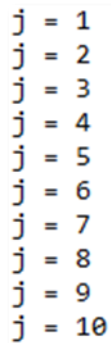
```
i = 1  
i = 2  
i = 3  
i = 4  
i = 5  
i = 6  
i = 7  
i = 8  
i = 9  
i = 10
```

图 6-31: 使用 while 循环输出 1~10

其实使用 while 循环输出 1~10 不止有以上这一种方式,还有其他方式,例如:

```
1 int j = 1;
2 while(j <= 10){
3     System.out.println("j = " + j);
4     j++;
5 }
```

运行结果如下图所示：



```
j = 1
j = 2
j = 3
j = 4
j = 5
j = 6
j = 7
j = 8
j = 9
j = 10
```

图 6-32：使用 while 循环输出 1~10

当然，大家还可以想想有没有其它的写法，可以自己写一写，例如：

```
1 int i = 0;
2 while(i < 10){
3     System.out.println("i = " + (++i));
4 }
```

再如：

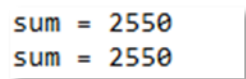
```
1 int j = 1;
2 while(j <= 10){
3     System.out.println("j = " + j++);
```

4	}
---	---

使用 while 循环计算 1~100 所有偶数的和，应该怎么做呢？

1	int sum = 0;
2	int i = 0;
3	while(i <= 100){
4	if(i % 2 == 0){
5	sum += i;
6	}
7	i++;
8	}
9	System.out.println("sum = " + sum);
10	
11	sum = 0;
12	int j = 0;
13	while(j <= 100){
14	sum += j;
15	j += 2;
16	}
17	System.out.println("sum = " + sum);

运行结果如下图所示：



```
sum = 2550
sum = 2550
```

图 6-33: while 循环计算 1~100 所有偶数和

实际上 while 循环可以看做是 for 循环的另一种变形写法，本质上是一样的，执行效率上

也是一样的，硬要说它们有什么不同的话，首先 `while` 循环语法结构比 `for` 更简单，`for` 循环的计数器比 `while` 更清楚一些，另外 `for` 循环的计数器对应的变量可以在 `for` 循环结束之后就释放掉，但是 `while` 循环的计数器对应的变量声明在 `while` 循环外面，扩大了该变量的作用域。总之，不管是 `for` 还是 `while`，大家都必须掌握，因为这两个循环使用最多。

6.4.3 do...while（掌握）

`do..while` 循环是 `while` 循环的变形，它们的区别在于 `do..while` 循环可以保证循环体执行次数至少为 1 次，也就是说 `do..while` 循环的循环体执行次数是 1~N 次，它有点儿先斩后奏的意思，而 `while` 循环的循环体执行次数为 0~N 次。

为什么 `do..while` 循环可以保证至少执行一次呢，它和 `while` 循环的区别在哪里呢？实际上是因为在开始执行 `while` 循环的时候，最先执行的是条件判断，只有条件为 `true` 的时候才会执行循环体，既然是这样，那么条件就有可能为 `false`，这个时候就会导致循环体执行次数为 0 次，俗话说，还没开始就结束了。而 `do..while` 循环最先执行的不是条件判断，它会先执行循环体，然后再进行条件判断，这样就可以保证循环体至少执行一次喽！

接下来我们一起来看看 `do..while` 循环的语法结构，以及执行顺序，如下图所示：

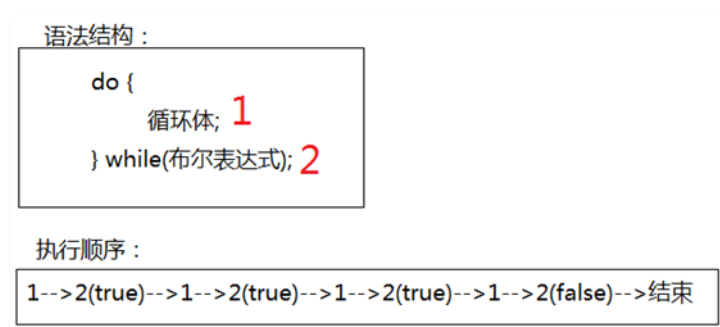


图 6-34: `do..while` 语法结构和执行顺序

或者参见下图：

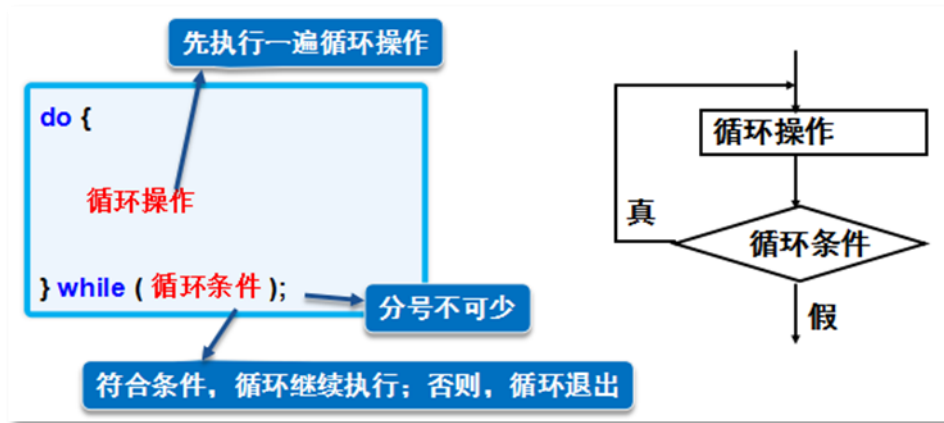


图 6-35: do..while 语法结构和执行顺序

上图中清晰的描述了 do..while 循环执行顺序，这里就不再赘述，但需要注意的是 do..while 循环在最后的时候有一个半角的分号“;”，这个不能丢，丢掉之后编译器就报错了。接下来我们看一个 do..while 循环的典型案例。

示例代码：业务背景：我们通常在使用的一个系统的时候需要登录，假设用户名或者密码记不清楚了，你是不是需要不断的“反复的”输入用户名和密码，这就是一个非常典型的循环案例，而这个循环当中首先要做的第一件事儿不是判断用户名和密码是否正确，它的起点是先让用户输入用户名和密码，所以这个时候我们就需要使用 do..while 循环来完成。请看以下代码：

```

1  java.util.Scanner scanner = new java.util.Scanner(System.in);
2  String username;
3  String password;
4  do{
5      System.out.print("用户名: ");
6      username = scanner.next();
7      System.out.print("密码: ");
8      password = scanner.next();
9  }while(!username.equals("admin") || !password.equals("123"));
10 System.out.println("登录成功，欢迎"+ username + "回来！");
    
```

运行效果如下图所示：

```
用户名: zhangsan  
密码: 123  
用户名: lisi  
密码: 123  
用户名: admin  
密码: 456  
用户名: admin  
密码: 123  
登录成功, 欢迎admin回来!
```

图 6-36: do..while 案例

解释以上程序：先提示用户输入用户名和密码，然后判断用户名和密码，当用户名不是 admin 或者密码不是 123 的时候继续提示用户输入用户名和密码，直到用户输入的用户名是 admin 并且密码是 123 的时候循环结束，循环结束之后输出登录成功的信息，只要循环没有结束就表示用户名和密码还是不对，当然，在现实的系统当中我们不可能允许用户无限制的输入用户名和密码，通常会给几次输入机会，当机会用完之后还是没有登录成功，通常该账户就被锁定了，你不妨试试这种业务又应该如何完成。

总之 while 和 do..while 循环的区别就在于 do..while 会先执行一次循环体，然后再判断条件，这样 do..while 的循环体至少执行一次。而 while 循环则是先判断条件是否合法，如果不合法，则循环体就没有任何执行的机会。while 循环体执行次数是 0~N 次，但是 do..while 循环体执行次数则是 1~N 次。

6.5 转向语句

转向语句用于实现循环执行过程中程序流程的跳转，在 Java 中转向语句有 break 和 continue 语句。当然，还包括其它的，例如 return 语句，这里主要给大家说一下 break 和 continue 语句。

6.5.1 break（掌握）

使用 break 这一个单词就可以在 java 语言中自成一条 java 语句，break 语句的编写很简单，例如“break;”，那么它可以用在哪里呢？首先它可以使用在 switch 语句当中，用来终止 switch 语句的执行，这个之前我们用过，这里不再赘述，break 语句重点是使用在循环语句当中，用来终止/跳出循环。例如有这样一个业务：从键盘不断的接收用户输入的整数，只要用户输入

的数字在[0~100]之间，则将输入的数字累加，一旦用户输入的整数不在[0-100]的范围，则终止循环的执行，并输出计算结果。请看下面的代码：

```
1 java.util.Scanner scanner = new java.util.Scanner(System.in);
2 int sum = 0;
3 while(true){
4     System.out.print("请输入[0-100]的整数: ");
5     int num = scanner.nextInt();
6     if(num >= 0 && num <= 100){
7         sum += num;
8         System.out.println("累加" + num + "成功! ");
9     }else{
10        System.out.println("不在合法范围，无法累加"+ num);
11        break;
12    }
13 }
14 System.out.println("最终累加结果 = " + sum);
```

运行效果如下图所示：



```
请输入[0-100]的整数: 1
累加1成功!
请输入[0-100]的整数: 2
累加2成功!
请输入[0-100]的整数: 3
累加3成功!
请输入[0-100]的整数: 111
不在合法范围，无法累加111
最终累加结果 = 6
```

图 6-37: break 初步使用

通过以上程序我们得知当用户输入的数字不在[0-100]范围内的时候 break 语句执行，while

循环结束了。那么，当循环语句多层嵌套的时候 break 语句终止的是哪个循环呢？我们来看以下的程序：

```
1  for(int j = 1; j <= 2; j++){  
2      for(int i = 1; i <= 10; i++){  
3          if(i == 5){  
4              break;  
5          }  
6          System.out.println("i = " + i);  
7      }  
8  }
```

运行结果如下所示：

```
i = 1  
i = 2  
i = 3  
i = 4  
i = 1  
i = 2  
i = 3  
i = 4
```

图 6-38: break 终止的是哪个循环

分析以上程序，请看下图：

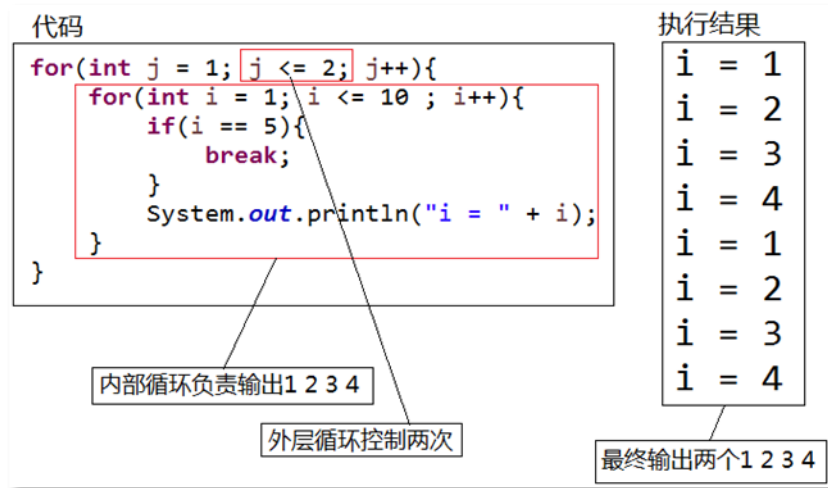


图 6-39：循环嵌套时，break 终止哪个？

通过上图的分析，可以得知 break 语句默认情况下只能终止离它“最近”的“一层”循环。以上的 break 语句则终止的是内部循环，不影响外部循环的执行。那么 break 语句可以用来终止指定的循环吗？请看以下代码：

```
1 first:for(int j = 1; j <= 2; j++){
2     for(int i = 1; i <= 10; i++){
3         if(i == 5){
4             break first;
5         }
6         System.out.println("i = " + i);
7     }
8 }
```

运行结果如下图所示：

```
i = 1
i = 2
i = 3
i = 4
```

图 6-40：终止指定的循环

通过以上程序的测试，我们可以得知当多层循环嵌套的时候，可以给每个循环设置标识，例如：first:for...、second:for...，当某个条件成立时，想终止指定循环的话，可以这样做：break first;或者 break second;，这样指定的循环就结束了。

总之，break 语句出现在循环当中用来终止循环的执行。例如：运动场上的运动员跑圈儿，跑了一圈又一圈，这显然是循环机制，假设比赛要求跑 10 圈儿，或者 20 圈儿，那么当计数器等于 10，或者 20 的时候，循环就结束了，如果中途发生意外呢，例如运动员晕倒了，那么此时在没有达到 10 圈儿或 20 圈儿的时候是不是也应该终止此循环的执行，要想让循环结束则执行 break 语句就可以了。

6.5.2 continue（掌握）

continue 语句也是单个单词自成一 Java 语句，例如“continue;”，它和 break 语句都是用来控制循环的，break 语句是用来终止循环的执行，而 continue 语句则是用来终止当前本次循环，直接进入下一次循环继续执行。请参照以下代码以及运行结果进行对比学习：

1	for(int i = 1; i <= 10; i++){
2	if(i == 5){
3	break;
4	}
5	System.out.println("break : i = " + i);
6	}
7	System.out.println("-----");
8	for(int i = 1; i <= 10; i++){
9	if(i == 5){
10	continue;
11	}
12	System.out.println("continue : i = " + i);
13	}

运行结果如下图所示：

```
break : i = 1
break : i = 2
break : i = 3
break : i = 4
-----
continue : i = 1
continue : i = 2
continue : i = 3
continue : i = 4
continue : i = 6
continue : i = 7
continue : i = 8
continue : i = 9
continue : i = 10
```

图 6-41: break 和 continue 对比

对以上代码以及执行结果进行分析，请看下图：

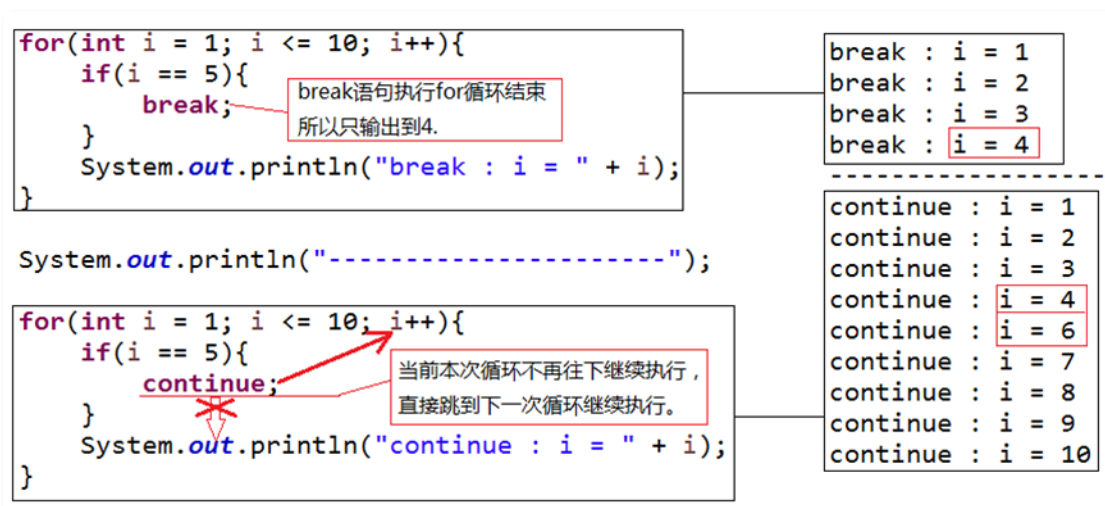


图 6-42: break 和 continue 对比

对于以上程序，当“`continue;`”语句执行的时候，当前本次循环剩下的代码就不再执行了（不再执行下面的输出语句），直接执行“`i++`”去了，而 `break` 就不同了，当以上程序“`break;`”语句执行之后整个 `for` 循环就结束了。

对于 `break` 语句有“`break first;`”这种写法，其实 `continue` 语句也是有这种语法机制的，这里就不再赘述了，大家可以自己编写程序测试一下。

对于 `break` 和 `continue` 语句的区别有这样一个小现实情景，大家可以借鉴理解一下：某公司销售部销售经理要进行人员招聘，预约了 10 个应聘者，这 10 个应聘者在办公室门口长凳上按照一定的次序坐着等待，销售经理一个一个轮流进行面试，当轮到第 3 个应聘者面试的时候，销售经理突然接到了一通电话，说家里有事了，那么此时销售经理就不得不终止今天的面试，此时销售经理执行了 `break` 语句，循环结束了，剩下的就不再面试了。那么假设销售经理没有接到这通电话则会继续轮流面试，他为每一个应聘者准备了 5 个问题，假设轮到第 6 个应聘者面试，在面试过程中问完第 1 个问题之后销售经理就知道这人不适合这个岗位，那么此时剩下的 4 个问题就不再问了（当前本次循环结束），直接对着门口喊了一句：下一个应聘的进来。这个过程就相当于销售经理执行了 `continue` 语句。

总之，`break` 用来终止循环，`continue` 用来中断当前本次循环，直接进入下一次循环继续执行。

6.6 章节小结

本章节内容在以后的开发中会经常的使用，因为软件很多时候都需要处理业务逻辑，那么处理逻辑的过程是需要控制语句来完成的，所以本章节内容非常重要。

本章节中所讲到的控制语句都很常用，如果说哪些使用频率最高，其中 `if` 语句、`for` 循环、`while` 循环使用最为频繁。对于这几个语句大家可以着重掌握。

6.7 难点解惑

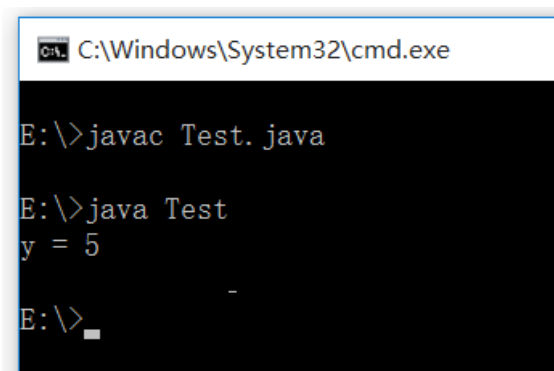
本章节内容虽然很重要，但是每一个都不是很复杂，比较容易掌握，如果难点，对于初学者来说 `switch` 语句也不是那么容易掌握的。请看以下代码：

1	<code>public class Test{</code>
2	<code> public static void main(String[] args){</code>
3	<code> int y = 3;</code>
4	<code> switch(y){</code>
5	<code> default :</code>
6	<code> y++;</code>
7	<code> case 3 :</code>
8	<code> y++;</code>
9	<code> case 5 :</code>

10	y++;
11	break;
12	}
13	System.out.println("y = " + y);
14	}
15	}

我们来分析一下以上程序，首先语法上没有错误，可以编译通过，有人可能认为 `default` 还可以写在这个位置？是的，`default` 可以写在任何位置，但它的执行时机是不变的，永远都是在所有分支都没有匹配成功的时候才会执行，对于以上程序来说 `default` 分支是不会执行的，因为 `y` 最初等于 3，与第一个分支会匹配成功。当第一个分支匹配成功之后，执行 `y++`，此时 `y` 等于 4。由于没有 `break` 语句，会发生 `case` 穿透现象，继续执行第二个分支 `y++`，`y` 的值最终是 5。

我们对以上程序进行编译和运行，来看一下我们的分析是否是正确的，请看下图：



```
C:\Windows\System32\cmd.exe

E:\>javac Test.java

E:\>java Test
y = 5

E:\>
```

图 6-43：Test 程序的编译和运行

通过以上的测试结果，可以看出，我们的分析是正确的。

6.8 章节习题

第一题：编写 java 程序，用循环结构打印如下的数值列表：

```
1  10 100 1000
2  20 200 2000
3  30 300 3000
4  40 400 4000
```

5 50 500 5000

第二题：打印 2 到 10000 的所有素数，每行显示 8 个素数。

第三题：编写程序，计算 5 的阶乘。

第四题：控制台输入年龄，根据年龄输出不同的提示。

第五题：编写程序输出下图菱形。



第六题：篮球从 5 米高的地方掉下来，每次弹起的高度是原来的 30%，经过几次弹起，篮球的高度是 0.1 米。

6.9 习题答案

第一题答案：

1	public class Test {
2	public static void main(String[] args) {
3	for(int i=1 ;i<=5 ;i++){
4	int temp = i;
5	for(int j=1;j<=4 ;j++){
6	System.out.print(temp + " ");
7	temp *= 10;
8	}
9	System.out.println();
10	}

11	}
12	}

第二题答案:

1	public class Test {
2	public static void main(String[] args) {
3	int count = 0;
4	for(int i=2;i<=10000;i++){
5	boolean isPrimeNum = true;
6	for(int j=2;j<i;j++){
7	if(i % j == 0){
8	isPrimeNum = false;
9	break;
10	}
11	}
12	if(isPrimeNum){
13	System.out.print(i + " ");
14	count++;
15	if(count == 8){
16	System.out.println();
17	count = 0;
18	}
19	}
20	}
21	}
22	}

第三题答案:


```
1 public class Test {
2     public static void main(String[] args) {
3         int result = 1;
4         for(int i=1;i<=5;i++){
5             result *= i;
6         }
7         System.out.println("5 的阶乘 = " + result);
8     }
9 }
```

第四题答案:

```
1 public class Test {
2     public static void main(String[] args) {
3         java.util.Scanner s = new java.util.Scanner(System.in);
4         System.out.print("请输入年龄: ");
5         int age = s.nextInt();
6         if(age < 0 || age > 150){
7             throw new RuntimeException("对不起, 年龄值不合法!");
8         }
9         String grade = "你是老年人";
10        if(age < 5){
11            grade = "可爱的小 baby";
12        }else if(age < 10){
13            grade = "你还是个小屁孩";
14        }else if(age < 15){
15            grade = "小少年, 不错哦";
16        }else if(age < 18){
17            grade = "你是个青少年";
18        }else if(age < 35){
```

19	grade = "你是个青年人";
20	}else if(age < 50){
21	grade = "你已是中年人";
22	}
23	System.out.println(grade);
24	}
25	}

第五题答案:

1	public class Test {
2	public static void main(String[] args) {
3	int lay = 9;
4	for (int m = 1; m <= (lay + 1) / 2; m++){
5	for (int b = 1; b <= (lay + 1) / 2 - m; b++){
6	System.out.print("");
7	}
8	for (int c = 1; c <= m * 2 - 1; c++) {
9	System.out.print("*");
10	}
11	System.out.println();
12	}
13	for (int d = (lay + 1) / 2 - 1; d >= 1; d--){
14	for (int b = 1; b <= (lay + 1) / 2 - d; b++){
15	System.out.print("");
16	}
17	for (int c = (lay + 1) / 2 - d; c <= (lay + 1) / 2 - 2 + d; c++){
18	System.out.print("*");
19	}
20	System.out.println();

21	}
22	}
23	}

第六题答案:

1	public class Test {
2	public static void main(String[] args) {
3	double height = 5.0;
4	int count = 0;
5	while(height > 0.1){
6	height *= 0.3;
7	count++;
8	}
9	System.out.println("弹起次数 = " + count);
10	}
11	}

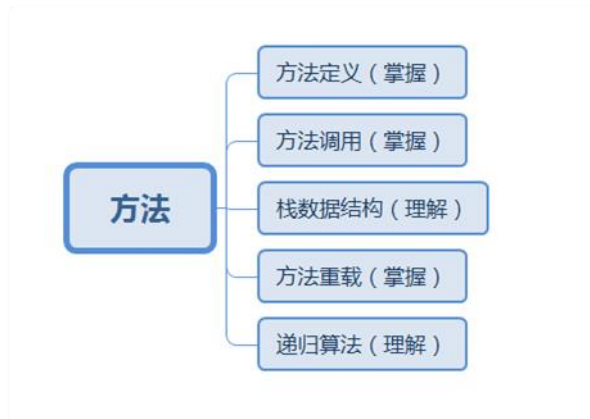
7. 第七章 方法

7.1 章节目标与知识框架

7.1.1 章节目标

理解方法的本质以及作用；掌握方法的定义；掌握方法如何调用；理解栈数据结构；理解方法执行过程中内存是如何变化的；掌握方法的重载机制；掌握方法递归算法。

7.1.2 知识框架



7.2 方法（掌握）

7.2.1 方法的本质以及作用（理解）

我们先不讲方法是什么，先来看一段代码，分析以下程序存在哪些缺点，应该如何去改进：

```
1 public static void main(String[] args) {  
2     //请计算 10 和 20 的和  
3     int a = 10;  
4     int b = 20;  
5     int c = a + b;  
6     System.out.println(a + "+" + b + "=" + c);  
7     //请计算 666 和 888 的和  
8     int x = 666;  
9     int y = 888;  
10    int z = x + y;  
11    System.out.println(x + "+" + y + "=" + z);  
12    //请计算 888 和 999 的和  
13    int m = 888;
```

```
14     int n = 999;
15     int e = m + n;
16     System.out.println(m + "+" + n + "=" + e);
17 }
```

以上代码完成了三个求和的功能，每一次求和的时候都把代码重新写了一遍，显然代码没有得到“重复利用”，表面上看是三个功能，但实际上只是“一个”求和功能，只不过每一次参与求和的实际数值不同。java 中有没有一种方式或者语法，能让我们把功能性代码写一次，然后给这个功能性代码传递不同的数据，来完成对应的功能呢？答案是：当然有。这就需要我们掌握 java 语言中的方法机制，接下来大家看看改进之后的代码（这里先不需要掌握方法是怎么定义的以及怎么调用的，只要看以下代码就行，此小节是为了让大家理解方法的本质以及作用）：

```
1  public static void main(String[] args) {
2      //调用求和方法计算 10 和 20 的和
3      sumInt(10, 20);
4      //调用求和方法计算 666 和 888 的和
5      sumInt(666, 888);
6      //调用求和方法计算 888 和 999 的和
7      sumInt(888, 999);
8  }
9  //专门负责求和的方法
10 public static void sumInt(int a, int b){
11     int c = a + b;
12     System.out.println(a + "+" + b + "=" + c);
13 }
```

运行结果如下图所示：

```
10+20=30  
666+888=1554  
888+999=1887
```

图 7-1：求和结果

通过以上程序我们可以看出，其实方法也没什么神秘的，方法其实就是一段普通的代码片段，并且这段代码可以完成某个特定的功能，而且可以被重复的调用/使用。java 中的方法又叫做 **method**，在 C 语言中叫做函数。

从现在开始大家以后在写代码的时候就要有定义方法的意思了，只要是独立出来的功能，我们都可以定义为单独的一个方法来完成，如果以后需要使用此功能时直接调用这个方法就行了，不要把所有的代码都扔到 **main** 方法当中，这样会导致程序的“复用性”很差。

7.2.2 方法的定义以及调用（掌握）

通过以上内容的学习，可以看出方法是一段可以完成某个特定功能的并且可以被重复利用的代码片段。接下来我们来学习一下方法应该怎么定义以及怎么调用。

定义/声明方法的语法格式如下所示：

```
[修饰符列表] 返回值类型 方法名(形式参数列表){  
    方法体;  
}
```

例如代码：

```
public static void sumInt(int a , int b){  
    int c = a + b;  
    System.out.println(a + "+" + b + "=" + c);  
}
```

public static 是修饰符列表；

void 是返回值类型；

sumInt 是方法名；

(int a , int b)是形式参数列表，简称形参，每一个形参都是局部变量；

形参后面使用一对儿大括号括起来的是方法体，方法体是完成功能的核心代码，方法体中的代码有执行顺序的要求，遵循自上而下的顺序依次逐行执行，不存在跳行执行的情况。

再如代码：

```
public static int sumInt(int a, int b){  
    int c = a + b;  
    return c;  
}
```

以上程序中 `sumInt` 之前的 `int` 是返回值类型。

接下来我将列出方法的相关规则，其中一些规则目前可能需要大家死记硬背，还有一些规则希望大家在理解的前提下进行记忆：

① [修饰符列表]，此项是可选项，不是必须的，目前大家统一写成 `public static`，后面的课程会详细讲解。

② 返回值类型，此项可以是 `java` 语言当中任何一种数据类型，包括基本数据类型，也包括所有的引用数据类型，当然，如果一个方法执行结束之后不准备返回任何数据，则返回值类型必须写 `void`。返回值类型例如：`byte, short, int, long, float, double, boolean, char, String, void` 等。

③ 方法名，此项需要是合法的标识符，开发规范中要求方法名首字母小写，后面每个单词首字母大写，遵循驼峰命名方式，见名知意，例如：`login`、`getUsername`、`findAllUser` 等。

④ 形式参数列表(`int a, int b`)，此项又被称为形参，其实每一个形参都是“局部变量”，形参的个数为 0~N 个，如果是多个参数，则采用半角“,”进行分隔，形参中起决定性作用的是参数的数据类型，参数名就是变量名，变量名是可以修改的，也就是说(`int a, int b`)也可以写成(`int x, int y`)。

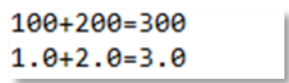
⑤ 方法体，由一对儿大括号括起来，在形参的后面，这个大括号当中的是实现功能的核心代码，方法体由 `java` 语句构成，方法体当中的代码只能遵循自上而下的顺序依次逐行执行，不能跳行执行，核心代码在执行过程中如果需要外部提供数据，则通过形参进行获取。

整体来说方法的声明语法是很简单的，我相信每个人都能记住，其实我觉得方法的定义难度最大的不是语法，而是方法在定义的时候，返回值类型定为什么类型比较合适？方法的形式参数列表中定义几个参数合适？每个参数的数据类型定义为什么比较合适？以上的一系列问题实际上还是需要和具体的功能结合在一起才能决定，当然，这不是一天两天的事儿，不是说这一章节的内容学完之后就真正的会定义方法了，我们只能说语法会了，还需要后期不断的做项目，写代码才能找到感觉，找到编程思路。到那时，你自然就会定义返回值类型、形式参数列表了。

当一个方法声明之后，我们应该如何去让这个方法执行呢，当然，这个时候就需要亲自去调用这个方法了，调用方法的语法格式是（前提是方法的修饰符列表中含有 `static` 关键字）：“类名.方法名(实际参数列表);”，例如以下代码：

```
1 public class MethodTest {
2     public static void main(String[] args) {
3         MethodTest.sumInt(100, 200);
4         MethodTest.sumDouble(1.0, 2.0);
5     }
6     public static void sumInt(int x, int y){
7         System.out.println(x + "+" + y + "=" + (x + y));
8     }
9     public static void sumDouble(double a, double b){
10        System.out.println(a + "+" + b + "=" + (a + b));
11    }
12 }
```

运行结果如下图所示：



```
100+200=300
1.0+2.0=3.0
```

图 7-2：方法如何调用

需要注意的是，方法在调用的时候，实际传给这个方法的数据被称为实际参数列表，简称实参，java 语法中有这样的规定：实参和形参必须一一对应，所谓的一一对应就是，个数要一样，数据类型要对应相同。例如：实参(100, 200)对应的形参(int x, int y)，如果不是一一对应则编译器就会报错。当然也可能会存在自动类型转换，例如：实参(100, 200)也可以传递给这样的形参(long a, long b)，这里我们先不谈这个。

实际上方法在调用的时候，有的情况下“类名.”是可以省略的，我们来看看什么情况下它可以省略不写：


```
1 public class MethodTest03 {
2     public static void main(String[] args) {
3         sumInt(100, 200); // “类名.” 可以省略
4         sumDouble(1.0, 2.0); // “类名.” 可以省略
5         //doOther(); //编译报错
6         Other.doOther(); // “类名.” 不能省略
7     }
8     public static void sumInt(int x, int y){
9         System.out.println(x + "+" + y + "=" + (x + y));
10    }
11    public static void sumDouble(double a, double b){
12        System.out.println(a + "+" + b + "=" + (a + b));
13    }
14 }
```

```
1 public class Other{
2     public static void doOther(){
3         System.out.println("Other doOther...");
4     }
5 }
```

运行结果如下图所示：

```
100+200=300
1.0+2.0=3.0
Other doOther...
```

图 7-3：“类名.” 什么时候可以省略

通过以上程序的分析，我们得知，当在 a()方法执行过程中调用 b()方法的时候，并且 a()

方法和 b() 方法在同一个类当中，此时“类名.”可以省略不写，但如果 a() 方法和 b() 方法不在同一个类当中，“类名.”则不能省略。

7.2.3 方法返回值详解（掌握）

每个方法都是为了完成某个特定的功能，例如：登录功能、求和功能等，既然是功能，那么当这个功能完成之后，大多数情况下都会有一个结果的，比如，登录成功了或者失败了（true/false），求和之后最后的结果是 100 或者 200，当然也有极少数的情况下是没有结果的。这个结果本质上就是一个数据，那么既然是一个数据，就一定会会有对应的类型，所以在方法定义的时候需要指定该方法的返回值类型。

java 语言中方法的返回值类型可以是任何一种数据类型，包括基本数据类型，也包括引用数据类型，例如：byte,short,int,long,float,double,boolean,char,String,Student（自定义类）等。当然，如果这个方法在执行完之后不需要返回任何数据，返回值类型必须写 void 关键字，不能空着不写。

关于方法的返回值在使用的时候有哪些注意事项呢，我们来看几段代码：

1	public static int method1(){
2	
3	}

以上程序在编译的时候，报错了，错误信息是“缺少返回语句”，为什么呢？这是因为该方法在声明的时候指定了方法结束之后返回 int 类型的数据，但是以上方法体中并没有编写“返回数据”的代码。也就是说当一个方法在声明的时候返回值类型不是 void 的情况下，要求方法体中必须有负责“返回数据”的语句。这样的语句怎么写呢？答案是：“return 数据;”（大家还记得第六章控制语句中的返回语句吗？这个就是），并且要求这个“数据”的类型要和声明的返回值类型一致，要不然编译器就会报错了。代码这样写就没问题了：

1	public static int method1(){
2	/*
3	return 1;
4	return 0;
5	return 100;

6	*/
7	int a = 100;
8	int b = 200;
9	return a + b;
10	}

如果代码这样写呢？

1	public static int method1(){
2	return 1;
3	System.out.println("hello world!");
4	}

编译以上程序，我们可以看到编译器报错了，提示的错误信息是：“System.out.println("hello world!");”这行代码是无法访问的语句。这是为什么呢？因为在方法中如果一旦执行了带有“return”关键字的语句，此方法就会结束，所以以上的程序中“System.out.println("hello world!");”这行代码是永远无法执行到的，所以编译报错了。得到的结论是：带有return关键字的语句只要执行，所在的方法则执行结束。

那这样写呢？

1	public static int method1(){
2	boolean flag = true;
3	if(flag){
4	return 1;
5	}
6	}

还是编译报错，错误信息是“缺少返回语句”，为什么呢？实际上方法在声明的时候指定了返回值类型为int类型，java语法则要求方法必须能够“百分百的保证”在结束的时候返回int类型的数据，以上程序“return 1;”出现在if语句的分支当中，对于编译器来说，它只知道

“return 1;”有可能执行，也有可能不执行，所以 java 编译器报错了，不允许程序员这样编写代码。来看以下代码：

1	public static int method1(){
2	boolean flag = true;
3	if(flag){
4	return 1;
5	}else{
6	return 0;
7	}
8	}

这样就能编译通过了，为什么呢？这是因为编译器能够检测出以上的 if..else..语句必然会有一个分支执行，这样就不缺少返回语句了。其实以上代码也可以这样写：

1	public static int method1(){
2	boolean flag = true;
3	if(flag){
4	return 1;
5	}
6	return 0;
7	}

以上代码也可以达到同样的效果，因为“return 1;”如果不执行，则一定会执行“return 0;”，当然，如果执行了“return 1;”则方法一定会结束，“return 0;”也就没有执行的机会了。再看下面的代码：

1	public static int method1(){
2	boolean flag = true;
3	if(flag){

4	return 1;
5	System.out.println("第 1 行");
6	}
7	System.out.println("第 2 行");
8	return 0;
9	System.out.println("第 3 行");
10	}

以上程序还是编译报错，哪里出错了，为什么呢？其中“第 1 行”和“第 3 行”代码没有执行机会，编译报错，但“第 2 行”是有执行机会的。通过以上程序我们可以得出这样的结论：在同一个“域”中，“return”语句后面是不能写任何代码的，因为它无法执行到。

如果只是单纯的完成以上代码的功能我们也可以这样写：

1	public static int method1(){
2	boolean flag = true;
3	return flag ? 1 : 0;
4	}

所以，一个功能的实现，代码可以写很多种不同的方式，慢慢培养吧同学们，这需要一个过程，千万戒骄戒躁。我们接着看关于返回值还有哪些注意事项：

1	public static void method2(){
2	return 10;
3	}

以上代码编译报错，为什么呢？这是因为一个方法在声明的时候返回值类型定义为 void，这就表示该方法执行结束的时候不能返回任何数据，而以上程序中编写了“return 10;”这样的代码，自然编译器会报错的。也就是说前后说法要一致，声明的时候有返回值，那么代码编写的时候就必须有“return 值;”这样的语句。如果声明的时候没有返回值，那么方法结束的时候就不能编写“return 值;”这样的语句。那么，大家再来看看以下的代码：

```
1 public static void method2(){
2     return;
3 }
```

经过测试，以上的代码编译通过了，也就是说当一个方法的返回值类型是 `void` 的时候，方法体当中允许出现 “`return;`” 语句（注意：不允许出现 “`return 值;`”），这个语法的作用主要是用来终止方法的执行。当一个方法的返回值类型是 `void` 的时候，在方法执行过程中，如果满足了某个条件，则这个方法可能就没必要往下继续执行了，想终止这个方法的执行，此时执行 “`return;`” 就行了。有一些同学在最初的学习过程中，对 `break` 和 `return` 认识的不够清晰，我们来研究一下，请看以下代码：

```
1 public static void main(String[] args) {
2     for(int i = 1; i <= 10; i++){
3         if(i == 5){
4             break;
5         }
6         System.out.println("i = " + i);
7     }
8     System.out.println("hello world!");
9 }
```

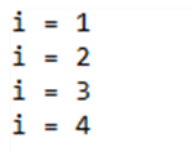
运行结果如下图所示：

```
i = 1
i = 2
i = 3
i = 4
hello world!
```

图 7-4：break 测试

```
1 public static void main(String[] args) {  
2     for(int i = 1; i <= 10; i++){  
3         if(i == 5){  
4             return;  
5         }  
6         System.out.println("i = " + i);  
7     }  
8     System.out.println("hello world!");  
9 }
```

运行结果如下图所示：



```
i = 1  
i = 2  
i = 3  
i = 4
```

图 7-5: return 测试

通过以上的测试，我们可以得出 `break` 和 `return` 根本不是一个级别的，`break` 用来终止循环，`return` 用来终止一个方法的执行。接下来再看一段代码：

```
1 public static int method3(){  
2     return;  
3 }
```

经过测试，以上代码编译报错，错误信息是：缺少返回值，为什么呢？这是因为方法在声明的时候规定了返回 `int` 类型的值，但是在方法体当中执行了“`return;`”语句，并没有返回具体的值，所以“`return;`”只能出现在返回值类型是 `void` 的方法中。

接下来我们来看看，当一个方法执行结束之后怎么接收这个返回值呢？请看下面代码：

```
1 public static void main(String[] args) {  
2     //可以编译也可以正常运行  
3     sumInt(10, 20);  
4     int retValue = sumInt(100, 200);  
5     System.out.println("计算结果 = " + retValue);  
6     //编译报错，返回值类型是 int，不能采用 byte 接收  
7     //byte retValue2 = sumInt(1, 2);  
8 }  
9 public static int sumInt(int a, int b){  
10     return a + b;  
11 }
```

运行结果如下图所示：



图 7-6：测试怎么接收返回值

通过以上的测试我们得知，方法执行结束之后的返回值我们可以接收，也可以不接收，不是必须的，但大多数情况下方法的返回值还是需要接收的，要不然就没有意义了，另外方法的返回值在接收的时候采用变量的方式接收，要求这个变量的数据类型和返回值类型一致（当然，也可以遵守自动类型转换规则）。

7.2.4 栈数据结构（理解）

要想理解方法执行过程中内存的分配，我们需要先学习一下栈数据结构，那么什么是数据结构呢？其实数据结构是一门独立的学科，不仅是在 java 编程中需要使用，在其它编程语言中也会使用，在大学的计算机课程当中，数据结构和算法通常作为必修课出现，而且是在学习任何一门编程语言之前先进行数据结构和算法的学习。数据结构是计算机存储、组织数据的方式。数据结构是指相互之间存在一种或多种特定关系的数据元素的集合。通常情况下，精心选择的数据结构可以带来更高的运行或者存储效率。数据结构往往同高效的检索算法和索引技术有关。

常见的数据结构有哪些呢？例如：栈、队列、链表、数组、树、图、堆、散列表等。目前我们先来学习一下栈（stack）数据结构，这是一种非常简单的数据结构。如下图所示：

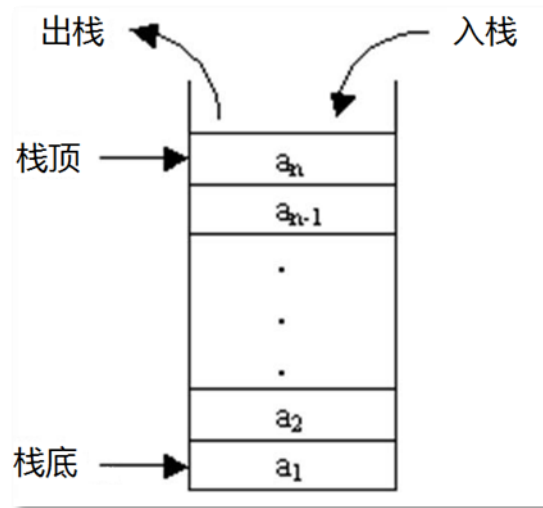


图 7-7：栈数据结构

栈（stack）又名堆栈，它是一种运算受限的线性表。其限制是：仅允许在表的一端进行插入和删除运算。这一端被称为栈顶，相对地，把另一端称为栈底。向一个栈插入新元素又称作进栈、入栈或压栈（push），它是把新元素放到栈顶元素的上面，使之成为新的栈顶元素；从一个栈删除元素又称作出栈、退栈或弹栈（pop），它是把栈顶元素删除掉，使其相邻的元素成为新的栈顶元素。如下图所示：

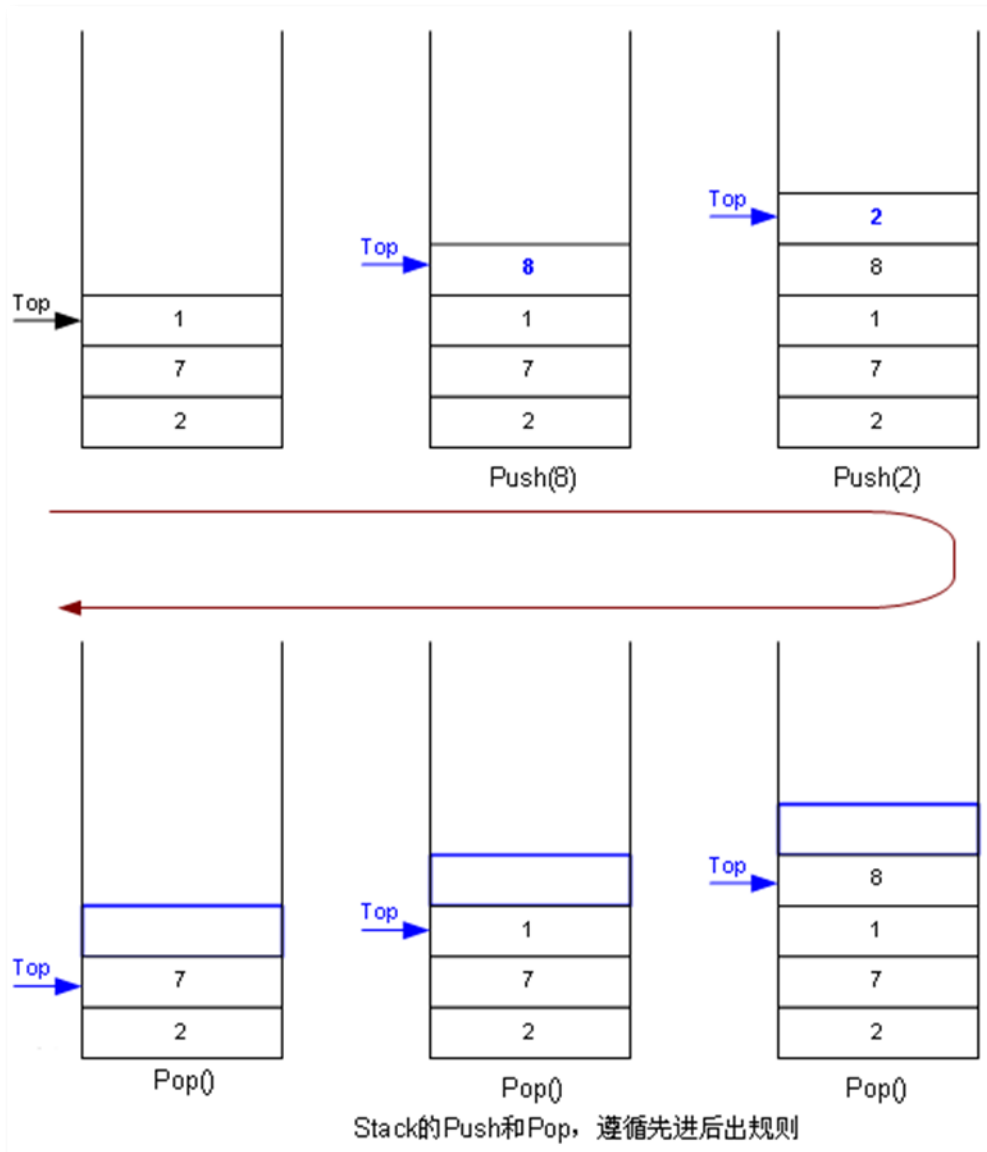


图 7-8：栈数据结构

通过以上的学习，我们可以得知栈数据结构存储数据有这样的特点：先进后出，或者后进先出原则。也就是说最先进去的元素一定是最后出去，最后进去的元素一定是最先出去，因为一端是开口的，另一端是封闭的。

对于栈数据结构，目前我们了解这么多就可以了，等学完“方法执行的时候内存是如何变化的”，到那个时候大家再思考一个问题，为什么方法执行过程的内存要采用栈这种数据结构呢，为什么不选择其它数据结构呢？

7.2.5 方法执行过程中内存的变化（理解）

以上内容中讲解了方法是什么，怎么定义，怎么调用，目前来说大家实际上掌握这些内容就行了，接下来的内容大家尽量去学，实在是掌握不了，也没有关系，后期的内容会对这一部分的知识点进行不断的讲解，慢慢的大家就会了，其实在学习编程的过程中会遇到很多这样的情况，没事，大家不要心急，学习后面内容的过程中你会对前面的内容豁然开朗。

以下要讲解的是程序的内存，例如：代码片段被存储在什么位置？方法调用的时候，在哪里开辟内存空间等等。所以这一部分内容还是很高端大气上档次的。不过话又说回来，要想真正掌握 java，内存的分析是必要的。一旦掌握内存的分配，在程序没有运行之前我们就可以很精准的预测到程序的执行结果。

好了，接下来我们开始学习方法执行过程中内存是如何变化的？我们先来看一张图片：

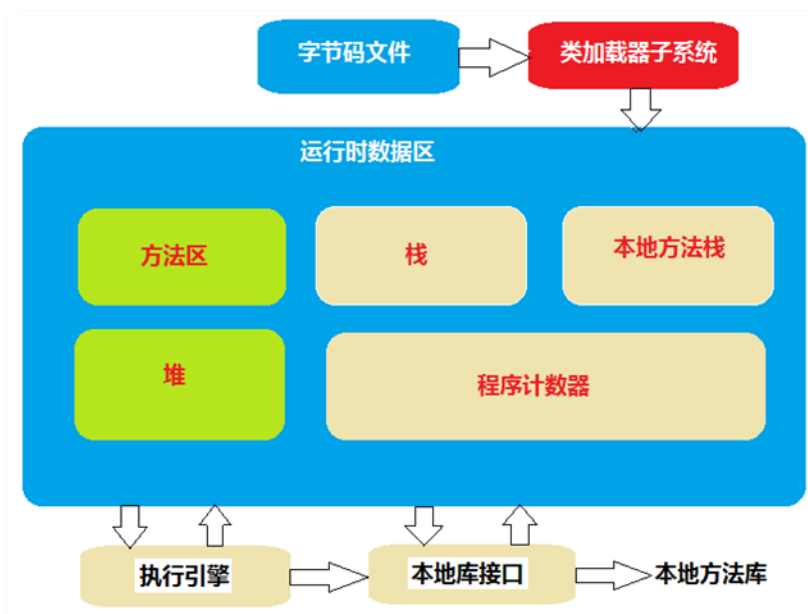


图 7-9: JVM 内存结构图

上图是一张标准的 java 虚拟机内存结构图，目前我们只看其中的“栈”和“方法区”，其它的后期研究，方法区中存储类的信息，或者也可以理解为代码片段，方法在执行过程中需要的内存空间在栈中分配。java 程序开始执行的时候先通过类加载器子系统找到硬盘上的字节码 (class) 文件，然后将其加载到 java 虚拟机的方法区当中，开始调用 main 方法，main 方法被调用的瞬间，会给 main 方法在“栈”内存中分配所属的活动空间，此时发生压栈动作，main 方法的活动空间处于栈底。

也就是说，方法只定义不去调用的话，只是把它的代码片段存储在方法区当中，java 虚拟机是不会在栈内存当中给该方法分配活动空间的，只有在调用的瞬间，java 虚拟机才会在“栈内存”当中给该方法分配活动空间，此时发生压栈动作，直到这个方法执行结束的时候，这个方法在栈内存中所对应的活动空间就会释放掉，此时发生弹栈动作。由于栈的特点是先进后出，所以最先调用的方法（最先压栈）一定是最后结束的（最后弹栈）。比如：main 方法最先被调用，那么它一定是最后一个结束的。换句话说：main 方法结束了，程序也就结束了（目前来说是这样）。

接下来我们来看一段代码，同时画出内存结构图，以及使用文字描述该程序的内存变化：

1	public class MethodTest {
2	public static void main(String[] args) {
3	System.out.println("main begin");
4	m1();
5	System.out.println("main over");
6	}
7	public static void m1() {
8	System.out.println("m1 begin");
9	m2();
10	System.out.println("m1 over");
11	}
12	public static void m2() {
13	System.out.println("m2 begin");
14	System.out.println("m2 over");
15	}
16	}

运行结果如下图所示：

```
main begin
m1 begin
m2 begin
m2 over
m1 over
main over
```

图 7-10：方法执行过程中内存变化测试程序

通过上图的执行结果我们了解到，main 方法最先被调用，但是它是最后结束的，其中 m2 方法最后被调用，但它是最先结束的。大家别忘了调用的时候分配内存是压栈，结束的时候是释放内存弹栈哦。为什么会是上图的结果呢，我们来看看它执行的内存变化，请看下图：

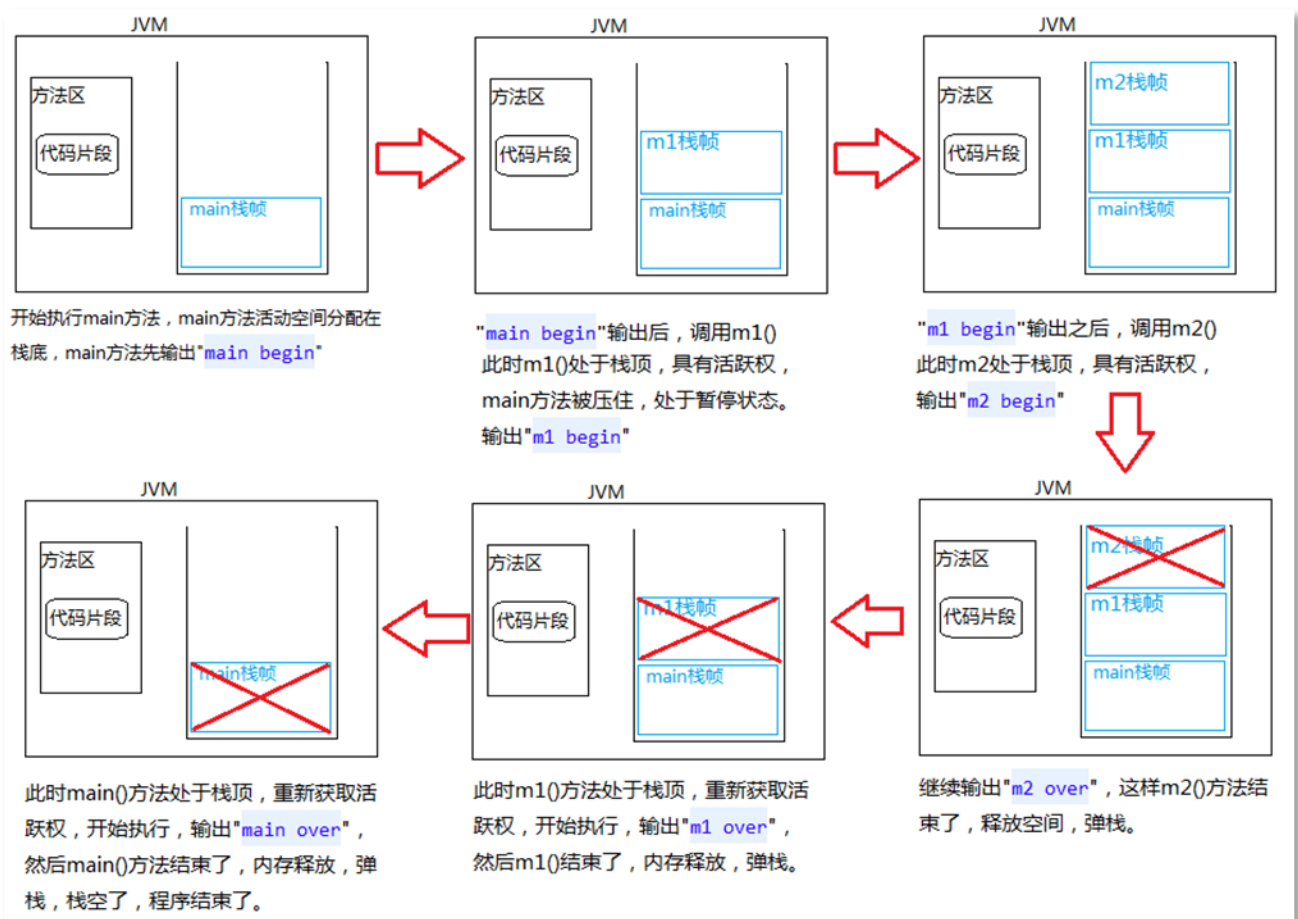


图 7-11：方法执行过程中内存的变化

通过上图的分析，可以很快明白，为什么输出结果是这样的顺序，接下来我们再采用文字的方式描述它的内存变化：

- ① 类加载器将 class 文件加载到方法区。
- ② 开始调用 main 方法,在栈内存中给 main 方法分配空间,开始执行 main 方法,输出"main begin"。
- ③ 调用 m1()方法,在栈内存中给 m1()方法分配空间, m1()方法处于栈顶,具备活跃权,输出"m1 begin"。
- ④ 调用 m2()方法,在栈内存中给 m2()方法分配空间, m2()方法处于栈顶,具备活跃权,输出"m2 begin",继续输出"m2 over"。
- ⑤ m2()方法执行结束,内存释放,弹栈。
- ⑥ m1()方法这时处于栈顶,具备活跃权,输出"m1 over"。
- ⑦ m1()方法执行结束,内存释放,弹栈。
- ⑧ main()方法这时处于栈顶,具备活跃权,输出"main over"。
- ⑨ main()方法执行结束,内存释放,弹栈。
- ⑩ 栈空了,程序结束。

大家是否还记得之前的课程中曾经提到方法体当中的代码是有执行顺序的,必须遵循自上而下的顺序依次逐行执行,当前行代码必须执行结束之后,下一行代码才能执行,不能跳行执行,还记得吗?现在再和栈数据结构一起联系起来思考一下,为什么要采用栈数据结构呢?是不是只有采用这种先进后出的栈数据结构才可以保证代码的执行顺序呢!此时,你是不是感觉程序的设计者在此处设计的非常巧妙呢!

7.3 方法重载/overload (掌握)

关于方法重载是什么,以及怎么进行重载,这些我们目前先不去研究,先来看看以下代码不使用方法重载机制,存在哪些缺点?

1	public static void main(String[] args) {
2	int x1 = 10;
3	int x2 = 20;
4	int retValue1 = sumInt(x1 , x2);
5	System.out.println(x1 + "+" + x2 + "=" + retValue1);
6	long y1 = 10L;
7	long y2 = 20L;
8	long retValue2 = sumLong(y1 , y2);
9	System.out.println(y1 + "+" + y2 + "=" + retValue2);
10	double z1 = 10.0;

```
11    double z2 = 20.0;
12    double retValue3 = sumDouble(z1, z2);
13    System.out.println(z1 + "+" + z2 + "=" + retValue3);
14 }
15 public static int sumInt(int a , int b){
16     return a + b;
17 }
18 public static long sumLong(long a , long b){
19     return a + b;
20 }
21 public static double sumDouble(double a , double b){
22     return a + b;
23 }
```

运行结果如下图所示：

```
10+20=30
10+20=30
10.0+20.0=30.0
```

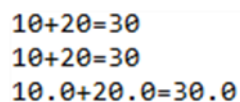
图 7-12：没有重载，分析缺点

我们可以看到以上三个方法功能“相似”，都是求和，只不过参与求和的数据类型不同，因此定义了三个方法，分别起了三个不同的方法名。这种方式会增加程序员编程的压力，因为程序员起码要记忆三个方法名，另外代码也不是很美观。怎么解决呢？我们来看看使用方法重载机制之后会是怎样，请看以下代码以及运行结果：

```
1    public static void main(String[] args) {
2        int x1 = 10;
3        int x2 = 20;
4        int retValue1 = sum(x1 , x2);
5        System.out.println(x1 + "+" + x2 + "=" + retValue1);
```

```
6    long y1 = 10L;
7    long y2 = 20L;
8    long retValue2 = sum(y1 , y2);
9    System.out.println(y1 + "+" + y2 + "=" + retValue2);
10   double z1 = 10.0;
11   double z2 = 20.0;
12   double retValue3 = sum(z1, z2);
13   System.out.println(z1 + "+" + z2 + "=" + retValue3);
14 }
15 public static int sum(int a , int b){
16     return a + b;
17 }
18 public static long sum(long a , long b){
19     return a + b;
20 }
21 public static double sum(double a , double b){
22     return a + b;
23 }
```

运行结果如下图所示：



```
10+20=30
10+20=30
10.0+20.0=30.0
```

图 7-13：使用重载机制

以上代码使用了方法重载机制，我们可以看到，三个“功能相似”的方法名字都叫做 `sum`，只不过方法的形参不同，这样对于程序员来说，调用方法时所需要记忆的方法名更少一些，代码更加美观一些。

接下来我们正式的研究一下方法的重载机制：什么是方法重载？什么情况下我们考虑使用方法重载？在代码角度来看，满足什么条件的时候构成方法重载？

那么，什么是方法重载呢？方法重载（overload）是指在一个类中定义多个同名的方法，但要求每个方法具有不同的参数的类型或参数的个数。调用重载方法时，Java 编译器能通过检查调用的方法的参数类型和个数选择一个恰当的方法。方法重载通常用于创建完成一组任务相似但参数的类型或参数的个数不同的方法。调用方法时通过传递给它们的不同个数和类型的实参来决定具体使用哪个方法。

什么情况下我们考虑使用方法重载呢？在同一个类当中，如果多个功能是相似的，可以考虑将它们的方法名定义的一致，使用方法重载机制，这样便于程序员的调用，以及代码美观，但相反，如果两个方法所完成的功能完全不同，那么方法名也一定要不一样，这样才是合理的。

代码满足什么条件的时候构成方法重载呢？满足以下三个条件：

- ① 在同一个类当中。
- ② 方法名相同。
- ③ 参数列表不同：个数不同算不同，顺序不同算不同，类型不同也算不同。

接下来我们来看看以下程序哪些方法构成了方法重载，哪些没有：

```
1 public class OverloadTest {
2     //方法1
3     public static void doSome(int i){
4
5     }
6     //方法2
7     public static void doSome(long i){
8
9     }
10    //方法3
11    public static void doSome(int i , long j){
12
13    }
14    //方法4
15    public static void doSome(long j, int i){
16
17    }
18    //方法5
19    public static void doSome(long x, int y){
20
21    }
22    //方法6
23    public static int doSome(int i){
24        return 1;
25    }
26    //方法7
27    void doSome(int i){
28
29    }
30 }
```

图 7-14：哪些方法重载了，哪些没有

编译结果如下图所示：

```
OverloadTest.java:19 : 错误: 已在类 OverloadTest02中定义了方法 doSome(long,int)
    public static void doSome(long x, int y){
                        ^
OverloadTest.java:23 : 错误: 已在类 OverloadTest02中定义了方法 doSome(int)
    public static int doSome(int i){
                        ^
OverloadTest.java:27 : 错误: 已在类 OverloadTest02中定义了方法 doSome(int)
    void doSome(int i){
        ^
3 个错误
```

图 7-15: 编译错误信息提示

通过观察以上代码以及测试结果我们得知，方法 5 和方法 4 是一样的，这不是方法重载，这叫“方法重复（哈哈）”，因为之前我们就说过方法形参中起决定性作用的是参数的数据类型，参数的名字随意，因为每一个形参都是局部变量，变量名自然是随意的。其中方法 6 和方法 1 相同，显然方法的重载和方法的返回值类型没有关系，这也是合理的，因为之前我们提过，方法执行结束之后的返回值我们可以接收也可以不接收。另外方法 7 和方法 1 也无法构成重载，显然方法重载和修饰符无关。

总之，方法 1 和方法 2 要想构成方法重载，首先它们在同一个类当中，方法名一样，参数列表不同（类型、个数、顺序），这样 java 虚拟机在运行的时候就可以分清楚去调用哪个方法了。其实，最终要调用哪个方法，还是取决于调用的时候传递的实际参数列表。所以在 java 编程中要区分两个方法，首先看方法名，如果方法名一致，则继续看它们的形式参数列表。

接下来我们来看一下方法重载在实际开发中的应用，你有没有觉得每一次输出的时候“System.out.println();”这行代码很麻烦，我们来封装一个工具类，请看以下代码：

1	public class U {
2	public static void p(){
3	System.out.println();
4	}
5	public static void p(int data){
6	System.out.println(data);
7	}
8	public static void p(long data){

```
9      System.out.println(data);
10   }
11   public static void p(float data){
12       System.out.println(data);
13   }
14   public static void p(double data){
15       System.out.println(data);
16   }
17   public static void p(boolean data){
18       System.out.println(data);
19   }
20   public static void p(char data){
21       System.out.println(data);
22   }
23   public static void p(String data){
24       System.out.println(data);
25   }
26   }
```

```
1   public class UTest {
2       public static void main(String[] args) {
3           U.p("hello world!");
4           U.p(10);
5           U.p(9.0);
6           U.p(false);
7           U.p('国');
8           int a = 10;
9           int b = 20;
10          int c = a + b;
11          U.p(a + "+" + b + "=" + c);
12      }
```

12	}
13	}

运行结果如下图所示：

```
hello world!  
10  
9.0  
false  
国  
10+20=30
```

图 7-16：测试工具类 U

看到以上的代码，你是不是感觉以后要打印数据到控制台就很方便了，代码再也不需要写这么多“System.out.println();”，你只需要“U.p();”，当然，你需要把 U.java 文件编译生成的 U.class 文件拷贝到你的硬盘当中，一直携带着，什么时候需要的话，把 U.class 文件放到 classpath 当中就可以使用了。

7.4 方法递归（理解）

什么是方法递归？我们先来看一段代码：

1	public class RecursionTest01 {
2	public static void main(String[] args) {
3	m();
4	}
5	public static void m(){
6	System.out.println("m begin");
7	m();
8	System.out.println("m over");
9	}
10	}

以上代码的执行结果如下图所示：

```
m begin
m begin
m begin
m begin
m begin
m begin
Exception in thread "main" java.lang.StackOverflowError
    at sun.nio.cs.UTF_8$Encoder.encodeLoop(UTF_8.java:691)
    at java.nio.charset.CharsetEncoder.encode(CharsetEncoder.java:579)
    at sun.nio.cs.StreamEncoder.implWrite(StreamEncoder.java:271)
    at sun.nio.cs.StreamEncoder.write(StreamEncoder.java:125)
```

图 7-17：递归执行结果

我们可以看到以上代码的执行过程中，一直输出“m begin”，“m over”一次也没有输出，直到最终发生了错误：java.lang.StackOverflowError，这个错误是栈内存溢出错误，错误发生后，JVM 退出了，程序结束了。

实际上以上代码在 m()方法执行过程中又调用了 m()方法，方法自身调用自身，这就是方法递归调用。以上程序实际上等同于以下的伪代码（说明问题，但是无法执行的代码）：

```

public class RecursionTest01 {
    public static void main(String[] args) {
        m();
    }
    public static void m(){
        System.out.println("m begin");  输出"m begin"
        m();
        System.out.println("m over");
    }
    public static void m(){
        System.out.println("m begin");  输出"m begin"
        m();
        System.out.println("m over");
    }
    public static void m(){
        System.out.println("m begin");  输出"m begin"
        m();
        System.out.println("m over");
    }
    public static void m(){
        System.out.println("m begin");  输出"m begin"
        m();
        System.out.println("m over");
    }
    ...
}

```

图 7-18: 说明递归执行原理的伪代码

通过伪代码我们可以看出，`m()`方法一直在被调用（方法中的代码必须遵循自上而下的顺序依次逐行执行，不能跳行执行），对于栈内存来说一直在进行压栈操作，`m()`方法从未结束过，所以没有弹栈操作，即使栈内存足够大（也是有限的内存），总有一天栈内存会不够用的，这个时候就会出现栈内存溢出错误。通过以上研究得出递归必须要有合法的结束条件，没有结束条件就一定会发生 `StackOverflowError`。我们再来看看有结束条件的递归，例如以下代码：

```
public class RecursionTest01 {
    public static void main(String[] args) {
        m();
    }
    public static void m(){
        System.out.println("m begin"); (1)输出"m begin"
        m();
        System.out.println("m over"); 输出"m over"(3)
    }
    public static void m(){
        System.out.println("m begin"); (2) 输出"m begin"
        m();
        System.out.println("m over"); 输出"m over"(2)
    }
    public static void m(){
        System.out.println("m begin"); (3) 输出"m begin"
        m();
        System.out.println("m over"); 输出"m over"(1)
    }
    public static void m(){
        if(...){
            return;
        }
    }
}
```

(结束了)

图 7-19：递归的过程中满足了某个条件，递归结束了

综上所述，递归其实就是方法在执行的过程中调用了另一个方法，而另一个方法则是自己本身。在代码角度来看就是在 `a()` 方法中调用 `a()` 方法，使用递归须谨慎，因为递归在使用的时候必须有结束条件，没有结束条件就会导致无终止的压栈，栈内存最终必然会溢出，程序因错误的发生而终止。

大家再来思考一个问题，一个递归程序有合法有效的结束条件就一定不会发生栈内存溢出错误吗？在实际开发中遇到这个错误应该怎么办？

一个递归程序有的时候存在合法有效的终止条件，但由于递归的太深，在还没有等到条件成立的时候，栈内存已经发生了溢出，这种情况也是存在的，所以实际开发中我们尽可能使用循环来代替递归算法，原则是：能不用递归尽量不用，能用循环代替的尽可能使用循环。当然，如果在开发中遇到了由于使用递归导致栈内存溢出错误的发生，首先，我们要检查递归的终止条件是否合法，如果是合法的还是发生栈内存溢出错误，那么我们可以尝试调整堆栈空间的大小。怎么调整堆栈大小呢，大家可以研究一下下图中的一些参数，这里就不再讲解内存大小的调整了，这不是初级程序员应该掌握的。


```
C:\Users\Administrator>java -X
-Xmixed          混合模式执行 <默认>
-Xint            仅解释模式执行
-Xbootclasspath:<用 ; 分隔的目录和 zip/jar 文件>
                  设置搜索路径以引导类和资源
-Xbootclasspath/a:<用 ; 分隔的目录和 zip/jar 文件>
                  附加在引导类路径末尾
-Xbootclasspath/p:<用 ; 分隔的目录和 zip/jar 文件>
                  置于引导类路径之前
-Xdiag           显示附加诊断消息
-Xnoclassgc      禁用类垃圾收集
-Xincgc          启用增量垃圾收集
-Xloggc:<file>   将 GC 状态记录在文件中 <带时间戳>
-Xbatch          禁用后台编译
-Xms<size>       设置初始 Java 堆大小
-Xmx<size>       设置最大 Java 堆大小
-Xss<size>       设置 Java 线程堆栈大小
-Xprof           输出 cpu 配置文件数据
-Xfuture         启用最严格的检查, 预期将来的默认值
-Xrs             减少 Java/VM 对操作系统信号的使用 <请参阅文档>
-Xcheck:jni      对 JNI 函数执行其他检查
-Xshare:off      不尝试使用共享类数据
-Xshare:auto     在可能的情况下使用共享类数据 <默认>
-Xshare:on       要求使用共享类数据, 否则将失败。
-XshowSettings   显示所有设置并继续
-XshowSettings:all
                  显示所有设置并继续
-XshowSettings:vm 显示所有与 vm 相关的设置并继续
-XshowSettings:properties
                  显示所有属性设置并继续
-XshowSettings:locale
                  显示所有与区域设置相关的设置并继续
```

图 7-20: java 虚拟机内存设置参数

接下来我们来研究一下在不使用递归的前提下, 完成 1~N 的求和, 这个应该很简单, 请看下面代码:

1	public class RecursionTest02 {
2	public static void main(String[] args) {
3	int n = 5;
4	int result = accumulate(n);
5	System.out.println("1 到" + n + "的和是: " + result);
6	}


```
7 public static int accumulate(int n){
8     int result = 0;
9     for(int i = 1;i <= n; i++){
10         result += i;
11     }
12     return result;
13 }
14 }
```

运行结果如下图所示：

1到5的和是：15

图 7-21：不使用递归计算 1~N 的和

那么，使用递归应该怎么写呢？请看以下代码：

```
1 public class RecursionTest03 {
2     public static void main(String[] args) {
3         int n = 5;
4         int result = accumulate(n);
5         System.out.println("1 到" + n + "的和是： " + result);
6     }
7     public static int accumulate(int n){
8         if(n == 1){
9             return 1;
10        }
11        return n + accumulate(n - 1);
12    }
13 }
```

运行结果如下图所示：

1到5的和是： 15

图 7-22：使用递归计算 1~N 的和

我们来使用伪代码对以上代码的执行过程进行分析，请看以下伪代码：

1	public static int accumulate(int n){ //假设 n 是 5
2	if(n == 1){
3	return 1;
4	}
5	return n + accumulate(n - 1);
6	//return 5 + accumulate(4); return 5 + 4 + 3 + 2 + 1;
7	}
8	public static int accumulate(int n){
9	if(n == 1){
10	return 1;
11	}
12	return n + accumulate(n - 1);
13	//return 4 + accumulate(3); return 4 + 3 + 2 + 1;
14	}
15	public static int accumulate(int n){
16	if(n == 1){
17	return 1;
18	}
19	return n + accumulate(n - 1);
20	//return 3 + accumulate(2); return 3 + 2 + 1;
21	}

```

22 public static int accumulate(int n){
23     if(n == 1){
24         return 1;
25     }
26     return n + accumulate(n - 1);
27     //return 2 + accumulate(1);           return 2 + 1;
28 }
29 public static int accumulate(int n){
30     if(n == 1){
31         return 1; //这行代码执行了       return 1;
32     }
33 }

```

以上程序的内存变化是这样的，请看下图：

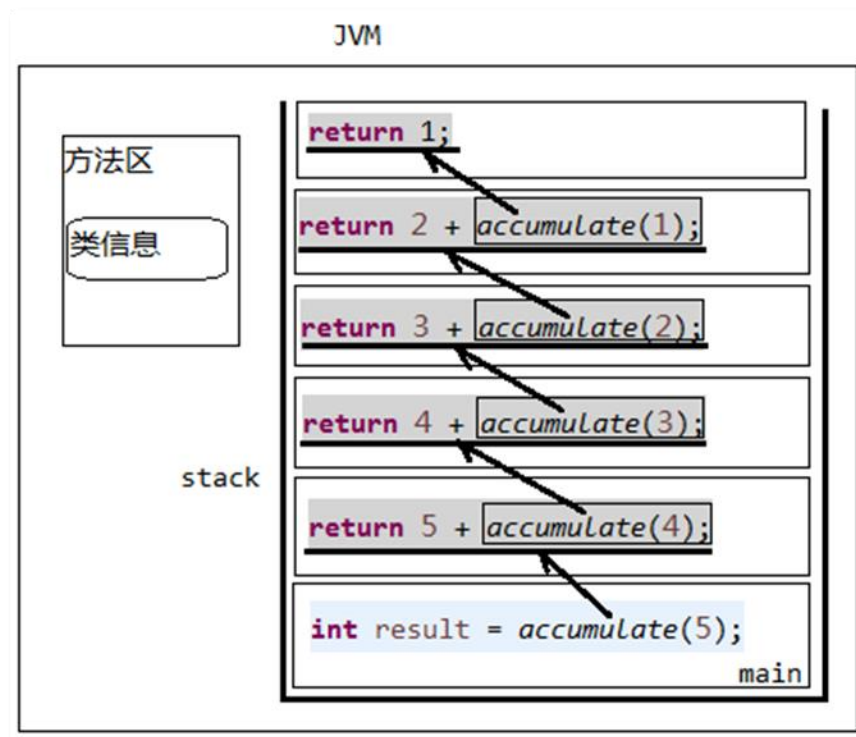


图 7-23：1~N 递归求和内存图

为了加强大家对递归算法的理解，我们再来看一张图：

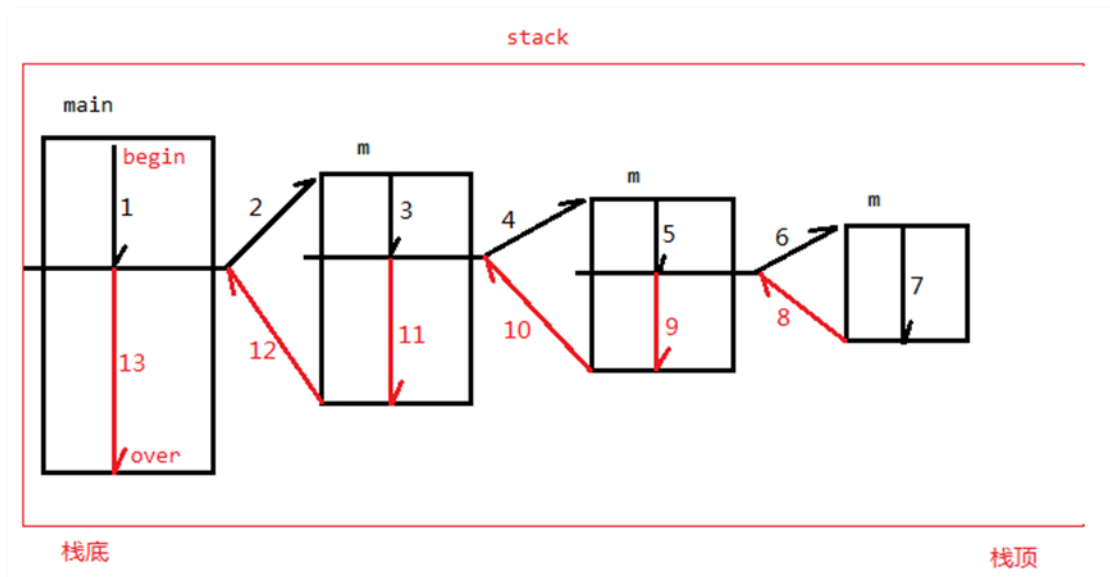


图 7-24：另一种形式的递归内存图

其实大家把上图逆时针旋转 90 度，你会看到一个栈数据结构对吗？

7.5 章节小结

本章节内容主要讲解了 Java 中方法相关的语法机制，其实这个在 C 语言中被称为函数，一般在开发中我们都会把独立的功能单独定义成一个方法，在需要的时候调用就行了，这样可以写过的代码得到重复利用。

本章节中对于方法的作用是需要大家理解的，对于方法的定义与调用是需要大家必须掌握的，另外要求大家对方法的形参和返回值存在的意义有深刻的理解。还有大家要想彻底掌握 Java 程序中的方法，还需要对方法执行过程中内存的变化有深刻的理解。另外还要掌握方法的重载机制，以及方法的递归算法。尤其是递归时的内存变化有助于你对递归执行原理的理解。

7.6 难点解惑

对于本章节内容来说，要求理解的内容较多，其中包括两个难点，第一个难点是：方法执行过程中内存的变化；第二个难点是：递归算法。其实只要对方法执行过程中内存的变化有很深入的理解，所有方法执行过程中内存变化都能画出来，递归算法也就理解了。对于画内存图

在这里嘱咐大家几个关键点，以帮助你掌握方法的内存变化以及对递归的理解：第一个关键点是方法体当中的代码必须遵循自上而下的顺序依次逐行执行，当前行代码不结束，下一行代码是绝对不会执行的；第二个关键点是一定要理解栈数据结构的原理是遵循先进后出、后进先出原则，每当方法调用时分配空间，此时“进”栈，每当方法执行结束时释放空间，此时“出”栈。永远都是最后执行的方法最先结束，最先执行的方法最后结束。

7.7 章节习题

第一题：使用递归方式计算 N 的阶乘。

第二题：编写程序，模拟用户登录功能，程序开始运行时先在 DOS 命令窗口中初始化登录页面，提醒用户输入用户名和密码，当用户输入用户名为 **admin**，密码为 **123** 的时候登录成功，打印欢迎信息，当用户输入的用户名和密码不正确打印错误提示信息并退出系统。对于以上的程序大家尽可能定义相关方法来完成，不要将所有代码都放到 **main** 方法当中。

第三题：通过方法重载、方法重复利用完成以下功能：

- 定义一个方法，该方法可以选出 2 个 **int** 类型较大的数据，返回值是较大的数据。
- 再定义一个方法，该方法可以选出 3 个 **int** 类型中较大的数据，返回值是较大的数据。
- 要求使用方法重载机制，要求代码体现出重复利用。
- **main** 方法中编写程序进行测试。

7.8 习题答案

第一题答案：

1	public class Test {
2	public static void main(String[] args) {
3	int n = 5;
4	int result = factorial(n);
5	System.out.println(n + "的阶乘 = " + result);
6	}
7	
8	public static int factorial(int n) {
9	if(n == 1){

10	return 1;
11	}
12	return n * factorial(n - 1);
13	}
14	}

第二题答案:

1	public class Test {
2	public static void main(String[] args) {
3	//初始化界面
4	initUI();
5	}
6	public static void login(String username, String password) {
7	if("admin".equals(username) && "123".equals(password)){
8	System.out.println("登录成功, 欢迎"+username+"回来! ");
9	}else{
10	System.out.println("对不起, 用户名或者密码错误! ");
11	}
12	}
13	public static void initUI() {
14	java.util.Scanner s = new java.util.Scanner(System.in);
15	System.out.println("欢迎使用本系统, 请登录! ");
16	System.out.print("用户名: ");
17	String username = s.next();
18	System.out.print("密码: ");
19	String password = s.next();
20	//登录
21	login(username, password);
22	}

23	}
----	---

第三题答案:

1	public class Test {
2	public static void main(String[] args) {
3	System.out.println(getBiger(10, 20));
4	System.out.println(getBiger(5,6,4));
5	}
6	public static int getBiger(int a , int b){
7	return a > b ? a : b;
8	}
9	public static int getBiger(int a , int b , int c){
10	return getBiger(a , b) > c ? getBiger(a , b) : c;
11	}
12	}

8. 第八章 认识面向对象

8.1 章节目标与知识框架

8.1.1 章节目标

了解面向对象，知道类和对象的区别，会进行类的定义。

8.1.2 知识框架



8.2 面向过程和面向对象的区别（了解）

“面向过程”(Procedure Oriented)是一种以过程为中心的编程思想，简称 OP。“面向过程”也可称之为“面向记录”编程思想，就是分析出解决问题所需要的步骤，然后用函数把这些步骤一步一步实现，使用的时候一个一个依次调用就可以了。所以面向过程的编程方式关注点不在“事物”上，而是做这件事分几步，先做什么，后做什么。例如：早晨起来：起床、穿衣、洗漱、上班，只要按照这个步骤来，就能实现“一天”的功能，整个这个过程中关注的是一步一步怎么做，并没有关注“人”这个事物。再例如：开门、调整座椅、系好安全带、踩离合、启动、挂档、给油，只要按照这个步骤来，车就走了，显然关注点还是在步骤上，只要实现每一步就行，整个过程并没有关注“汽车”这个事物。

“面向对象”(Object Oriented)是一种以对象为中心的编程思想，简称 OO。随着计算机技术的不断提高，计算机被用于解决越来越复杂的问题。一切事物皆对象，通过面向对象的方式，将现实世界的事物抽象成对象。通过面向对象的方法，更利于用人理解的方式对复杂系统进行分析、设计与编程。同时，面向对象能有效提高编程的效率，通过封装技术，可以像搭积木的一样快速开发出一个全新的系统。面向对象将对象作为程序的基本单元，将程序和数据封装其中，以提高软件的重用性、灵活性和扩展性。

使用面向对象编程思想开发系统，在现代开发中会将面向对象贯穿整个过程，一般包括：OOA/OOD/OOP：

- ① OOA：面向对象分析（Object-Oriented Analysis）
- ② OOD：面向对象设计（Object-Oriented Design）
- ③ OOP：面向对象编程（Object-Oriented Programming）

面向过程和面向对象有什么关系呢？面向过程其实是最为实际的一种思考方式，就算是面向对象的方法也是含有面向过程的思想。可以说面向过程是一种基础的方法。它考虑的是实际

地实现。一般的面向过程是从上往下步步求精。面向对象主要是把事物给对象化，对象包括属性与行为。当程序规模不是很大时，面向过程的方法还会体现出一种优势。因为程序的流程很清楚，按着模块与函数的方法可以很好的组织。但对于复杂而庞大的系统来说，面向过程就显得很无力了。

为了帮助大家理解面向过程和面向对象，我们再来设想一个场景，假如说编写一段程序，模拟一个人抽烟的场景，采用面向过程的方式是这样的：买烟->买打火机->找能够抽烟的场合->点燃香烟->开抽，只要按照这个流程一步一步来，就可以实现抽烟场景，采用面向对象的方式关注点就不一样了，我们会想这个场景都有什么事物参与，每个事物应该有什么行为，然后将这些事物组合在一起，来描述这个场景，例如：一个会抽烟的人（对象）+香烟（对象）+打火机（对象）+允许抽烟的场所（对象），将以上4个对象组合在一起，就实现了抽烟场景，其中采用面向对象的方式开发具有很强的扩展力，例如：人这个对象是可以更换的，打火机也是可以更换的，香烟的品牌也是可以更换的，包括抽烟的场合也是可以更换的。如果采用面向过程方式开发，一步依赖另一步，任何一步都不能变化，变化其中一步则整个软件都会受到影响。

网上发现了一篇文章，说了一下 OP 与 OO 的不同，并且打了一个比喻，通俗易懂。有人这么形容 OP 和 OO 的不同：用面向过程的方法写出来的程序是一份蛋炒饭，而用面向对象写出来的程序是一份盖浇饭。所谓盖浇饭，北京叫盖饭，东北叫烩饭，广东叫碟头饭，就是在一碗白米饭上面浇上一份盖菜，你喜欢什么菜，你就浇上什么菜。我觉得这个比喻还是比较贴切的。蛋炒饭制作的细节，我不太清楚，因为我没当过厨师，也不会做饭，但最后的一道工序肯定是把米饭和鸡蛋混在一起炒匀。盖浇饭呢，则是把米饭和盖菜分别做好，你如果要一份红烧肉盖饭呢，就给你浇一份红烧肉；如果要一份青椒土豆盖浇饭，就给浇一份青椒土豆丝。蛋炒饭的好处就是入味均匀，吃起来香。如果恰巧你不爱吃鸡蛋，只爱吃青菜的话，那么唯一的办法就是全部倒掉，重新做一份青菜炒饭了。盖浇饭就没这么多麻烦，你只需要把上面的盖菜拔掉，更换一份盖菜就可以了。盖浇饭的缺点是入味不均，可能没有蛋炒饭那么香。到底是蛋炒饭好还是盖浇饭好呢？其实这类问题都很难回答，非要比个上下高低的话，就必须设定一个场景，否则只能说是各有所长。如果大家都不是美食家，没那么多讲究，那么从饭馆角度来讲的话，做盖浇饭显然比蛋炒饭更有优势，他可以组合出来任意多的组合，而且不会浪费。盖浇饭的好处就是"菜""饭"分离，从而提高了制作盖浇饭的灵活性。饭不满意就换饭，菜不满意换菜。用软件工程的专业术语就是"可维护性"比较好，"饭"和"菜"的耦合度比较低。蛋炒饭将"蛋""饭"搅和在一起，想换"蛋""饭"中任何一种都很困难，耦合度很高，以至于"可维护性"比较差。软件工程追求的目标之一就是可维护性，可维护性主要表现在3个方面：可理解性、可测试性和可修改性。面向对象的好处之一就是显著的改善了软件系统的可维护性。

对于编程语言来说，基于 C 语言的编程是面向过程的，C++只能说一半面向过程一半面向对象，java 语言就是一门完全面向对象的编程语言。有 C++基础的同学，学习 java 应该很快，因为 java 底层是 C++语言实现的。当然，除了 java 语言之外，还有很多都是完全面向对象的

编程语言，例如：C#、Python 等。

对于面向过程和面向对象的理解，目前阶段来说还是很难的，毕竟大家现在还停留在只会定义变量，写个 if 语句阶段，慢慢来吧，我们需要不断的学习后面的内容，然后再加深对面向对象的理解。

8.3 面向对象三大特征（了解）

面向对象具有三大特征，这三大特征目前大家只需要记住，后面我会进行一一讲解：

- ① 封装（Encapsulation）
- ② 继承（Inheritance）
- ③ 多态（Polymorphism）

任何一门面向对象的编程语言都具备以上三大特征，例如：python、C#、java 等。

8.4 类

8.4.1 类和对象的概念（理解）

面向对象之所以能够成为主流，那是因为人习惯以对象的方式认识现实世界，例如我说：老虎。那你大脑中马上呈现出一个老虎的样子，对吧。

软件存在的意义就是为了解决现实世界当中的问题，它必然模拟现实世界，也就是说现实世界中有什么，软件中就对应有什么。

面向对象编程思想中关注点是“对象”或者“事物”，那么在编程语言当中要想创建对象则必须先有类，那么类和对象分别是什么，它们的区别和联系是什么呢？

类是现实世界当中具有共同特征的事物进行抽象形成的模板或概念。而对象是实际存在的个体。例如：“汽车”就是一个类（所有的汽车都有方向盘、发动机、都能形式，这是它们的共同特征），“你家的那个汽车”就是一个真实存在的对象。或者说“明星”是一个类，“刘德华”就是一个对象。“沈腾”、“赵本山”、“宋丹丹”都是实际存在的对象，他们都属于“笑星”类，类描述事物的共同特征，那么“笑星”类都有哪些共同特征呢？笑星类都有姓名、性别、年龄等状态信息（属性），他们还有一个共同的行为就是“演出”（方法）。但当具体到某个对象上之后，我们发现姓名是不同的，性别是不同的，年龄也是不同的，演出的效果也是不同的。所以我们在访问姓名、性别、年龄的时候，必须先有笑星对象，通过真实存在的笑星对象去访问他的属性，包括“演出”的时候，只有“笑星”类是不行的，必须先有笑星对象，让笑星对象去执行“演出”这个动作。

通过类可以创建对象，对象又被称为实例（instance），这个过程也可以称为实例化。对象

1、2、3 具有共同特征，进行抽象形成了类，所以从对象到类称为抽象。如下图所示：

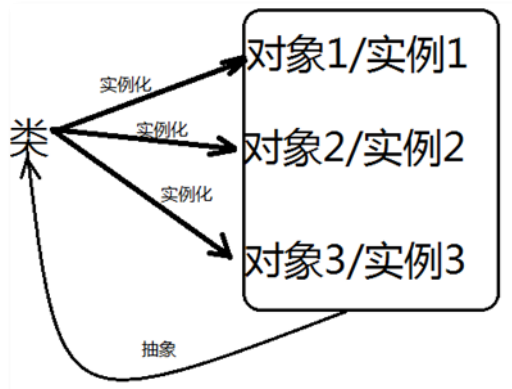


图 8-1：类和对象

通过以上的描述，我们得知：类 = 属性 + 方法，而属性描述的是状态，方法描述的是行为动作。行为动作以方法的形式存在，那属性以什么形式存在呢？例如：姓名、性别、年龄，大家想起之前学习的变量了吗？变量用来存储数据。不错，对象的属性以变量形式存在，并且这里所说的变量是我们之前提过的“成员变量当中的实例变量”。为什么是实例变量呢，实例变量就是对象级别的变量，这样的变量要求必须先存在对象，通过对象才能访问。例如：“中国人”这个类，有一个属性是“身份证号”，每一个中国人的“身份证号”都是不一样的，所以身份证号必须使用一个真实存在的“中国人对象”来访问。不能使用“中国人”这个类去访问身份证号。一个类可以实例化 N 多个对象，假设通过“中国人”这个类创建了 100 个“中国人对象”，那么“身份证号”必然会有 100 个实例变量空间去存储。

8.4.2 类的发现和设计（了解）

理解了类和对象的概念之后，我们开始进行类的设计，那么，应该怎么在现实世界当中发现类呢？例如有这样的背景：开发学生选课系统，要求能够单独对学生信息、课程信息进行维护，还要求能够维护某学生选择某些课程。根据以上的描述，我们可以看到上面的描述中有很多名词，例如：学生、课程等。从这些名词当中就可以发现类，例如：学生类、课程类。所有的学生都有学号、姓名、性别、出生日期等属性，所有的课程都有课程编号、课程名字等属性。

如果我们发现了类，并且发现了类中的属性和方法，那么应该以什么形式展现出来呢，在团队协作开发中应该如何让其他项目组成员知晓你的设计呢，恐怕这个时候就需要使用 UML 了。Unified Modeling Language (UML) 又称统一建模语言或标准建模语言，是始于 1997 年一个 OMG 标准，它是一个支持模型化和软件系统开发的图形化语言，为软件开发的所有阶段提供

模型化和可视化支持，包括由需求分析到规格，到构造和配置。面向对象的分析与设计(OOA&D, OOAD)方法的发展在 80 年代末至 90 年代中出现了一个高潮，UML 是这个高潮的产物。它不仅统一了 Booch、Rumbaugh 和 Jacobson 的表示方法，而且对其作了进一步的发展，并最终统一为大众所接受的标准建模语言。UML 规范用来描述建模的概念有：类、对象、关联、职责、行为、接口、用例、包、顺序、协作，以及状态。

其实软件开发和现实生活当中的建造大楼是一样的，在建造大楼之前需要进行前期的设计，这个时候就需要建筑师画图纸，图纸上画的都是一些符合某些标准的符号，负责建筑的人员一定是能够看懂这些标准符号的。而 UML 就是在软件开发方面的一种图标式语言，程序员在进行系统设计的时候，需要画出 UML 建模图，程序员根据 UML 建模图进行开发。

那么能够实现 UML 图的工具有哪些呢？例如：IBM Rational Rose、PowerDesigner、StarUML、MS Visio 等，我们接下来使用 Rational Rose 工具画一个类出来。请看下图：

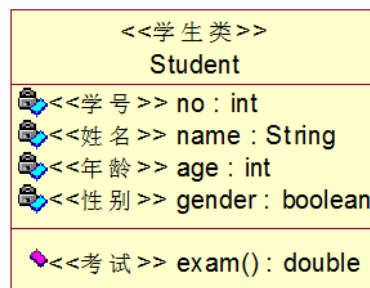


图 8-2：学生类的设计

通过以上类图，我们可以看到一个学生有学号、姓名、年龄、性别属性，并且有一个考试的方法。其中学号采用整数型，姓名采用字符串类型，年龄采用整数型，性别采用布尔型，考试返回值类型设计为浮点型 double。

请大家注意，该小节内容属于了解内容，目前还不需要掌握怎么画这些图，只要知道从名词中发现类，系统开发初期需要进行类的设计，而设计的时候就需要使用 UML 进行建模，了解几个常见的建模工具就行了。

8.4.3 类的定义（掌握）

以上所讲内容使用java语言完全可以实现，因为java语言是一门完全面向对象的编程语言，当然，使用其他面向对象的编程语言也可以实现。当进行了类的设计之后，接下来就可以根据 UML 图进行代码的编写了，在代码级别上实现一个类，类怎么定义呢？

```
[修饰符] class 类名 {  
    类体 = 属性 + 方法  
}
```

以上为类的简单定义，实际上一个完整的类的定义要比以上语法复杂一些，以后再慢慢补充，先从简单的开始。

接下来，根据 UML 图，使用代码将“学生类”进行实现（只实现属性），代码如下图所示：

1	public class Student {
2	//学号
3	int no;
4	//姓名
5	String name;
6	//年龄
7	int age;
8	//性别
9	boolean sex;
10	}

以上程序当中 no、name、age、sex 都是属性，它们都是成员变量中的实例变量，所谓实例变量就是对象级别的变量，这些属性要想访问，必须先创建对象才能访问，不能直接通过类去访问，因为每一个学生的学号都是不一样的。没有学生对象，谈何学号！

8.5 章节小结

通过本章节内容的学习，需要大家理解类是什么，对象是什么，类和对象有什么区别。必须要掌握的是要会定义一个类，代码要会写。

8.6 难点解惑

本章节的难点主要还是在于概念的理解上, 对于一个有 Java 开发经验的程序员来说也许很简单, 但对于一个从未写过程序的人来说, 简直是太抽象了, 尤其是在开发中如何发现类, 如何设计类, 在这里我想说的是, 大家不必过于急躁, 类的发现和设计是需要大家以后不断的在实战中积累的, 通过不断的写项目, 慢慢的对面向对象的理解会越来越深入。

8.7 章节习题

第一题: 设计日期类, 每个日期对象都可以描述年月日信息。

第二题: 设计男人类, 每个男人都有身份证号、姓名、性别、女人。设计女人类, 每个女人都

有身份证号、姓名、性别、男人。

第三题: 设计银行账户类, 每个账户都有账号、密码、余额等信息。

第四题: 设计微信账号类, 每个微信账号都有微信号、手机号、昵称等信息。

8.8 习题答案

第一题答案:

1	public class Date {
2	int year;
3	int month;
4	int day;
5	}

第二题答案:

1	public class Man {
2	String idCard;
3	String name;

4	boolean sex;
5	Woman girl;
6	}

1	public class Woman {
2	String idCard;
3	String name;
4	boolean sex;
5	Man boy;
6	}

第三题答案:

1	public class Account {
2	String actno;
3	String password;
4	double balance;
5	}

第四题答案:

1	public class WebChatAccount {
2	String webChatId;
3	String phone;
4	String nickname;
5	}

9. 第九章 对象的创建和使用

9.1 章节目标与知识框架

9.1.1 章节目标

理解构造方法以及重载机制，通过构造方法可以完成对象的创建，并且能够通过引用访问对象的内存，了解 Java 虚拟机内存管理，能够画出程序执行过程的内存图，并了解空指针异常是如何发生的，以及方法调用时参数是如何传递的。

9.1.2 知识框架



9.2 对象的创建和使用（掌握）

9.2.1 对象的创建

类定义之后，就可以使用类这个“模板”来创造“对象”了，一个类是可以创建多个对象的哦！怎么创建呢，语法是什么？其实语法格式很简单：`new 类名()`，这样就可以完成对象的创建了。俗话说，你想要什么 java 都可以给你，想要啥你就 new 啥。请看下面代码：


```
1 public class StudentTest {
2     public static void main(String[] args) {
3         //创建一个学生对象
4         new Student();
5         //再创建一个学生对象
6         new Student();
7     }
8 }
```

为了使用对象更加方便，建议使用变量接收一下？例如以下代码：

```
1 public class StudentTest {
2     public static void main(String[] args) {
3         //创建一个学生对象
4         Student s1 = new Student();
5         //再创建一个学生对象
6         Student s2 = new Student();
7         //以上代码其实和这行代码差不多
8         int i = 10;
9     }
10 }
```

以上代码最初接触的时候，大家肯定会感觉非常陌生，这也是正常的，`Student s1 = new Student()`实际上和`int i = 10`是类似的，对于`int i = 10`来说，`int`是一种基本数据类型，`i`是变量名，`10`是`int`类型的字面量。那对于`Student s1 = new Student()`来说，其中`Student`是一种引用数据类型，`s1`是变量名，`new Student()`执行之后是一个`Student`类型的对象。

大家要注意了，java 语言当中凡是使用 `class` 关键字定义的类都属于引用数据类型，类名本身就是这种引用数据类型的类型名。

9.2.2 对象的使用

创建了对象之后怎么去访问这个对象的属性呢，或者说学生对象现在有了，怎么去访问他的学号、姓名、性别、年龄等信息呢。请看以下代码：

```
1 public class StudentTest {
2     public static void main(String[] args) {
3         //创建一个学生对象
4         Student s1 = new Student();
5         //再创建一个学生对象
6         Student s2 = new Student();
7         //以上代码其实和这行代码差不多
8         int i = 10;
9
10        int no1 = s1.no;
11        System.out.println("学号: " + no1);
12        String name1 = s1.name;
13        System.out.println("姓名: " + name1);
14        int age1 = s1.age;
15        System.out.println("年龄: " + age1);
16        boolean sex1 = s1.sex;
17        System.out.println("性别: " + sex1);
18
19        int no2 = s2.no;
20        System.out.println("学号: " + no2);
21        String name2 = s2.name;
22        System.out.println("姓名: " + name2);
23        int age2 = s2.age;
24        System.out.println("年龄: " + age2);
25        boolean sex2 = s2.sex;
```

```
26      System.out.println("性别: " + sex2);
27
28      //当然, 也可以不使用 no1,no2 这样的变量接收
29      System.out.println("学号 = " + s1.no);
30      System.out.println("姓名 = " + s1.name);
31      System.out.println("年龄 = " + s1.age);
32      System.out.println("性别 = " + s1.sex);
33
34      System.out.println("学号 = " + s2.no);
35      System.out.println("姓名 = " + s2.name);
36      System.out.println("年龄 = " + s2.age);
37      System.out.println("性别 = " + s2.sex);
38  }
39 }
```

运行结果如下图所示:

```
学号: 0
姓名: null
年龄: 0
性别: false
学号: 0
姓名: null
年龄: 0
性别: false
学号 = 0
姓名 = null
年龄 = 0
性别 = false
学号 = 0
姓名 = null
年龄 = 0
性别 = false
```

图 9-1: 对象的创建和使用

接下来解释一下以上的输出结果，通过以上的 `Student` 类可以创建很多学生对象，假设通过 `Student` 类实例化了两个学生对象，那必然会有两个不同的学号，以上程序中并没有给学号赋值，但是获取了到的学号都是 0，这是怎么回事呢？这是因为在 java 语言当中，当实例变量没有手动赋值，在创建对象的时候，也就是说在 `new` 的时候，系统会对实例变量默认赋值，它们的默认值请参考下表：

数据类型	默认值
byte	0
short	0
int	0
long	0L
float	0.0f
double	0.0
boolean	false
char	\u0000
引用类型	null

9.3 对象创建和使用的深层次解密

9.3.1 java 虚拟机内存管理（理解）

为了更好的理解上面的程序，先来看看 java 虚拟机是如何管理它的内存的，请看下图：

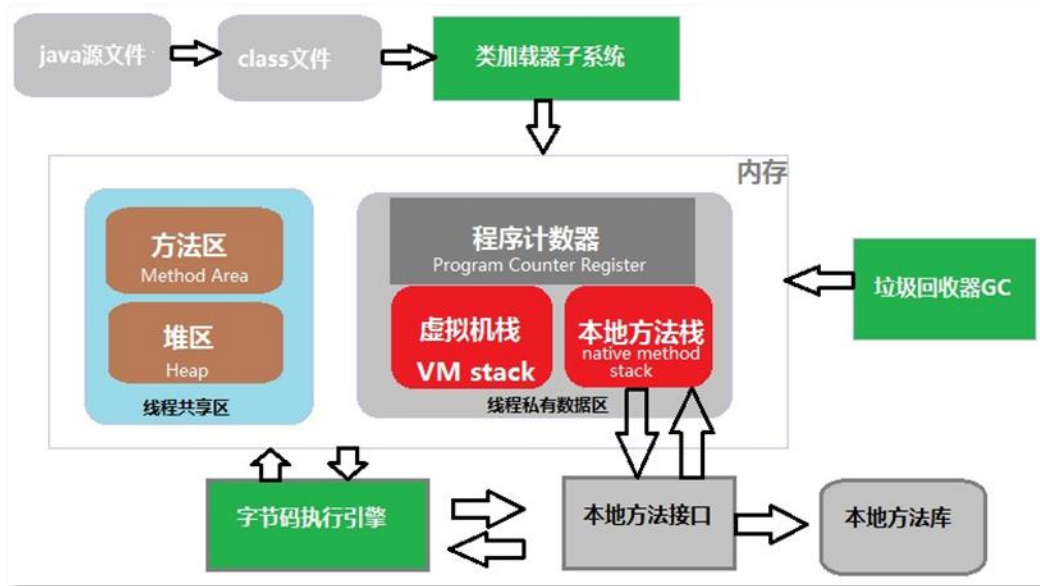


图 9-2: java 虚拟机内存管理

- ① 程序计数器：
 - 1) 概念：可以看做当前线程所执行的字节码的行号指示器。
 - 2) 特点：线程私有的内存
- ② java 虚拟机栈（重点）：
 - 1) 概念：描述的是 java 方法执行的内存模型。（每个方法在执行的时候会创建一个**栈帧**，用于存储**局部变量表**，操作数栈，动态链接，方法出口等信息。每个方法从调用直至完成的过程，就对应一个栈帧从入栈到出栈的过程。）
 - 2) 特点：线程私有，生命周期和线程相同。这个区域会出现两种异常：
 StackOverflowError 异常：若线程请求的深度大于虚拟机所允许的深度。
 OutOfMemoryError 异常：若虚拟机可以动态扩展，如果扩展是无法申请到足够的内存。
- ③ 本地方法栈：
 - 1) 概念：它与虚拟机栈所发挥的作用是相似的，区别是 java 虚拟机栈为执行 java 方法服务，而本地方法栈是为本地方法服务。
 - 2) 特点：线程私有，也会抛出两类异常：StackOverflowError 和 OutOfMemoryError。
- ④ java 堆（重点）：
 - 1) 概念：是被所有线程共享的一块区域，在虚拟机启动时创建。
 - 2) 特点：线程共享，存放的是**对象实例**（所有的对象实例和数组），GC 管理的主要区域。可以处于物理上不连续的内存空间。
- ⑤ 方法区（重点）：
 - 1) 概念：存储已被虚拟机加载的**类信息**、**常量**、**静态变量**，即时编译器编译后的代码等数据。

- 2) 特点：线程共享的区域，抛出异常 OutOfMemory 异常：当方法区无法满足内存分配需求的时候。

以上所描述内容，有看得懂的，也有看不懂的，例如：线程、本地方法等，这个需要大家在学习后面内容之后，返回来再看一看，那个时候你就全部明白了。针对于目前来说，大家必须要知道 java 虚拟机有三块主要的内存空间，分别是“虚拟机栈（后面简称栈）”、“方法区”、“堆区”，方法区存储类的信息，栈中存储方法执行时的栈帧以及局部变量，堆区中主要存储 new 出来的对象，以及对象内部的实例变量。其中垃圾回收器主要针对的是堆内存，方法区中最先有数据，因为程序执行之前会先进行类加载。栈内存活动最频繁，因为方法不断的执行并结束，不断的进行压栈弹栈操作。将目前阶段需要掌握的内存空间使用一张简单的图表示出来，这个图是大家需要掌握的：

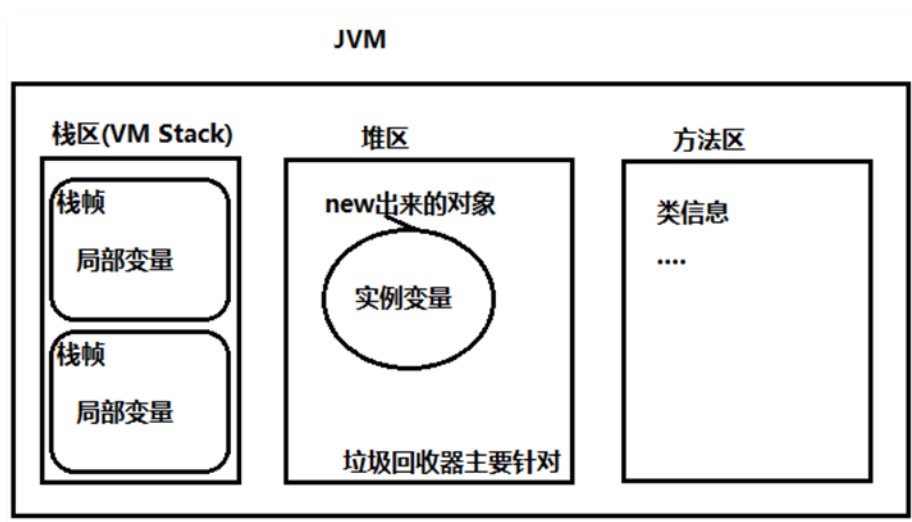


图 9-3: java 虚拟机内存管理简图

大概了解了 java 虚拟机内存分配之后，来看看以下代码在执行过程中，内存是如何变化的：

```

1 public class StudentTest {
2     public static void main(String[] args) {
3         int i = 10;
4         Student s1 = new Student();
5     }
6 }

```

以上代码在执行过程中内存的变化如下图所示：

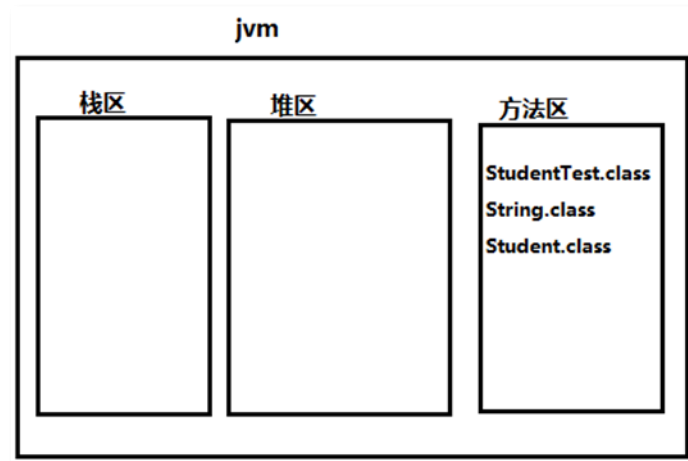


图 9-4：第一步进行类加载

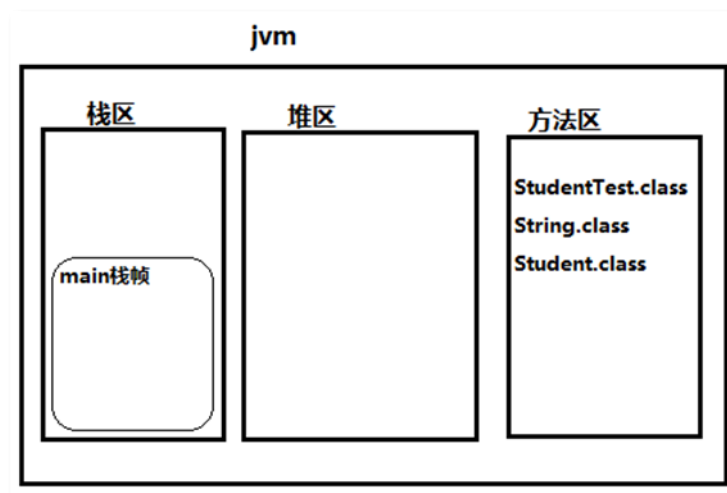


图 9-5：第二步 main 方法调用，给 main 方法分配栈帧（压栈）

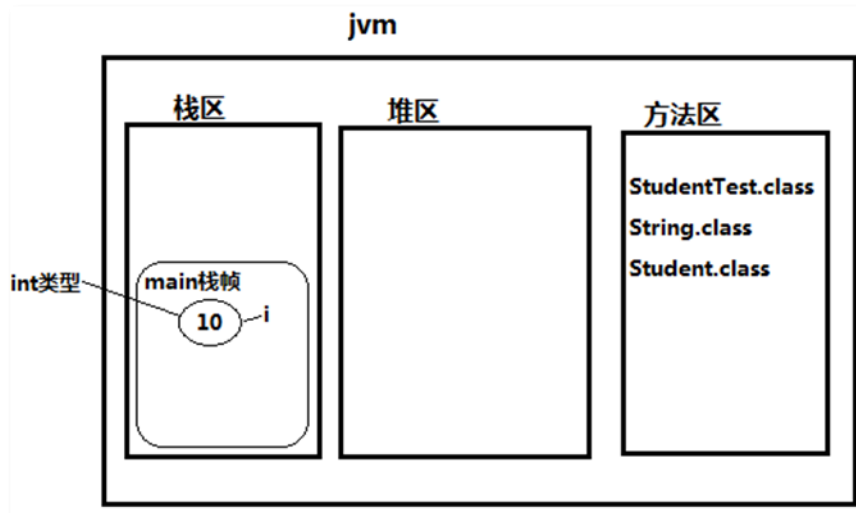


图 9-6：第三步执行 `int i = 10`，局部变量

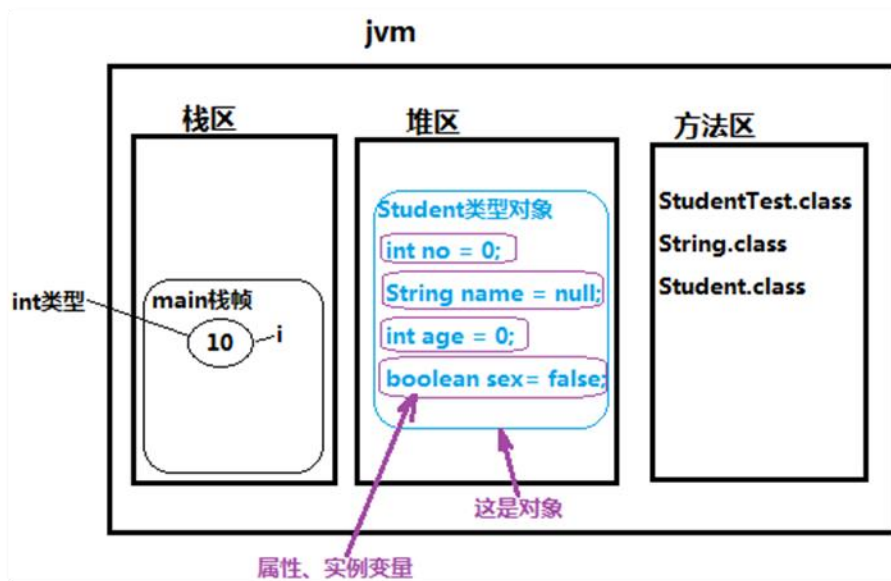


图 9-7：第四步执行 `new Student()`，在堆中创建对象，同时初始化实例变量

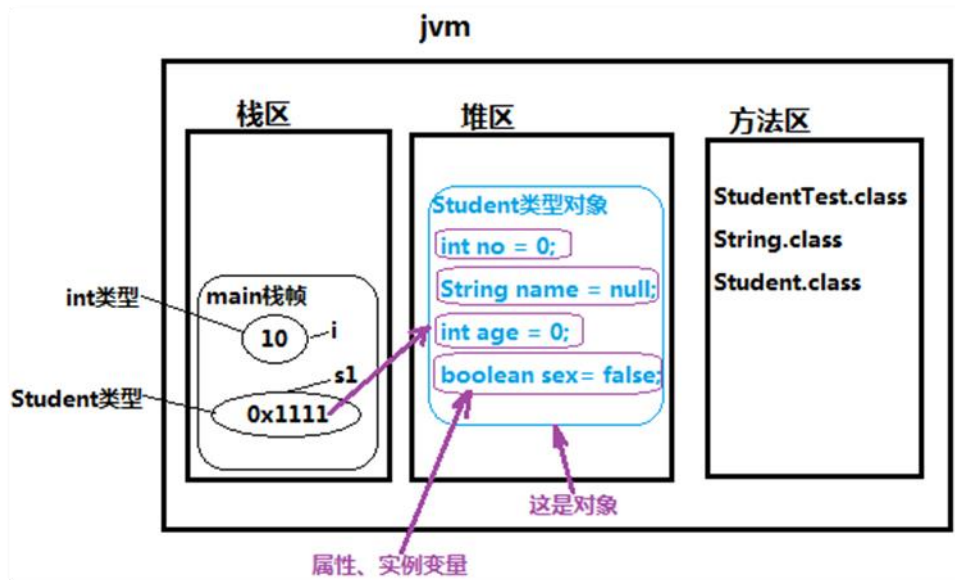


图 9-8：第五步将堆区中学生对象的内存地址赋值给局部变量 s1

注意：上图所描述内存图有些地方帮助大家更好的理解，有些位置画的不是很精确，随着后面内容的学习我们再进行进一步修改，目前上图已经够大家用了。

上图中 **i** 变量和 **s1** 变量都是局部变量，都在栈内存当中，只不过 **i** 变量是基本数据类型 **int**，而 **s1** 变量是引用数据类型 **Student**。

上图中堆区当中的称为“对象”，该“对象”内部 **no**、**name**、**age**、**sex** 都是实例变量/属性，这些变量在 **new** 对象的时候初始化，如果没有手动赋值，系统会赋默认值。

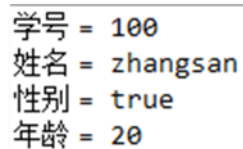
上图堆区中“对象”创建完成之后，该对象在堆区当中的内存地址是：**0x1111**，程序中的“=”将 **0x1111** 这个堆内存地址赋值给 **s1** 变量，也就是说 **s1** 变量保存了堆内存对象的内存地址，我们对于这种变量有一种特殊的称呼，叫做“引用”。也就是说对于 **Student s1 = new Student()** 代码来说，**s1** 不是对象，是一个引用，对象实际上是在堆区当中，**s1** 变量持有这个对象的内存地址。

java 中没有指针的概念（指针是 C 语言当中的机制），所以 java 程序员没有权利直接操作堆内存，只能通过“引用”去访问堆内存中的对象，例如：**s1.no**、**s1.name**、**s1.sex**、**s1.age**。访问一个对象的内存，其实就是访问该对象的实例变量，而访问实例变量通常包括两种形式，要么就是读取数据，要么就是修改数据，例如：**System.out.println(s1.no)** 这就是读取数据，**s1.no = 100** 这就是修改数据。请看以下代码：

1	public class StudentTest {
---	----------------------------

```
2      public static void main(String[] args) {  
3          int i = 10;  
4          Student s1 = new Student();  
5          s1.no = 100;  
6          s1.name = "zhangsan";  
7          s1.sex = true;  
8          s1.age = 20;  
9          System.out.println("学号 = " + s1.no);  
10         System.out.println("姓名 = " + s1.name);  
11         System.out.println("性别 = " + s1.sex);  
12         System.out.println("年龄 = " + s1.age);  
13     }  
14 }
```

运行结果如下所示：



```
学号 = 100  
姓名 = zhangsan  
性别 = true  
年龄 = 20
```

图 9-9：修改实例变量之后的执行结果

执行了以上程序之后，堆内存对象的实例变量发生了变化，如下图所示：

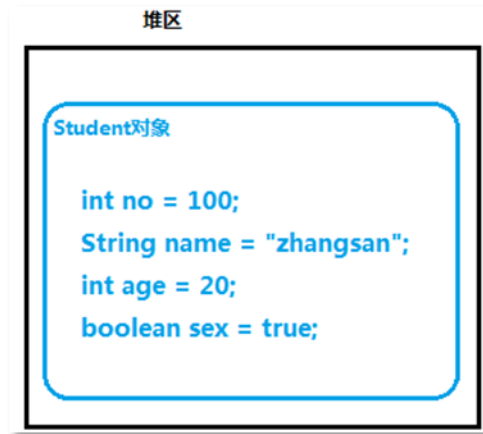


图 9-10：实例变量执行赋值运算之后的内存图

如果基于以上的代码再创建一个对象，内存图会是怎么的呢？先看代码：

```
1 public class StudentTest {
2     public static void main(String[] args) {
3         int i = 10;
4         Student s1 = new Student();
5         Student s2 = new Student();
6     }
7 }
```

JVM 内存结构图如下所示：

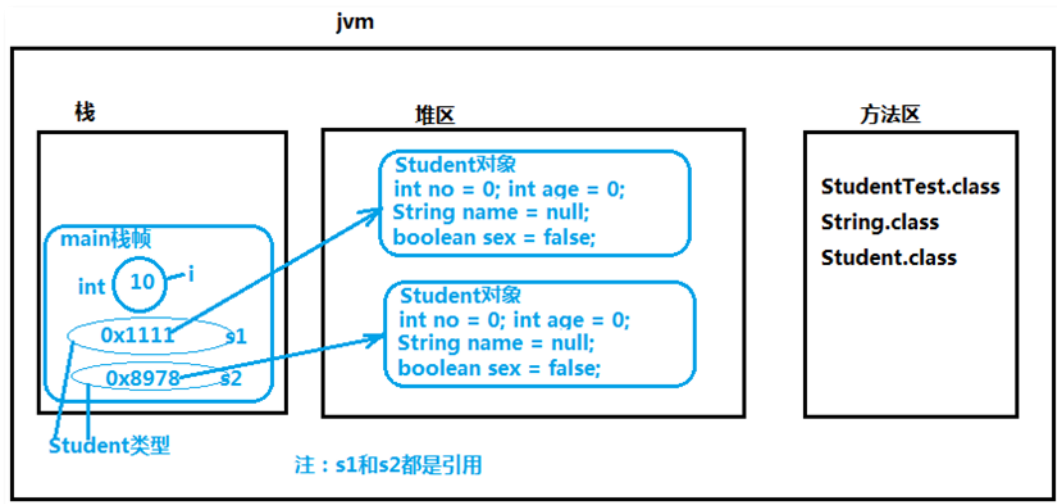


图 9-11: 创建多个对象的内存结构图

通过上图的学习，可以看出假设 new 出 100 个学生对象，会有 100 个 no，100 个 age...是这样吧。

通过以上内容的学习，需要每位同学掌握：局部变量存储在哪里？实例变量存储在哪里？实例变量在什么时候初始化？对象和引用有什么区别？在 java 中怎么访问堆内存当中的对象？这些你都掌握了吗。

9.3.2 构造方法 Constructor（掌握）

什么是构造方法？构造方法怎么定义？构造方法怎么调用？构造方法有什么作用？构造方法可以重载吗？接下来学习一下。

构造方法是类中特殊的方法，通过调用构造方法来完成对象的创建，以及对象属性的初始化操作。

构造方法怎么定义，请看以下的语法格式：

```
[修饰符列表] 构造方法名(形式参数列表){
    构造方法体;
}
```

- ① 构造方法名和类名一致。
- ② 构造方法用来创建对象，以及完成属性初始化操作。
- ③ 构造方法返回值类型不需要写，写上就报错，包括 void 也不能写。
- ④ 构造方法的返回值类型实际上是当前类的类型。
- ⑤ 一个类中可以定义多个构造方法，这些构造方法构成方法重载。

怎么调用构造方法呢，语法格式是：new 构造方法名(实际参数列表);接下来，看以下代码：

```
public class Date {
    int year; //年
    int month; //月
    int day; //日
}

public class DateTest {
    public static void main(String[] args) {
        Date time = new Date();
        System.out.println(time);
        String msg = DateTest.getMessage();
        System.out.println(msg);
    }

    public static String getMessage(){
        return "隔壁老王在练腰";
    }
}
```

这个构造方法在哪儿，怎么看不见

返回值类型
???????

这里调用的是构造方法

普通方法调用

返回值类型

图 9-12：调用无参数构造方法

以上程序运行结果如下图所示：

```
Date@15db9742
隔壁老王在练腰
```

图 9-13：调用无参数构造方法的运行结果

以上程序的输出结果中虽然第一行看不懂，但起码说明程序是能够正常执行的。我们看到以上 Date 类当中的代码，并没有发现任何构造方法，为什么可以调用呢？接下来我们来测试一下，把构造方法显示的定义出来：

1	public class Date {
2	int year; //年
3	int month; //月
4	int day; //日

```
5 //构造方法(无参数构造方法)
6 public Date(){
7     System.out.println("Date 类无参数构造方法执行");
8 }
9 }
```

```
1 public class DateTest {
2     public static void main(String[] args) {
3         System.out.println("main begin");
4         new Date();
5         System.out.println("main over");
6     }
7 }
```

运行结果如下图所示：

```
main begin
Date类无参数构造方法执行
main over
```

图 9-14：测试无参数构造方法

通过以上程序执行结果确实看到了“new Date()”确实调用了 Date 类当中的无参数构造方法。再看以下程序：

```
1 public class Date {
2     int year; //年
3     int month; //月
4     int day; //日
5     //构造方法(有参数构造方法)
```

```
6      public Date(int year){
7          System.out.println("带有参数 year 的构造方法");
8      }
9  }
```

```
1  public class DateTest {
2      public static void main(String[] args) {
3          System.out.println("main begin");
4          new Date();
5          System.out.println("main over");
6      }
7  }
```

编译报错了，错误信息如下图所示：

```
C:\>javac DateTest.java
DateTest.java:4: 错误: 无法将类 Date 中的构造器 Date 应用到给定类型;
    new Date();
        ^
需要: int
找到: 没有参数
原因: 实际参数列表和形式参数列表长度不同
1 个错误
```

图 9-15: 编译器错误信息提示

通过以上的测试，得出这样一个结论（这是 java 中语法的规定，记住就行）：当一个类没有显示的定义任何构造方法的时候，系统默认提供无参数构造方法，当显示的定义构造方法之后，系统则不再提供无参数构造方法。无参数构造方法又叫做缺省构造器，或者默认构造方法。一般在开发中为了方便编程，建议程序员手动的将无参数构造方法写上，因为不写无参数构造方法的时候，这个默认的构造方法很有可能就不存在了，另外也是因为无参数构造方法使用的频率较高。例如以下代码：

```
1 public class Date {
2     int year; //年
3     int month; //月
4     int day; //日
5     //构造方法(无参数构造方法)
6     public Date(){
7         System.out.println("Date 类无参数构造方法执行");
8     }
9     //构造方法(有参数构造方法)
10    public Date(int year1){
11        System.out.println("带有参数 year 的构造方法");
12    }
13    //构造方法(有参数构造方法)
14    public Date(int year1 , int month1){
15        System.out.println("带有参数 year,month 的构造方法");
16    }
17    //构造方法(有参数构造方法)
18    public Date(int year1 , int month1 , int day1){
19        System.out.println("带有参数 year,month,day 的构造方法");
20    }
21 }
```

```
1 public class DateTest {
2     public static void main(String[] args) {
3         System.out.println("main begin");
4         new Date();
5         new Date(2008);
6         new Date(2008 , 8);
7         new Date(2008 , 8 , 8);
8         System.out.println("main over");
9     }
10 }
```


9	}
10	}

运行结果如下图所示：

```
main begin
Date类无参数构造方法执行
带有参数year的构造方法
带有参数year,month的构造方法
带有参数year,month,day的构造方法
main over
```

图 9-16：调用所有构造方法

通过以上的测试可以看出一个类当中可以定义多个构造方法，构造方法是支持重载机制的，具体调用哪个构造方法，那要看调用的时候传递的实际参数列表符合哪个构造方法了。构造方法虽然在返回值类型方面不写任何类型，但它执行结束之后实际上会返回该对象在堆内存当中的内存地址，这个时候可以定义变量接收对象的内存地址，这个变量就是之前所学的“引用”，请看以下代码：

1	public class DateTest {
2	public static void main(String[] args) {
3	System.out.println("main begin");
4	Date time1 = new Date();
5	System.out.println(time1);
6	Date time2 = new Date(2008);
7	System.out.println(time2);
8	Date time3 = new Date(2008 , 8);
9	System.out.println(time3);
10	Date time4 = new Date(2008 , 8 , 8);
11	System.out.println(time4);
12	System.out.println("main over");

13	}
14	}

运行结果如下图所示：

```
main begin
Date类无参数构造方法执行
Date@15db9742
带有参数year的构造方法
Date@6d06d69c
带有参数year,month的构造方法
Date@7852e922
带有参数year,month,day的构造方法
Date@4e25154f
main over
```

图 9-17：构造方法的返回值

以上程序中 time1,time2,time3,time4 都是引用，输出这些引用的结果是“Date@xxxxx”，对于这个结果目前可以把它等同看做是对象的内存地址（严格来说不是真实的对象内存地址）。通过这个引用就可以访问对象的内存了，例如以下代码：

1	public class DateTest {
2	public static void main(String[] args) {
3	System.out.println("main begin");
4	Date time1 = new Date();
5	System.out.println(time1.year + "年" + time1.month + "月" + time1.day +
6	"日");
7	Date time2 = new Date(2008);
8	System.out.println(time2.year + "年" + time2.month + "月" + time2.day +
9	"日");
10	Date time3 = new Date(2008, 8);
11	System.out.println(time3.year + "年" + time3.month + "月" + time3.day +

```
12  "日");
13      Date time4 = new Date(2008, 8, 8);
14      System.out.println(time4.year + "年" + time4.month + "月" + time4.day +
15  "日");
16      System.out.println("main over");
17  }
18 }
```

运行结果如下图所示：

```
main begin
Date类无参数构造方法执行
0年0月0日
带有参数year的构造方法
0年0月0日
带有参数year,month的构造方法
0年0月0日
带有参数year,month,day的构造方法
0年0月0日
main over
```

图 9-18：通过“引用”访问属性

为什么无论通过哪个构造方法创建 Date 对象，最终的结果都是“0 年 0 月 0 日”呢？这是因为所有的构造方法在执行过程中没有给对象的属性手动赋值，系统则自动赋默认值，实际上大部分情况下我们需要在构造方法中手动的给属性赋值，这本来就是构造方法的主要的职责，要不然重载多次构造方法就没有意义了，以上的代码应该这样写，请看：

```
1  public class Date {
2      int year; //年
3      int month; //月
4      int day; //日
5      public Date(){
6      }
```

```
7      public Date(int year1){
8          year = year1;
9      }
10     public Date(int year1 , int month1){
11         year = year1;
12         month = month1;
13     }
14     public Date(int year1 , int month1 , int day1){
15         year = year1;
16         month = month1;
17         day = day1;
18     }
19 }
```

```
1  public class DateTest {
2      public static void main(String[] args) {
3          System.out.println("main begin");
4          Date time1 = new Date();
5          System.out.println(time1.year + "年" + time1.month + "月" + time1.day +
6 "日");
7          Date time2 = new Date(2008);
8          System.out.println(time2.year + "年" + time2.month + "月" + time2.day +
9 "日");
10         Date time3 = new Date(2008 , 8);
11         System.out.println(time3.year + "年" + time3.month + "月" + time3.day +
12 "日");
13         Date time4 = new Date(2008 , 8 , 8);
14         System.out.println(time4.year + "年" + time4.month + "月" + time4.day +
15 "日");
16         System.out.println("main over");
17     }
18 }
```

17	}
18	}

运行结果如下图所示：

```
main begin
0年0月0日
2008年0月0日
2008年8月0日
2008年8月8日
main over
```

图 9-19：通过构造方法给属性赋值

为什么第一个日期输出的是“0 年 0 月 0 日”？这是因为调用的是无参数构造方法创建的第一个日期对象，在无参数构造方法中没有给属性赋值，则系统赋默认值，所以年月日都是 0。那为什么第二个日期输出的是“2008 年 0 月 0 日”呢？这是因为调用的是“`public Date(int year1){ year = year1; }`”构造方法创建的第二个日期对象，在这个构造方法当中只是显示的给 Date 对象的 year 属性赋值，month 和 day 仍然是系统赋默认值，所以 month 和 day 都是 0。第三个日期对象是“2008 年 8 月 8 日”，这是因为在这个对应的构造方法中显示的为 year,month,day 三个属性都赋值了。在编写以上构造方法的时候需要注意变量名的问题，请看下图：

这样写可以：

```
public Date(int year1 , int month1 , int day1){
    year = year1;
    month = month1;
    day = day1;
}

public Date(int a , int b , int c){
    year = a;
    month = b;
    day = c;
}
```

这样写不行：java遵循就近原则

```
public Date(int year , int month , int day){
    year = year;
    month = month;
    day = day;
}
```

这里的两个year都是局部变量year，和属性year没有关系，month和day也是。

图 9-20： java 遵循就近原则

通过以上内容的学习得知，构造方法的作用是专门用来创建对象同时给属性赋值的，它的语法很简单，比普通方法还要简单，因为构造方法名和类名一致，还不需要写返回值类型，使用 `new` 就可以调用了。在一个类当中可以同时定义多个构造方法，它们之间构成重载关系。这样就做到了在 `java` 中你想要什么就 `new` 什么，每一次 `new` 都会在堆内存中创建对象，并且对象内部的实例变量被初始化了。

一定要注意，实例变量没有手动赋值的时候系统会默认赋值，但不管是手动赋值还是系统赋默认值，都是在构造方法执行的时候才会进行赋值操作，类加载的时候并不会初始化实例变量的空间，那是因为实例变量是对象级别的变量，没有对象，哪来实例变量，这也是为什么实例变量不能采用“类名”去访问的原因。

1	<code>public class DateTest {</code>
2	<code> public static void main(String[] args) {</code>

3	System.out.println(Date.year);
4	}
5	}

编译报错了：

```
C:\>javac DateTest.java
DateTest.java:3: 错误: 无法从静态上下文中引用非静态 变量 year
    System.out.println(Date.year);
                        ^
1 个错误
```

图 9-21：实例变量不能直接采用“类名”访问

9.3.3 空指针异常（掌握）

当一个空的引用去访问实例变量会出现什么问题吗？请看以下代码：

1	public class Balloon {
2	//颜色
3	String color;
4	//气体
5	String gas;
6	//构造方法
7	public Balloon(){
8	
9	}
10	public Balloon(String_color , String_gas){
11	color = _color;
12	gas = _gas;
13	}

14 }

```
1 public class BalloonTest {  
2     public static void main(String[] args) {  
3         Balloon ball = new Balloon("红色", "氢气");  
4         System.out.println("气球颜色是: "+ ball.color);  
5         System.out.println("气球中的气体是: "+ ball.gas);  
6         ball = null;  
7  
8         //空指针异常  
9         System.out.println("气球颜色是: "+ ball.color);  
10    }  
11 }
```

运行结果如下图所示:

```
气球颜色是: 红色  
气球中的气体是: 氢气  
Exception in thread "main" java.lang.NullPointerException  
    at com.bjpowernode.operator.BalloonTest.main(BalloonTest.java:9)
```

图 9-22: 空指针异常演示

java.lang.NullPointerException 被称为空指针异常, 在java 编程当中属于很常见的异常, 接下来研究一下以上程序执行过程的内存图是如何变化的。请看下图:

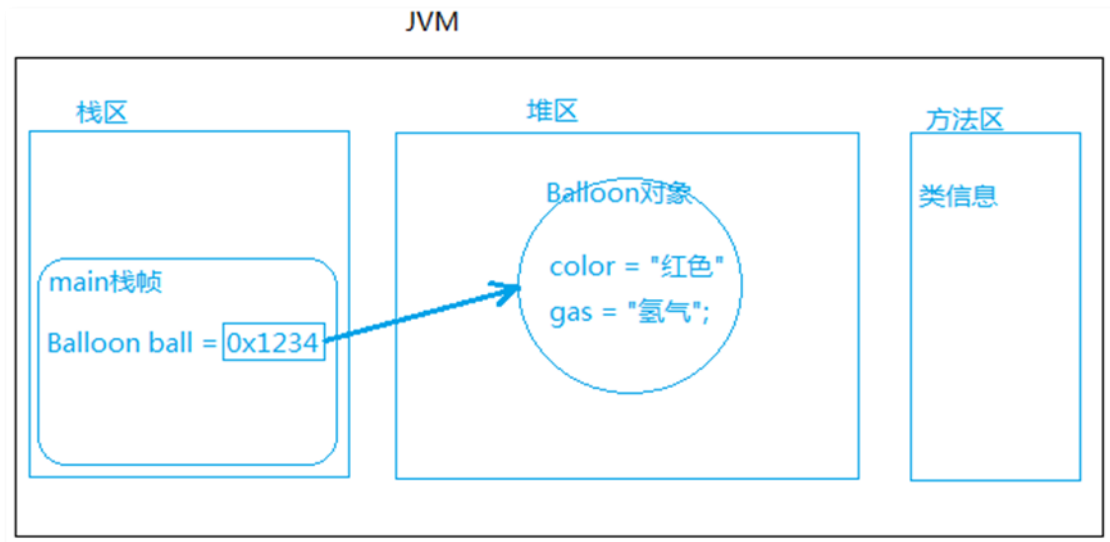


图 9-23: `Balloon ball = new Balloon("红色" , "氢气");`

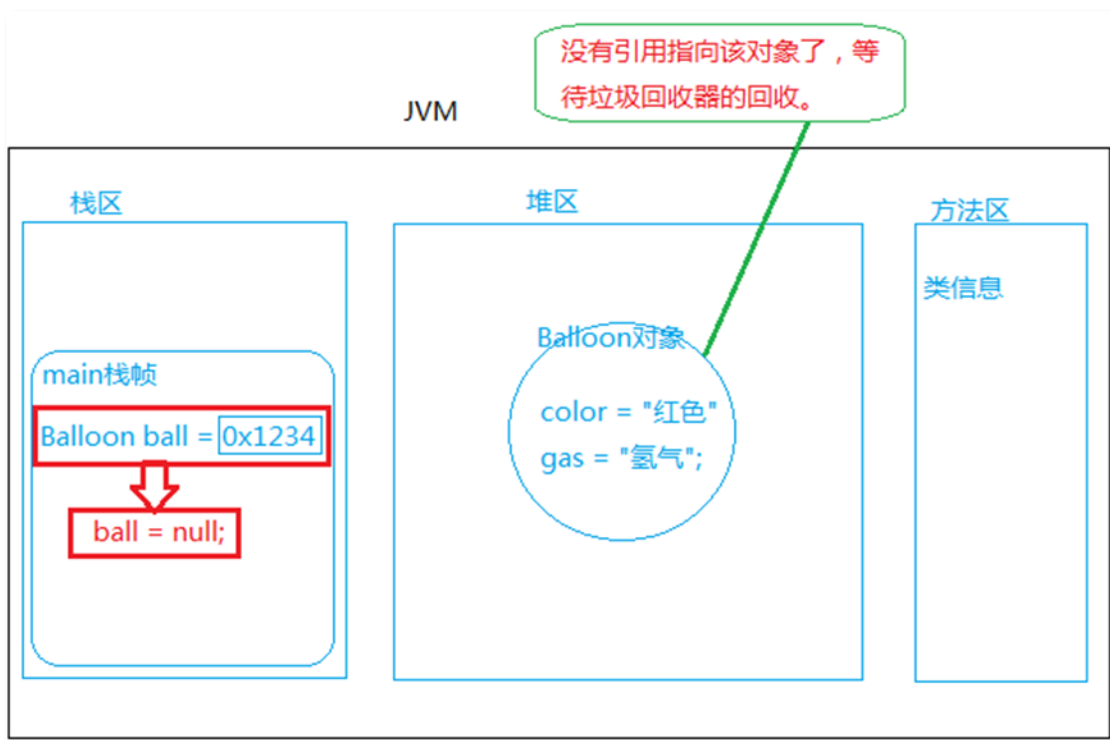


图 9-24: `ball = null;`

以上程序语法正确，编译通过，因为程序在编译阶段检测出“引用 ball”属于 Balloon 类

型，在 Balloon 类中有 color 属性，所以编译器允许通过 ball 引用去访问 color 属性，例如以上代码的 ball.color。但是程序在运行阶段会通过 ball 引用查找堆内存当中的对象，因为 color 是实例变量，该变量存储在 java 对象内部，当 ball = null 执行之后表示“引用 ball”不再保存 java 对象的内存地址，换句话说通过 ball 引用已经无法找到堆内存当中的 java 对象了，对于程序来说这个时候就没有办法正常访问了，这种情况下就会发生空指针异常。就好比一个小孩儿放风筝，通过拽线来操控风筝，结果线断了，再拽风筝线的时候，已经无法再操控风筝了，这对于小孩儿来说是一种异常。而 java 程序中把这种异常叫做 NullPointerException。

总之，当一个“空的引用”去访问“对象相关/实例相关”数据的时候，此时一定会发生空指针异常。

9.3.4 当实例变量是一个引用（理解）

在以上内容学习的过程当中，其实大家已经接触过实例变量是引用的情况，不知道吧！例如在 Student 学生类当中有一个属性“String name;”，这个属性/实例变量 name 描述的是学生的姓名，name 变量的数据类型是 String 类型，String 类型不属于基本数据类型的范畴，也就是说 String 类型属于引用数据类型，换句话说 String 类型应该对应一个 String.class 文件才对，String 是一个类，和我们自己定义的类没什么区别，是这样吗？一起来看看 JDK 的 java 源代码：

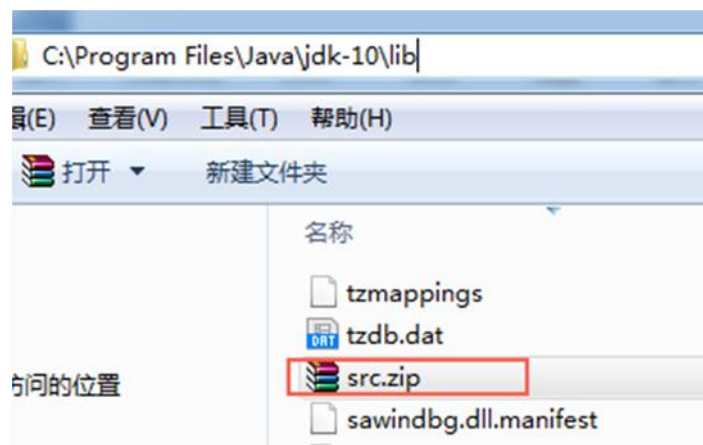


图 9-25: jdk java 源代码位置

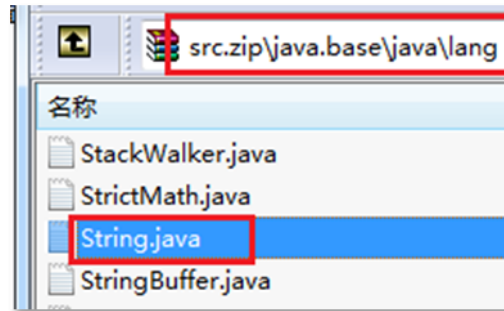


图 9-26: String 类源代码位置

```
public final class String
    implements java.io.Serializable

    /**
     * The value is used for char
     *
     * @implNote This field is to
     * constant folding if String
     * field after construction v
     */
```

图 9-27: String 类源代码

通过查看源代码得知，其实 String 是一个 class，和我们定义的类没有区别，它和基本数据类型还是不一样的（int i = 10，i 变量是基本数据类型，i 变量中存储的就是 10），也就是说 String name = “zhangsang”，实际上 name 变量中存储的并不是”zhangsang”这个字符串，因为 name 是一个引用，那 name 中必然存储的是”zhangsang”字符串对象的内存地址。因为之前我们说过引用的概念，什么是引用：引用就是一个变量，只不过该变量中存储的是 java 对象的内存地址。也就是说，以前所讲内容中使用内存图描述字符串的时候都是有偏差的。我们来修正一下，请看代码：

1	public class Student {
2	int no;
3	String name;
4	int age;
5	boolean sex;

```
6      public Student(int _no,String _name,int _age,boolean _sex){  
7          no = _no;  
8          name = _name;  
9          age = _age;  
10         sex = _sex;  
11     }  
12 }
```

```
1  public class StudentTest {  
2      public static void main(String[] args) {  
3          Student s1 = new Student(110 , "zhangsan" , 20 , true);  
4          System.out.println("学号: " + s1.no);  
5          System.out.println("姓名: " + s1.name);  
6          System.out.println("年龄: " + s1.age);  
7          System.out.println("性别: " + (s1.sex ? "男" : "女"));  
8      }  
9  }
```

以上程序的运行结果请看下图：



```
学号: 110  
姓名: zhangsan  
年龄: 20  
性别: 男
```

图 9-28：运行结果

将以上内存结构图画出来，请看：

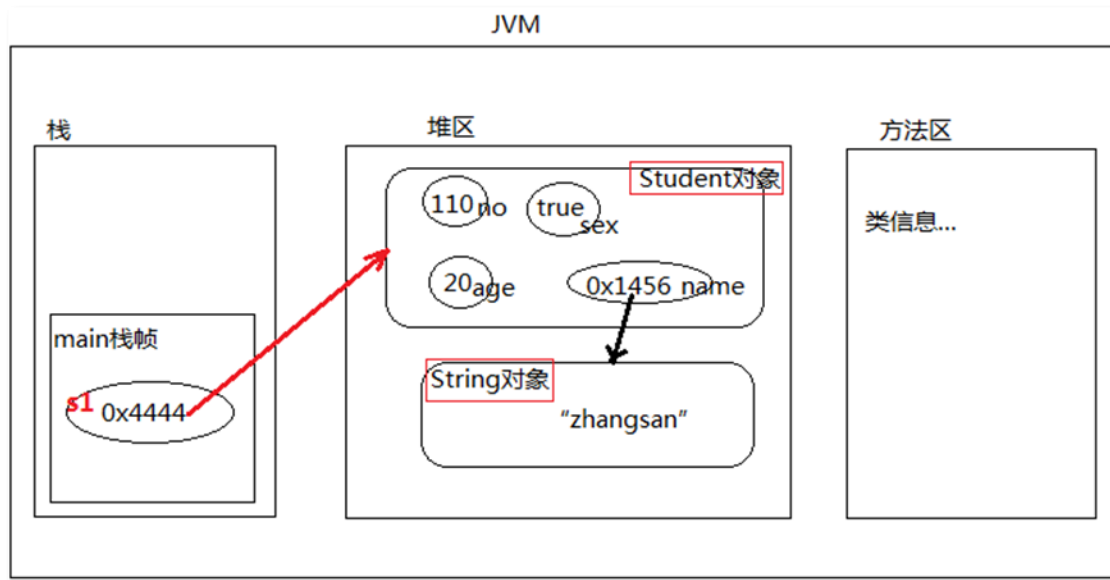


图 9-29：修正字符串对象的内存

通过上图可以看到，Student 对象当中的 name 这个“属性/实例变量”是一个引用，保存的不是“zhangsan”字符串，而是字符串对象的内存地址。（按照实际来说，字符串“zhangsan”是在方法区的字符串常量池当中，这个后期再继续进行修正）。接下来，我们再来看看当属性是其他类型引用的时候，请看代码：

```

1 public class Date {
2     int year;
3     int month;
4     int day;
5     public Date(){
6     }
7     public Date(int year1 , int month1 , int day1){
8         year = year1;
9         month = month1;
10        day = day1;
11    }
12 }
```

```
1 public class Vip {
2     int id;
3     String name;
4     Date birth;
5     public Vip(){
6     }
7     public Vip(int _id,String _name,Date _birth){
8         id = _id;
9         name = _name;
10        birth = _birth;
11    }
12 }
```

```
1 public class VipTest {
2     public static void main(String[] args) {
3         Date d = new Date(1983, 5, 6);
4         Vip v = new Vip(123, "jack", d);
5         System.out.println("编号="+ v.id);
6         System.out.println("姓名="+ v.name);
7         System.out.println("生日="+ v.birth.year + "年" + v.birth.month + "月" +
8 v.birth.day + "日");
9     }
10 }
```

运行结果如下图所示：

```
编号=123  
姓名=jack  
生日=1983年5月6日
```

图 9-30：运行结果

以上程序 main 方法的内存结构图如下所示：

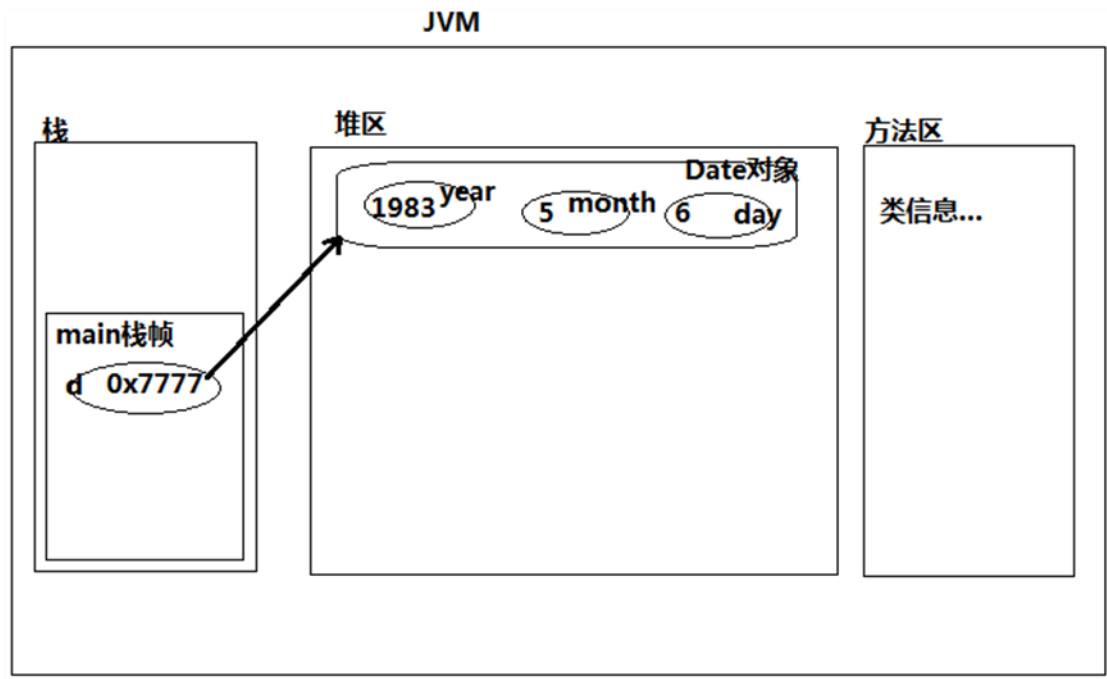


图 9-31：Date d = new Date(1983 , 5 , 6);

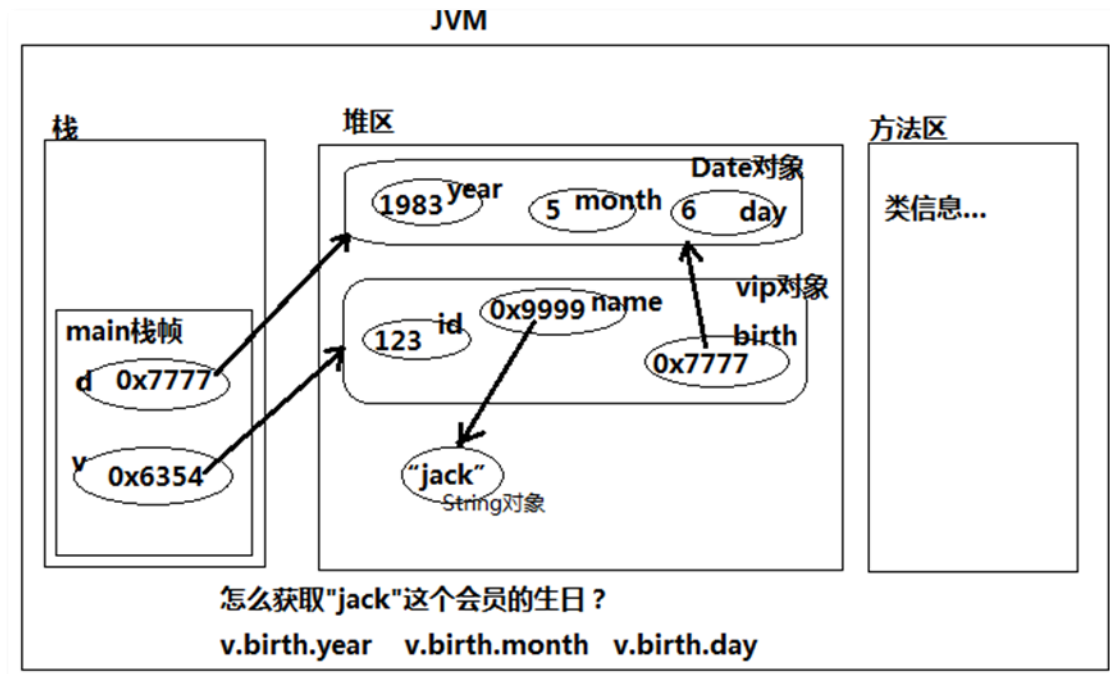


图 9-32: `Vip v = new Vip(123 , "jack" , d);`

通过以上内容的学习大家掌握当对象的属性是一个引用的时候内存图是怎样的了吗？其实只要记住一点，任何“引用”当中存储一定是对象的内存地址，“引用”不一定只是以局部变量的形式存在，例如以上程序，其中 `Vip` 类当中的 `birth` 属性就是一个“引用”，它是一个实例变量。当一个对象的属性是引用的时候应该如何访问这个引用所指向的对象呢？这里其实有一个规律，大家记住就行：类当中有什么就可以“.”什么，例如：`Vip` 类中有 `birth` 属性，那么就可以 `v.birth`，那么 `v.birth` 是 `Date` 类型，`Date` 类当中有 `year` 属性，那么就可以 `v.birth.year`，你懂了吗？

9.3.5 方法调用时参数的传递问题（理解）

方法在调用的时候参数是如何传递的呢？其实在调用的时候参数传递给方法，这个过程就是赋值的过程，参数传递和“赋值规则”完全相同，只不过参数传递在代码上看不见“=”运算符。我们先来深入的研究一下“赋值规则”吧！

1	<code>public class AssignmentTest {</code>
2	<code> public static void main(String[] args) {</code>
3	<code> //基本数据类型</code>


```
4      int a = 10;
5      int b = a; //a 赋值给 b, a 把什么给了 b?
6      //引用数据类型
7      Bird bird1 = new Bird("polly");
8      //bird1 赋值给 bird2, bird1 把什么给了 bird2?
9      Bird bird2 = bird1;
10     }
11 }
```

```
1 class Bird {
2     String name;
3     public Bird(){}
4     public Bird(String _name){
5         name = _name;
6     }
7 }
```

在以上程序当中,有两个疑问,第一个:**a** 赋值给 **b**, **a** 把什么给了 **b**? 第二个:**bird1** 赋值给 **bird2**, **bird1** 把什么给了 **bird2**?

其实 **a**, **b**, **bird1**, **bird2** 就是 4 个普通的变量,唯一的区别只是 **a** 和 **b** 都是基本数据类型的变量, **bird1** 和 **bird2** 都是引用数据类型的变量(或者说都是引用), **a** 变量中保存的那个“值”是 **10**, **bird1** 变量中保存的那个“值”是 **0x8888**(java 对象内存地址),本质上来说 **10** 和 **0x8888** 都是“值”,只不过一个“值”是整数数字,另一个“值”是 java 对象的内存地址,大家不要把内存地址特殊化,它也是一个普通的值。

那么“赋值”是什么意思呢,顾名思义,赋值就是把“值”赋上去。**a** 赋值给 **b**,本质上不是把 **a** 给了 **b**,而是把 **a** 变量中保存的“值 **10**”复制了一份给了 **b**。**bird1** 赋值给 **bird2** 本质上不是把 **bird1** 给了 **bird2**,而是把 **bird1** 变量中保存的“值 **0x8888**”复制了一份给了 **bird2**。请看以下内存图的变化:

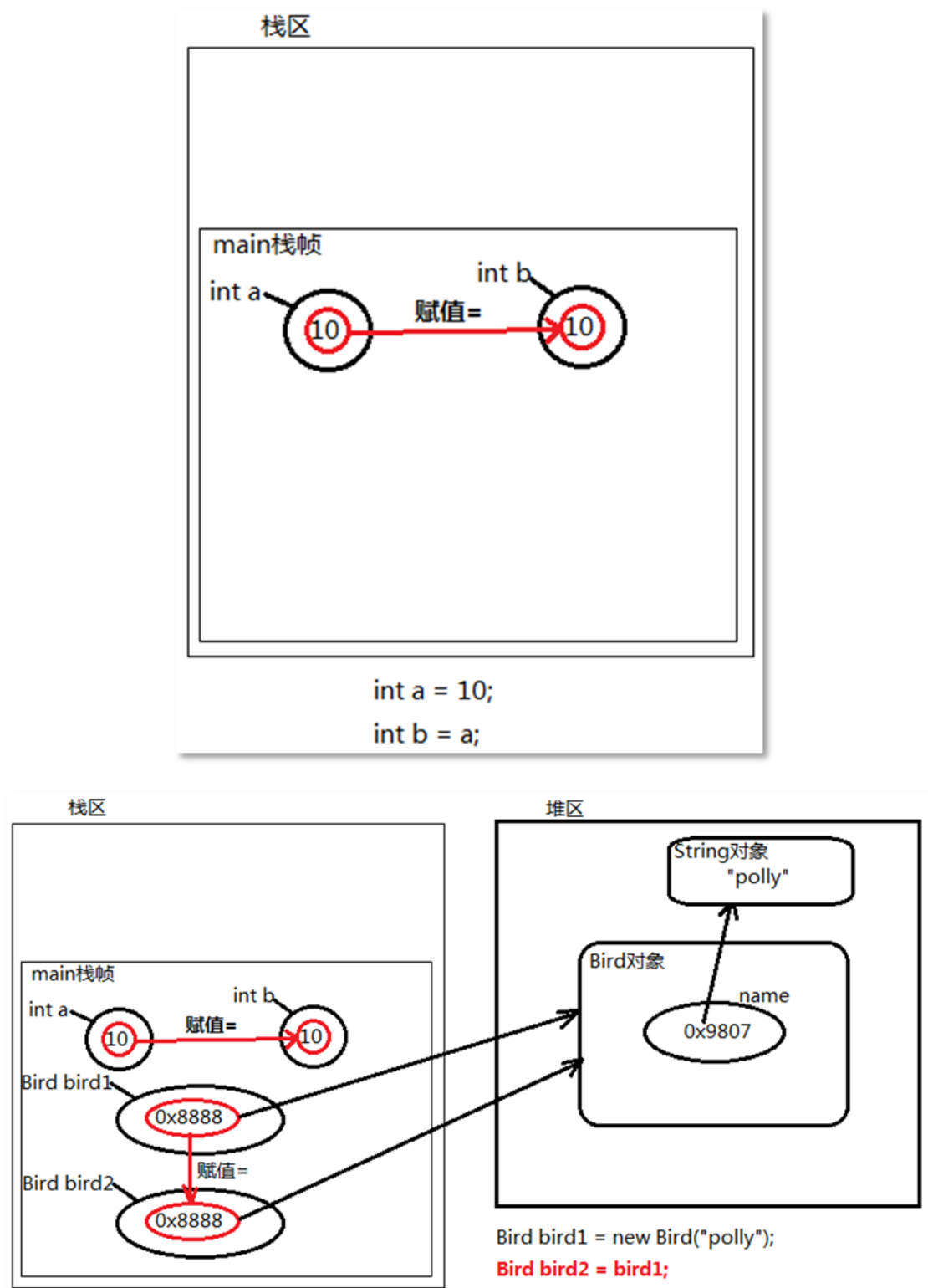


图 9-33: 赋值原理图

通过以上内存图我们可以看出“赋值”运算的时候实际上和变量的数据类型无关，无论是基本数据类型还是引用数据类型，一律都是将变量中保存的“值”复制一份，然后将复制的这个“值”赋上去。他们的区别在于，如果是基本数据类型则和堆内存当中的对象无关，如果是引用数据类型由于传递的这个值是 java 对象的内存地址，所以会导致两个引用指向同一个堆内存中的 java 对象，通过任何一个引用去访问堆内存当中的对象，此对象内存都会受到影响。我们来验证一下，让 a++，a 应该变成了 11，但是 b 不会变，让 bird1.name = “波利”，然后输出 bird2.name 的结果肯定也是”波利”，请看代码以及运行结果：

1	public class AssignmentTest {
2	public static void main(String[] args) {
3	//基本数据类型
4	int a = 10;
5	int b = a; //a 赋值给 b，a 把什么给了 b?
6	a++;
7	System.out.println("a = " + a);
8	System.out.println("b = " + b);
9	//引用数据类型
10	Bird bird1 = new Bird("polly");
11	//bird1 赋值给 bird2，bird1 把什么给了 bird2?
12	Bird bird2 = bird1;
13	System.out.println("bird1's name = " + bird1.name);
14	System.out.println("bird2's name = " + bird2.name);
15	bird1.name = "波利";
16	System.out.println("bird1's name = " + bird1.name);
17	System.out.println("bird2's name = " + bird2.name);
18	}
19	}

1	class Bird {
2	String name;

```
3    public Bird(){  
4    public Bird(String _name){  
5        name = _name;  
6    }  
7 }
```

运行结果如下图所示：

```
a = 11  
b = 10  
bird1's name = polly  
bird2's name = polly  
bird1's name = 波利  
bird2's name = 波利
```

图 9-34：赋值原理测试

上面我就提到了，方法调用时参数的传递和赋值运算符的原理完全相同，那么请大家根据以上内容所学，画出以下程序的内存图，以及推算它们的执行结果：

```
1  public class ParameterTransferTest01 {  
2      public static void main(String[] args) {  
3          int i = 10;  
4          add(i);  
5          System.out.println("main's i = " + i);  
6      }  
7      public static void add(int i){  
8          i++;  
9          System.out.println("add's i = " + i);  
10     }  
11 }
```

```
1 public class ParameterTransferTest02 {
2     public static void main(String[] args) {
3         User user = new User(20);
4         add(user);
5         System.out.println("main's user age = " + user.age);
6     }
7     public static void add(User user) {
8         user.age++;
9         System.out.println("add's user age = " + user.age);
10    }
11 }
```

```
1 class User{
2     int age;
3     public User(){}
4     public User(int _age){
5         age = _age;
6     }
7 }
```

9.4 章节小结

通过本章节内容的学习，需要大家掌握的是一个类定义好之后，怎么创建对象，对象创建之后怎么去使用这个对象，代码要会写。另外还需要掌握构造方法怎么定义、怎么调用。需要理解构造方法在开发中的作用。

在本章节内容中大家尤其要对 Java 虚拟机内存管理这块要有深入的理解。要知道 Java 虚拟机中的栈和堆各自存储什么，要能够画出程序执行过程中内存的变化。要知道空指针异常是如何发生的，为什么会出现空指针。

除以上所描述之外，大家还需要掌握方法调用时参数是如何传递的，一定要弄明白这个知识点，在以后的开发中会频繁的的进行方法的调用，而方法调用时需要传递参数，参数传递时内

存是如何变化的，弄明白这些有助于你对程序运行结果的把控。

9.5 难点解惑

对于初学者来说，本章节内容是比较艰难的一个章节，主要是因为涉及到程序执行过程中内存的变化。本章节中要想搞明白所有的难点，只需要搞定一点就行，那就是亲手画出每个程序执行时的内存图。

在这里我要提醒大家的有两点：第一先要记住 Java 虚拟机内存管理这块有哪些内存空间，每一个空间中都存储什么；第二代码要遵循一行一行逐行执行，每执行一行则需要在内存方面发生一些变化。只要大家能把握以上两点，对于空指针异常、实例变量为引用类型、方法调用时参数传递问题等迎刃而解。

9.6 章节习题

第一题：定义丈夫类 Husband 和妻子类 Wife，丈夫类的属性包括：身份证号，姓名，出生日期，妻子。妻子类的属性包括：身份证号，姓名，出生日期，丈夫。分别给这两个类提供构造方法（无参数构造方法和有参数构造方法都要提供），编写测试程序，创建丈夫对象，然后再创建妻子对象，丈夫对象关联妻子对象，妻子对象关联丈夫对象，要求能够输出这个“丈夫对象”的妻子的名字，或者能够输出这个“妻子对象”的丈夫的名字。要求能够画出程序执行过程的内存图。并且要求在程序中演示出空指针异常的效果。

9.7 习题答案

第一题答案：

1	public class Husband {
2	String idCard;
3	String name;
4	String birth;
5	Wife w;
6	public Husband() {
7	super();
8	}
9	public Husband(String _idCard, String _name, String _birth) {

```
10         super();
11         this.idCard = _idCard;
12         this.name = _name;
13         this.birth = _birth;
14     }
15 }
```

```
1 public class Wife {
2     String idCard;
3     String name;
4     String birth;
5     Husband h;
6     public Wife() {
7         super();
8     }
9     public Wife(String _idCard, String _name, String _birth) {
10         super();
11         this.idCard = _idCard;
12         this.name = _name;
13         this.birth = _birth;
14     }
15 }
```

```
1 public class Test {
2     public static void main(String[] args) {
3         Husband zhangsan = new Husband("11111", "zhangsan", "2020-10-01");
4         Wife lisi = new Wife("22222", "lisi", "2020-10-02");
5         zhangsan.w = lisi;
6         lisi.h = zhangsan;
```

7	System.out.println(zhangsan.name + "的妻子是" + zhangsan.w.name);
8	System.out.println(lisi.name + "丈夫是" + lisi.h.name);
9	zhangsan.w = null;
10	lisi.h = null;
11	
12	System.out.println(zhangsan.name + "的妻子是" + zhangsan.w.name);
13	System.out.println(lisi.name + "丈夫是" + lisi.h.name);
14	}
15	}

执行结果如下图所示：

```
zhangsan的妻子是lisi  
lisi丈夫是zhangsan  
Exception in thread "main" java.lang.NullPointerException  
    at test.Test.main(Test.java:12)
```

10. 第十章 封装

10.1 章节目标与知识框架

10.1.1 章节目标

理解为什么要进行封装？封装有什么好处？封装的代码怎么实现？

10.1.2 知识框架



10.2 封装（掌握）

10.2.1 封装的理解

封装是面向对象的三大特征之一，什么是封装？封装有什么好处？怎么封装，代码怎么写？这是大家这一章节要学习的内容。

封装从字面上来理解就是包装的意思，专业点就是信息隐藏，是指利用抽象数据类型将数据和基于数据的操作封装在一起，使其构成一个不可分割的独立实体，数据被保护在抽象数据类型的内部，尽可能地隐藏内部的细节，只保留一些对外接口使之与外部发生联系。系统的其他对象只能通过包裹在数据外面的已经授权的操作来与这个封装的对象进行交流和交互。也就是说用户是无需知道对象内部的细节，但可以通过该对象对外提供的接口来访问该对象。

在现实世界当中我们可以看到很多事物都是封装好的，比如“鼠标”，外部有一个壳，将内部的原件封装起来，至于鼠标内部的细节是什么，我们不需要关心，只需要知道鼠标对外提供了左键、右键、滚动滑轮这三个简单的操作。对于用户来说只要知道左键、右键、滚动滑轮都能完成什么功能就行了。为什么鼠标内部的原件要在外部包装一个“壳”呢，起码内部的原件是安全的，不是吗。再如“数码相机”，外部也有一个壳，将内部复杂的结构包装起来，对外提供简单的按键，这样每个人都可以很快的学会照相了，因为它的按键很简单，另外照相机内部精密的原件也受到了壳儿的保护，不容易坏掉。

根据以上的描述，可以得出封装有什么好处呢？封装之后就形成了独立实体，独立实体可以在不同的环境中重复使用，显然封装可以降低程序的耦合度，提高程序的扩展性，以及重用性或复用性，例如“鼠标”可以在 A 电脑上使用，也可以在 B 电脑上使用。另外封装可以隐藏内部实现细节，站在对象外部是看不到内部复杂结构的，对外只提供了简单的安全的操作入口，所以封装之后，实体更安全了。

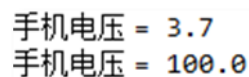
10.2.2 不封装存在的问题

我们来看一段代码，在不进行封装的前提下，存在什么问题：

```
1 public class MobilePhone {  
2     //电压：手机正常电压在 3~5V  
3     double voltage;  
4 }
```

```
1 public class MobilePhoneTest {  
2     public static void main(String[] args) {  
3         MobilePhone phone = new MobilePhone();  
4         phone.voltage = 3.7;  
5         System.out.println("手机电压 = " + phone.voltage);  
6         phone.voltage = 100;  
7         System.out.println("手机电压 = " + phone.voltage);  
8     }  
9 }
```

运行结果如下图所示：



```
手机电压 = 3.7  
手机电压 = 100.0
```

图 10-1：未进行封装的程序测试

以上程序 `MobilePhone` 类未进行封装，其中的电压属性 `voltage` 对外暴露，在外部程序当中可以对 `MobilePhone` 对象的电压 `voltage` 属性进行随意访问，导致了它的不安全，例如手机的正常电压是 3~5V，但是以上程序已经将手机电压设置为 100V，这个时候显然是要出问题的，但这个程序编译以及运行仍然是正常的，没有出现任何问题，这是不对的。

10.2.3 怎么封装

为了保证内部数据的安全，这个时候就需要进行封装了，封装的第一步就是将应该隐藏的数据隐藏起来，起码在外部是无法随意访问这些数据的，怎么隐藏呢？我们可以使用 java 语言中的 `private` 修饰符，`private` 修饰的数据表示私有的，私有的数据只能在本类当中访问。请看程序：

1	<code>public class MobilePhone {</code>
2	<code> //电压：手机正常电压在 3~5V</code>
3	<code> private double voltage;</code>
4	<code>}</code>

1	
2	<code>public class MobilePhoneTest {</code>
3	<code> public static void main(String[] args) {</code>
4	<code> MobilePhone phone = new MobilePhone();</code>
5	<code> phone.voltage = 3.7;</code>
6	<code> System.out.println("手机电压 = " + phone.voltage);</code>
7	<code> phone.voltage = 100;</code>
8	<code> System.out.println("手机电压 = " + phone.voltage);</code>
9	<code> }</code>
10	<code>}</code>

以上程序编译报错了，请看下图：

```
C:\>javac MobilePhoneTest.java
MobilePhoneTest.java:5: 错误: voltage可以在MobilePhone中访问private
    phone.voltage = 3.7;
    ^
MobilePhoneTest.java:6: 错误: voltage可以在MobilePhone中访问private
    System.out.println("手机电压 = " + phone.voltage);
    ^
MobilePhoneTest.java:7: 错误: voltage可以在MobilePhone中访问private
    phone.voltage = 100;
    ^
MobilePhoneTest.java:8: 错误: voltage可以在MobilePhone中访问private
    System.out.println("手机电压 = " + phone.voltage);
    ^
4 个错误
```

图 10-2: private 修饰的数据无法在外部程序中直接访问

通过以上的测试，手机对象的电压属性确实受到了保护，在外部程序中无法访问了。但从当前情况来看，`voltage` 属性有点儿太安全了，一个对象的属性无法被外部程序访问，自然这个数据就没有存在的价值了。所以这个时候就需要进入封装的第二步了：对外提供公开的访问入口，让外部程序统一通过这个入口去访问数据，我们可以在这个入口处设立关卡，进行安全控制，这样对象内部的数据就安全了。

对于“一个”属性来说，我们对外应该提供几个访问入口呢？通常情况下我们访问对象的某个属性，不外乎读取（`get`）和修改（`set`），所以对外提供的访问入口应该有两个，这两个方法通常被称为 `set` 方法和 `get` 方法（请注意：`set` 和 `get` 方法访问的都是某个具体对象的属性，不同的对象调用 `get` 方法获取的属性值不同，所以 `set` 和 `get` 方法必须有对象的存在才能调用，这样的方法定义的时候不能使用 `static` 关键字修饰，被称为实例方法。实例方法必须使用“引用”的方式调用。还记得之前我们接触的方法都是被 `static` 修饰的，这些方法直接采用“类名”的方式调用，而不需要创建对象，在这里显然是不行的）。请看以下代码：

1	<code>public class MobilePhone {</code>
2	
3	<code> //电压：手机正常电压在 3~5V</code>
4	<code> private double voltage;</code>
5	<code> public MobilePhone(){</code>
6	
7	<code> }</code>
8	<code> public void setVoltage(double _voltage){</code>

```
9         if(_voltage < 3 || _voltage > 5){
10             //当电压低于 3V 或者高于 5V 时抛出异常，程序则终止
11             throw new RuntimeException("电压非法，请爱护手机！");
12         }
13         //程序如果能执行到此处说明以上并没有发生异常，电压值合法
14         voltage = _voltage;
15     }
16     public double getVoltage(){
17         return voltage;
18     }
19 }
```

```
1 public class MobilePhoneTest {
2     public static void main(String[] args) {
3         MobilePhone phone = new MobilePhone();
4         phone.setVoltage(3.7);
5         System.out.println("手机电压 : " + phone.getVoltage());
6         phone.setVoltage(100);
7         System.out.println("手机电压 : " + phone.getVoltage());
8     }
9 }
```

运行结果如下图所示：

```
手机电压 : 3.7
Exception in thread "main" java.lang.RuntimeException: 电压非法，请爱护手机!
    at com.bjpowernode.operator.MobilePhone.setVoltage(MobilePhone.java:11)
    at com.bjpowernode.operator.MobilePhoneTest.main(MobilePhoneTest.java:8)
```

图 10-3：对封装之后的测试

通过以上程序，可以看出 MobilePhone 的 voltage 属性不能在外部程序中随意访问了，只能调用 MobilePhone 的 setVoltage() 方法来修改电压，调用 getVoltage() 方法来读取电压，在 setVoltage() 方法中编写了安全控制代码，当电压低于 3V，或者高于 5V 的时候，程序抛出了异常，不允许修改电压值，程序结束了。只有合法的时候，才允许程序修改电压值。（异常机制在后续的内容中会学到，不要着急。）

总之，在 java 语言中封装的步骤应该是这样的：需要被保护的属性使用 private 进行修饰，给这个私有的属性对外提供公开的 set 和 get 方法，其中 set 方法用来修改属性的值，get 方法用来读取属性的值。并且 set 和 get 方法在命名上也是有规范的，规范中要求 set 方法名是 set + 属性名（属性名首字母大写），get 方法名是 get + 属性名（属性名首字母大写）。其中 set 方法有一个参数，用来给属性赋值，set 方法没有返回值，一般在 set 方法内部编写安全控制程序，因为毕竟 set 方法是修改内部数据的，而 get 方法不需要参数，返回值类型是该属性所属类型（先记住，以后讲：另外 set 方法和 get 方法都不带 static 关键字，不带 static 关键字的方法称为实例方法，这些方法调用的时候需要先创建对象，然后通过“引用”去调用这些方法，实例方法不能直接采用“类名”的方式调用。），例如以下代码：

1	public class Product {
2	private int no;
3	private String name;
4	private double price;
5	public Product(){
6	
7	}
8	public Product(int _no, String _name, double _price){
9	no = _no;
10	name = _name;
11	price = _price;
12	}
13	public int getNo() {
14	return no;
15	}
16	public void setNo(int _no) {
17	no = _no;

```
18     }
19     public String getName() {
20         return name;
21     }
22     public void setName(String _name) {
23         name = _name;
24     }
25     public double getPrice() {
26         return price;
27     }
28     public void setPrice(double _price) {
29         price = _price;
30     }
31 }
```

```
1 public class ProductTest {
2     public static void main(String[] args) {
3         Product p1 = new Product(10000, "小米 5S", 2000.0);
4         System.out.println("商品编号: " + p1.getNo());
5         System.out.println("商品名称: " + p1.getName());
6         System.out.println("商品单价: " + p1.getPrice());
7         p1.setNo(70000);
8         p1.setName("小米 6");
9         p1.setPrice(2100.0);
10        System.out.println("商品编号: " + p1.getNo());
11        System.out.println("商品名称: " + p1.getName());
12        System.out.println("商品单价: " + p1.getPrice());
13    }
14 }
```


运行结果如下图所示：

```
商品编号：10000  
商品名称：小米5S  
商品单价：2000.0  
商品编号：70000  
商品名称：小米6  
商品单价：2100.0
```

图 10-4: set 和 get 方法测试

有的读者可能会有这样的疑问：构造方法中已经给属性赋值了，为什么还要提供 set 方法呢？注意了同学们，这是两个完全不同的时刻，构造方法中给属性赋值是在创建对象的时候完成的，当对象创建完毕之后，属性可能还是会被修改的，后期要想修改属性的值，这个时候就必须调用 set 方法了。

10.3 章节小结

通过本章节内容的学习，需要理解为什么要封装，封装有什么好处，以及掌握封装的代码应该如何编写。

10.4 难点解惑

对于本章节内容来说，比较难以理解的是在之前所写的程序中所有的方法都是带有 static 关键字的，为什么在进行封装时，提供的 setter 和 getter 方法就不能添加 static 关键字了呢？这块内容在后期学习 static 关键字之后自然就好理解了，这里我简单解释一下：带有 static 关键字的方法是静态方法，不需要创建对象，直接通过“类”来调用。对于没有 static 关键字的方法被称为实例方法，这些方法执行时要求必须先创建对象，然后通过“引用”的方式来调用。而对于封装来说，setter 和 getter 方法都是访问对象内部的属性，所以 setter 和 getter 方法在定义时不能添加 static 关键字。

10.5 章节习题

第一题：定义“人”类，“人”类包括这些属性：姓名、性别、年龄等。使用封装的方式编写程序，要求对年龄进行安全控制，合法的年龄范围为[0~100]，其他值表示不合法。

10.6 习题答案

第一题答案：

```
3 public class People {
4     private String name;
5     private boolean sex;
6     private int age;
7     public String getName() {
8         return name;
9     }
10    public void setName(String _name) {
11        name = _name;
12    }
13    public boolean isSex() {
14        return sex;
15    }
16    public void setSex(boolean _sex) {
17        sex = _sex;
18    }
19    public int getAge() {
20        return age;
21    }
22    public void setAge(int _age) {
23        if(_age < 0 || _age > 100){
24            throw new RuntimeException("年龄不合法！");
25        }
26        age = _age;
27    }
28 }
```

```
3 public class PeopleTest {
4     public static void main(String[] args) {
5         People p = new People();
6         p.setName("张三");
7         p.setAge(20);
8         p.setSex(true);
9
10        System.out.println("姓名 = " + p.getName());
11        System.out.println("性别 = " + (p.isSex() ? "男" : "女"));
12        System.out.println("年龄 = " + p.getAge());
13
14        p.setAge(-100);
15    }
16 }
```

执行结果如下图所示：

```
姓名 = 张三
性别 = 男
年龄 = 20
Exception in thread "main" java.lang.RuntimeException: 年龄不合法!
    at com.lwz.wx.bean.kf.People.setAge(People.java:24)
    at com.lwz.wx.bean.kf.PeopleTest.main(PeopleTest.java:14)
```

11. 第十一章 this 和 static

11.1 章节目标与知识框架

11.1.1 章节目标

理解 this 是什么，this 能用在哪儿，不能用在哪儿，this 什么时候可以省略，什么时候不能省略，以及怎么通过构造方法调用当前类中其它的构造方法。掌握静态代码块的执行时机，变量什么时候声明为静态变量，什么时候声明为实例变量，方法什么时候声明为实例方法，什么时候声明为静态方法，以及静态方法中为何不能直接访问实例变量和实例方法。

11.1.2 知识框架



11.2 this (掌握)

11.2.1 this 是什么

this 是 java 语言中的一个关键字，它存储在内存的什么地方呢，一起来看一段程序：

1	public class Customer {
2	private String name;

3	public Customer(){
4	}
5	public Customer(String _name){
6	name = _name;
7	}
8	public void setName(String _name){
9	name = _name;
10	}
11	public String getName(){
12	return name;
13	}
14	}

1	public class CustomerTest {
2	public static void main(String[] args) {
3	Customer jack = new Customer("jack");
4	Customer rose = new Customer("rose");
5	}
6	}

以上程序的内存结构图如下所示：

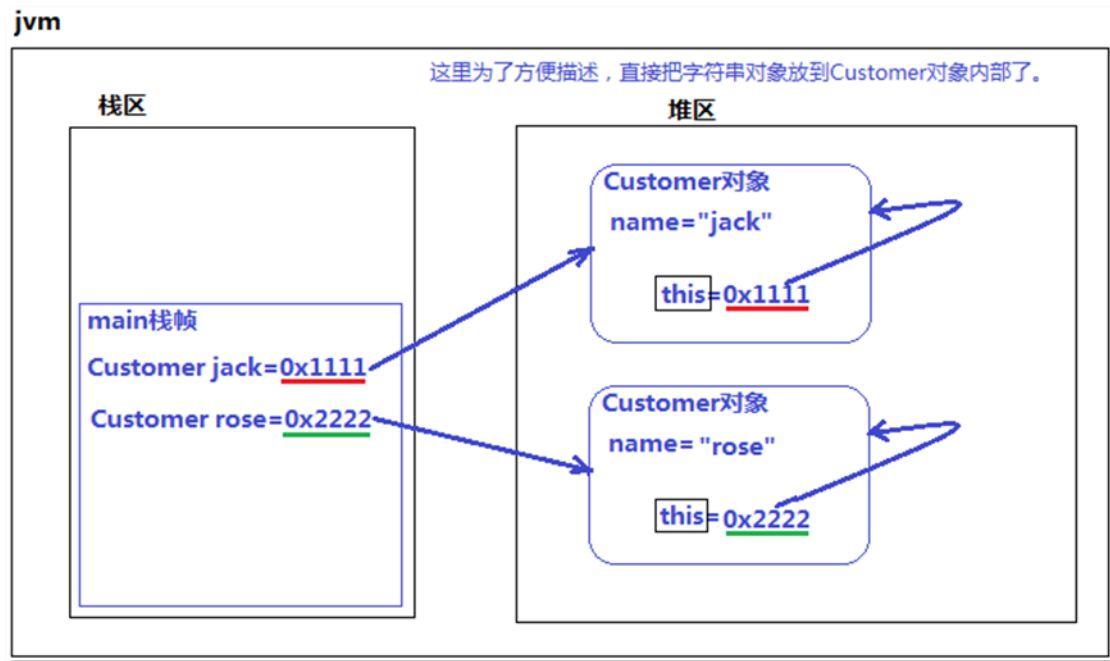


图 11-1: this 内存图

this 可以看做一个变量，它是一个引用，存储在 Java 虚拟机堆内存的对象内部，**this** 这个引用保存了当前对象的内存地址指向自身，任何一个堆内存的 java 对象都有一个 **this**，也就是说创建 100 个 java 对象则分别对应 100 个 **this**。通过以上的内存图，可以看出“jack 引用”保存的内存地址是 0x1111，对应的“**this** 引用”保存的内存地址也是 0x1111，所以“jack 引用”和“**this** 引用”是可以划等号的。也就是说访问对象的时候 **jack.name** 和 **this.name** 是一样的，都是访问该引用所指向对象的 **name** 属性。

this 指向“当前对象”，也可以说 **this** 代表“当前对象”，**this** 可以使用在实例方法中以及构造方法中，语法格式分别为“**this.**”和“**this(..)**”。**this** 不能出现在带有 **static** 的方法当中。

11.2.2 this 使用在实例方法中

我们来看看 **this** 是否可以出现在 **static** 的方法当中，请看以下代码以及编译结果：

1	public class ThisInStaticMethod {
2	public static void main(String[] args) {
3	ThisInStaticMethod.method();
4	}

5	public static void method(){
6	System.out.println(this);
7	}
8	}

编译报错，如下图所示：

```
C:\>javac ThisInStaticMethod.java
ThisInStaticMethod.java:6: 错误: 无法从静态上下文中引用非静态 变量 this
    System.out.println<this>;
                        ^
1 个错误
```

图 11-2: static 的方法中不能使用 this

通过以上的测试得知 this 不能出现在 static 的方法当中，这是为什么呢？首先 static 的方法，在调用的时候是不需要创建对象的，直接采用“类名”的方式调用，也就是说 static 方法执行的过程中是不需要“当前对象”参与的，所以 static 的方法中不能使用 this，因为 this 代表的就是“当前对象”。

大家是否还记得在之前的“封装”过程中，曾编写属性相关的 set 和 get 方法，set 和 get 方法在声明的时候不允许带 static 关键字，我们把这样的方法叫做实例方法，说到实例方法，大家肯定想到了实例变量，没错，实例变量和实例方法都是对象相关，必须有对象的存在，然后通过“引用”去访问。

为什么 set 和 get 方法设计为实例方法呢？那是因为 set 和 get 方法操作的是实例变量，“不同的对象”调用 get 方法最终得到的数据是不同的，例如 zhangsan 调用 getName()方法得到的名字是 zhangsan，lisi 调用 getName()方法得到的名字是 lisi，显然 get 方法是一个对象级别的方法，不能直接采用“类名”调用，必须先创建对象，再通过“引用”去访问。

this 可以出现在实例方法当中，因为实例方法在执行的时候一定是对象去触发的，实例方法一定是对象才能去调用的，而 this 恰巧又代表“当前对象”，所以“谁”去调用这个实例方法 this 就是“谁”。测试一下，请看以下代码：

1	public class Customer {
2	private String name;

```
3 public Customer(){
4 }
5 public Customer(String _name){
6     name = _name;
7 }
8 public void setName(String _name){
9     name = _name;
10 }
11 public String getName(){
12     return name;
13 }
14 public void shopping(){
15     System.out.println("shopping() --> " + this);
16 }
17 }
```

```
1 public class CustomerTest {
2     public static void main(String[] args) {
3         Customer jack = new Customer("jack");
4         System.out.println("main() ---> " + jack);
5         jack.shopping();
6         System.out.println("=====");
7         Customer rose = new Customer("rose");
8         System.out.println("main() ---> " + rose);
9         rose.shopping();
10    }
11 }
```

运行结果如下图所示：

```
main() ---> Customer@15db9742
shopping() --> Customer@15db9742
=====
main() ---> Customer@6d06d69c
shopping() --> Customer@6d06d69c
```

图 11-3: 测试 this

以上代码的输出结果具体是什么不重要，重要的是可以看出谁和谁是相等的。运行结果和代码结合起来分析一下 this：

```
name = _name;

String getName(){
return name;

void shopping(){
System.out.println("shopping() --> " + this);

CustomerTest {
static void main(String[] args) {
Customer jack = new Customer("jack");
System.out.println("main() ---> " + jack);
jack.shopping();
System.out.println("=====");
Customer rose = new Customer("rose");
System.out.println("main() ---> " + rose);
rose.shopping();
```

this代表当前对象，jack调用shopping()方法，this就是jack，rose调用shopping()方法，this就是rose

main() ---> Customer@15db9742
shopping() --> Customer@15db9742
=====
main() ---> Customer@6d06d69c
shopping() --> Customer@6d06d69c

图 11-4: this

指向了当前对象

通过以上内容的学习得知，this 可以使用在实例方法当中，它指向当前正在执行这个动作的对象。

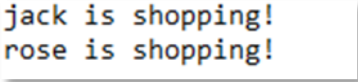
大家是否还记得实例变量怎么访问？正规的访问方式是采用“引用.”去访问。请看下面的代码：

1	public class Customer {
2	private String name;
3	public Customer(){


```
4      }  
5      public Customer(String _name){  
6          name = _name;  
7      }  
8      public void setName(String _name){  
9          name = _name;  
10     }  
11     public String getName(){  
12         return name;  
13     }  
14     public void shopping(){  
15         System.out.println(name + " is shopping!");  
16     }  
17 }
```

```
1 public class CustomerTest {  
2     public static void main(String[] args) {  
3         Customer jack = new Customer("jack");  
4         jack.shopping();  
5         Customer rose = new Customer("rose");  
6         rose.shopping();  
7     }  
8 }
```

运行结果如下图所示：



```
jack is shopping!  
rose is shopping!
```

图 11-5：测试结果

将以上部分代码片段取出来进行分析：

1	public class Customer {
2	private String name; //实例变量
3	...
4	public void shopping(){
5	//jack 调用 shopping, 当前对象是 jack
6	//rose 调用 shopping, 当前对象是 rose
7	//name 是实例变量, 不用“引用”可以访问? (以上结果表示可以)
8	System.out.println(name + " is shopping!");
9	//正规的访问方式应该是“引用.name”, 比如
10	//System.out.println(jack.name + " is shopping!");
11	//或者
12	//System.out.println(rose.name + " is shopping!");
13	//对不起, jack 和 rose 在 main 方法当中, 在这里不可见, 不能用
14	//难道是这样???
15	System.out.println(this.name + " is shopping!");
16	}
17	}

1	public class CustomerTest {
2	public static void main(String[] args) {
3	Customer jack = new Customer("jack");
4	jack.shopping();
5	Customer rose = new Customer("rose");
6	rose.shopping();
7	}
8	}

把完整的代码拿过来：

```
1 public class Customer {
2     private String name;
3     public Customer(){
4     }
5     public Customer(String _name){
6         name = _name;
7     }
8     public void setName(String _name){
9         name = _name;
10    }
11    public String getName(){
12        return name;
13    }
14    public void shopping(){
15        System.out.println(name + " is shopping!");
16        System.out.println(this.name + " is shopping!");
17    }
18 }
```

```
1 public class CustomerTest {
2     public static void main(String[] args) {
3         Customer jack = new Customer("jack");
4         jack.shopping();
5         System.out.println("=====");
6         Customer rose = new Customer("rose");
7         rose.shopping();
8     }
9 }
```

运行结果如下图所示：

```
jack is shopping!  
jack is shopping!  
=====  
rose is shopping!  
rose is shopping!
```

图 11-6：测试结果

通过以上的测试我们得知：`System.out.println(name + " is shopping!")` 和 `System.out.println(this.name + " is shopping!")` 是等效的。也就是说在 `shopping()` 这个“实例方法”当中直接访问“实例变量”`name` 就表示访问当前对象的 `name`。换句话说在实例方法中可以直接访问当前对象的实例变量，而“`this.`”是可以省略的。“`this.`”什么时候不能省略呢？请看以下代码：

```
1 public class Customer {  
2     private String name;  
3     public Customer(){  
4     }  
5     public Customer(String _name){  
6         name = _name;  
7     }  
8     public void setName(String _name){  
9         name = _name;  
10    }  
11    public String getName(){  
12        return name;  
13    }  
14    public void shopping(){  
15        System.out.println(name + " is shopping!");  
}
```

16	}
17	}

你有没有看到 `name=_name` 这样的代码很丑陋，怎样可以优雅一些呢？请看：

1	public class Customer {
2	private String name;
3	public Customer(){
4	}
5	public Customer(String name){
6	this.name = name;//这里的“this.”不能省略
7	}
8	public void setName(String name){
9	this.name = name;//这里的“this.”不能省略
10	}
11	public String getName(){
12	return name; //这里的“this.”可以省略
13	}
14	public void shopping(){
15	//这里的“this.”可以省略
16	System.out.println(name + " is shopping!");
17	}
18	}

以上代码当中 `this.name = name`，其中 `this.name` 表示实例变量 `name`，等号右边的 `name` 是局部变量 `name`，此时如果省略“`this.`”，则变成 `name = name`，这两个 `name` 都是局部变量（`java` 遵守就近原则），和实例变量 `name` 无关了，显然是不可以省略“`this.`”的。

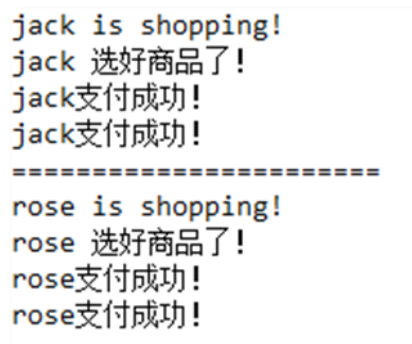
最终的结论是，`this` 不能出现在 `static` 的方法中，可以出现在实例方法中，代表当前对象，大部分情况下 `this` 都是可以省略的，只有当在实例方法中区分局部变量和实例变量的时候不能省略。

接下来我们再来扩展一下 `this` 的使用，请看代码：

```
1 public class Customer {
2     private String name;
3     public Customer(){
4     }
5     public Customer(String name){
6         this.name = name;
7     }
8     public void setName(String name){
9         this.name = name;
10    }
11    public String getName(){
12        return name;
13    }
14    //实例方法
15    public void shopping(){
16        System.out.println(name + " is shopping!");
17        System.out.println(name + " 选好商品了! ");
18        //pay()支付方法是实例方法，实例方法需要使用“引用”调用
19        //那么这个“引用”是谁呢？
20        //当前对象在购物，肯定是当前对象在支付，所以引用是 this
21        this.pay();
22        //同样“this.”可以省略
23        pay();
24    }
25    //实例方法
26    public void pay(){
27        System.out.println(name + "支付成功! ");
28    }
29 }
```

1	public class CustomerTest {
2	public static void main(String[] args) {
3	Customer jack = new Customer("jack");
4	jack.shopping();
5	System.out.println("=====");
6	Customer rose = new Customer("rose");
7	rose.shopping();
8	}
9	}

运行结果如下图所示：



```
jack is shopping!  
jack 选好商品了!  
jack支付成功!  
jack支付成功!  
=====  
rose is shopping!  
rose 选好商品了!  
rose支付成功!  
rose支付成功!
```

图 11-7：测试结果

通过以上的测试，可以看出在一个实例方法当中可以直接去访问其它的实例方法，方法是对象的一种行为描述，实例方法中直接调用其它的实例方法，就表示“当前对象”完成了一系列行为动作。例如在实例方法 `shopping()` 中调用另一个实例方法 `pay()`，这个过程就表示 `jack` 在选购商品，选好商品之后，完成支付环节，其中选购商品是一个动作，完成支付是另一个动作。接下来继续扩展，请看以下代码：

1	
2	public class ThisTest {

```
3      int i = 10;
4      public static void main(String[] args) {
5          System.out.println(i);
6      }
7  }
```

以上代码编译报错了，请看：

```
C:\>javac ThisTest.java
ThisTest.java:5: 错误: 无法从静态上下文中引用非静态 变量 i
    System.out.println(i);
                      ^
1 个错误
```

图 11-8：编译错误提示信息

为什么会编译报错，在 main 方法中为什么无法直接访问变量 i？我们来分析一下，首先 i 变量是实例变量，实例变量要想访问必须先创建对象，然后通过“引用”去访问，main 方法是 static 的方法，也就是说 main 方法是通过“类名”去调用的，在 main 方法中没有“当前对象”的概念，也就是说 main 方法中不能使用 this，所以编译报错了。那应该怎么修改呢？请看：

```
1  public class ThisTest {
2      int i = 10;
3      public static void main(String[] args) {
4          //这肯定是不行的，因为 main 方法带有 static，不能用 this
5          //System.out.println(this.i);
6          //可以自己创建一个对象
7          ThisTest tt = new ThisTest();
8          //通过引用访问
9          System.out.println(tt.i);
10     }
```



```
11 }
```

运行结果如下图所示：



图 11-9：测试结果

通过以上的测试得知，在 `static` 的方法中不能直接访问实例变量，要访问实例变量必须先自己创建一个对象，通过“引用”可以去访问，不能通过 `this` 访问，因为在 `static` 方法中是不能存在 `this` 的。其实这种设计也是有道理的，因为 `static` 的方法在执行的时候是采用“类名”去调用，没有对象的参与，自然也不会存在当前对象，所以 `static` 的方法执行过程中不存在 `this`。那么在 `static` 方法中能够直接访问实例方法吗？请看以下代码：

```
1
2 public class ThisTest {
3     public static void main(String[] args) {
4         doSome();
5     }
6     public void doSome(){
7         System.out.println("do some...");
8     }
9 }
```

编译报错了，请看下图：



图 11-10：编译报错提示信息

为什么在 main 方法中无法直接调用实例方法 doSome()呢？很简单，因为实例方法必须先创建对象，通过引用去调用，在以上的 main 方法中并没有创建对象，更没有 this。所以在 main 方法中无法直接访问实例方法。结论就是：在 static 的方法中不能直接访问实例方法。怎么修改呢？同样需要先创建对象，请看：

```
1 public class ThisTest {
2     public static void main(String[] args) {
3         ThisTest tt = new ThisTest();
4         tt.doSome();
5     }
6     public void doSome(){
7         System.out.println("do some...");
8     }
9 }
```

运行结果如下图所示：

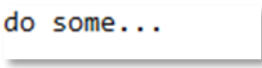


图 11-11：运行结果

综上所述，我们需要记住这样的一个结论：this 不能使用在 static 的方法中，可以使用在实例方法中，代表当前对象，多数情况下 this 是可以省略不写的，但是在区分局部变量和实例变量的时候不能省略，在实例方法中可以直接访问当前对象实例变量以及实例方法，在 static 方法中无法直接访问实例变量和实例方法。

11.2.3 this 使用在构造方法中

this 还有另外一种用法，使用在构造方法第一行（只能出现在第一行，这是规定，记住就行），通过当前构造方法调用本类当中其它的构造方法，其目的是为了代码复用。调用时的语法格式是：this(实际参数列表)，请看以下代码：

```
1 public class Date {
2     private int year;
3     private int month;
4     private int day;
5     //业务要求，默认创建的日期为 1970 年 1 月 1 日
6     public Date(){
7         this.year = 1970;
8         this.month = 1;
9         this.day = 1;
10    }
11    public Date(int year,int month,int day){
12        this.year = year;
13        this.month = month;
14        this.day = day;
15    }
16    public int getYear() {
17        return year;
18    }
19    public void setYear(int year) {
20        this.year = year;
21    }
22    public int getMonth() {
23        return month;
24    }
25    public void setMonth(int month) {
26        this.month = month;
27    }
28    public int getDay() {
29        return day;
```

```
30     }
31     public void setDay(int day) {
32         this.day = day;
33     }
34 }
```

```
1  public class DateTest {
2      public static void main(String[] args) {
3          Date d1 = new Date();
4          System.out.println(d1.getYear() + "年 " + d1.getMonth() + "月 " +
5 d1.getDay() + "日");
6          Date d2 = new Date(2008, 8, 8);
7          System.out.println(d2.getYear() + "年 " + d2.getMonth() + "月 " +
8 d2.getDay() + "日");
9      }
10 }
```

运行结果如下图所示：

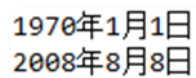


图 11-12： 运行结果

我们来看看以上程序的无参数构造方法和有参数构造方法：

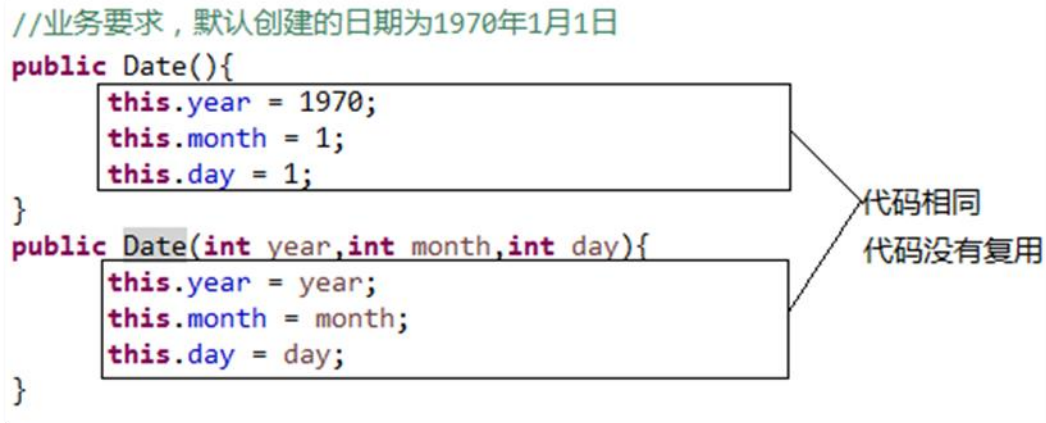


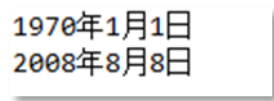
图 11-13：无参数构造和有参数构造对比

通过上图可以看到无参数构造方法中的代码和有参数构造方法中的代码是一样的，按照以上方式编写，代码没有得到重复使用，这个时候就可以在无参数构造方法中使用“this(实际参数列表);”来调用有参数的构造方法，这样就可以让代码得到复用了，请看：

1	public class Date {
2	private int year;
3	private int month;
4	private int day;
5	//业务要求，默认创建的日期为 1970 年 1 月 1 日
6	public Date(){
7	this(1970, 1, 1);
8	}
9	public Date(int year,int month,int day){
10	this.year = year;
11	this.month = month;
12	this.day = day;
13	}
14	public int getYear() {
15	return year;

16	}
17	public void setYear(int year) {
18	this.year = year;
19	}
20	public int getMonth() {
22	return month;
23	}
25	public void setMonth(int month) {
26	this.month = month;
27	}
28	public int getDay() {
29	return day;
30	}
31	public void setDay(int day) {
32	this.day = day;
33	}
34	}

还是使用以上的 main 方法进行测试，运行结果如下：



1970年1月1日
2008年8月8日

图 11-14：运行结果

在 this() 上一行尝试添加代码，请看代码以及编译结果：

1	public class Date {
2	private int year;
3	private int month;

```
4     private int day;
5     //业务要求，默认创建的日期为 1970 年 1 月 1 日
6     public Date(){
7         System.out.println("...");
8         this(1970, 1, 1);
9     }
10    public Date(int year,int month,int day){
11        this.year = year;
12        this.month = month;
13        this.day = day;
14    }
15    public int getYear() {
16        return year;
17    }
18    public void setYear(int year) {
19        this.year = year;
20    }
22    public int getMonth() {
23        return month;
24    }
25    public void setMonth(int month) {
26        this.month = month;
27    }
28    public int getDay() {
29        return day;
30    }
31    public void setDay(int day) {
32        this.day = day;
33    }
34 }
```

```
C:\>javac Date.java
Date.java:8: 错误: 对this的调用必须是构造器中的第一个语句
    this(1970, 1, 1);
    ^
1 个错误
```

图 11-15: 编译报错信息

通过以上测试得出: `this()`语法只能出现在构造方法第一行, 这个大家记住就行了。

11.3 static (掌握)

11.3.1 static 概述

`static` 是 java 语言中的关键字, 表示“静态的”, 它可以用来修饰变量、方法、代码块等, 修饰的变量叫做静态变量, 修饰的方法叫做静态方法, 修饰的代码块叫做静态代码块。在 java 语言中凡是用 `static` 修饰的都是类相关的, 不需要创建对象, 直接通过“类名”即可访问, 即使使用“引用”去访问, 在运行的时候也和堆内存当中的对象无关。

本小节要求大家主要掌握: 什么时候将变量定义为静态变量? 什么时候把方法定义为静态方法? 静态代码块什么时候使用? 类体中的代码有执行顺序要求吗? 空引用访问静态变量和静态方法会出现空指针异常吗?

11.3.2 静态变量

java 中的变量包括: 局部变量和成员变量, 在方法体中声明的变量为局部变量, 有效范围很小, 只能在方法体中访问, 方法结束之后局部变量内存就释放了, 在内存方面局部变量存储在栈当中。在类体中定义的变量为成员变量, 而成员变量又包括实例变量和静态变量, 当成员变量声明时使用了 `static` 关键字, 那么这种变量称为静态变量, 没有使用 `static` 关键字称为实例变量, 实例变量是对象级别的, 每个对象的实例变量值可能不同, 所以实例变量必须先创建对象, 通过“引用”去访问, 而静态变量访问时不需要创建对象, 直接通过“类名”访问。实例变量存储在堆内存当中, 静态变量存储在方法区当中。实例变量在构造方法执行过程中初始化, 静态变量在类加载时初始化。那么变量在什么情况下会声明为静态变量呢? 请看以下代码, 定义一个“男人”类:


```
1 public class Man {
2     //身份证号
3     int idCard;
4     //性别（所有男人的性别都是“男”）
5     //true 表示男，false 表示女
6     boolean sex = true;
7     public Man(int idCard){
8         this.idCard = idCard;
9     }
10 }
```

```
1 public class ManTest {
2     public static void main(String[] args) {
3         Man jack = new Man(100);
4         System.out.println(jack.idCard + "," + (jack.sex ? "男" : "女"));
5         Man sun = new Man(101);
6         System.out.println(sun.idCard + "," + (sun.sex ? "男" : "女"));
7         Man cok = new Man(102);
8         System.out.println(cok.idCard + "," + (cok.sex ? "男" : "女"));
9     }
10 }
```

运行结果如下图所示：



```
100,男
101,男
102,男
```

图 11-16： 运行结果

我们来看一下以上程序的内存结构图：

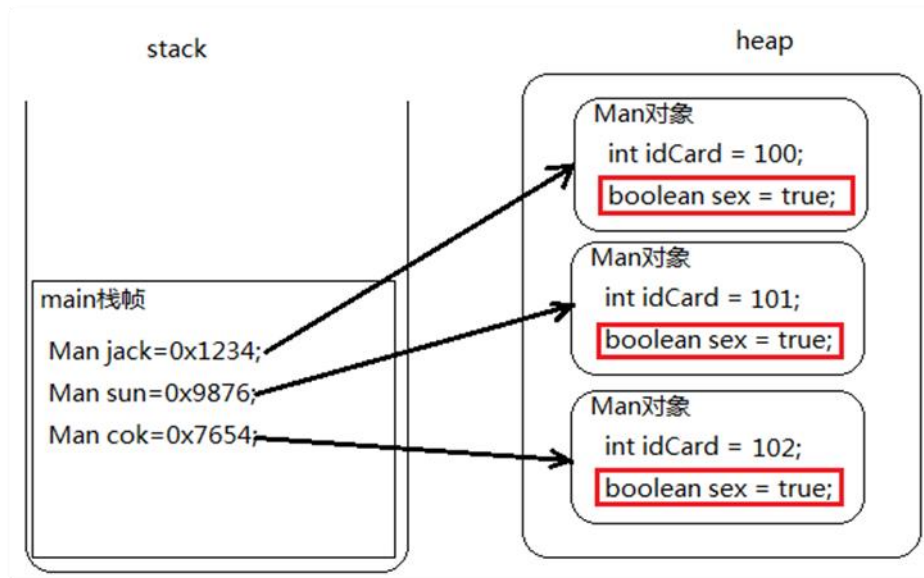


图 11-17：内存结构图

“男人”创建的所有“男人对象”，每一个“男人对象”的身份证号都不一样，该属性应该每个对象持有一份，所以应该定义为实例变量，而每一个“男人对象”的性别都是“男”，不会随着对象的改变而变化，性别值永远都是“男”，这种情况下，性别这个变量还需要定义为实例变量吗，有必要让每一个“男人对象”持有一份吗，这样岂不是浪费了大量的堆内存空间，所以这个时候建议将“性别=男”属性定义为类级别的属性，声明为静态变量，上升为“整个族”的数据，这样的变量不需要创建对象直接使用“类名”即可访问。请看代码：

```

1 public class Man {
2     //身份证号
3     int idCard;
4     //性别（所有男人的性别都是“男”）
5     //true 表示男，false 表示女
6     static boolean sex = true;
7     public Man(int idCard){
8         this.idCard = idCard;
9     }

```

10	}
----	---

1	public class ManTest {
2	public static void main(String[] args) {
3	Man jack = new Man(100);
4	System.out.println(jack.idCard + "," + (Man.sex ? "男" : "女"));
5	Man sun = new Man(101);
6	System.out.println(sun.idCard + "," + (Man.sex ? "男" : "女"));
7	Man cok = new Man(102);
8	System.out.println(cok.idCard + "," + (Man.sex ? "男" : "女"));
9	}
10	}

运行结果如下图所示：



```
100,男
101,男
102,男
```

图 11-18： 运行结果

我们来看一下以上程序的内存结构图：

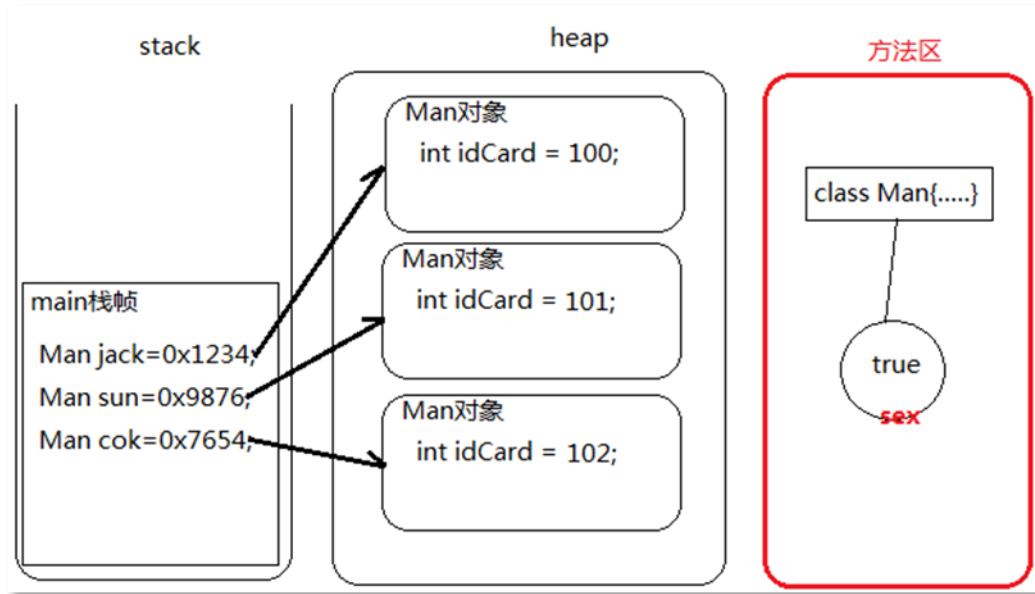


图 11-19：静态变量内存图

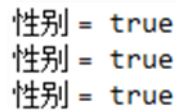
通过以上内容的学习我们得知，当一个类的所有对象的某个“属性值”不会随着对象的改变而变化的时候，建议将该属性定义为静态属性（或者说把这个变量定义为静态变量），静态变量在类加载的时候初始化，存储在方法区当中，不需要创建对象，直接通过“类名”来访问。如果静态变量使用“引用”来访问，可以吗，如果可以的话，这个访问和具体的对象有关系吗？来看以下代码：

```

1 public class ManTest {
2     public static void main(String[] args) {
3         //静态变量比较正式的访问方式
4         System.out.println("性别 = " + Man.sex);
5         //创建对象
6         Man jack = new Man(100);
7         //使用“引用”来访问静态变量可以吗？
8         System.out.println("性别 = " + jack.sex);
9         //对象被垃圾回收器回收了
10        jack = null;
11        //使用“引用”还可以访问吗？
    
```

12	System.out.println("性别 = " + jack.sex);
13	}
14	}

运行结果如下图所示：



```
性别 = true
性别 = true
性别 = true
```

图 11-20：静态变量使用“引用”访问

通过以上代码以及运行结果可以看出，静态变量也可以使用“引用”去访问，但实际上在执行过程中，“引用”所指向的对象并没有参与，如果是空引用访问实例变量，程序一定会发生空指针异常，但是以上的程序编译通过了，并且运行的时候也没有出现任何异常，这说明虽然表面看起来是采用“引用”去访问，但实际上在运行的时候还是直接通过“类”去访问的。静态方法是这样吗？请看以下代码：

1	public class Man {
2	//身份证号
3	int idCard;
4	//性别（所有男人的性别都是“男”）
5	//true 表示男，false 表示女
6	static boolean sex = true;
7	public Man(int idCard){
8	this.idCard = idCard;
9	}
10	//静态方法
11	public static void printInfo(){
12	System.out.println("-----" + (Man.sex ? "男" : "女") + "-----");
13	}

14 }

```
1 public class ManTest {
2     public static void main(String[] args) {
3         //静态变量比较正式的访问方式
4         System.out.println("性别 = " + Man.sex);
5         //创建对象
6         Man jack = new Man(100);
7         //使用“引用”来访问静态变量可以吗？
8         System.out.println("性别 = " + jack.sex);
9         //对象被垃圾回收器回收了
10        jack = null;
11        //使用“引用”还可以访问吗？
12        System.out.println("性别 = " + jack.sex);
13        //静态方法比较正式的访问方式
14        Man.printInfo();
15        //访问静态方法可以使用引用吗？并且空的引用可以吗？
16        jack.printInfo();
17    }
18 }
```

运行结果如下图所示：

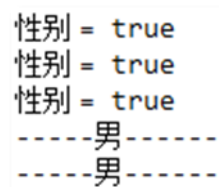


图 11-21：静态方法可以使用引用访问吗

通过以上代码测试得知，静态变量和静态方法比较正式的方式是直接采用“类名”访问，但实际上使用“引用”也可以访问，并且空引用访问静态变量和静态方法并不会出现空指针异常。实际上，在开发中并不建议使用“引用”去访问静态相关的成员，因为这样会让程序员困惑，因为采用“引用”方式访问的时候，程序员会认为你访问的是实例相关的成员。

总之，所有实例相关的，包括实例变量和实例方法，必须先创建对象，然后通过“引用”的方式去访问，如果空引用访问实例相关的成员，必然会出现空指针异常。所有静态相关的，包括静态变量和静态方法，直接使用“类名”去访问。虽然静态相关的成员也能使用“引用”去访问，但这种方式并不被主张。

11.3.3 静态代码块

静态代码块的语法格式是这样的：

```
类{  
    //静态代码块  
    static{  
        java 语句;  
    }  
}
```

静态代码块在类加载时执行，并且只执行一次。开发中使用不多，但离了它有的时候还真是没法写代码。静态代码块实际上是 java 语言为程序员准备的一个特殊的时刻，这个时刻就是类加载时刻，如果你想类加载的时候执行一段代码，那么这段代码就有的放矢了。例如我们要在类加载的时候解析某个文件，并且要求该文件只解析一次，那么此时就可以把解析该文件的代码写到静态代码块当中了。我们来测试一下静态代码块：

1	public class StaticTest01 {
2	//静态代码块
3	static{
4	System.out.println(2);
5	}
6	//静态代码块

7	static{
8	System.out.println(1);
9	}
10	//main 方法
11	public static void main(String[] args) {
12	System.out.println("main execute!");
13	}
14	//静态代码块
15	static{
16	System.out.println(0);
17	}
18	}

运行结果如下图所示：

```
2
1
0
main execute!
```

图 11-22：静态代码块运行结果

通过以上的测试可以得知一个类当中可以编写多个静态代码块（尽管大部分情况下只编写一个），并且静态代码块遵循自上而下的顺序依次执行，所以有的时候放在类体当中的代码是有执行顺序的（大部分情况下类体当中的代码没有顺序要求，方法体当中的代码是有顺序要求的，方法体当中的代码必须遵守自上而下的顺序依次逐行执行），另外静态代码块当中的代码在 `main` 方法执行之前执行，这是因为静态代码块在类加载时执行，并且只执行一次。再来看一下以下代码：

1	public class StaticTest02 {
2	int i = 100;

3	static{
4	System.out.println(i);
5	}
6	}

编译结果如下图所示：

```
C:\>javac StaticTest02.java
StaticTest02.java:4: 错误: 无法从静态上下文中引用非静态 变量 i
    System.out.println(i);
                       ^
1 个错误
```

图 11-23：静态代码块中访问实例变量编译报错

为什么编译报错呢？那是因为 `i` 变量是实例变量，实例变量必须先创建对象才能访问，静态代码块在类加载时执行，这个时候对象还没有创建呢，所以 `i` 变量在这里是不能这样访问的。可以考虑在 `i` 变量前添加 `static`，这样 `i` 变量就变成静态变量了，静态变量访问时不需要创建对象，直接通过“类”即可访问，例如以下代码：

1	public class StaticTest02 {
2	static int i = 100;
3	static{
4	System.out.println("静态变量 i = " + i);
5	}
6	public static void main(String[] args) {
7	}
8	}

运行结果如下图所示：

```
静态变量i = 100
```

图 11-24：静态代码块中访问静态变量

代码修改为这样呢？

```
1 public class StaticTest02 {  
2     static {  
3         System.out.println("静态变量 i = " + i);  
4     }  
5     static int i = 100;  
6 }
```

编译报错了，请看下图：

```
C:\>javac StaticTest02.java  
StaticTest02.java:3: 错误: 非法前向引用  
        System.out.println("静态变量i = " + i);  
                               ^  
1 个错误
```

图 11-25：编译报错信息

通过测试，可以看到有的时候类体当中的代码也是有顺序要求的（类体当中定义两个独立的方法，这两个方法是没有先后顺序要求的），静态代码块在类加载时执行，静态变量在类加载时初始化，它们在同一时间发生，所以必然会有顺序要求，如果在静态代码块中要访问*i*变量，那么*i*变量必须放到静态代码块之前。

11.3.4 静态方法

方法在什么情况下会声明为静态的呢？方法实际上描述的是行为动作，我认为当某个动作在触发的时候需要对象的参与，这个方法应该定义为实例方法，例如：每个玩篮球的人都会打篮球，但是你打篮球和科比打篮球最终的效果是不一样的，显然打篮球这个动作存在对象差异化，该方法应该定义为实例方法。再如：每个高中生都有考试的行为，但是你考试和学霸考试最终的结果是不一样的，一个上了“家里蹲大学”，一个上了“清华大学”，显然这个动作也是需要对象参与才能完成的，所以考试这个方法应该定义为实例方法。

以上描述是从设计思想角度出发来进行选择，其实也可以从代码的角度来进行判断，当方法体中需要直接访问当前对象的实例变量或者实例方法的时候，该方法必须定义为实例方法，因为只有实例方法中才有 `this`，静态方法中不存在 `this`。请看代码：

```
1 public class Customer {  
2     String name;  
3     public Customer(String name){  
4         this.name = name;  
5     }  
6     public void shopping(){  
7         //直接访问当前对象的 name  
8         System.out.println(name + "正在选购商品！");  
9         //继续让当前对象去支付  
10        pay();  
11    }  
12    public void pay(){  
13        System.out.println(name + "正在支付！");  
14    }  
15 }
```

```
1 public class CustomerTest {  
2     public static void main(String[] args) {  
3         Customer jack = new Customer("jack");  
4         jack.shopping();  
5         Customer rose = new Customer("rose");  
6         rose.shopping();  
7     }  
8 }
```

运行结果如下图所示：

```
jack正在选购商品!  
jack正在支付!  
rose正在选购商品!  
rose正在支付!
```

图 11-26: 运行结果

在以上的代码中，不同的客户购物，最终的效果都不同，另外在 `shopping()` 方法中访问了当前对象的实例变量 `name`，以及调用了实例方法 `pay()`，所以 `shopping()` 方法不能定义为静态方法，必须声明为实例方法。

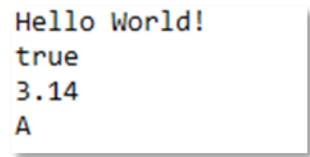
另外，在实际的开发中，“工具类”当中的方法一般定义为静态方法，因为工具类就是为了方便大家的使用，将方法定义为静态方法，比较方便调用，不需要创建对象，直接使用类名就可以访问。请看以下工具类，为了简化“`System.out.println();`”代码而编写的工具类：

```
1 public class U {  
2     public static void p(int data){  
3         System.out.println(data);  
4     }  
5     public static void p(long data){  
6         System.out.println(data);  
7     }  
8     public static void p(float data){  
9         System.out.println(data);  
10    }  
11    public static void p(double data){  
12        System.out.println(data);  
13    }  
14    public static void p(boolean data){  
15        System.out.println(data);  
16    }  
17    public static void p(char data){
```

```
18         System.out.println(data);
19     }
20     public static void p(String data){
21         System.out.println(data);
22     }
23 }
```

```
1 public class HelloWorld {
2     public static void main(String[] args) {
3         U.p("Hello World!");
4         U.p(true);
5         U.p(3.14);
6         U.p('A');
7     }
8 }
```

运行结果如下图所示：



```
Hello World!
true
3.14
A
```

图 11-27：测试工具类

11.4 章节小结

通过本章节内容的学习，要求大家掌握 Java 中两个关键字的用法，一个是 `this`，一个是 `static`。其中 `this` 要理解的是 `this` 是什么，内存中存储在哪里，要掌握 `this` 在实例方法和构造方法中的用法。对于 `static` 来说，要理解 `static` 代表什么含义，要掌握静态变量、静态代码块、静态方法的使用。

11.5 难点解惑

在本章节中，对于 `static` 来说，代码的执行顺序是一个需要大家注意的地方，一般来说方法体当中的代码是有执行顺序要求的，之前所接触的程序中，类体当中的程序没有顺序要求，但自从认识了 `static` 之后，我们发现类体当中的代码也有执行顺序的要求了，尤其是 `static` 修饰的静态变量，以及 `static` 修饰的静态代码块他们是有先后顺序的，这里需要嘱咐大家的是 `static` 修饰的静态变量以及静态代码块都是在类加载时执行，并且遵循自上而下的顺序依次逐行执行。

11.6 章节习题

第一题：选择题

1	public class Test {
2	static int value = 9;
3	public static void main(String[] args) throws Exception{
4	new Test().printValue();
5	}
6	public void printValue(){
7	int value = 69;
8	System.out.println(this.value);
9	}
10	}

- A. 编译错误
- B. 打印 9
- C. 打印 69
- D. 运行时抛出异常

第二题：判断下面代码的输出结果，并说明原因

```
1 public class User {  
2     private String name;  
3     public User(){  
4     }  
5     public void setName(String name){  
6         name = name;  
7     }  
8     public String getName(){  
9         return name;  
10    }  
11 }
```

```
1 public class UserTest {  
2     public static void main(String[] args) {  
3         User user = new User();  
4         user.setName("zhangsan");  
5         System.out.println(user.getName());  
6     }  
7 }
```

第三题：找出下面代码的错误，并说明为什么错了

```
1 public class Test {  
2     int i;  
3     static int j;  
4     public void m1(){  
5         System.out.println(i);  
6         System.out.println(j);  
7         m2();  
8     }  
9 }
```

8	m3();
9	}
10	public void m2(){
11	}
12	public static void m3(){
13	System.out.println(i);
14	System.out.println(j);
15	m2();
16	m4();
17	}
18	public static void m4(){
19	}
20	}

11.7 习题答案

第一题答案：B

第二题答案：最终的输出结果是：null。原因：setName 方法体当中的 name = name 是把局部变量 name 赋值给局部变量 name，和实例变量 name 无关，所以 getName() 方法获取的实例变量值是 null

第三题答案：第 13,15 行编译报错，因为静态方法中无法直接访问实例变量 i 和实例方法 m2()。

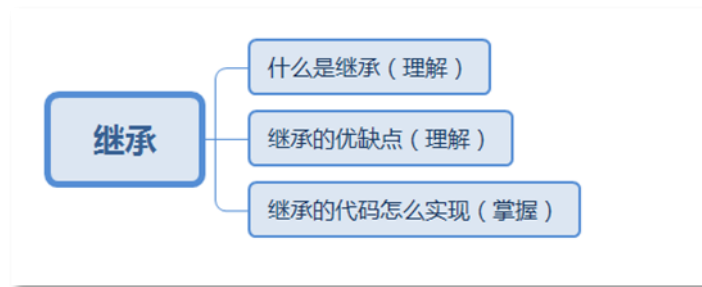
12. 第十二章 继承（Inheritance）

12.1 章节目标与知识框架

12.1.1 章节目标

了解继承在 java 中有什么作用，以及在代码上如何实现继承。

12.1.2 知识框架



12.2 继承

12.2.1 继承概述（理解）

继承是面向对象三大特征之一，封装居首位，封装之后形成了独立体，独立体 A 和独立体 B 之间可能存在继承关系。其实程序中的继承灵感来自于现实生活，在现实生活中继承处处可见，例如，儿子继承了父亲的财产，儿子不需要努力就很有钱。

生活中的继承：

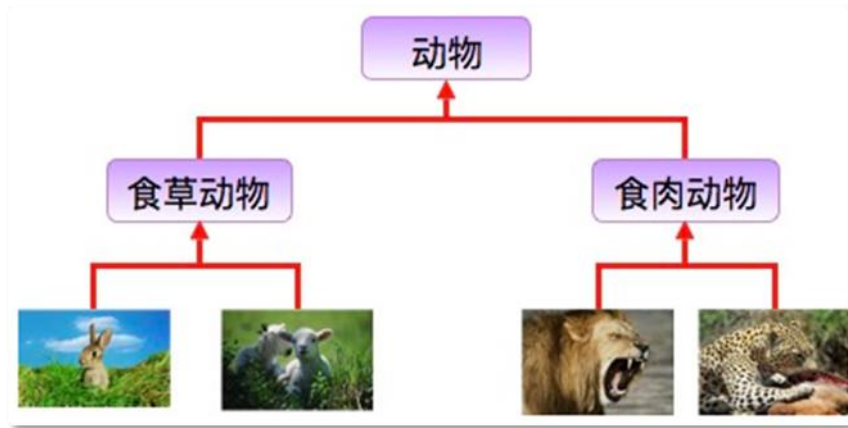


图 12-1：现实世界中的继承

继承时子类继承父类的特征和行为，使得子类对象（实例）具有父类的属性，或子类从父类继承方法，使得子类具有与父类相同的行为。兔子和羊属于食草动物类，狮子和豹属于食肉动物类。食草动物和食肉动物又是属于动物类。所以继承需要符合的关系是：is-a（Bird is-a Animal），父类更通用，子类更具体。虽然食草动物和食肉动物都是属于动物，但是两者的属性和行为上有差别，所以子类会具有父类的一般特性也会具有自身的特性。

为什么要使用继承机制？在不同的类中也可能会有共同的特征和动作，可以把这些共同的特征和动作放在一个类中，让其它类共享。因此可以定义一个通用类，然后将其扩展为其它多个特定类，这些特定类继承通用类中的特征和动作。继承是 Java 中实现软件重用的重要手段，避免重复，易于维护。

12.2.2 如何继承（掌握）

java 中继承的语法格式：

```
class 类名 extends 父类名{
    类体;
}
```

为什么需要继承，接下来我们用以下这个需求来说明一下：

1	public class Account { //银行账户类
---	--------------------------------

```
2      //账号
3      private String actno;
4      //余额
5      private double balance;
6      //账号和余额的 set 和 get 方法
7      public String getActno() {
8          return actno;
9      }
10     public void setActno(String actno) {
11         this.actno = actno;
12     }
13     public double getBalance() {
14         return balance;
15     }
16     public void setBalance(double balance) {
17         this.balance = balance;
18     }
19 }
```

```
1 public class CreditAccount { //信用账户类
2     //账号
3     private String actno;
4     //余额
5     private double balance;
6     //账号和余额的 set 和 get 方法
7     public String getActno() {
8         return actno;
9     }
10    public void setActno(String actno) {
11        this.actno = actno;
```

```
12     }
13     public double getBalance() {
14         return balance;
15     }
16     public void setBalance(double balance) {
17         this.balance = balance;
18     }
19     //信誉度（特有属性）
20     private double credit;
21     //信誉度的 set 和 get 方法
22     public double getCredit() {
23         return credit;
24     }
25     public void setCredit(double credit) {
26         this.credit = credit;
27     }
28 }
```

以上两个类分别描述了“银行账户类”和“信用账户类”，信用账户类除了具有银行账户类的特征之外还有自己的特性，按照以上代码的编写方式，程序将会非常的臃肿，我们将上面的程序修改为继承方式，请看代码：

```
1 public class Account { //银行账户类
2     //账号
3     private String actno;
4     //余额
5     private double balance;
6     //账号和余额的 set 和 get 方法
7     public String getActno() {
8         return actno;
```

9	}
10	public void setActno(String actno) {
11	this.actno = actno;
12	}
13	public double getBalance() {
14	return balance;
15	}
16	public void setBalance(double balance) {
17	this.balance = balance;
18	}
19	}

1	public class CreditAccount extends Account{ //信用账户类
2	//信誉度（特有属性）
3	private double credit;
4	//信誉度的 set 和 get 方法
5	public double getCredit() {
6	return credit;
7	}
8	public void setCredit(double credit) {
9	this.credit = credit;
10	}
11	}

1	public class AccountTest {
2	public static void main(String[] args) {
3	CreditAccount act = new CreditAccount();
4	act.setActno("11111111");
5	act.setBalance(9000.0);

6	System.out.println(act.getActno() + "信用账户， 余额" + act.getBalance()
7	+ "元");
8	}
9	}

运行结果如下图所示：

111111111信用账户， 余额9000.0元

图 12-2： 继承测试

通过以上的代码，我们可以看到继承可以解决代码臃肿的问题。换句话说，继承解决了代码复用的问题（代码复用就是代码的重复利用），这是继承机制最基本的作用。

继承的作用中除了可以让代码复用之外，还有非常重要的两个作用，那就是有了继承之后才会衍生出方法的覆盖和多态机制。这两个作用我们在后续的课程中会详细讲解。目前先理解一下继承可以做到代码复用的效果。

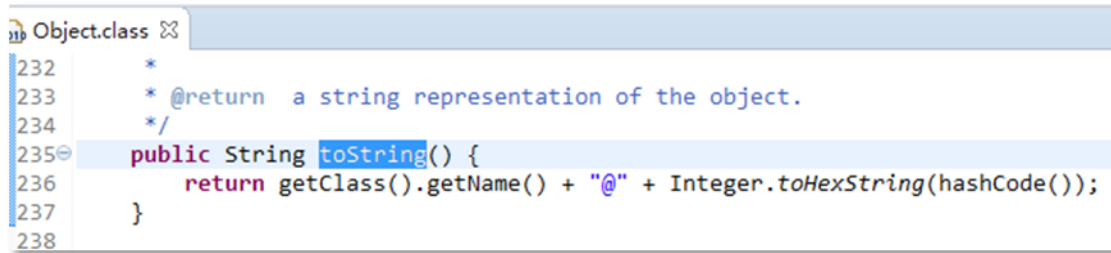
12.2.3 继承的相关特性（掌握）

那么，对于 Java 的继承有哪些特性需要大家理解和记忆呢？我来罗列一下：

- ① B 类继承 A 类，则称 A 类为超类(superclass)、父类、基类，B 类则称为子类(subclass)、派生类、扩展类。
- ② java 中的继承只支持单继承，不支持多继承，C++中支持多继承，这也是 java 体现简单性的一点，换句话说，java 中不允许这样写代码：class B extends A,C{ }。
- ③ 虽然 java 中不支持多继承，但有的时候会产生间接继承的效果，例如：class C extends B, class B extends A, 也就是说，C 直接继承 B，其实 C 还间接继承 A。
- ④ java 中规定，子类继承父类，除构造方法和被 private 修饰的数据不能继承外，剩下都可以继承。
- ⑤ java 中的类没有显示的继承任何类，则默认继承 Object 类，Object 类是 java 语言提供的根类（老祖宗类），也就是说，一个对象与生俱来就有 Object 类型中所有的特征。
- ⑥ 继承也存在一些缺点，例如：CreditAccount 类继承 Account 类会导致它们之间的耦合度非常高，Account 类发生改变之后会马上影响到 CreditAccount 类。

12.2.4 对继承自 Object 类的方法的测试（理解）

接下来我们来测试一下从 Object 类中继承过来的方法，先来看一下 Object 类的部分源代码：



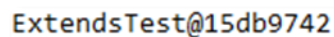
```
Object.class
232  *
233  * @return a string representation of the object.
234  */
235  public String toString() {
236      return getClass().getName() + "@" + Integer.toHexString(hashCode());
237  }
238
```

图 12-3：Object 类中 toString() 方法源代码

我们来尝试调用这个方法，请看代码以及执行结果：

1	public class ExtendsTest{
2	public static void main(String[] args) {
3	ExtendsTest et = new ExtendsTest();
4	String s = et.toString();
5	System.out.println(s);
6	}
7	}

运行结果如下图所示：



```
ExtendsTest@15db9742
```

图 12-4：toString() 方法执行结果

虽然输出结果看不懂，但是起码看到这个字符串当中确实是有一个“@”符号的，toString() 方法确实被 ExtendsTest 类继承过来了。

12.3 章节小结

通过本章节的学习，要求大家理解什么是继承，为什么要继承，继承有什么好处，继承的特性有哪些。另外要求大家掌握的是继承的代码如何实现。还有 Java 中任何一个类的定义都会自带 Object 类的特征，这是因为 Object 是所有类的超类。

12.4 难点解惑

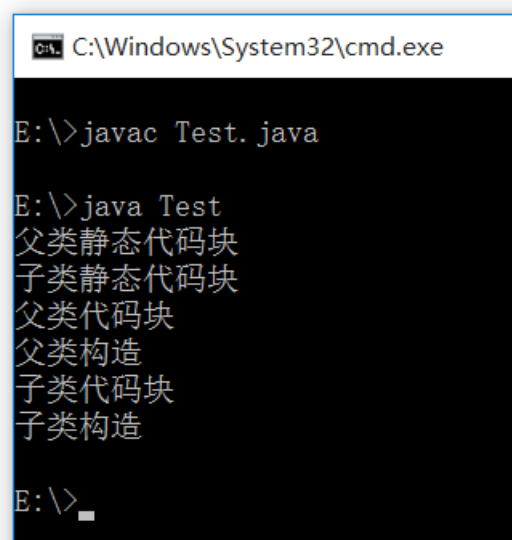
对于本章节内容来说，难点主要是类和类继承之后的代码执行顺序，请看程序：

```
public class Test {
    public static void main(String[] args) {
        new H2();
    }
}
class H1 {
    {
        System.out.println("父类代码块");
    }
    public H1(){
        System.out.println("父类构造");
    }
    static{
        System.out.println("父类静态代码块");
    }
}
class H2 extends H1 {
    static{
        System.out.println("子类静态代码块");
    }
    {
```



```
        System.out.println("子类代码块");
    }
    public H2(){
        System.out.println("子类构造");
    }
}
```

我们对以上的程序进行分析，子类 H2 继承 H1，new H2()执行的时候，会先进行类加载，先加载 H2 的父类 H1，所以 H1 当中的静态代码块先执行，然后再执行 H2 中的静态代码块，静态代码块执行结束之后，不会马上执行构造方法，代码块会先执行，Java 中有一条规则：子类构造方法执行前先执行父类的构造方法（学习 super 之后大家就知道了），所以父类 H1 的代码块先执行，再执行 H1 的构造方法，然后再执行 H2 的代码块，最后执行 H2 的构造方法。我们对以上程序进行编译并运行，请看下图结果：



```
C:\Windows\System32\cmd.exe
E:\>javac Test.java
E:\>java Test
父类静态代码块
子类静态代码块
父类代码块
父类构造
子类代码块
子类构造
E:\>
```

图 12-5：Test 程序的编译和运行结果

通过以上的测试结果，可以看出以上我们的分析是正确的。

12.5 章节习题

第一题：定义猴子类，猴子有名字和性别等属性，并且定义猴子说话的方法，定义人类，人有名字和性别等属性，并且定义人说话的方法。使用继承，让代码具有复用性。

第二题：定义动物类，动物有名字属性，并且定义动物移动的方法，定义鱼类，鱼有名字属性，

有颜色属性，并且定义鱼移动的方法。使用继承，让代码具有复用性。

12.6 习题答案

第一题答案：

```
1 public class Monkey {
2     private String name;
3     private boolean sex;
4     public String getName() {
5         return name;
6     }
7     public void setName(String name) {
8         this.name = name;
9     }
10    public boolean isSex() {
11        return sex;
12    }
13    public void setSex(boolean sex) {
14        this.sex = sex;
15    }
16    public Monkey() {
17        super();
18    }
19    public Monkey(String name, boolean sex) {
20        super();
21        this.name = name;
22        this.sex = sex;
23    }
24    public void speak(){
25        System.out.println(name + "咿咿呀呀！");
```

26	}
27	}

1	public class People extends Monkey{
2	public void speak(){
3	System.out.println(this.getName() + ",呦， 小伙会说话了！");
4	}
5	}

第二题答案：

1	public class Animal {
2	private String name;
3	
4	public String getName() {
5	return name;
6	}
7	
8	public void setName(String name) {
9	this.name = name;
10	}
11	
12	public Animal() {
13	super();
14	}
15	
16	public Animal(String name) {
17	super();
18	this.name = name;

```
19     }  
20  
21     public void move(){  
22         System.out.println(name + " is moving!");  
23     }  
24 }
```

```
1 public class Fish extends Animal{  
2     public void move(){  
3         System.out.println(this.getName() + "正在水里欢快的游来游去！");  
4     }  
5 }
```

13. 第十三章 方法覆盖和多态（Polymorphism）

13.1 章节目标与知识框架

13.1.1 章节目标

理解在什么情况下我们需要进行方法覆盖？掌握在满足什么条件的时候构成方法覆盖？什么是多态，代码怎么写？向上转型和向下转型都是什么？多态在开发中有什么作用？

13.1.2 知识框架



13.2 方法覆盖 Override

13.2.1 什么时候需要方法覆盖 (理解)

学习方法覆盖之前，我们先来回顾一下方法重载（overload），什么情况下考虑使用方法重载呢？在同一个类当中，如果功能相似，尽可能将方法名定义得相同，这样方便调用的同时代码也会美观。那么，代码满足什么条件的时候能够构成方法重载呢？只要在同一个类当中，方法名相同，参数列表不同（类型、个数、顺序），即构成方法重载。

带着同样的疑问去学习方法覆盖，什么是方法覆盖？什么情况下考虑方法覆盖？代码怎么写的时候就构成了方法覆盖呢？接下来看一段代码：

```
1 public class People {
2     private String name;
3     public String getName() {
4         return name;
5     }
6     public void setName(String name) {
7         this.name = name;
8     }
9     public void speakHi(){
```

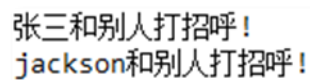
10	System.out.println(this.name + "和别人打招呼!");
11	}
12	}

1	public class ChinaPeople extends People {
2	//中国人
3	}

1	public class AmericaPeople extends People {
2	//美国人
3	}

1	public class PeopleTest {
2	public static void main(String[] args) {
3	ChinaPeople cp = new ChinaPeople();
4	cp.setName("张三");
5	cp.speakHi();
6	AmericaPeople ap = new AmericaPeople();
7	ap.setName("jackson");
8	ap.speakHi();
9	}
10	}

运行结果如下图所示：



```
张三和别人打招呼!  
jackson和别人打招呼!
```

图 13-1：运行结果

“中国人”调用 `speakHi()` 方法希望输出的结果是“你好，我叫张三，很高兴见到你！”，“美国人”调用 `speakHi()` 方法更希望输出的结果是“Hi,My name is jackson,Nice to meet you!”，可见 `ChinaPeople` 和 `AmericaPeople` 从父类中继承过来的 `speakHi()` 方法已经不够子类使用了，那这个时候应该怎么办呢？当然，此时就需要使用方法覆盖机制了。

13.2.2 怎么进行方法覆盖（掌握）

针对上一小节的问题，我们应该怎么进行方法覆盖呢，请看以下代码：

```
1 public class People {  
2     private String name;  
3     public String getName() {  
4         return name;  
5     }  
6     public void setName(String name) {  
7         this.name = name;  
8     }  
9     public void speakHi(){  
10        System.out.println(this.name + "和别人打招呼!");  
11    }  
12 }
```

```
1 public class ChinaPeople extends People {  
2     public void speakHi(){  
3         System.out.println("你好，我叫"+this.getName()+"，很高兴认识你！");  
4     }  
5 }
```

```
1 public class AmericaPeople extends People {  
2     public void speakHi(){
```

```
3         System.out.println("Hi,My name is "+this.getName()+",Nice to meet  
4 you!");  
5     }  
6 }
```

```
1 public class PeopleTest {  
2     public static void main(String[] args) {  
3         ChinaPeople cp = new ChinaPeople();  
4         cp.setName("张三");  
5         cp.speakHi();  
6         AmericaPeople ap = new AmericaPeople();  
7         ap.setName("jackson");  
8         ap.speakHi();  
9     }  
10 }
```

运行结果如下图所示：

```
你好，我叫张三，很高兴认识你！  
Hi,My name is jackson,Nice to meet you!
```

图 13-2：方法覆盖之后的运行结果

以上程序中 ChinaPeople 和 AmericaPeople 将从 People 类中继承过来的 speakHi() 方法进行了覆盖，我们也看到了当 speakHi() 方法发生覆盖之后，子类对象会调用覆盖之后的方法，不会再去调用之前从父类中继承过来的方法。

那么，到底在什么情况下我们会考虑使用方法覆盖呢？通过以上内容的学习，我们了解到只有当从父类中继承过来的方法无法满足当前子类业务需求的时候，需要将父类中继承过来的方法进行覆盖。换句话说，父类中继承过来的方法已经不够用了，子类有必要将这个方法重新再写一遍，所以方法覆盖又被称为方法重写。当该方法被重写之后，子类对象一定会调用重写之后的方法。

13.2.3 方法覆盖的条件及注意事项（掌握）

那么，当程序具备哪些条件的时候，就能构成方法覆盖呢？

- ① 方法覆盖发生在具有继承关系的父子类之间，这是首要条件；
- ② 覆盖之后的方法与原方法具有相同的返回值类型、相同的方法名、相同的形式参数列表；

另外，在使用方法覆盖的时候，需要有哪些注意事项呢？

- ① 由于覆盖之后的方法与原方法一模一样，建议在开发的时候采用复制粘贴的方式，不建议手写，因为手写的时候非常容易出错，比如在 `Object` 类当中有 `toString()` 方法，该方法中的 `S` 是大写的，在手写的时候很容易写成小写 `tostring()`，这个时候你会认为 `toString()` 方法已经被覆盖了，但由于方法名不一致，导致最终没有覆盖，这样就尴尬了；
- ② 私有的方法不能被继承，所以不能被覆盖；
- ③ 构造方法不能被继承，所以也不能被覆盖；
- ④ 覆盖之后的方法不能比原方法拥有更低的访问权限，可以更高（学习了访问控制权限修饰符之后你就明白了）；
- ⑤ 覆盖之后的方法不能比原方法抛出更多的异常，可以相同或更少（学习了异常之后就明白了）；
- ⑥ 方法覆盖只是和方法有关，和属性无关；
- ⑦ 静态方法不存在覆盖（不是静态方法不能覆盖，是静态方法覆盖意义不大，学习了多态机制之后就明白了）；

以上的注意事项还需要大家记忆，多下点功夫吧。接下来我们再来看一段代码，对方法覆盖加深一下印象，业务需求是这样的：定义一个动物类，所有动物都有移动的行为，其中猫类型的对象在移动的时候输出“猫在走猫步！”，鸟儿类型的对象在移动的时候输出“鸟儿在飞翔！”，但是猫类型的对象具有一个特殊的行为，抓老鼠，这个行为不是所有动物对象都有的，是猫类型对象特有的：

1	<code>public class Animal {</code>
2	<code> public void move(){</code>
3	<code> System.out.println("动物在移动！");</code>
4	<code> }</code>
5	<code>}</code>

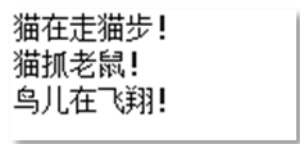
1	<code>public class Cat extends Animal{</code>
2	<code> public void move(){</code>

```
3         System.out.println("猫在走猫步！");
4     }
5     public void catchMouse(){
6         System.out.println("猫抓老鼠！");
7     }
8 }
```

```
1 public class Bird extends Animal{
2     public void move(){
3         System.out.println("鸟儿在飞翔！");
4     }
5 }
```

```
1 public class Test {
2     public static void main(String[] args) {
3         Cat cat = new Cat();
4         cat.move();
5         cat.catchMouse();
6         Bird bird = new Bird();
7         bird.move();
8     }
9 }
```

运行结果如下图所示：



```
猫在走猫步！
猫抓老鼠！
鸟儿在飞翔！
```

图 13-3： 方法覆盖演示

对方法覆盖总结一下，当父类中继承过来的方法无法满足当前子类业务需求的时候，子类有必要将父类中继承过来的方法进行覆盖/重写。方法覆盖发生在具有继承关系的父子类之间，方法覆盖的时候要求相同的返回值类型、相同的方法名、相同的形式参数列表。方法覆盖之后子类对象在调用的时候一定会执行覆盖之后的方法。

13.3 多态

13.3.1 多态基础语法（掌握）

多态（Polymorphism）属于面向对象三大特征之一，它的前提是封装形成独立体，独立体之间存在继承关系，从而产生多态机制。多态是同一个行为具有多个不同表现形式或形态的能力。现实中，比如我们按下 F1 键这个动作：

- 如果当前在 Flash 界面下弹出的就是 AS 3 的帮助文档；
- 如果当前在 Word 下弹出的就是 Word 帮助；
- 如果当前在 Windows 下弹出的就是 Windows 帮助和支持。

多态就是“同一个行为”发生在“不同的对象上”会产生不同的效果。那么在 java 中多态是如何体现的呢？

在 java 中允许这样的两种语法出现，一种是向上转型（Upcasting），一种是向下转型（Downcasting），向上转型是指子类型转换为父类型，又被称为自动类型转换，向下转型是指父类型转换为子类型，又被称为强制类型转换。请看下图：

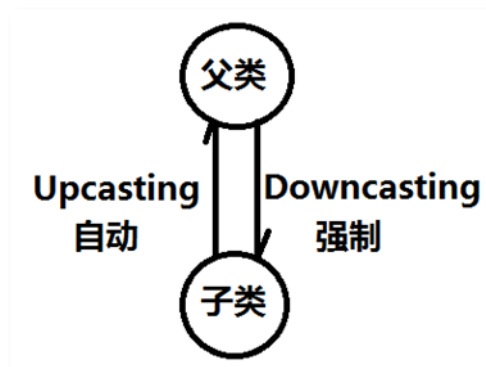


图 13-4：向上转型和向下转型

在 java 语言中有这样的一个规定，无论是向上转型还是向下转型，两种类型之间必须要有

继承关系，没有继承关系情况下进行向上转型或向下转型的时候编译器都会报错，这一点要死记硬背哦！

接下来我们来看一段代码：

1	public class Animal {
2	public void move(){
3	System.out.println("Animal move!");
4	}
5	}

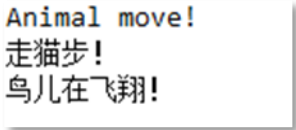
1	public class Cat extends Animal{
2	//方法覆盖
3	public void move(){
4	System.out.println("走猫步！");
5	}
6	//子类特有
7	public void catchMouse(){
8	System.out.println("抓老鼠！");
9	}
10	}

1	public class Bird extends Animal{
2	//方法覆盖
3	public void move(){
4	System.out.println("鸟儿在飞翔！");
5	}
6	//子类特有
7	public void sing(){
8	System.out.println("鸟儿在歌唱！");

9	}
10	}

1	public class Test01 {
2	public static void main(String[] args) {
3	//创建 Animal 对象
4	Animal a = new Animal();
5	a.move();
6	//创建 Cat 对象
7	Cat c = new Cat();
8	c.move();
9	//创建鸟儿对象
10	Bird b = new Bird();
11	b.move();
12	}
13	}

运行结果如下图所示：



```
Animal move!  
走猫步!  
鸟儿在飞翔!
```

图 13-5：运行结果

其实在 java 中还允许这样写代码，请看：

1	public class Test02 {
2	public static void main(String[] args) {
3	Animal a1 = new Cat();

4	a1.move();
5	Animal a2 = new Bird();
6	a2.move();
7	}
8	}

运行结果如下图所示：

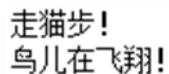


图 13-6：运行结果

以上程序演示的就是多态，多态就是“同一个行为（move）”作用在“不同的对象上”会有不同的表现结果。java 中之所以有多态机制，是因为java 允许一个父类型的引用指向一个子类型的对象。也就是说允许这种写法：Animal a2 = new Bird()，因为 Bird is a Animal 是能够说通的。其中 Animal a1 = new Cat()或者 Animal a2 = new Bird()都是父类型引用指向了子类型对象，都属于向上转型（Upcasting），或者叫做自动类型转换。

我来解释一下这段代码片段【Animal a1 = new Cat();a1.move();】：java 程序包括编译和运行两个阶段，分析 java 程序一定要先分析编译阶段，然后再分析运行阶段，在编译阶段编译器只知道 a1 变量的数据类型是 Animal，那么此时编译器会去 Animal.class 字节码中查找 move()方法，发现 Animal.class 字节码中存在 move()方法，然后将该 move()方法绑定到 a1 引用上，编译通过了，这个过程我们可以理解为“静态绑定”阶段完成了。紧接着程序开始运行，进入运行阶段，在运行的时候实际上在堆内存中 new 的对象是 Cat 类型，也就是说真正在 move 移动的时候，是 Cat 猫对象在移动，所以运行的时候就会自动执行 Cat 类当中的 move()方法，这个过程可以称为“动态绑定”。但无论是什么时候，必须先“静态绑定”成功之后才能进入“动态绑定”阶段。

来看以下的一段代码以及编译结果：

1	public class Test03 {
2	public static void main(String[] args) {
3	Animal a = new Cat();

4	a.catchMouse();
5	}
6	}

编译结果:

```
C:\>javac Test03.java
Test03.java:4: 错误: 找不到符号
        a.catchMouse<>;
           ^
符号:   方法 catchMouse<>
位置:  类型为Animal的变量 a
1 个错误
```

图 13-7: 编译错误信息

有人认为 Cat 猫是可以抓老鼠的呀, 为什么会编译报错呢? 那是因为 “Animal a = new Cat();” 在编译的时候, 编译器只知道 a 变量的数据类型是 Animal, 也就是说它只会去 Animal.class 字节码中查找 catchMouse()方法, 结果没找到, 自然 “静态绑定” 就失败了, 编译没有通过。就像以上描述的错误信息一样: 在类型为 Animal 的变量 a 中找不到方法 catchMouse()。

那么, 假如说我就是想让这只猫去抓老鼠, 以上代码应该如何修改呢? 请看以下代码:

1	public class Test04 {
2	public static void main(String[] args) {
3	//向上转型
4	Animal a = new Cat();
5	//向下转型: 为了调用子类对象特有的方法
6	Cat c = (Cat)a;
7	c.catchMouse();
8	}
9	}

运行结果如下图所示：

抓老鼠！

图 13-8：向下转型

我们可以看到直接使用 a 引用是无法调用 catchMouse()方法的,因为这个方法属于子类 Cat 中特有的行为,不是所有 Animal 动物都可以抓老鼠的,要想让它去抓老鼠,就必须做向下转型 (Downcasting),也就是使用强制类型转换将 Animal 类型的 a 引用转换成 Cat 类型的引用 c (Cat c = (Cat)a;),使用 Cat 类型的 c 引用调用 catchMouse()方法。

通过这个案例,可以得出:只有在访问子类型中特有数据的时候,需要先进行向下转型。其实向下转型就是用在这种情形之下。那么向下转型会存在什么风险吗?请看以下代码:

```
3 public class Test05 {  
4     public static void main(String[] args) {  
5         Animal a = new Bird();  
6         Cat c = (Cat)a;  
7     }  
8 }
```

以上代码可以编译通过吗?答案是可以的,为什么呢?那是因为编译器只知道 a 变量是 Animal 类型,Animal 类和 Cat 类之间存在继承关系,所以可以进行向下转型(前面提到过,只要两种类型之间存在继承关系,就可以进行向上或向下转型),语法上没有错误,所以编译通过了。但是运行的时候会出问题吗,因为毕竟 a 引用指向的真实对象是一只小鸟。来看运行结果:

```
Exception in thread "main" java.lang.ClassCastException: com.Bird cannot be cast to com.Cat  
at com.Test05.main(Test05.java:6)
```

图 13-9：类型转换异常

以上的异常是很常见的 ClassCastException,翻译为类型转换异常,这种异常通常出现在向

下转型的操作过程当中，当类型不兼容的情况下进行转型出现的异常，之所以出现此异常是因为在程序运行阶段 a 引用指向的对象是一只小鸟，然后我们要将一只小鸟转换成一只猫，这显然是不合理的，因为小鸟和猫之间是没有继承关系的。为了避免这种异常的发生，建议在进行向下转型之前进行运行期类型判断，这就需要我们学习一个运算符了，它就是 instanceof。

instanceof 运算符的语法格式是这样的：

(引用 instanceof 类型)

instanceof 运算符的运算结果是布尔类型，可能是 true，也可能是 false，假设(c instanceof Cat) 结果是 true 则表示在运行阶段 c 引用指向的对象是 Cat 类型，如果结果是 false 则表示在运行阶段 c 引用指向的对象不是 Cat 类型。有了 instanceof 运算符，向下转型就可以这样写了：

1	public class Test05 {
2	public static void main(String[] args) {
3	Animal a = new Bird();
4	if(a instanceof Cat){
5	Cat c = (Cat)a;
6	c.catchMouse();
7	}
8	}
9	}

以上程序运行之后不再发生异常，并且什么也没有输出，那是因为 if 语句的条件并没有成立，因为在运行阶段 a 引用指向的对象不是 Cat 类型，所以(a instanceof Cat)是 false，自然就不会进行向下转型了，也不会出现 ClassCastException 异常了。在实际开发中，java 中有这样一条默认规范需要大家记住：在进行任何向下转型的操作之前，要使用 instanceof 进行判断，这是一个很好的编程习惯。就像下面的代码：

1	public class Test05 {
2	public static void main(String[] args) {
3	Animal a = new Bird();
4	if(a instanceof Cat){
5	Cat c = (Cat)a;

```
6         c.catchMouse();
7     }else if(a instanceof Bird){
8         Bird b = (Bird)a;
9         b.sing();
10    }
11 }
12 }
```

运行结果如下图所示：

鸟儿在歌唱！

图 13-10：向下转型前判断

到这里大家理解什么是多态了吗？其实多态存在的三个必要条件分别是：

- ① 继承
- ② 方法覆盖
- ③ 父类型引用指向子类型对象

多态显然是离不开方法覆盖机制的，多态就是因为编译阶段绑定父类当中的方法，程序运行阶段自动调用子类对象上的方法，如果子类对象上的方法没有进行重写，这个时候创建子类对象就没有意义了，自然多态也就没有意义了，只有子类将方法重写之后调用到子类对象上的方法产生不同效果时，多态就形成了。实际上方法覆盖机制和多态机制是捆绑的，谁也离不开谁，多态离不开方法覆盖，方法覆盖离开了多态也就没有意义了。

接下里就来看看之前没有解决的问题：方法覆盖主要是说实例方法，静态方法为什么不谈方法覆盖？

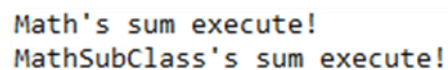
```
1 public class OverrideTest {
2     public static void main(String[] args) {
3         Math.sum();
4         MathSubClass.sum();
5     }
```

```
6 }
```

```
1 public class Math{
2     public static void sum(){
3         System.out.println("Math's sum execute!");
4     }
5 }
```

```
1 public class MathSubClass extends Math{
2     //尝试覆盖从父类中继承过来的静态方法
3     public static void sum(){
4         System.out.println("MathSubClass's sum execute!");
5     }
6 }
```

运行结果如下图所示：



```
Math's sum execute!
MathSubClass's sum execute!
```

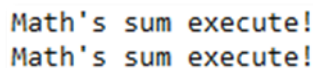
图 13-11：尝试覆盖静态方法

我们发现貌似也发生了覆盖，在程序运行的时候确实也调用了“子类 MathSubClass”的 sum 方法，但这种“覆盖”有意义吗？其实上面的课程我们已经说过了，方法覆盖和多态机制联合起来才有意义，我们来看看这种“覆盖”是否能够达到“多态”的效果，请看代码：

```
1 public class OverrideTest {
2     public static void main(String[] args) {
3         Math m = new MathSubClass();
4         m.sum();
5     }
6 }
```

5	m = null;
6	m.sum();
7	}
8	}

运行结果如下图所示：



```
Math's sum execute!
Math's sum execute!
```

图 13-12：运行结果

通过以上的代码，我们发现虽然创建了子类型对象“new MathSubClass()”，但是程序在运行的时候仍然调用的是 Math 类当中的 sum 方法，甚至 m = null 的时候再去调用 m.sum() 也没有出现空指针异常，这说明静态方法的执行压根和对象无关，既然和对象无关那就表示和多态无关，既然和多态无关，也就是说静态方法的“覆盖”是没有意义的，所以通常我们不谈静态方法的覆盖。

13.3.2 多态在开发中的作用（理解）

以上学习了多态的基础语法，多态在实际开发中有什么作用呢？我们先来了解一个业务背景：请设计一个系统，描述主人喂养宠物的场景，首先在这个场景当中应该有“宠物对象”，宠物对象应该有一个吃的行为，另外还需要一个“主人对象”，主人对象应该有一个喂的行为，请看代码：

1	//宠物狗
2	public class Dog {
3	String name;
4	public Dog(String name){
5	this.name = name;
6	}
7	//吃的行为

```
8     public void eat(){
9         System.out.println(this.name + "在啃肉骨头! ");
10    }
11 }
```

```
1 //主人
2 public class Master {
3     //喂养行为
4     public void feed(Dog dog){
5         //主人喂养宠物，宠物就吃
6         System.out.println("主人开始喂食儿");
7         dog.eat();
8         System.out.println("主人喂食儿完毕");
9     }
10 }
```

```
1 public class Test {
2     public static void main(String[] args) {
3         //创建狗对象
4         Dog dog = new Dog("二哈");
5         //创建主人对象
6         Master master = new Master();
7         //喂养
8         master.feed(dog);
9     }
10 }
```

运行结果如下图所示：

```
主人开始喂食儿  
二哈在啃肉骨头!  
主人喂食儿完毕
```

图 13-13：运行结果

以上程序编译和运行都很正常，输出结果也是对的，那么存在什么问题吗？假设后期用户提出了新的需求，软件可能面临着功能扩展，这个扩展会很方便吗？假设现在主人家里又来了一个宠物猫，那该怎么办呢？请看代码：

在以上代码的基础之上，新增了一个 Cat 类，来表示宠物猫，这个对于程序来说是可以接受的：

```
1 //宠物猫
2 public class Cat {
3     String name;
4     public Cat(String name){
5         this.name = name;
6     }
7     //吃的行为
8     public void eat(){
9         System.out.println(this.name + "在吃鱼！");
10    }
11 }
```

另外，除了增加一个 Cat 类之外，我们还需要“修改”Master 主人类的源代码，这件事儿是我们程序员无法容忍的，因为修改之前写好的源代码就面临着重新编译、重新全方位的测试，这是一个巨大的工作，维护成本很高，也很麻烦：

```
1 //主人
2 public class Master {
3     //喂养行为
```

```
4      public void feed(Dog dog){
5          //主人喂养宠物，宠物就吃
6          System.out.println("主人开始喂食儿");
7          dog.eat();
8          System.out.println("主人喂食儿完毕");
9      }
10     //喂养行为
11     public void feed(Cat cat){
12         //主人喂养宠物，宠物就吃
13         System.out.println("主人开始喂食儿");
14         cat.eat();
15         System.out.println("主人喂食儿完毕");
16     }
17 }
```

```
1 public class Test {
2     public static void main(String[] args) {
3         //创建狗对象
4         Dog dog = new Dog("二哈");
5         //创建主人对象
6         Master master = new Master();
7         //喂养
8         master.feed(dog);
9         //创建猫对象
10        Cat cat = new Cat("汤姆");
11        //喂养
12        master.feed(cat);
13    }
14 }
```

运行结果如下图所示：

```
主人开始喂食儿
二哈在啃肉骨头!
主人喂食儿完毕
主人开始喂食儿
汤姆在吃鱼!
主人喂食儿完毕
```

图 13-14：运行结果

在软件开发过程中，有这样的一个开发原则：开闭原则。开闭原则（OCP）是面向对象设计中“可复用设计”的基石，是面向对象设计中最重要原则之一，其它很多的设计原则都是实现开闭原则的一种手段。1988 年，勃兰特·梅耶（Bertrand Meyer）在他的著作《面向对象软件构造（Object Oriented Software Construction）》中提出了开闭原则，它的原文是这样：

“Software entities should be open for extension, but closed for modification”。翻译过来就是：“软件实体应当对扩展开放，对修改关闭”。这句话说得略微有点专业，我们把它讲得更通俗一点，也就是：软件系统中包含的各种组件，例如模块（Modules）、类（Classes）以及功能（Functions）等等，应该在不修改现有代码的基础上，引入新功能。开闭原则中“开”，是指对于组件功能的扩展是开放的，是允许对其进行功能扩展的；开闭原则中“闭”，是指对于原有代码的修改是封闭的，即修改原有的代码对外部的使用是透明的。

以上程序在扩展的过程当中就违背了 OCP 原则，因为在扩展的过程当中修改了已经写好的 Master 类，怎样可以解决这个问题呢？多态可以解决，请看代码：

1	//宠物类
2	public class Pet {
3	String name;
4	//吃的行为
5	public void eat(){
6	}
7	}

1	//宠物猫
---	-------


```
2 public class Cat extends Pet{
3     public Cat(String name){
4         this.name = name;
5     }
6     //吃的行为
7     public void eat(){
8         System.out.println(this.name + "在吃鱼！");
9     }
10 }
```

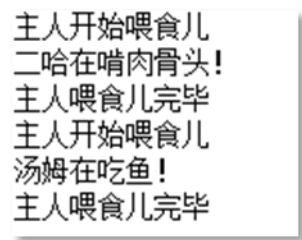
```
1 //宠物狗
2 public class Dog extends Pet{
3     public Dog(String name){
4         this.name = name;
5     }
6     //吃的行为
7     public void eat(){
8         System.out.println(this.name + "在啃肉骨头！");
9     }
10 }
```

```
1 //主人
2 public class Master {
3     //喂养行为
4     public void feed(Pet pet){
5         //主人喂养宠物，宠物就吃
6         System.out.println("主人开始喂食儿");
7         pet.eat();
8         System.out.println("主人喂食儿完毕");
9     }
10 }
```

9	}
10	}

1	public class Test {
2	public static void main(String[] args) {
3	//创建狗对象
4	Dog dog = new Dog("二哈");
5	//创建主人对象
6	Master master = new Master();
7	//喂养
8	master.feed(dog);
9	//创建猫对象
10	Cat cat = new Cat("汤姆");
11	//喂养
12	master.feed(cat);
13	}
14	}

运行结果如下图所示：



```
主人开始喂食儿
二哈在啃肉骨头!
主人喂食儿完毕
主人开始喂食儿
汤姆在吃鱼!
主人喂食儿完毕
```

图 13-15：使用多态机制

在以上程序中，Master 类中的方法 feed(Pet pet)的参数类型定义为更加抽象的 Pet 类型，而不是具体 Dog 宠物，或者 Cat 宠物，显然 Master 类和具体的 Dog、Cat 类解耦合了，依赖性弱了，这就是我们通常所说的面向抽象编程，尽量不要面向具体编程，面向抽象编程会让你的代

码耦合度降低，扩展能力增强，从而符合 OCP 的开发原则。假如说这会再来一个新的宠物猪呢，我们只需要这样做，新增加一个“宠物猪类”，然后宠物猪类 **Pig** 继承宠物类 **Pet**，并重写 **eat()**方法，然后修改一下测试类就行了，整个过程我们是不需要修改 **Master** 类的，只是额外增加了一个新的类：

```
1 public class Pig extends Pet {
2     public Pig(String name){
3         this.name = name;
4     }
5     //吃的行为
6     public void eat(){
7         System.out.println(this.name + "在吃粥！");
8     }
9 }
```

```
1 public class Test {
2     public static void main(String[] args) {
3         //创建狗对象
4         Dog dog = new Dog("二哈");
5         //创建主人对象
6         Master master = new Master();
7         //喂养
8         master.feed(dog);
9         //创建猫对象
10        Cat cat = new Cat("汤姆");
11        //喂养
12        master.feed(cat);
13        //创建宠物猪对象
14        Pig pig = new Pig("小猪猪");
15        master.feed(pig);
16    }
17 }
```

16	}
17	}

运行结果如下图所示：

```
主人开始喂食儿  
二哈在啃肉骨头!  
主人喂食儿完毕  
主人开始喂食儿  
汤姆在吃鱼!  
主人喂食儿完毕  
主人开始喂食儿  
小猪猪在吃粥!  
主人喂食儿完毕
```

图 13-16：运行结果

以上程序中到底哪里使用了多态机制呢？请看下图：

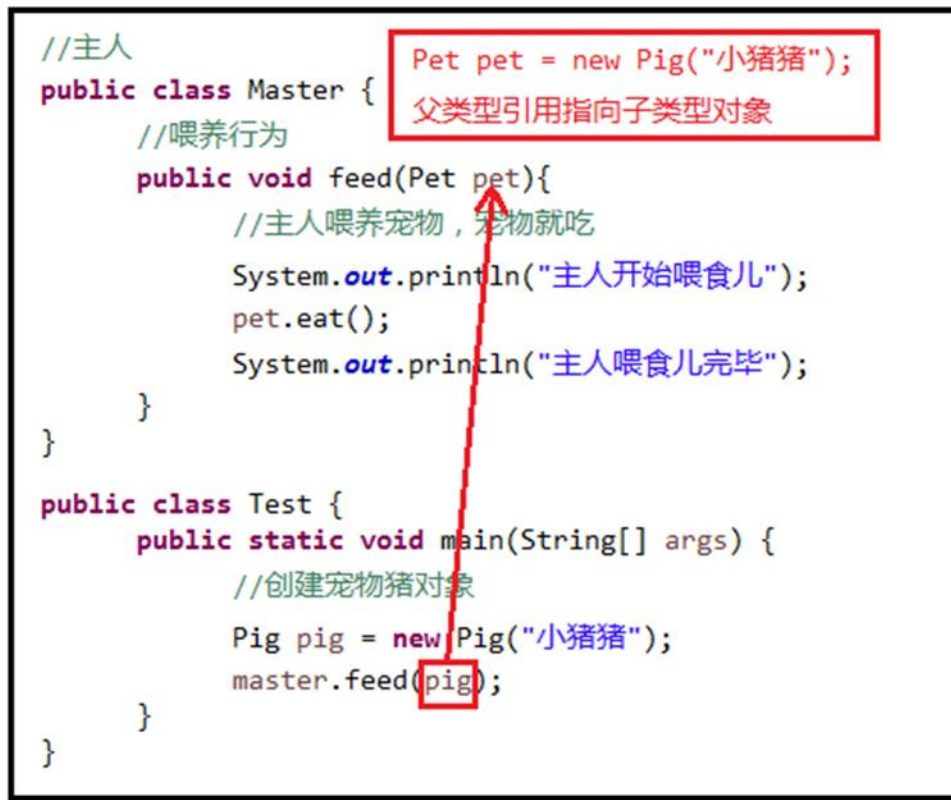


图 13-17：哪里使用了多态机制

通过以上内容的学习，我们可以看到多态在开发中联合方法覆盖一起使用，可以降低程序的耦合度，提高程序的扩展力。在开发中尽可能面向抽象编程，不要面向具体编程，好比电脑主板和内存条的关系一样，主板和内存条件之间有一个抽象的符合某个规范的插槽，不同品牌的内存条都可以插到主板上使用，2个G的内存条和4个G的内存条都可以插上，但最终的表现结果是不同的，2个G的内存条处理速度慢一些，4个G的快一些，这就是多态，所谓多态就是同一个行为作用到不同的对象上，最终的表现结果是不同的，主要的要求就是对象是可以进行灵活切换的，灵活切换的前提就是解耦合，解耦合依赖多态机制。

13.4 章节小结

通过本章节内容的学习，大家需要理解什么时候考虑使用方法覆盖，理解满足什么条件时能够构成方法覆盖，以及掌握怎么进行方法覆盖。另外还需要掌握多态相关的基础语法，理解多态在开发中起到了解耦合的作用，提倡面向抽象编程，不要面向具体编程。理解 OCP 开发原则等。

13.5 难点解惑

本章节的难点就是对多态机制的理解，多态的代码表现是父类型引用指向子类型对象，对于多态的理解一定要分为编译阶段和运行阶段来进行分析，编译阶段只是看父类型中是否存在要调用的方法，如果父类中不存在，则编译器会报错，编译阶段和具体 new 的对象无关。但是在运行阶段就要看底层具体 new 的是哪个类型的子对象了，new 的这个子类型对象可以看做“真实对象”，自然在运行阶段就会调用真实对象的相关方法。例如代码：`Animal a = new Cat(); a.move();`，在编译阶段编译器只能检测到 a 的类型是 `Animal`，所以一定会去 `Animal` 类中找 `move()` 方法，如果 `Animal` 中没有 `move()` 方法，则编译器会报错，即使 `Cat` 中有 `move()` 方法，也会报错，因为编译器只知道 a 的类型是 `Animal` 类，只有在运行的时候，实际创建的真实对象是 `Cat`，那么在运行的时候就会自动调用 `Cat` 对象的 `move()` 方法。这样就可以达到多种形态，也就是说编译阶段一种形态，运行阶段的时候是另一种形态。这也就是多态的含义。

13.6 章节习题

第一题：实现愤怒的小鸟

我们有很多种小鸟，每种小鸟都有飞的行为，还有一个弹弓，弹弓有一个弹射的行为，弹射时把小鸟弹出去，之后小鸟使用自己飞行的行为飞向小猪（不要求编写小猪的代码）。不同种类的小鸟有不同飞行的方式：

红火：红色小鸟，飞行方式：正常

蓝冰：蓝色小鸟，飞行方式：分成 3 个

黄风：黄色小鸟，飞行方式：加速。

第二题：计算不同类型的员工薪资

定义员工类 `Employee`，员工包含姓名 `name`、出生月份 `birthMonth` 两个属性，员工有获取指定月份工资的方法（`getSalary(int month)`），如果该月员工生日，公司补助 250 元。

定义有固定工资的员工类 `SalariedEmployee`，有月薪 `monthlySalary` 属性。

定义小时工类 `HourlyEmployee`，包含工作小时数 `hours` 和每小时的工资 `hourlySalary` 属性，如果每月工作超过 160 小时，超过的部分按 1.5 倍工资发放。

定义销售人员类 `SalesEmployee`，包含月销售额 `sales` 和提成比例 `comm` 属性。

第三题：某汽车租赁公司有多种汽车可以出租，计算汽车租金

`Vehicle` 是所有车的父类，属性：品牌、车牌号，有返回总租金的方法：`public double getSumRent(int days){}`

小轿车类 `Car` 是 `Vehicle` 的子类，属性：车型（两厢、三厢、越野），两厢每天 300，

三厢每天 350，越野每天 500。

多座汽车类 Bus 是 Vehicle 的子类，属性：座位数，座位数 ≤ 16 的每天 400，座位数 > 16 的每天 600。

编写测试类，根据用户选择不同的汽车，计算总租金。

13.7 习题答案

第一题答案：

1	//小鸟
2	public class Bird {
3	//飞
4	public void fly(){
5	
6	}
7	}

1	public class RedBird extends Bird{
2	public void fly(){
3	System.out.println("正常的飞翔");
4	}
5	}

1	public class BlueBird extends Bird{
2	public void fly(){
3	System.out.println("变成了 3 只小鸟一起飞");
4	}
5	}

1	public class YellowBird extends Bird{
---	---------------------------------------

2	public void fly(){
3	System.out.println("加速飞翔");
4	}
5	}

1	//弹弓
2	public class Slingshot {
3	//射
4	public void shot(Bird bird){
5	//鸟儿飞
6	bird.fly();
7	}
8	}

1	public class Test {
2	public static void main(String[] args) {
3	Bird redBird = new RedBird();
4	Bird blueBird = new BlueBird();
5	Bird yellowBird = new YellowBird();
6	Slingshot ss = new Slingshot();
7	ss.shot(redBird);
8	ss.shot(blueBird);
9	ss.shot(yellowBird);
10	}
11	}

执行结果如下图所示：

正常的飞翔
变成了3只小鸟一起飞
加速飞翔

第二题答案:

```
1 public class Employee {
2     private String name;
3     private int birthMonth;
4     public String getName() {
5         return name;
6     }
7     public void setName(String name) {
8         this.name = name;
9     }
10    public int getBirthMonth() {
11        return birthMonth;
12    }
13    public void setBirthMonth(int birthMonth) {
14        this.birthMonth = birthMonth;
15    }
16    public double getSalary(int month){
17        return 0.0;
18    }
19 }
```

```
1 public class SalariedEmployee extends Employee {
2     private double monthlySalary;
3     public double getMonthlySalary() {
```

```
4         return monthlySalary;
5     }
6     public void setMonthlySalary(double monthlySalary) {
7         this.monthlySalary = monthlySalary;
8     }
9     public double getSalary(int month){
10         if(month == this.getBirthMonth()){
11             return monthlySalary + 250;
12         }
13         return monthlySalary;
14     }
15 }
```

```
1 public class HourlyEmployee extends Employee {
2     private int hours;
3     private double hourlySalary;
4     public int getHours() {
5         return hours;
6     }
7     public void setHours(int hours) {
8         this.hours = hours;
9     }
10    public double getHourlySalary() {
11        return hourlySalary;
12    }
13    public void setHourlySalary(double hourlySalary) {
14        this.hourlySalary = hourlySalary;
15    }
16    public double getSalary(int month){
17        double money = 0.0;
```

```
18         if(hours <= 160){
19             money = hourlySalary * hours;
20         }else{
21             money = hourlySalary * 160 + hourlySalary * (hours - 160) * 1.5;
22         }
23         if(month == this.getBirthMonth()){
24             money += 250;
25         }
26         return money;
27     }
28 }
```

```
1 public class SalesEmployee extends Employee {
2     private double sales;
3     private double comm;
4     public double getSales() {
5         return sales;
6     }
7     public void setSales(double sales) {
8         this.sales = sales;
9     }
10    public double getComm() {
11        return comm;
12    }
13    public void setComm(double comm) {
14        this.comm = comm;
15    }
16    public double getSalary(int month){
17        if(month == this.getBirthMonth()){
18            return sales * comm + 250;
```

```
19     }
20     return sales * comm;
21 }
22 }
```

```
1 public class Test {
2     public static void main(String[] args) {
3         SalariedEmployee e1 = new SalariedEmployee();
4         e1.setName("张三");
5         e1.setBirthMonth(2);
6         e1.setMonthlySalary(5000.0);
7         System.out.println(e1.getName() + "的 2 月份工资是" + e1.getSalary(2));
8         HourlyEmployee e2 = new HourlyEmployee();
9         e2.setName("李四");
10        e2.setBirthMonth(2);
11        e2.setHourlySalary(50);
12        e2.setHours(170);
13        System.out.println(e2.getName() + "的 2 月份工资是" + e2.getSalary(2));
14        SalesEmployee e3 = new SalesEmployee();
15        e3.setName("王五");
16        e3.setBirthMonth(2);
17        e3.setSales(10000.0);
18        e3.setComm(0.5);
19        System.out.println(e3.getName() + "的 2 月份工资是" + e3.getSalary(2));
20    }
21 }
```

执行结果如下图所示：

张三的2月份工资是5250.0
李四的2月份工资是9000.0
王五的2月份工资是5250.0

第三题答案:

```
1 public class Vehicle {
2     private String brand;
3     private String licensePlateNumber;
4     public String getBrand() {
5         return brand;
6     }
7     public void setBrand(String brand) {
8         this.brand = brand;
9     }
10    public String getLicensePlateNumber() {
11        return licensePlateNumber;
12    }
13    public void setLicensePlateNumber(String licensePlateNumber) {
14        this.licensePlateNumber = licensePlateNumber;
15    }
16    public Vehicle() {
17        super();
18    }
19    public double getSumRent(int days){
20        return 0.0;
21    }
22 }
```

```
1 public class Car extends Vehicle {
2
3     //车型：两厢(300/天)、三厢(350/天)、越野(500/天)
4     private String models;
5
6     public String getModels() {
7         return models;
8     }
9
10    public void setModels(String models) {
11        this.models = models;
12    }
13
14    public Car() {
15        super();
16    }
17
18    public double getSumRent(int days){
19        if("两厢".equals(models)){
20            return 300 * days;
21        }else if("三厢".equals(models)){
22            return 350 * days;
23        }else{
24            return 500 * days;
25        }
26    }
27 }
```

```
1 public class Bus extends Vehicle {
2     //座位数
```

```
3     private int seats;
4
5     public int getSeats() {
6         return seats;
7     }
8
9     public void setSeats(int seats) {
10        this.seats = seats;
11    }
12
13    public Bus() {
14        super();
15    }
16
17    public double getSumRent(int days){
18        if(seats <= 16){
19            return 400 * days;
20        }else{
21            return 600 * days;
22        }
23    }
24 }
```

```
1 public class Test {
2     public static void main(String[] args) {
3         Car c = new Car();
4         c.setBrand("宝马");
5         c.setLicensePlateNumber("京 A 88888");
6         c.setModels("三厢");
7         System.out.println("品 牌： " + c.getBrand() + "， 车 牌 号： " +
```

```
8 c.getLicensePlateNumber() + ", 10 天总租金: " + c.getSumRent(10));
9
10     Bus b = new Bus();
11     b.setBrand("海格");
12     b.setLicensePlateNumber("京 B 88888");
13     b.setSeats(60);
14     System.out.println("品 牌: " + b.getBrand() + ", 车 牌 号: " +
15 b.getLicensePlateNumber() + ", 10 天总租金: " + b.getSumRent(10));
16 }
17 }
```

执行结果如下图所示:

```
品牌: 宝马, 车牌号: 京A 88888, 10天总租金: 3500.0
品牌: 海格, 车牌号: 京B 88888, 10天总租金: 6000.0
```

14. 第十四章 super

14.1 章节目标与知识框架

14.1.1 章节目标

掌握 super 都可以用在哪里? 理解 super 在内存方面的存储位置。掌握怎么通过子类的构造方法调用父类的构造方法。super 什么时候可以省略, 什么时候不能省略?

14.1.2 知识框架



14.2 super

14.2.1 super 概述（理解）

super 和 this 可以对比着学习：

① this

- this 是一个引用，保存内存地址指向自己。
- this 出现在实例方法中，谁调用这个实例方法，this 就代表谁，this 代表当前正在执行这个动作的对象。
- this 不能出现在静态方法中。
- this 大部分情况下可以省略，在方法中区分实例变量和局部变量的时候不能省略。
- “this(实际参数列表)” 出现在构造方法第一行，通过当前的构造方法去调用本类当中其它的构造方法。

② super

- 严格来说，super 其实并不是一个引用，它只是一个关键字，super 代表了当前对象中从父类继承过来的那部分特征。this 指向一个独立的对象，super 并不是指向某个“独立”的对象，假设张大明是父亲，张小明是儿子，有这样一句话：大家都说张小明的眼睛、鼻子和父亲的很像。那么也就是说儿子继承了父亲的眼睛和鼻子特征，那么眼睛和鼻子肯定最终还是长在儿子的身上。假设 this 指向张小明，那么 super 就代表张小明身上的眼睛和鼻子。换句话说 super 其实是 this 的一部分。如下图所示：张大明和张小明其实是两个独立的对象，两个对象内存方面没有联系，super 只是代表张小明对象身上的眼睛和鼻子，因为这个是从父类中继承过来的，在内存方面使用了 super 关键字进行了标记，对于下图来说“this.眼睛”和“super.眼睛”都是访问的同一块内存空间。

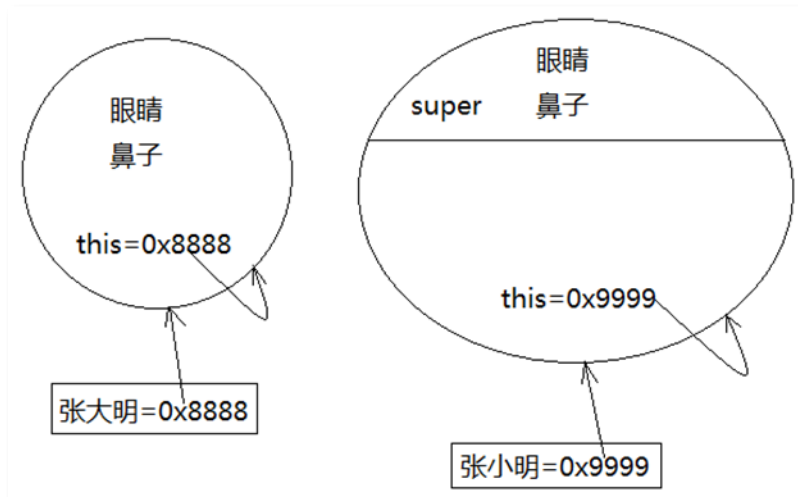


图 14-1: super 内存图

- `super` 和 `this` 都可以使用在实例方法当中。
- `super` 不能使用在静态方法当中，因为 `super` 代表了当前对象上的父类型特征，静态方法中没有 `this`，肯定也是不能使用 `super` 的。
- `super` 也有这种用法：“`super(实际参数列表);`”，这种用法是通过当前的构造方法调用父类的构造方法。

接下来，我们来测试一下：

```
1 public class SuperTest01 extends Object{
2     //实例方法
3     public void doSome(){
4         System.out.println(this);
5         System.out.println(super);
6     }
7 }
```

编译报错了：

```
C:\>javac SuperTest01.java
SuperTest01.java:5: 错误: 需要','
        System.out.println(super);
                           ^
SuperTest01.java:5: 错误: 需要'>'
        System.out.println(super);
                           ^
SuperTest01.java:5: 错误: 需要';'
        System.out.println(super);
                           ^
SuperTest01.java:7: 错误: 解析时已到达文件结尾
>
^
4 个错误
```

图 14-2: this 可以单独输出，super 不能

通过以上的测试，可以看出 this 是可以单独使用的引用，但 super 无法输出，编译器提示 super 要使用必须是“super.xxx”，显然 super 并不指向独立的对象，并不是保存某个对象的内存地址。

再来看另外的一个测试：

1	public class SuperTest02 extends Object{
2	//静态方法
3	public static void doSome(){
4	System.out.println(this);
5	System.out.println(super.toString());
6	}
7	}

编译报错了：

```
C:\>javac SuperTest02.java
SuperTest02.java:4: 错误: 无法从静态上下文中引用非静态 变量 this
    System.out.println(this);
                        ^
SuperTest02.java:5: 错误: 无法从静态上下文中引用非静态 变量 super
    System.out.println(super.toString());
                        ^
2 个错误
```

图 14-3: this 和 super 都不能使用在静态方法中

通过以上的测试，可以看出 this 和 super 都是无法使用在静态方法当中的。

14.2.2 super 使用在构造方法中（掌握）

super 使用在构造方法中，语法格式为：super(实际参数列表)，这行代码和“this(实际参数列表)”都是只允许出现在构造方法第一行（这一点记住就行了），所以这两行代码是无法共存的。“super(实际参数列表)”这种语法表示子类构造方法执行过程中调用父类的构造方法。我们来看一段代码：

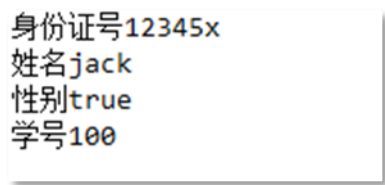
1	public class People {
2	String idCard;
3	String name;
4	boolean sex;
5	public People(){
6	}
7	public People(String idCard,String name,boolean sex){
8	this.idCard = idCard;
9	this.name = name;
10	this.sex = sex;
11	}
12	}

1	public class Student extends People{
---	--------------------------------------

```
2    //学号是子类特有的
3    int sno;
4    public Student(){
5
6    }
7    public Student(String idCard,String name,boolean sex,int sno){
8        this.idCard = idCard;
9        this.name = name;
10       this.sex = sex;
11       this.sno = sno;
12    }
13 }
```

```
1 public class StudentTest {
2     public static void main(String[] args) {
3         Student s = new Student("12345x","jack",true,100);
4         System.out.println("身份证号" + s.idCard);
5         System.out.println("姓名" + s.name);
6         System.out.println("性别" + s.sex);
7         System.out.println("学号" + s.sno);
8     }
9 }
```

运行结果如下图所示：



```
身份证号12345x
姓名jack
性别true
学号100
```

图 14-4： 运行结果

我们把上面的代码片段拿过来放在一起看看：

父类的构造方法：

1	public People(String idCard,String name,boolean sex){
2	this.idCard = idCard;
3	this.name = name;
4	this.sex = sex;
5	}

子类的构造方法：

1	public Student(String idCard,String name,boolean sex,int sno){
2	this.idCard = idCard;
3	this.name = name;
4	this.sex = sex;
5	this.sno = sno;
6	}

你有没有察觉到子类的构造方法前三行代码和父类构造方法中的代码是一样的？接下来把子类的构造方法修改一下，然后再运行测试程序：

1	public Student(String idCard,String name,boolean sex,int sno){
2	super(idCard,name,sex);
3	this.sno = sno;
4	}

运行结果如下图所示：

```
身份证号12345x  
姓名jack  
性别true  
学号100
```

图 14-5：运行结果

通过以上代码的学习，“super(实际参数列表);”语法表示调用父类的构造方法，代码复用性增强了，另外一方面也是模拟现实世界当中的“要想有儿子，必须先有父亲”的道理。不过这里的“super(实际参数列表)”在调用父类构造方法的时候，从本质上来说并不是创建一个“独立的父类对象”，而是为了完成当前对象的父类特征特征的初始化操作。（或者说通过子类的构造方法调用父类的构造方法，是为了让张小明身上长出具有他父亲特点的鼻子和眼睛，鼻子和眼睛初始化完毕之后，具有父亲的特点，但最终还是长在张小明的身上）。

接下来，再来看一段代码：

```
1 public class A {  
2     public A(){  
3         System.out.println("A 类的无参数构造方法执行");  
4     }  
5 }
```

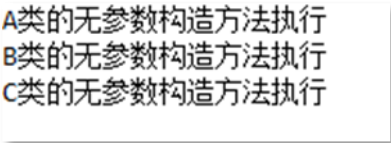
```
1 public class B extends A {  
2     public B(){  
3         System.out.println("B 类的无参数构造方法执行");  
4     }  
5 }
```

```
1 public class C extends B {  
2     public C(){  
3         System.out.println("C 类的无参数构造方法执行");  
4     }  
5 }
```

5	}
---	---

1	public class Test {
2	public static void main(String[] args) {
3	new C();
4	}
5	}

运行结果如下图所示：



A类的无参数构造方法执行
B类的无参数构造方法执行
C类的无参数构造方法执行

图 14-6: super()的测试

通过以上运行结果可以得出以下的等效代码：

1	public class A {
2	public A(){
3	//这里调用的是 Object 类中的无参数构造方法
4	//因为 A 类的父类是 Object
5	super();
6	System.out.println("A 类的无参数构造方法执行");
7	}
8	}

1	public class B extends A {
2	public B(){
3	super();

4	System.out.println("B 类的无参数构造方法执行");
5	}
6	}

1	public class C extends B {
2	public C(){
3	super();
4	System.out.println("C 类的无参数构造方法执行");
5	}
6	}

运行结果如下图所示：

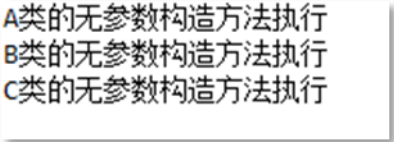


图 14-7：super()的测试

通过以上代码的测试我们得出，当一个构造方法第一行没有显示的调用“super(实际参数列表)”的话，系统默认调用父类的无参数构造方法“super()”。当然前提是“this(实际参数列表)”也没有显示的去调用(因为super()和this()都只能出现在构造方法第一行，所以不能并存)。我们可以通过以下程序再次测试一下：

1	public class A {
2	//有参数构造方法定义之后
3	//系统则不再提供无参数构造方法
4	public A(String s){
5	}
6	}

```
1 public class B extends A {  
2     public B(){  
3     }  
4 }
```

编译报错了：

```
C:\>javac B.java  
B.java:2: 错误: 无法将类 A 中的构造器 A 应用到给定类型;  
    public B() {  
            ^  
需要: String  
找到: 没有参数  
原因: 实际参数列表和形式参数列表长度不同  
1 个错误
```

图 14-8：编译报错信息

以上程序为什么会编译报错呢？原因是 B 类的构造方法第一行默认会调用“super()”，而 super() 会调用父类 A 的无参数构造方法，但由于父类 A 中提供了有参数构造方法，导致无参数构造方法不存在，从而编译报错了。所以在实际开发中还是建议程序员将无参数构造方法显示的定义出来，这样就可以避免对象的创建失败了。

另外，通过以上内容的学习，还可以得出这样的结论：在 java 语言当中无论是创建哪个 java 对象，老祖宗 Object 类中的无参数构造方法是必然执行的。

接下来我们再来看一下：一个 java 对象在创建过程中比较完整的内存图是如何变化的，请先看以下代码：

```
1 public class People {  
2     String name;  
3     boolean sex;  
4     public People(String name,boolean sex){  
5         this.name = name;  
6         this.sex = sex;  
7     }
```

```
8 }
```

```
1 public class Worker extends People {  
2     //子类特有的工资属性  
3     double salary;  
4     public Worker(String name,boolean sex,double salary){  
5         super(name,sex);  
6         this.salary = salary;  
7     }  
8 }
```

```
1 public class WorkerTest {  
2     public static void main(String[] args) {  
3         Worker w = new Worker("jack",true,10000.0);  
4         System.out.println("姓名: " + w.name);  
5         System.out.println("性别: " + w.sex);  
6         System.out.println("工资: " + w.salary);  
7     }  
8 }
```

运行结果如下图所示：

```
姓名: jack  
性别: true  
工资: 10000.0
```

图 14-9：运行结果

以上程序创建 Worker 对象时构造方法的执行顺序是：

- ① 先执行 Object 类的无参数构造方法；

- ② 再执行 People 类的构造方法;
- ③ 最后执行 Worker 类的构造方法;

注意：虽然执行了三个构造方法，但是对象实际上只创建了一个 Worker。

以上程序的内存结构图是这样变化的：

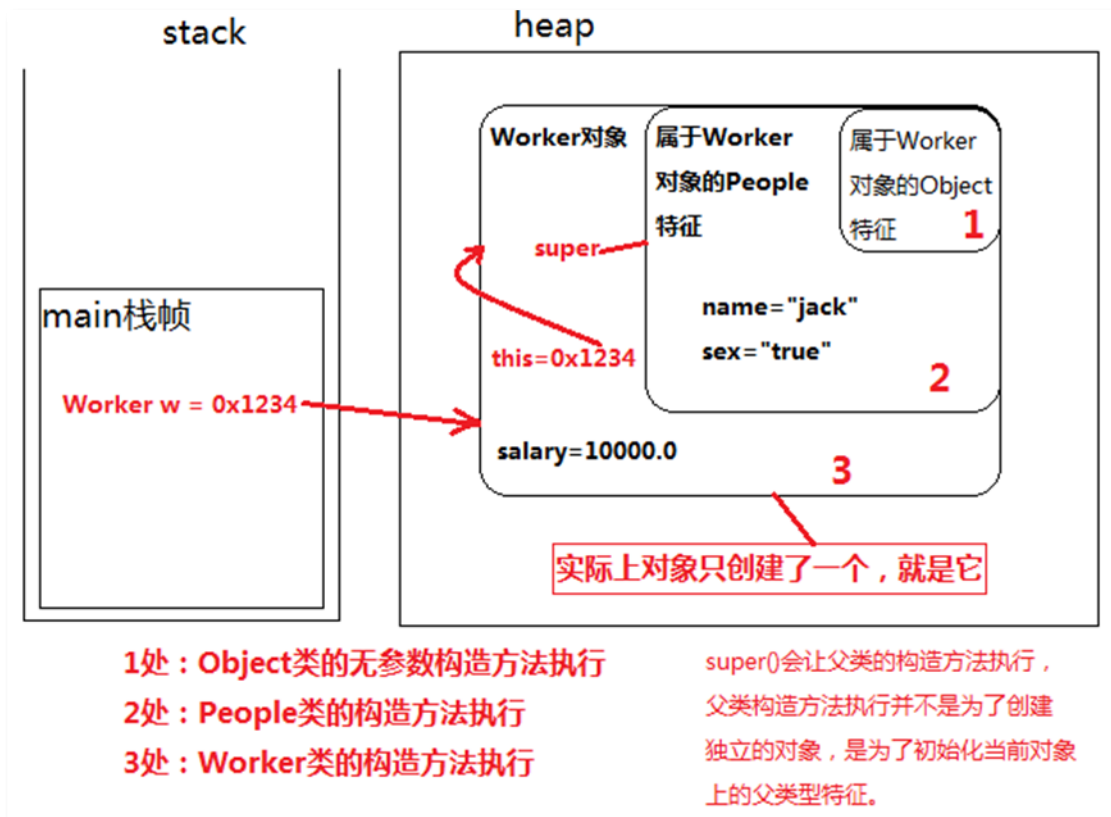


图 14-10：程序执行内存图

通过以上内容的学习，super()的作用主要是：第一，调用父类的构造方法，使用这个构造方法来给当前子类对象初始化父类型特征；第二，代码复用。

14.2.3 super 使用在实例方法中（掌握）

super 和 this 都可以使用在实例方法中，并且都不能使用在静态方法当中，“this”大部分情况下都是可以省略的，只有在方法中区分局部变量和实例变量的时候不能省略。那“super”什么时候可以省略，什么时候不能省略呢？

```
1 //书
2 public class Book {
3     //书名
4     String name;
5     //构造方法
6     public Book(){
7         super();
8     }
9     public Book(String name){
10        super();
11        this.name = name;
12    }
13 }
```

```
1 //纸质书
2 public class PaperBook extends Book {
3     //构造方法
4     public PaperBook(){
5         super();
6     }
7     public PaperBook(String name){
8         super();
9         this.name = name;
10    }
11    //打印书名
12    public void printName(){
13        System.out.println("this.name->书名 : " + this.name);
14        System.out.println("super.name->书名 : " + super.name);
15    }
16 }
```

```
1 public class BookTest {  
2     public static void main(String[] args) {  
3         PaperBook book1 = new PaperBook("零基础学 Java 卷 I");  
4         book1.printName();  
5     }  
6 }
```

运行结果如下图所示：

```
this.name->书名： 零基础学Java卷I  
super.name->书名： 零基础学Java卷I
```

图 14-11: super 和 this

我们发现 printName()方法中的 super.name 和 this.name 最终输出结果是一样的，这是为什么呢？请看以上程序执行的内存图：

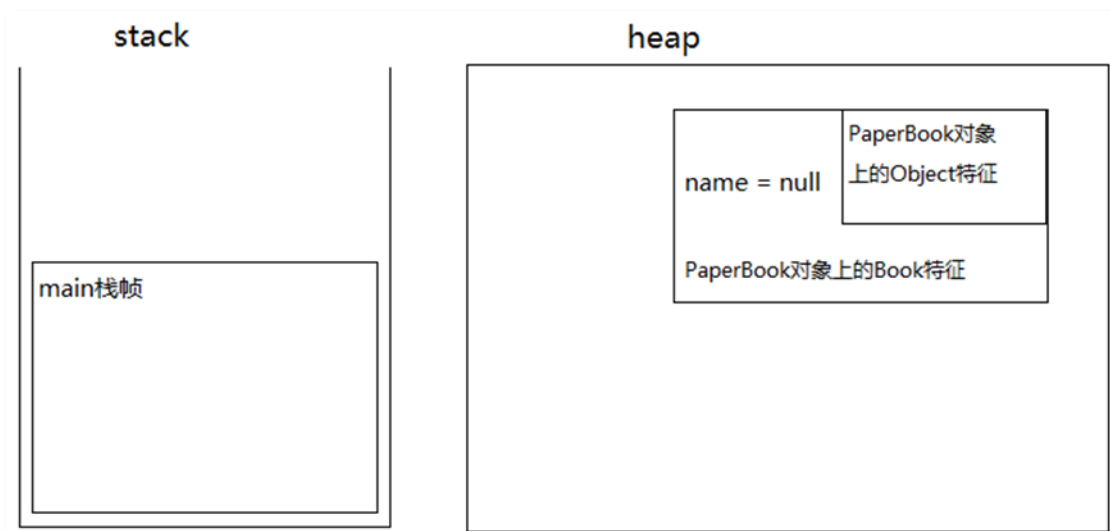


图 14-12: 父类的构造方法执行结束之后的内存图

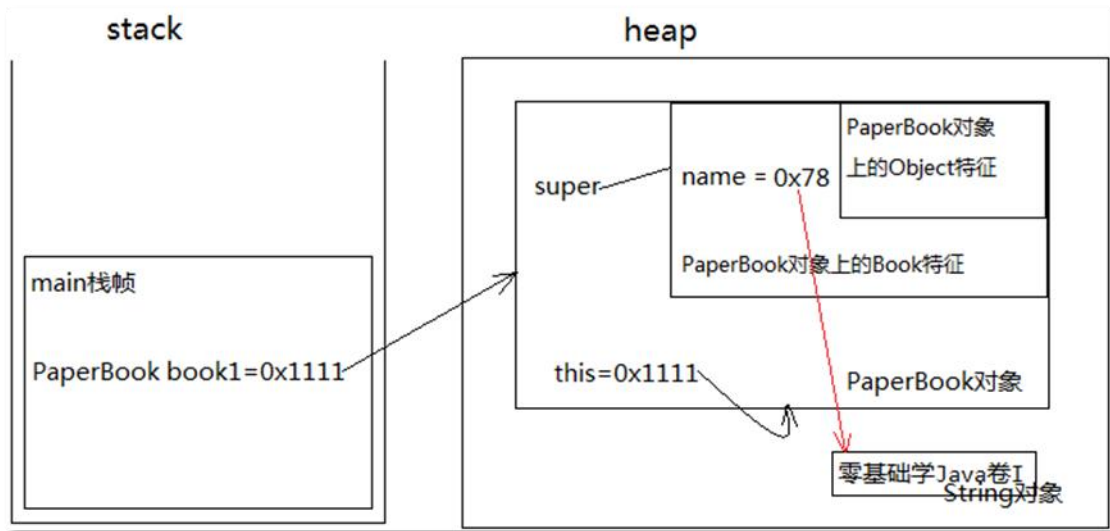


图 14-13: 子类的构造方法执行结束之后的内存图

通过以上内存结构图发现 `this.name` 和 `super.name` 实际上是同一块内存空间，所以它们的输出结果是完全一样的。接下来，我们再把以上的 `PaperBook` 类修改一下：

```

1 //纸质书
2 public class PaperBook extends Book {
3     String name; //在子类中也定义了一个 name 属性
4     //构造方法
5     public PaperBook(){
6         super();
7     }
8     public PaperBook(String name){
9         super();
10        this.name = name; //这里的 this.name 代表子类的 name
11    }
12    //打印书名
13    public void printName(){
14        System.out.println("this.name->书名 : " + this.name);
15        System.out.println("super.name->书名 : " + super.name);

```

16	}
17	}

运行结果如下图所示：

```
this.name->书名： 零基础学Java卷I  
super.name->书名： null
```

图 14-14： super 和 this 的区别

为什么 super.name 是 null 呢，我们一起来看看以上程序的内存图：

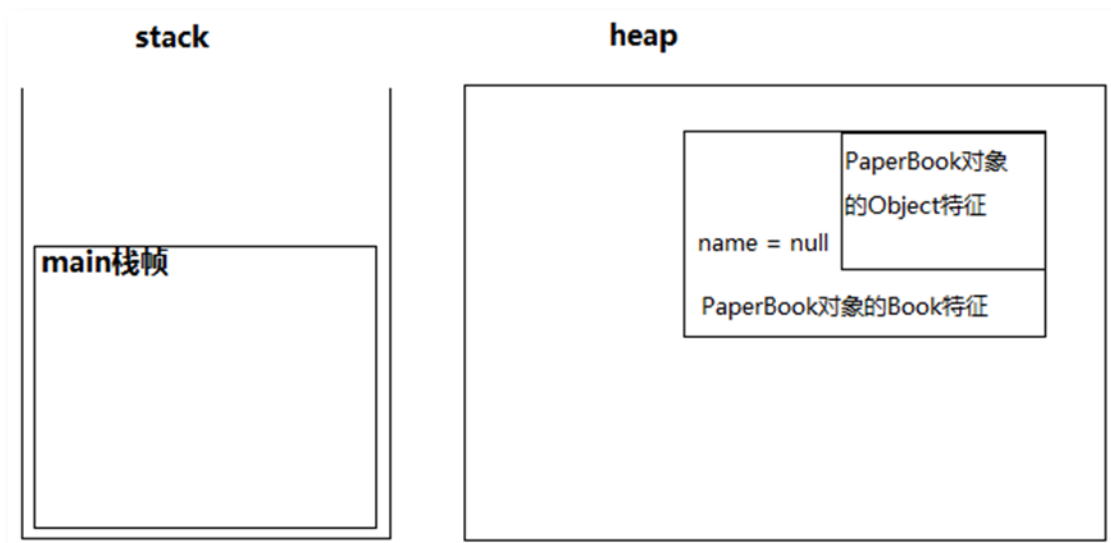


图 14-15： 父类 `Book` 的构造方法执行之后的内存图

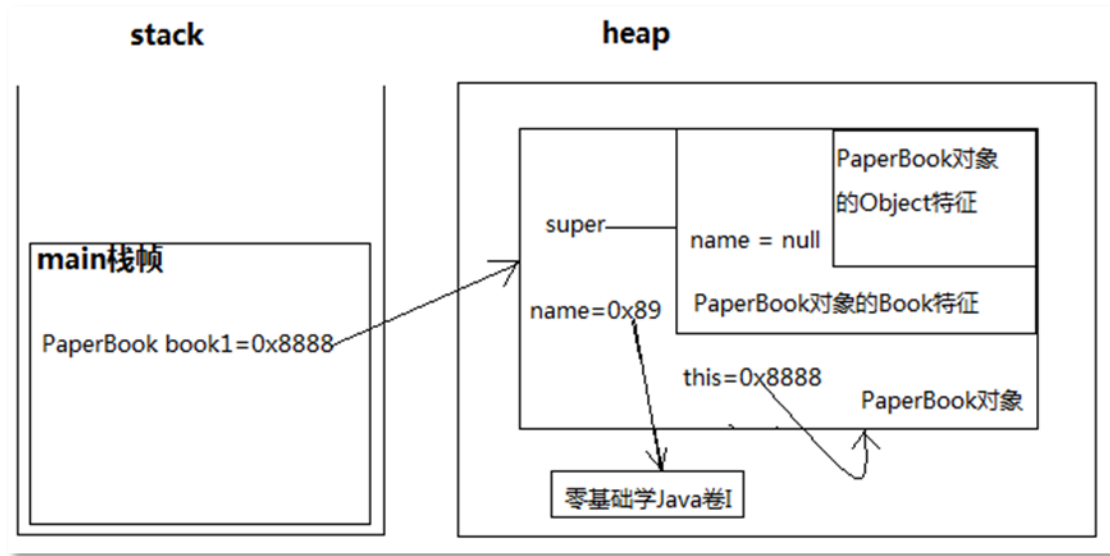


图 14-16: 子类 `PaperBook` 的构造方法执行结束之后的内存图

通过以上内存图可以清楚的看到，父类 `Book` 的构造方法在执行的时候给 `super.name` 赋值 `null`，子类 `PaperBook` 的构造方法在执行的时候给 `this.name` 赋值“零基础学 Java 卷 I”，由于在子类 `PaperBook` 中定义了重名的变量 `name` 导致在当前对象中有两个 `name`，一个是从父类中继承过来的，一个是自己的，如果此时想访问父类中继承过来的 `name` 则必须使用 `super.name`，当直接访问 `name` 或者 `this.name` 都表示访问当前对象自己的 `name`。

通过以上的学习，大家知道 `super` 在什么情况下不能省略了吗？当父类中有该实例变量，子类中又重新定义了同名的实例变量，如果想在子类中访问父类的实例变量，`super` 不能省略。实例方法是这样吗？我们可以来测试一下，请看代码：

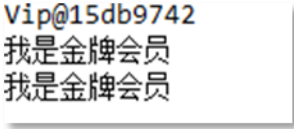
```

1 public class Vip {
2     //Vip 默认继承 Object
3     //重写从 Object 类中继承过来的 toString()方法
4     public String toString(){
5         return "我是金牌会员";
6     }
7     public void test(){
8         System.out.println(super.toString());
9         System.out.println(this.toString());
    
```

```
10      System.out.println(toString());
11  }
12 }
```

```
1 public class VipTest {
2     public static void main(String[] args) {
3         Vip vip = new Vip();
4         vip.test();
5     }
6 }
```

运行结果如下图所示：

A screenshot of a Java application's output. It shows three lines of text: 'Vip@15db9742' in blue, '我是金牌会员' in black, and another '我是金牌会员' in black. The text is displayed in a simple, monospaced font against a white background with a light gray border.

Vip@15db9742
我是金牌会员
我是金牌会员

图 14-17：运行结果

通过以上测试得出最终结论：父类和子类中有同名实例变量或者有同名的实例方法，想在子类中访问父类中的实例变量或实例方法，则 `super` 是不能省略的，其它情况都可以省略。

14.3 章节小结

通过本章节内容的学习，主要理解 `super` 代表什么，主要掌握 `super` 关键字的用法，例如在实例方法中 `super` 怎么用，在构造方法中 `super` 又应该怎么用。`super` 和 `this` 可以对比学习。

14.4 难点解惑

Java 中 `super` 存储的是一个父对象的内存地址吗？`this` 保存了内存地址指向了当前对象，那么 `super` 也是保存了内存地址指向了当前对象的父对象吗？

这个理解是错误的，在 Java 程序中创建 Java 对象的时候会调用构造方法，在构造方法执

行之前会先调用父类的构造方法，在这里说明一下，调用父类的构造方法实际上并不是创建父类对象，只是为了完成初始化当前子类对象的父类型特征。所以严格意义上来说 `super` 并不指向任何对象，`super` 只是代表了当前对象中的那部分父类型特征，单独输出 `super`，例如 `System.out.println(super);`是无法编译的。

14.5 章节习题

第一题：请判断以下代码的输出结果

```
1 public class Text {
2     public static int k = 0;
3     public static Text t1 = new Text("t1");
4     public static Text t2 = new Text("t2");
5     public static int i = print("i");
6     public static int n = 99;
7     public int j = print("j");
8     static {
9         print("静态块");
10    }
11    public Text(String str) {
12        System.out.println(++k + ":" + str + "    i=" + i + "    n=" + n);
13        ++i;
14        ++n;
15    }
16    public static int print(String str) {
17        System.out.println(++k + ":" + str + "    i=" + i + "    n=" + n);
18        ++n;
19        return ++i;
20    }
21    public static void main(String args[]) {
22        new Text("init");
```

23	}
24	}

14.6 习题答案

第一题答案:

```
1:j    i=0    n=0
2:t1   i=1    n=1
3:j    i=2    n=2
4:t2   i=3    n=3
5:i    i=4    n=4
6:静态块 i=5    n=99
7:j    i=6    n=100
8:init i=7    n=101
```